

Python 3 : objectif jeux



Didier Müller, août 2016

www.nymphomath.ch



Introduction

0.1. Buts et moyens

Comme son nom l'indique, ce livre a pour but d'apprendre le langage Python en programmant principalement des jeux, qui serviront de prétexte pour découvrir les différentes facettes du langage Python. La difficulté sera progressive : on commencera par des jeux très simples et sans fioritures, pour aller vers des jeux de plus en plus « jolis » et sophistiqués. Chaque jeu permettra d'introduire de nouvelles notions, tout en répétant celles déjà vues auparavant.

0.2. Programmer ?

La **programmation** consiste à « expliquer » en détails à un ordinateur ce qu'il doit faire, sachant qu'il ne comprend évidemment pas une langue humaine, mais seulement ce qu'on appelle un **langage de programmation** (par exemple C, Python, Java, etc.). En d'autres termes, il faudra traduire une idée simple (par exemple « trier des nombres dans l'ordre croissant »), en un vrai raisonnement parfaitement structuré et détaillé, que l'on appelle un **algorithme**.

Un langage de programmation a de nombreux points communs avec un langage humain : il a une syntaxe (l'orthographe des mots) et une grammaire (la façon d'agencer les mots). La différence la plus importante est qu'un langage informatique ne tolère **aucune** erreur, alors que l'on peut mal parler une langue tout en se faisant comprendre quand même.

0.2.1. Types d'erreurs

La programmation est une tâche complexe, et on y commet de nombreuses erreurs.

Erreurs de syntaxe

Un programme ne peut être exécuté que si sa syntaxe est parfaitement correcte. Dans le cas contraire, le processus s'arrête (on parle communément de « plantage ») et vous obtenez un message d'erreur. Le terme syntaxe se réfère aux règles que les auteurs du langage ont établies pour la structure du programme. **Tous** les détails ont de l'importance : le respect des majuscules et des minuscules, l'orthographe, la ponctuation, ...

Erreurs sémantiques

Le second type d'erreur est l'erreur sémantique ou erreur de logique. S'il existe une erreur de ce type dans un de vos programmes, il n'y aura aucun message d'erreur, mais le résultat ne sera pas celui que vous attendiez.

Erreurs à l'exécution

Le troisième type d'erreur est l'erreur en cours d'exécution (*Run-time error*), qui apparaît seulement lorsque votre programme fonctionne déjà, mais que des circonstances particulières se

Pour des raisons anecdotiques, les erreurs de programmation s'appellent des « bugs ». *bug* est à l'origine un terme anglais servant à désigner de petits insectes gênants, tels les punaises. Il est arrivé à plusieurs reprises que des cadavres de ces insectes provoquent des court-circuits et donc des pannes incompréhensibles.

présentent (par exemple, votre programme essaie de lire un fichier qui n'existe plus).



Recherche des erreurs et expérimentation

Débuguer efficacement un programme demande beaucoup de perspicacité et ce travail ressemble à une enquête policière. Vous examinez les résultats, et vous devez émettre des hypothèses pour reconstituer les processus et les événements qui ont logiquement entraîné ces résultats.

0.2.2. Les 5 règles d'or pour bien programmer

Programmer ne suffit pas. Il faut **bien** programmer, de sorte que quelqu'un qui voudra réutiliser votre programme puisse le faire facilement. En un mot, le programme doit être **compréhensible**. La première chose consistera à **décomposer un problème compliqué en plusieurs sous-problèmes simples**, donc à découper le programme en plusieurs sous-programmes.

Voici les règles d'or à respecter :



Règle numéro 1.

Ne pas écrire de longs sous-programmes (pas plus de 10-15 lignes de code).



Règle numéro 2.

Chaque sous-programme doit avoir un objectif clair.



Règle numéro 3.

Ne jamais utiliser les fonctionnalités du langage dont on n'est pas sûr du résultat ou du rôle.



Règle numéro 4 (jamais respectée).

Éviter à tout prix le copier/coller, source d'innombrables erreurs.



Règle numéro 5.

Écrire des commentaires pour expliquer les parties les plus subtiles du programme.

Vous trouverez sur le site web compagnon (voir § 0.4) un extrait des PEP8 PEP257 (Python Extension Proposal) originaux qui reprend les points importants pour l'apprentissage des bonnes méthodes de programmation en Python (voir § 0.3). Les auteurs originaux sont Guido **van Rossum** et Barry **Warsaw**.

0.2.3. Citations de sagesse

« L'enseignement de l'informatique ne peut faire de personne un programmeur expert plus que l'étude des pinceaux et du pigment peut faire de quelqu'un un peintre expert. » - Eric S. Raymond

« Programmer, c'est comme se donner des coups de pied dans le visage, tôt ou tard, votre nez va saigner. » - Kyle Woodbury

« Toujours codez comme si le gars qui finit par maintenir votre code sera un psychopathe violent qui sait où vous vivez. » - John Woods

« Je ne suis pas un excellent programmeur. Je suis juste un bon programmeur avec d'excellentes habitudes. » - Kent Beck

« N'importe quel idiot peut écrire du code qu'un ordinateur peut comprendre. Les bons programmeurs écrivent du code que les humains peuvent comprendre. » - Martin Fowler

« La vérité ne peut être trouvée qu'à un endroit : le code. » - Robert C. Martin



Guido von Rossum

0.3. Python

Python est un langage portable, extensible, gratuit, qui permet (sans l'imposer) une approche modulaire et orientée objet de la programmation. Python est développé depuis 1989 par Guido **van Rossum** et de nombreux contributeurs bénévoles.

Il est apprécié par les pédagogues qui y trouvent un langage où la syntaxe permet une initiation aisée aux concepts de base de la programmation. Enfin, le langage Python gagne en popularité car il est le langage favori des « data scientists », notamment. Il permet également de faire du Web aisément avec *Django*.

0.3.1. Principales caractéristiques du langage

Détaillons quelques caractéristiques de Python :

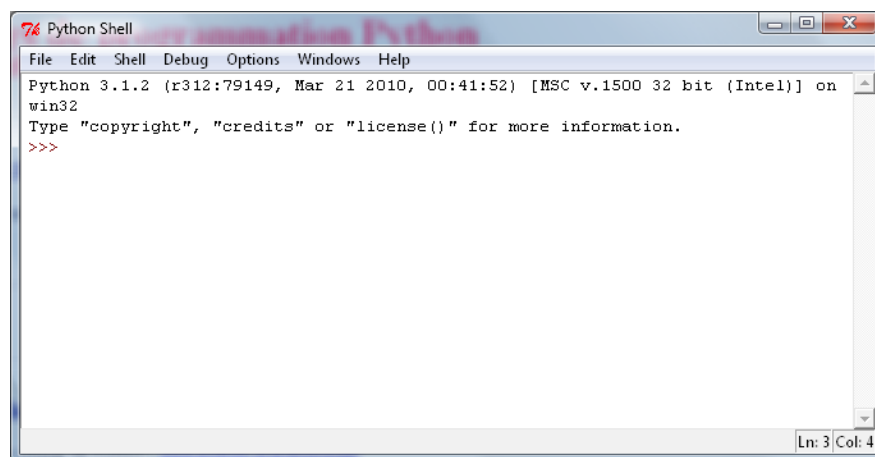
- Python est **portable** : il fonctionne non seulement sur Linux, mais aussi sur MacOS, et les différentes variantes de Windows.
- Python est **gratuit**, mais on peut l'utiliser sans restriction dans des projets commerciaux.
- Python convient aussi bien à des **scripts** d'une dizaine de lignes qu'à des **projets complexes** de plusieurs dizaines de milliers de lignes.
- La **syntaxe** de Python est très simple et, combinée à des **types de données évolués** (listes, dictionnaires,...), conduit à des programmes à la fois très compacts et très lisibles. À fonctionnalités égales, un programme Python (abondamment commenté et présenté selon les canons standards) est souvent de 3 à 5 fois plus court qu'un programme C ou C++ (ou même Java) équivalent, ce qui représente en général un temps de développement de 5 à 10 fois plus court et une facilité de maintenance largement accrue.
- Python est **orienté objet**. Il supporte l'**héritage multiple** et la **surcharge des opérateurs**.
- Python est un langage qui **continue à évoluer**, soutenu par une communauté d'utilisateurs enthousiastes et responsables, dont la plupart sont des supporters du logiciel libre. Nous utiliserons dans ce cours la **version 3**. Il est à noter qu'un programme écrit en Python 2 n'est pas compatible avec la version 3, et nécessitera quelques modifications.

0.3.2. Installation de Python 3

Un guide d'installation est disponible sur le site compagnon (voir § 0.4).

Allez sur le site officiel de Python : www.python.org/download/

1. Choisissez la dernière version (non bêta) adaptée à votre système d'exploitation.
2. Dans le programme d'installation, le plus simple est de cliquer chaque fois sur *Next* jusqu'au bouton *Finish*.
3. Python est maintenant installé. Vous le trouverez dans votre liste de programmes. Pour le démarrer, choisissez dans le répertoire Python le programme **IDLE (Python GUI)**. La fenêtre suivante devrait alors apparaître :



La version (ici 3.1.2) peut varier.

0.4. Site web compagnon

Vous trouverez sur le site associé à ce cours les programmes des jeux, afin d'éviter de perdre du temps à les recopier et afin de pouvoir les tester facilement. Vous trouverez aussi les corrigés des exercices et les ressources à utiliser.

L'adresse est www.nymphomath.ch/pj/

0.5. Contenu du cours

Dans ce cours, les jeux et autres amusements auront une place prépondérante, même si, de temps à autre, surtout dans les premiers chapitres, on trouvera des exercices concernant d'autres sujets.

Première partie : bases de Python

Au chapitre 1, nous aborderons un élément essentiel des jeux : le **hasard**. Nous aborderons du même coup en douceur quelques concepts importants de la programmation : l'affichage, les opérateurs, les boucles et les conditions.

Au chapitre 2, le jeu **Devine mon nombre** permettra de voir deux concepts fondamentaux de la programmation : les conditions et les boucles. Il sera aussi question de variables. On calculera plus loin les probabilités de perdre des armées lors de combats au **Risk**.

Au chapitre 3, on jouera à **Pierre, papier, ciseaux** contre l'ordinateur, ce qui nous permettra d'introduire les procédures et les fonctions.

On retrouvera ce jeu au chapitre 4, mais cette fois dans une version graphique. On calculera aussi des probabilités avec **Let's make a deal !**

Au chapitre 5, nous découvrirons le **jeu de Juniper Green**, où nous devrons utiliser des listes. Nous analyserons aussi le célèbre jeu pour enfants **Le verger**.

Au chapitre 6, nous aborderons un des piliers de la théorie des jeux : le **dilemme du prisonnier**, qui nous permettra de découvrir les dictionnaires et les fonctions lambda.

Au chapitre 7, nous utiliserons plusieurs jeux de lettres : **le pendu**, **le mot le plus long**, **le Scrabble** et **Motus**, ce qui nous amènera tout naturellement à étudier les fichiers et les chaînes de caractères.

Deuxième partie : graphisme

Nous programmerons une version graphique du pendu au chapitre 8.

Le chapitre 9 est un peu spécial, puisqu'il faudra écrire un programme qui nous permettra de jouer au **Memory** contre l'ordinateur et qu'il pourra déboucher sur un travail de groupe.

Dans le chapitre 10, nous utiliserons le **Blackjack** pour introduire l'importante notion de programmation orientée objet.

Le chapitre 11, exceptionnellement, ne concernera pas directement les jeux, mais sera quand même amusant : il sera en effet question de dessins et nous verrons comment utiliser des visages stylisés pour représenter des données statistiques.

Qui dit jeux dit souvent damiers. Au chapitre 12, nous approfondirons le chapitre précédent en dessinant différents damiers (qui nous permettront de programmer le jeu **Le loup et les moutons** et **la course de la dame**) et nous nous prendrons un moment pour le peintre Vasarely.

Le chapitre 13 traitera d'un classique de la programmation pour débutants : les automates cellulaires, et en particulier le célèbre **jeu de la vie** de **Conway**. Nous programmerons aussi le casse-tête **Gasp**.

Au chapitre 14, nous aborderons la récursivité avec le célèbre jeu du **démineur**. Nous verrons aussi comment **sortir d'un labyrinthe**.

Le chapitre 15 sera la cerise sur le gâteau. En effet, nous allons programmer un **jeu d'échecs**. En fait, pas tout à fait. Nous allons faire ce qui se fait très souvent en informatique : partir d'un programme (relativement simple) disponible sur le web, puis l'améliorer. Cela demandera évidemment de le comprendre avant de le retoucher... Nous créerons dans un premier temps une interface graphique, puis, dans un deuxième temps, nous essaierons de le rendre plus fort.

Enfin, au chapitre 16, nous verrons comment bouger des images lors d'une **course d'escargots**, et d'un **Space Invaders** simplifié. Nous retrouverons aussi avec nostalgie le premier jeu vidéo à avoir connu un succès populaire : **Pong**. Finalement, nous verrons comment simuler une **planche de Galton**.

0.6. Remarques sur les exercices

Ce livre est destiné à l'enseignement dans des classes. L'une des difficultés de l'enseignement de l'informatique est que chaque élève a son propre rythme, et que les niveaux des élèves lors des premières leçons peuvent être très différents. Cela signifie qu'après quelques leçons déjà, certains élèves peuvent avoir plusieurs chapitres d'avance sur les plus lents. Pour y remédier, il faut avoir quelques exercices supplémentaires pour occuper les plus rapides.

En face de chaque exercice, vous trouverez une de ces trois icônes :



indique un exercice relativement facile que tous les élèves devront faire.



indique un exercice que tous les élèves devront faire, mais qui est un peu plus difficile. S'ils y passent trop de temps, ils auront avantage à essayer de regarder et comprendre le corrigé.



indique un exercice difficile que les élèves le plus rapides pourront faire s'ils le souhaitent, en attendant les élèves les plus lents.

De plus, 256 exercices plus courts sont disponibles dans le **Défi Turing**, qui propose des problèmes de maths où l'informatique sera souvent nécessaire.

L'adresse est www.nymphomath.ch/turing/

Presque toutes les bases théoriques pour faire ces exercices seront acquises à la fin du chapitre 7.



0.7. Ce que vous avez appris dans l'introduction

- Ce qu'est la programmation et quels sont les types d'erreurs que vous rencontrerez.
- Les 5 règles d'or pour bien programmer.
- Les principales caractéristiques de Python 3 et comment l'installer.



Chapitre 1

Le hasard

1.1. Thèmes abordés dans ce chapitre

- les nombres pseudo-aléatoires
- le module `random`
- `print(...)`
- la boucle `for ... in range(...)`
- la condition `if ... elif ... else`
- opérateurs de comparaisons

1.2. Le hasard dans les jeux

Dans beaucoup de jeux, le hasard a une place plus ou moins importante. Un **jeu de hasard** est un jeu dont le déroulement est partiellement ou totalement soumis à la chance. Celle-ci peut provenir d'un tirage ou d'une distribution de cartes, d'un jet de dé, etc. Lorsque le jeu est totalement soumis au hasard, on parle de jeu de hasard pur. Lorsque le joueur doit déterminer son action en fonction d'événements aléatoires passés ou futurs et de probabilités, on parle plus volontiers de **jeu de hasard raisonné**.

1.2.1. Jeux de hasard pur

- La **bataille** se joue avec un jeu de 32 cartes ou de 52 cartes. Chaque joueur joue la première carte de son paquet mélangé aléatoirement et il existe une règle pour chaque combinaison de carte. Les joueurs n'ont alors rien à décider.
- Le **jeu de l'oie** est un jeu de plateau contenant un chemin de cases et se jouant avec deux dés. Les règles pour chaque case sont fixées, le joueur ne décide donc rien.
- Le **jeu de la Loterie** (ou Loto) se joue en pariant sur plusieurs nombres (choisis ou aléatoires). Chaque nombre est choisi de manière aléatoire et de manière équiprobable.

1.2.2. Jeux de hasard raisonné

- Le jeu de **poker** se joue avec un jeu de 52 cartes. La distribution des cartes est le seul élément aléatoire du jeu. La manière de miser, parier, bluffer, etc. est au choix du joueur.
- Dans la plupart des jeux de cartes, comme le **bridge** ou le **jass**, la distribution des cartes est l'élément aléatoire.
- Les cartes du jeu **Magic : l'assemblée** sont des cartes spéciales pour ce jeu, avec des actions spécifiques pour chacune d'entre elles. Le hasard provient du mélange de ces cartes, le paquet de cartes étant construit par les joueurs eux-mêmes. Pour certaines cartes, on utilise également un dé ou un jeu de pile ou face.

- Le **Backgammon** se joue sur un tablier (plateau) avec quinze pions et deux dés. L'avancement des pions sur le tablier s'effectue en fonction des valeurs des dés, mais le joueur choisit quels pions il avance.
- Dans le jeu de **Monopoly**, l'avancement des pions se fait de manière aléatoire avec un dé, des cartes actions sont tirées de manière aléatoire, cependant les stratégies d'achat de terrains et de maisons se font par les joueurs.
- Le **Yam's** (ou **Yahtzee**) se joue avec cinq dés. Le but est de réaliser des figures en trois lancers de dés. Le joueur choisit quels dés il relance.
- Dans certains jeux comme le **blackjack**, le joueur joue contre la banque qui a un jeu totalement aléatoire.

1.2.3. Autres

La stratégie optimale du jeu de pierre-papier-ciseaux (lorsque son adversaire joue de manière aléatoire équiprobable), est également de jouer de manière aléatoire équiprobable. Le jeu est alors un jeu de hasard pur. Cependant il est difficile de générer une suite de valeurs aléatoires par soi-même. Ainsi, si votre adversaire joue avec une certaine stratégie, vous pouvez chercher à obtenir une stratégie optimale. On peut parler de jeu de hasard raisonné. Ce jeu échappe donc au classement jeu de hasard pur/jeu de hasard raisonné.

Il existe évidemment aussi des jeux où le hasard n'a pas sa place : les échecs, les dames, le morpion, etc. Quoique... il faut bien déterminer d'une manière ou d'une autre qui commence.

1.3. Comment générer du hasard sur ordinateur?

En informatique, le « vrai » hasard n'existe pas ! On ne fait qu'imiter le hasard. C'est moins facile que l'on pourrait le croire, car un ordinateur n'improvise pas : il ne sait que suivre son programme. Il lui est donc difficile de produire, à partir d'une procédure purement mathématique, des chiffres réellement aléatoires de façon totalement imprévisible. Si on connaît la procédure mathématique, on peut obtenir la même suite de nombres. Ce n'est donc pas du hasard. Il faut se contenter de produire des séquences de nombres qui ont toutes les apparences du hasard. Dans la pratique, on se contente de nombres « pseudo-aléatoires » générés à partir d'une variable difficile à reproduire.

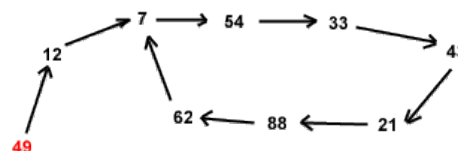
Pour obtenir une réelle dose d'imprévu, il faut faire intervenir dans le calcul une variable extérieure dont la valeur est imprévisible. L'une des méthodes les plus sûres consiste à relier un ordinateur à une source radioactive, puisqu'il est impossible de prédire le moment où un atome émettra un rayonnement radioactif.

1.3.1. Nombres pseudo-aléatoires

Une **séquence pseudo-aléatoire** (*Pseudo Random Sequence* en anglais) est une suite de nombres entiers $x_0, x_1, x_2, \dots$ prenant ses valeurs dans l'ensemble $M = \{0, 1, 2, \dots, m-1\}$. Le terme x_n ($n > 0$) est le résultat d'un calcul (à définir) sur le ou les termes précédents. Le premier terme x_0 est appelé le **germe** (*seed* en anglais).

Algorithme général de récurrence

1. Choisir x_0 dans M
2. Poser $x_{n+1} = f(x_n)$, pour $n > 0$
où f est une application adéquate de M dans M .



Une suite construite comme indiqué dans l'algorithme ci-dessus **contient toujours un cycle** de nombres se répétant à partir d'un certain endroit. Pour éviter de devoir employer plusieurs fois le même nombre au cours d'une simulation, on cherchera à rendre la longueur de ce cycle, appelée **période**, aussi grande que possible.

Définir la fonction f est tout un art. Le lecteur intéressé pourra en savoir plus en allant sur la page : www.nymphomath.ch/random

1.3.2. Le hasard dans Python

Le langage Python dispose d'un module `random`, qui contient plusieurs fonctions qui nous seront utiles pour générer du hasard, par exemple simuler le jet d'un dé, mélanger des cartes, tirer une carte au sort, etc.

1.4. Le module `random`

Il est temps d'écrire notre premier programme en Python. Traditionnellement, il est d'usage d'écrire « Hello world ! » à l'écran.

```
print("Hello world !")
```

On peut aussi utiliser des apostrophes au lieu des guillemets :

```
print('Hello world !')
```

Nous allons maintenant utiliser le module `random`. Pour cela, il faut d'abord l'importer :

```
from random import *
```

L'étoile indique que l'on importe toutes les fonctions que contient le module. On peut aussi énumérer les fonctions qu'il nous faut, par exemple s'il ne nous en faut qu'une :

```
from random import randint
```

La documentation complète de ce module est disponible dans la documentation officielle Python, à l'adresse : <http://docs.python.org/3.3/library/random.html>. Vous y trouverez la description des fonctions qui ne figurent pas ci-dessous. En effet, nous n'utiliserons que quelques-unes d'entre elles.

1.4.1. `random()`

Parmi les fonctions du module `random`, la plus simple s'appelle... `random()`. Remarquez les parenthèses ! Cette fonction retourne un nombre réel entre 0 (compris) et 1 (non compris).

L'exécution de ce programme produira un résultat différent à chaque fois :

```
from random import random
print(random())
```

a écrit : 0.23474765730874114

mais, si vous essayez, vous obtiendrez très certainement autre chose...

Il est parfois utile, pour déboguer un programme par exemple, ou pour faire plusieurs simulations identiques, de choisir le germe de la séquence pseudo-aléatoire :

```
from random import random, seed
seed(666,3)
print(random())
```

écrira chaque fois : 0.45611964897696833

Le paramètre 666 est le germe, 3 est la version de Python utilisée.

1.4.2. `randint()`

La fonction `randint(a,b)` retourne un nombre entier entre `a` et `b` (bornes comprises). On peut ainsi facilement simuler le jet d'un dé :

```
from random import randint
print(randint(1,6))
```

Il est fastidieux de toujours relancer le programme pour faire un lancer de dé. On peut faire 100 lancer avec une **boucle** :

```
from random import randint
for x in range(100):
    print(randint(1,6), end=" ")
```

a produit les 100 résultats suivants :

```
3 5 6 1 6 6 4 1 3 1 6 4 6 1 4 4 2 6 3 1 6 3 6 3 4 3 1 4 4 3 6 4 4 6 3 6 1 1
6 5 6 2 2 6 2 3 1 6 1 6 2 6 6 5 2 5 1 6 4 4 1 6 4 6 6 1 6 4 3 5 4 2 2 5 3 1
3 1 5 2 4 4 3 5 2 6 1 1 6 1 6 2 3 1 2 6 5 5 5
```

La **variable** `x` sert juste à compter de 0 à 99.

Remarquez bien le **décalage** vers la droite de la ligne `print(...)`. En Python, contrairement aux autres langages de programmation les plus courants, cette **indentation** n'est pas seulement esthétique, cela fait partie de la syntaxe ! Si ce *bloc* n'est pas indenté, une fenêtre avec le message « **expected an indented block** » apparaîtra.

L'instruction `end=" "` permet de séparer les valeurs par un espace plutôt que par un saut de ligne. On aurait pu séparer les valeurs par autre chose, une virgule par exemple : `end=","`. Essayez !



Exercice 1.1

1. Écrivez un programme qui simule le jet de trois dés à six faces (on additionne le résultat des trois dés).
2. Écrivez un programme qui simule 50 jets de trois dés à six faces.
3. Écrivez un programme qui simule 200 jets de deux dés à huit faces.

1.4.3. choice()

En Python, on peut créer et utiliser des **listes** (de lettres, de mots, de chiffres, ...). Il suffit de mettre les éléments entre crochets :

```
liste = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
```

La fonction `choice(liste)` permet de tirer au sort un de ces éléments :

```
from random import choice
liste = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
print(choice(liste))
```

a produit comme résultat : b

Dans une liste, les caractères doivent être mis en guillemets ou entre apostrophes. Ce n'est pas nécessaires avec les nombres.



Exercice 1.2

Trouvez un moyen de simuler un dé pipé à six faces :

- 1 apparaît dans 10 % des cas,
- 2, 3, 4 et 5 dans 15 % des cas
- 6 dans 30 % des cas.

Écrivez un programme qui simule 100 jets de ce dé.



Exercice 1.3

Écrivez un programme que génère un mot de passe de 8 caractères aléatoires, choisis parmi les 26 lettres minuscules et les 10 chiffres.

1.4.4. shuffle()

La fonction `shuffle(liste)` mélange les éléments d'une liste. Idéal pour mélanger un paquet de cartes !

```
from random import shuffle

liste = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
shuffle(liste)
print(liste)
```

a produit comme résultat : ['g', 'c', 'f', 'd', 'a', 'e', 'h', 'b']



Exercice 1.4

Le *Royal Rumble* est une bataille royale exposant 30 catcheurs durant un combat. La particularité du match est que les catcheurs ne sont que deux au départ et qu'un nouvel entrant arrive au bout d'un temps prédéfini, et ceci de façon régulière jusqu'à ce que tous les participants aient fait leur entrée.

Écrivez un programme qui donnera un ordre d'entrée aléatoire des catcheurs, dont voici la liste (2013) :

Wade Barrett, Daniel Bryan, Sin Cara, John Cena, Antonio Cesaro, Brodus Clay, Bo Dallas, The Godfather, Goldust, Kane, The Great Khali, Chris Jericho, Kofi Kingston, Jinder Mahal, Santino Marella, Drew McIntyre, The Miz, Rey Mysterio, Titus O'Neil, Randy Orton, David Otunga, Cody Rhodes, Ryback, Zack Ryder, Damien Sandow, Heath Slater, Sheamus, Tensai, Darren Young, Dolph Ziggler.

1.4.5. sample()

La fonction `sample(liste, k)` tire au sort k éléments d'une liste.

Contrairement à `shuffle()`, la liste n'a pas été modifiée par `sample()` !

```
from random import sample

liste = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
print(sample(liste, 3))
```

a produit comme résultat la sous-liste : ['b', 'h', 'g']



Exercice 1.5

Écrivez un programme qui simule un tirage *Euro Millions* : cinq numéros tirés au sort entre 1 et 50, suivi deux étoiles numérotées de 1 à 11.

Par exemple :



Aide : L'instruction `numeros = list(range(a,b))` crée la liste des entiers entre a (compris) et b (**non** compris).



Exercice 1.6

Le **bingo** est un jeu de société où les nombres tirés au sort sont annoncés les uns à la suite des autres, dans un ordre aléatoire. Supposons que les boules soient numérotées de 1 à 90.

Simulez une partie de bingo. Autrement dit, il faut mélanger les boules.

1.5. Les conditions

Imaginons que l'on veuille simuler un dé pipé à six faces, comme dans l'exercice 1.2, mais avec les probabilités suivantes :

- 1 apparaît dans 12.52 % des cas
- 2 apparaît dans 13.09 % des cas
- 3 apparaît dans 21.57 % des cas
- 4 apparaît dans 19.87 % des cas
- 5 apparaît dans 11.23 % des cas
- 6 apparaît dans 21.72 % des cas

Comme la somme des probabilités doit faire 100 %, c'est-à-dire 1, on va pouvoir utiliser `random()` avec une série de conditions `si... sinon si... sinon si... sinon...` (en Python `if... elif... elif... else...`).

L'idée est de découper l'intervalle $[0, 1[$ en 6 sous-intervalles de différentes largeurs. Si `random()` est plus petit que 0.1252 (12.52 %), alors on écrira 1. Si `random()` est compris entre 0.1252 et 0.2561 ($0.1252+0.1309$), alors on écrira 2. Et ainsi de suite.

```
from random import random

rnd = random()
if rnd < 0.1252:
    print(1)
elif rnd < 0.2561:
    print(2)
elif rnd < 0.4718:
    print(3)
elif rnd < 0.6705:
    print(4)
elif rnd < 0.7828:
    print(5)
else:
    print(6)
```

Remarquez bien le décalage du texte.

On a utilisé une **variable** `rnd` dans laquelle on a stocké le résultat de la fonction `random()`. On a n'avait pas le choix, puisque chaque fois que l'on appelle `random()`, on obtient un résultat différent.

Pour comparer des valeurs, on dispose des opérateurs suivants :

Symbole	Nom	Exemple	Résultat
<	Plus petit que	$0 < 6$	True
>	Plus grand que	$0 > 6$	False
<=	Plus petit ou égal à	$5 <= 5$	True
>=	Plus grand ou égal à	$5 >= 6$	False
==	Égal à	$10 == 10$	True
!=	Différent de	$10 != 11$	True

Tableau 1.1: opérateurs de comparaison

En Python, il est aussi possible de vérifier si un nombre est encadré par deux autres :

```
if 0.3 < rnd < 0.4:
```



Notez bien la différence entre `a=1`, qui est une **affectation** (c'est-à-dire que l'on donne la valeur 1 à la variable `a`), et `a==1` qui est une **comparaison** (on teste si la valeur de `a` est égale à 1).

Il n'y a que deux valeurs possibles pour le résultat d'une comparaison : `True` (vrai) ou `False` (faux) : ce sont des valeurs *booléennes*. Attention aux majuscules !



Exercice 1.7

Simulez 100 jets d'une pièce truquée qui, en moyenne, montre face dans 57.83 % des cas.

Exercice 1.8



Dans le jeu de cartes *Hearthstone*, il y a quatre types de rareté des cartes : il y a environ 1 % de cartes « légendaires », 4 % d'« épiques », 23 % de « rares » 72 % de « communes ».

Sachant que chaque paquet doit contenir au moins une carte rare ou supérieure, générez un paquet de 5 cartes tirées au sort en respectant ces proportions.

Un paquet sera simplement écrit sous la forme : L C R E C

Si un paquet ne contient que des communes, on indiquera que ce paquet n'est pas valide.



Exercice 1.9

Les cinq solides platoniciens peuvent être utilisés comme dés réguliers, car toutes leurs faces sont identiques.

Les cinq polyèdres réguliers convexes (solides de Platon)				
Tétraèdre	Hexaèdre ou Cube	Octaèdre	Dodécaèdre	Icosaèdre

On veut générer un nombre aléatoire de la manière suivante :

1. on lance le tétraèdre pour choisir lequel des quatre autres dés on va utiliser ;
2. on lance ce dé et on écrit le résultat.

Générez 100 nombres aléatoires en suivant ce protocole.



1.6. Ce que vous avez appris dans ce chapitre

- Le « vrai » hasard n'existe pas en informatique. On génère une suite de nombres qui a l'apparence du hasard (§ 1.3)
- C'est dans le module externe `random` que se trouvent les fonctions pseudo-aléatoires (§ 1.4)
- Vous savez écrire des choses à l'écran avec la fonction `print()`.
- Vous avez découvert trois concepts fondamentaux des langages de programmation, dont nous reparlerons plus en détails dans le chapitre 2 :
 - les boucles (§ 1.4.2),
 - les variables,
 - la condition `si... sinon` (§ 1.5).
- Vous savez comment définir une liste (§ 1.4.3). Le chapitre 5 sera essentiellement consacré à cette structure de données.
- Vous connaissez tous les opérateurs de comparaison (tableau 1.1). Attention, en particulier, à ne pas confondre `=` et `==`. C'est une erreur fréquente.



Chapitre 2

Devine mon nombre !

2.1. Thèmes abordés dans ce chapitre

- commentaires
- modules externes, import
- variables
- boucle `while`
- condition : `if... elif... else`
- la fonction de conversion `int`
- `input()`
- exceptions

2.2. Règles du jeu

Ce jeu est très simple. L'ordinateur tire un nombre au hasard entre 1 et 30 et vous avez cinq essais pour le trouver. Après chaque tentative, l'ordinateur vous dira si le nombre que vous avez proposé est trop grand, trop petit, ou si vous avez trouvé le bon nombre.

Exemple de partie

```
J'ai choisi un nombre entre 1 et 30
A vous de le deviner en 5 tentatives au maximum !
Essai no 1
Votre proposition : 15
Trop petit
Essai no 2
Votre proposition : 22
Trop grand
Essai no 3
Votre proposition : 17
Trop grand
Essai no 4
Votre proposition : 16
Bravo ! Vous avez trouvé 16 en 4 essais
```

Remarque : les nombres en gras ont été entrés au clavier par le joueur.

2.3. Code du programme



devine.py

<http://ow.ly/bVSzK>

```
# Devine mon nombre
from random import randint

nbr_essais_max = 5
nbr_essais = 1
borne_sup = 30
mon_nombre = randint(1, borne_sup) # nombre choisi par l'ordinateur
ton_nombre = 0 # nombre proposé par le joueur

print("J'ai choisi un nombre entre 1 et", borne_sup)
print("A vous de le deviner en", nbr_essais_max, "tentatives au maximum !")

while ton_nombre != mon_nombre and nbr_essais <= nbr_essais_max:
    print("Essai no ", nbr_essais)
    ton_nombre = int(input("Votre proposition : "))
    if ton_nombre < mon_nombre:
        print("Trop petit")
    elif ton_nombre > mon_nombre:
        print("Trop grand")
    else:
        print("Bravo ! Vous avez trouvé", mon_nombre, "en", nbr_essais, "essai(s)")
    nbr_essais += 1

if nbr_essais > nbr_essais_max and ton_nombre != mon_nombre:
    print("Désolé, vous avez utilisé vos", nbr_essais_max, "essais en vain.")
    print("J'avais choisi le nombre", mon_nombre, ".")
```

2.4. Analyse du programme

Reprenons ce programme ligne par ligne pour l'expliquer en détails.

2.4.1. Commentaires

```
# Devine mon nombre
```

Ceci est un commentaire. Les commentaires n'ont pas d'influence sur le programme lui-même ; ils sont là pour aider à la lecture et à la compréhension du code.



Règle 1

Le commentaire ne doit pas être redondant avec le code. Inutile de commenter des choses évidentes ! D'une manière générale, mieux le code est écrit, moins il y aura besoin de commentaires.



Règle 2

Pour déterminer ce qu'il faut indiquer dans le commentaire, se poser la question « pourquoi ? » et non pas « comment ? ». En effet, on arrivera souvent à comprendre ce que fait une fonction sans commentaires, mais on ne verra pas toujours son utilité.

2.4.2. Variables

```
nbr_essais_max = 5
nbr_essais = 1
borne_sup = 30
mon_nombre = randint(1, borne_sup) # nombre choisi par l'ordinateur
ton_nombre = 0 # nombre proposé par le joueur
```

Nous avons ici cinq variables qu'il faut initialiser. Cela signifie qu'il faut leur donner une valeur de départ. Si on ne le fait pas, l'interpréteur Python va envoyer le message d'erreur du genre :

NameError: name 'nbr_essais_max' is not defined

C'est au moment où l'on initialise une variable que l'interpréteur Python la crée. On peut voir une variable comme une boîte qui va contenir une valeur : ce peut être un nombre, une chaîne de caractères, une liste, etc. Écrire `nbr_essais = 1` a pour effet de déposer dans cette boîte la valeur entière 1. On ne pourra pas mettre autre chose que des nombres entiers dans cette variable par la suite.



Dans la variable `mon_nombre` va être stockée une valeur aléatoire entière, qui changera à chaque exécution du programme. Il est à noter que si l'on avait omis la ligne

```
from random import randint
```

l'interpréteur Python aurait écrit le message d'erreur : `NameError: name 'randint' is not defined`

```
print("J'ai choisi un nombre entre 1 et", borne_sup)
print("A vous de le deviner en", nbr_essais_max, "tentatives au maximum !")
```

Ces deux lignes écrivent à l'écran le texte entre guillemets, ainsi que les valeurs contenues dans les variables `borne_sup` et `nbr_essais_max`. En l'occurrence, on verra s'écrire sur l'écran :

```
J'ai choisi un nombre entre 1 et 30
A vous de le deviner en 5 tentatives au maximum !
```

Règles pour les noms des variables

Le nom d'une variable est composé des lettres de a à z, de A à Z, et des chiffres 0 à 9, mais il ne doit pas commencer par un chiffre.

Les lettres accentuées, les cédilles, les espaces, les caractères spéciaux tels que \$, #, @, etc. sont interdits, à l'exception du caractère `_` (souligné). Le tiret (-) est bien sûr interdit puisqu'il correspond aussi à la soustraction.

La casse est significative : `spam` et `Spam` sont des variables différentes !

Python compte 33 **mots réservés** qui ne peuvent pas non plus être utilisés comme noms de variable (ils sont utilisés par le langage lui-même) :



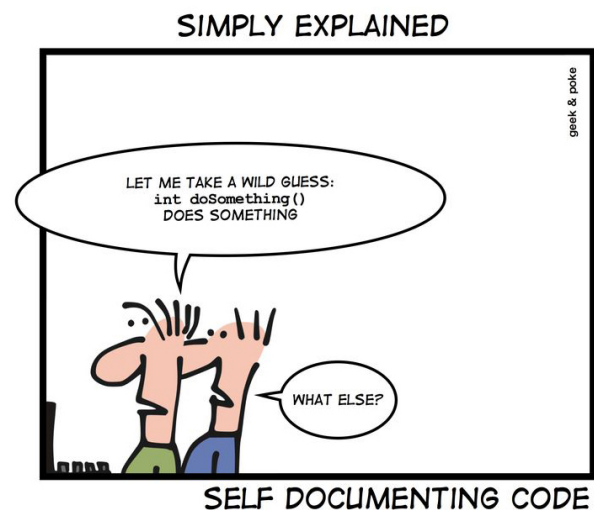
<code>and</code>	<code>as</code>	<code>assert</code>	<code>break</code>	<code>class</code>	<code>continue</code>	<code>def</code>
<code>del</code>	<code>elif</code>	<code>else</code>	<code>except</code>	<code>False</code>	<code>finally</code>	<code>for</code>
<code>from</code>	<code>global</code>	<code>if</code>	<code>import</code>	<code>in</code>	<code>is</code>	<code>lambda</code>
<code>None</code>	<code>nonlocal</code>	<code>not</code>	<code>or</code>	<code>pass</code>	<code>raise</code>	<code>return</code>
<code>True</code>	<code>try</code>	<code>while</code>	<code>with</code>	<code>yield</code>		



Prenez l'habitude d'écrire l'essentiel des noms de variables en caractères minuscules (y compris la première lettre). Il s'agit d'une convention largement respectée. N'utilisez les majuscules ou les soulignés qu'à l'intérieur du nom, pour en augmenter la lisibilité. Par exemple : `finDeMot` ou `fin_de_mot`.



Utilisez des noms de variable qui ont un sens afin d'augmenter la compréhension du programme. Cela vous évitera d'ajouter des commentaires pour expliquer l'utilité de ces variables.



Affectations

En Python, on peut assigner une valeur à plusieurs variables simultanément. Exemple :

```
a = b = 3
```

On peut aussi effectuer des affectations *parallèles* à l'aide d'un seul opérateur :

```
a, b = 3, 2.54
```



Dans cet exemple, les variables `a` et `b` prennent **simultanément** les nouvelles valeurs 3 et 2.54. **Cela est particulièrement utile quand on veut échanger les valeurs de deux variables.** Il suffit d'écrire :

```
a, b = b, a
```

Comme les affectations sont simultanées, les nouvelles valeurs de `a` et `b` seront respectivement 2.54 et 3.

Notons enfin au passage qu'une instruction du type :

```
a + 1 = 3
```

est tout à fait illégale !



Opérations sur les variables entières

Dans notre programme, toutes les variables sont du type entier. Les opérations que l'on peut faire avec les entiers sont les suivantes :

Les priorités sont les mêmes que sur une calculatrice standard. On peut utiliser des parenthèses pour changer les priorités.

Symbole	Nom	Exemple	Résultat
+	Addition	3+4	7
-	Soustraction	8-3	5
*	Multiplication	5*2	10
//	Division entière	14//3	4
%	Reste de la division entière	14%3	2
/	Division	14/3	4.666...
**	Élévation à la puissance	3**4	81

Tableau 2.1: opérateurs sur les nombres entiers

2.4.3. Boucle while (tant que)

```
while ton_nombre != mon_nombre and nbr_essais <= nbr_essais_max:
    print("Essai no ", nbr_essais)
    ton_nombre = int(input("Votre proposition : "))
    if ton_nombre < mon_nombre:
        print("Trop petit")
    elif ton_nombre > mon_nombre:
        print("Trop grand")
    else:
        print("Bravo ! Vous avez trouvé", mon_nombre, "en", nbr_essais, "essai(s)")
    nbr_essais += 1
```

En informatique, on dit *indenté* plutôt que *décalé à droite*.

Voici une *boucle Tant que*. Tant que la valeur stockée dans `mon_nombre` sera différente de la valeur stockée dans `ton_nombre` et que le nombre d'essais effectués sera inférieur ou égal au nombre d'essais maximum, alors toute la partie du code qui est indentée vers la droite sera exécutée en boucle.

Symbole	Nom	Exemple	Résultat
==	égal	3 == 3	True
!=	différent	3 != 4	True
>	supérieur	5 > 5	False
>=	supérieur ou égal	5 >= 5	True
<	inférieur	6 < 2	False
<=	inférieur ou égal	0 <= 1	True

Tableau 2.2: opérateurs de comparaison

2.4.4. Incrémentation

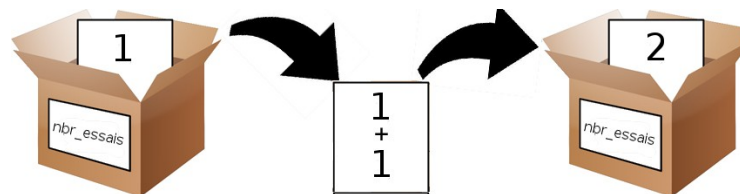
L'incrémentation est une des instructions les plus déroutantes pour un débutant :

```
nbr_essais += 1
```

On peut aussi écrire cette instruction ainsi, sans que cela soit beaucoup plus clair :

```
nbr_essais = nbr_essais + 1
```

Voici ce qui se passe. On prend la valeur de la variable `nbr_essais`, on y ajoute 1, puis on remet le résultat dans la variable `nbr_essais`. Donc, si `nbr_essais` avait la valeur 1, après l'instruction `nbr_essais += 1` il aura la valeur 2.



Beaucoup de boucles se terminent par une incrémentation, comme dans notre programme. Faute de quoi, la condition d'arrêt (ici `nbr_essais <= nbr_essais_max`) ne sera jamais satisfaite, ce qui provoquera une **boucle infinie**. C'est une des erreurs les plus courantes en programmation.



Exercice 2.1

Écrivez un programme qui demande à l'utilisateur d'écrire des mots.

Tant que l'utilisateur n'aura pas pressé sur la touche 'Enter' toute seule, l'ordinateur demandera un nouveau mot. Le programme écrira ce mot, précédé de numéro de rang.

Poliment, l'ordinateur écrira « Au revoir » avant que le programme se termine.

2.4.5. Boucles imbriquées

Il est tout à fait possible, et parfois nécessaire, **d'imbriquer** une boucle dans une autre boucle.

Par exemple, imaginons que l'on veuille écrire toutes les heures et minutes d'une journée. Cela commencera par 0 heure 0 minute, 0 heure 1 minute, 0 heure 2 minutes, ..., 0 heure 59 minutes, 1 heure 0 minute, 1 heure 1 minute, ...

On voit qu'une première boucle parcourra les heures, tandis que la deuxième parcourra les minutes, et que l'on incrémentera l'heure seulement quand 60 minutes seront passées.

Devine mon nombre !

```
h=0
while h<24:
    m=0
    while m<60:
        print(h, "heure(s) ", m, "minute(s)")
        m+=1
    h+=1
```



Exercice 2.2

Ajoutez une boucle au programme ci-dessus pour écrire les secondes.



Exercice 2.3

Partie 1

Écrivez un programme qui affiche les 20 premiers termes de la table de multiplication par 53. Le résultat commencera comme ceci :

```
1 x 53 = 53
2 x 53 = 106
3 x 53 = 159
```

Aide : Il existe une fonction de formatage qui permet d'aligner joliment les nombres en colonne. Par exemple,

```
print('{:4d}'.format(a))
```

écrira l'entier a sur 4 espaces et le calera sur la droite.

La page de référence pour le formatage est :

<http://docs.python.org/py3k/library/string.html#string-formatting>

Partie 2

Modifiez la partie 1 pour obtenir un programme qui affiche toutes les tables de multiplication de 2 à 12. Chaque table comprendra 12 lignes. Alignez joliment les nombres dans les tables.

2.4.6. Conversion de types

```
ton_nombre = int(input("Votre proposition : "))
```

La fonction `input` permet à l'utilisateur d'entrer une chaîne de caractères (*string* en anglais) au clavier. En écrivant `int(input())`, on transforme le type `string` en un type entier. La variable `ton_nombre` contiendra donc un nombre entier, qui pourra être utilisé comme tel.



Exercice 2.4 : livret

Écrivez un programme Python pour tester votre livret. Le programme demandera d'abord quel est le plus grand nombre (*Nmax*) qu'il pourra utiliser. Puis il proposera 10 questions de livret avec deux nombres tirés au sort dans l'intervalle `[2, Nmax]`.

En cas de réponse erronée, le programme reposera la même question, tant que la bonne réponse n'aura pas été donnée.

Exemple d'une partie

```

nombre maximum ? 15

6 x 12 = 72
14 x 7 = 98
8 x 9 = 72
5 x 12 = 60
13 x 14 = 183
Faux ! Réessayez !
13 x 14 = 192
Faux ! Réessayez !
13 x 14 = 182
4 x 5 = 20
9 x 7 = 63
8 x 8 = 64
12 x 14 = 168
10 x 12 = 120

```

2.4.7. Conditions

```

if ton_nombre < mon_nombre:
    print("Trop petit")
elif ton_nombre > mon_nombre:
    print("Trop grand")
else:
    print("Bravo ! Vous avez trouvé",mon_nombre,"en",nbr_essais,"essais")

```

Quand le joueur propose un nombre, il y a trois possibilités : soit son nombre est trop petit, soit il est trop grand, soit c'est le bon nombre. Ces trois possibilités correspondront à trois réponses différentes de l'ordinateur.

Cela se traduira en Python par l'utilisation des instructions `if... elif... else...`. On aurait pu écrire plusieurs instructions au lieu d'une. Il aurait suffi de garder le même décalage. Par exemple :

```

if ton_nombre < mon_nombre:
    print("Trop petit")
    print("Dommage!")

```

Exercice 2.5 : livret sous pression du temps



Améliorez le programme de l'exercice 2.4 : à la fin de la partie, affichez le nombre d'erreurs (avec la bonne orthographe) et le temps utilisé pour répondre aux dix questions.

Aide : dans le module `time` se trouve la fonction `time()`, qui donne le nombre de secondes écoulées depuis le 1^{er} janvier 1970 à 00:00:00.



Exercice 2.6 : calcul mental

Améliorez le programme de l'exercice 2.5. Cette fois-ci, on ne veut pas se contenter d'exercer les multiplications, mais aussi l'addition, la soustraction et la division entière. L'opération à tester sera tirée au sort pour chaque question.



Exercice 2.7

Écrivez un programme qui simule 1000 lancers d'une pièce de monnaie. Vous afficherez seulement le nombre de piles et le nombre de faces obtenus.

Écrivez une version avec une boucle `for` (voir chapitre 1) et une autre avec une boucle `while`.



Exercice 2.8 : les paquets de cartes *Hearthstone*

Améliorez le programme de l'exercice 1.8.

Générez n paquets de cartes *Hearthstone* valides.

Comptez le nombre total de cartes de chaque rareté afin de vérifier que les pourcentages obtenus sont « proches » des pourcentages théoriques (plus vous générerez de paquets, plus les pourcentages seront proches de la théorie).

Exemple de sortie avec 10 paquets (50 cartes) :

```
R C R C C
C R C C C
C R C C L !
C R R C C
C C C C C non valide
C C C R C
C C R R C
R C C L C !
C C C C C non valide
C C C R C
R C C C C
E C E C C
```

```
2 légendaire(s) : 4.00 %
2 épique(s) : 4.00 %
12 rare(s) : 24.00 %
34 communes : 68.00 %
```

Vous ajouterez un point d'exclamation après les paquets contenant au moins une légendaire.



Exercice 2.9

Modifiez le code du § 2.3 :

1. Avant de commencer le jeu, le programme demandera le prénom du joueur. Quand René trouvera le bon nombre, le programme le félicitera par la phrase :
Bravo, René ! Vous avez trouvé 16 en 4 essai(s).
2. À la fin de la partie, le programme proposera une nouvelle partie au joueur, qui répondra par oui ou non. Quand le joueur arrêtera de jouer, le programme indiquera le pourcentage de réussite et le nombre moyen de tentatives pour trouver le nombre (on ne comptabilisera le nombre de coups qu'en cas de réussite).

Exemple de partie

```
Quel est votre prénom ? René
J'ai choisi un nombre entre 1 et 30
A vous de le deviner en 5 tentatives au maximum !
Essai no 1
Votre proposition : 15
Bravo, René ! Vous avez trouvé 15 en 1 essai(s)
Voulez-vous rejouer (o/n) ? o
J'ai choisi un nombre entre 1 et 30
A vous de le deviner en 5 tentatives au maximum !
Essai no 1
Votre proposition : 1
Trop petit
Essai no 2
Votre proposition : 9
Trop petit
Essai no 3
Votre proposition : 14
Trop petit
Essai no 4
```

```
Votre proposition : 19
Trop petit
Essai no 5
Votre proposition : 21
Trop petit
Désolé, vous avez utilisé vos 5 essais en vain.
J'avais choisi le nombre 25 .
Voulez-vous rejouer (o/n) ? n
Pourcentage de réussite: 50.0 %
Nombre moyen de tentatives: 1.0
```

2.5. Exceptions



devine-try.py

<http://ow.ly/bVSzK>

Si l'utilisateur entre autre chose qu'un nombre entier, le programme va générer une erreur et s'arrêter. Pour éviter cela, il faut remplacer la ligne :

```
ton_nombre = int(input("Votre proposition : "))
```

par

```
while True:
    try:
        ton_nombre = int(input("Votre proposition : "))
        break
    except ValueError:
        print("Réponse non valide. Réessayez !")
```

C'est une boucle infinie (`while True`) dont le seul moyen de sortir (`break`) est d'entrer un nombre entier. Tant que l'exception `ValueError` est levée, un message d'erreur créé par l'utilisateur est affiché à l'écran et le programme repose la question.



Exercice 2.10

Invertissons les rôles ! C'est l'ordinateur qui essaiera de deviner le nombre que vous avez en tête et vous lui indiquerez si le nombre qu'il proposera est trop petit ou trop grand.

Convention : entrez 1 si le nombre de l'ordinateur est trop grand, 2 s'il est trop petit et 0 si l'ordinateur a trouvé le bon nombre.

Prévoyez les cas où l'utilisateur entre autre chose qu'une des trois réponses attendues.



Exercice 2.11 : Risk

Au jeu « Risk », les combats se déroulent ainsi : l'assaillant désigne par leurs noms le territoire visé et le territoire attaquant. L'assaillant prend trois dés de même couleur (rose). Il lance autant de dés qu'il engage d'armées (3 au maximum).

Exemple : Un assaillant a 4 armées sur l'Ontario. Il attaque le Québec avec maximum 3 armées à chaque coup de dés mais il peut attaquer, s'il le désire, avec seulement 1 ou 2 armées en jetant 1 ou 2 dés. Le défenseur, lui, ne peut lancer que 2 dés au maximum, même s'il a 3 armées ou plus sur son territoire.

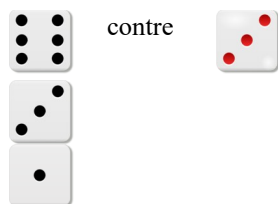


On compare séparément chacun de ses dés avec ceux de l'adversaire en commençant par le plus fort de chaque côté. Les dés les plus forts gagnent. Mais, en cas d'égalité de points, le défenseur l'emporte, même s'il lance moins de dés.

Devine mon nombre !

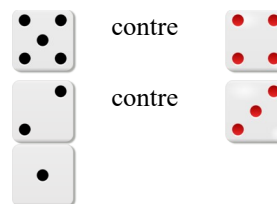
Voici plusieurs configurations de dés. L'assaillant a les dés noirs et le défenseur les dés rouges. L'assaillant peut attaquer avec 1, 2 ou 3 dés; le défenseur peut utiliser 1 ou 2 dés.

Exemple 1



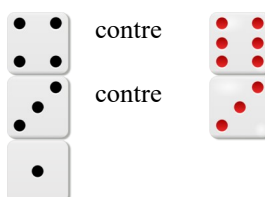
Le défenseur perd 1 armée.

Exemple 2



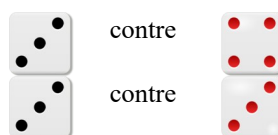
Chaque adversaire perd 1 armée.

Exemple 3



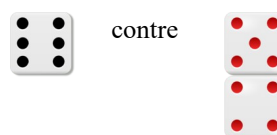
À égalité des points, le défenseur l'emporte. L'assaillant perd 2 armées.

Exemple 4



L'assaillant perd 2 armées.

Exemple 5



Le défenseur perd 1 armée.

Calculez à l'aide d'un programme et notez dans le tableau ci-après la probabilité de chacun des événements y figurant, en fonction du nombre de dés que chacun des deux joueurs choisit de lancer.

		Nombre de dés lancés par le défenseur	
		2	1
Nombre de dés lancés par l'assaillant	3	L'assaillant perd 2 armées : %	L'assaillant perd 1 armée : %
		Le défenseur perd 2 armées : %	Le défenseur perd 1 armée : %
		Chaque joueur perd 1 armée : %	
	2	L'assaillant perd 2 armées : %	L'assaillant perd 1 armée : %
		Le défenseur perd 2 armées : %	Le défenseur perd 1 armée : %
		Chaque joueur perd 1 armée : %	
	1	L'assaillant perd 1 armée : %	L'assaillant perd 1 armée : %
		Le défenseur perd 1 armée : %	Le défenseur perd 1 armée : %



2.6. Ce que vous avez appris dans ce chapitre

- On peut écrire dans le code d'un programme des commentaires qui aideront à sa lisibilité. Il faut savoir les utiliser avec intelligence (§ 2.4.1). Un bon programmeur écrit des programmes lisibles !
- Vous savez tout sur les variables (§ 2.4.2) et comment s'en servir : noms autorisés, initialisation, affectation, incrémentation (§ 2.4.4).
- Il y a plusieurs genres d'affectation en Python : simples, multiples, simultanées (§ 2.4.2).
- En plus de la boucle `for` vue rapidement au chapitre 1, il existe une boucle `while` (§ 2.4.3). On utilise plutôt la boucle `for` quand on sait d'avance combien d'itérations on devra effectuer (par exemple, lancer 100 fois un dé), mais une boucle `while` fait aussi l'affaire. La boucle `while` s'utilise quand on ne sait pas au départ le nombre d'itérations.
- On peut imbriquer des boucles.
- Il y a différents types de données : les chaînes de caractères, les entiers, les réels, les booléens.
- La fonction `input()` retourne toujours une chaîne de caractères (§ 2.4.6). On peut convertir cette chaîne de caractères en un entier (avec `int`) ou en un réel (avec `float`).
- Le tableau 2.1 résume toutes les opérations possibles avec les entiers et le tableau 2.2 les opérateurs de comparaison.
- Les exceptions sont utiles pour éviter des erreurs qui pourraient provoquer l'arrêt du programme (§ 2.5).
- Il existe une fonction de formatage qui permet d'aligner joliment les nombres en colonne (exercice 2.3).



Chapitre 3

Pierre, papier, ciseaux

3.1. Nouveaux thèmes abordés dans ce chapitre

- procédures, fonctions
- paramètres
- variables globales et locales
- effets de bord
- types mutables

3.2. Règles du jeu

Tout le monde connaît le jeu pierre-papier-ciseaux, aussi connu sous le nom de « chifoumi ». Deux joueurs se montrent simultanément leur main qui symbolisera une pierre (poing fermé), un papier (main tendue) ou des ciseaux (l'index et le majeur forment un V).



La pierre bat les ciseaux, les ciseaux battent le papier et le papier bat la pierre. Si les deux joueurs jouent le même symbole, il y a égalité.

Nous allons dans un ce chapitre écrire une version textuelle du jeu. Nous verrons au chapitre 4 comment utiliser les images ci-dessus.

Exemple de partie

```
Pierre-papier-ciseaux. Le premier à 5 a gagné !
1 : pierre, 2 : papier, 3 : ciseaux ? 3
Vous montrez ciseaux - Je montre pierre
vous 0    moi 1
1 : pierre, 2 : papier, 3 : ciseaux ? 2
Vous montrez papier - Je montre papier
vous 0    moi 1
1 : pierre, 2 : papier, 3 : ciseaux ? 2
```


Devine mon nombre !

```
Vous montrez papier - Je montre papier
vous 0    moi 1
1 : pierre, 2 : papier, 3 : ciseaux ? 2
Vous montrez papier - Je montre ciseaux
vous 0    moi 2
1 : pierre, 2 : papier, 3 : ciseaux ? 1
Vous montrez pierre - Je montre ciseaux
vous 1    moi 2
1 : pierre, 2 : papier, 3 : ciseaux ? 1
Vous montrez pierre - Je montre ciseaux
vous 2    moi 2
1 : pierre, 2 : papier, 3 : ciseaux ? 1
Vous montrez pierre - Je montre papier
vous 2    moi 3
1 : pierre, 2 : papier, 3 : ciseaux ? 1
Vous montrez pierre - Je montre papier
vous 2    moi 4
1 : pierre, 2 : papier, 3 : ciseaux ? 3
Vous montrez ciseaux - Je montre papier
vous 3    moi 4
1 : pierre, 2 : papier, 3 : ciseaux ? 2
Vous montrez papier - Je montre ciseaux
vous 3    moi 5
```

Remarque : les nombres en gras ont été entrés au clavier par le joueur.

3.3. Code du programme

```
# jeu pierre, papier, ciseaux
# l'ordinateur joue au hasard

from random import randint

def ecrire(nombre):
    if nombre == 1:
        print("pierre",end=" ")
    elif nombre == 2:
        print("papier",end=" ")
    else:
        print("ciseaux",end=" ")

def augmenter_scores(mon_coup,ton_coup):
    global mon_score, ton_score
    if mon_coup == 1 and ton_coup == 2:
        ton_score += 1
    elif mon_coup == 2 and ton_coup == 1:
        mon_score += 1
    elif mon_coup == 1 and ton_coup == 3:
        mon_score += 1
    elif mon_coup == 3 and ton_coup == 1:
        ton_score += 1
    elif mon_coup == 3 and ton_coup == 2:
        mon_score += 1
    elif mon_coup == 2 and ton_coup == 3:
        ton_score += 1

ton_score = 0
mon_score = 0
fin = 5
print("Pierre-papier-ciseaux. Le premier à",fin,"a gagné !")
no_manche = 0
while mon_score < fin and ton_score < fin:
    ton_coup = int(input("1 : pierre, 2 : papier, 3 : ciseaux ? "))
    while ton_coup < 1 or ton_coup > 3:
        ton_coup = int(input("1 : pierre, 2 : papier, 3 : ciseaux ? "))
    print("Vous montrez",end=" ")
    ecrire(ton_coup)
    mon_coup = randint(1,3)
```



ppc.py

<http://ow.ly/bWDSM>

```
print("- Je montre",end=" ")
ecrire(mon_coup)
print() # aller à la ligne
augmenter_scores(mon_coup,ton_coup)
print("vous",ton_score," moi",mon_score)
```

3.4. Analyse du programme

Reprenons ce programme ligne par ligne pour l'expliquer en détails.

```
# jeu pierre, papier, ciseaux
# l'ordinateur joue au hasard
```

Il est toujours utile de décrire ce que fait le programme au début du code.

```
from random import randint
```

Revoici le module `random` d'où nous importons la fonction `randint(a,b)`, qui renvoie un nombre entier compris entre les bornes `a` et `b`, les bornes étant comprises.

3.4.1. Procédures

Quand les programmes deviennent plus longs, ils deviennent évidemment plus difficiles à comprendre. Aussi est-il fortement conseillé de « découper » le programme en sous-programmes, aussi appelés **procédures** ou **fonctions** (nous verrons la différence entre les deux un peu plus loin). Idéalement, chaque sous-programme devrait être visible entièrement sur l'écran pour faciliter sa lecture, et donc sa compréhension.

Les procédures sont souvent appelées plusieurs fois dans un programme, et elles ont la plupart du temps des **paramètres**. Le résultat de la procédure dépendra du ou des paramètres.

Voici une procédure permettant d'écrire à l'écran soit « pierre », soit « papier », soit « ciseaux ». Cela dépendra du paramètre `nombre`. Ainsi `ecrire(2)` affichera « papier ».

```
def écrire(nombre):
    if nombre == 1:
        print("pierre",end=" ")
    elif nombre == 2:
        print("papier",end=" ")
    else:
        print("ciseaux",end=" ")
```

On remarquera un deuxième paramètre dans les procédures `print : end=" "`. Cela indique à Python qu'il faut continuer d'écrire sur la même ligne, après avoir inséré un espace.

La deuxième procédure du programme est chargée de mettre à jour les scores des joueurs en fonctions de leur coup.

```
def augmenter_scores(mon_coup,ton_coup):
    global mon_score, ton_score
    if mon_coup == 1 and ton_coup == 2:
        ton_score += 1
    elif mon_coup == 2 and ton_coup == 1:
        mon_score += 1
    elif mon_coup == 1 and ton_coup == 3:
        mon_score += 1
    elif mon_coup == 3 and ton_coup == 1:
        ton_score += 1
    elif mon_coup == 3 and ton_coup == 2:
        mon_score += 1
    elif mon_coup == 2 and ton_coup == 3:
        ton_score += 1
```

Les variables `mon_coup` et `ton_coup` sont **globales**. Cela signifie qu'elles ont été déclarées dans le programme principal, donc hors de la procédure. Dans un tel cas, la ligne

```
global mon_score, ton_score
```

est indispensable, sous peine du message d'erreur **UnboundLocalError: local variable 'mon_score' referenced before assignment**.

Les variables `mon_coup` et `ton_coup` ont été passées en arguments. Il n'y a donc pas lieu de les déclarer comme globales.

3.4.2. Effets de bord

En informatique, une procédure est dite à **effet de bord** si elle modifie autre chose que sa valeur de retour. Par exemple, une procédure peut modifier une variable globale, ou modifier un ou plusieurs de ses arguments. Les effets de bord rendent souvent le comportement des programmes difficile à comprendre et sont **à éviter** dans la mesure du possible.

La procédure `augmenter_scores(mon_coup, ton_coup)` est à effet de bord puisqu'elle modifie les variables globales `mon_score` et `ton_score`.

Comment éviter les effets de bord ?

On pourrait, plutôt que de les déclarer comme variables globales, les passer comme arguments de la fonction et les modifier. Mais, en Python, les arguments ne peuvent être modifiés que s'ils sont d'un type **mutable**.



Pour le moment, nous n'avons vu que les deux premiers types du tableau 3.1.

Types	Mutables
Chaîne de caractères	non
Valeur numérique	non
Liste	oui
Tuple	non
Dictionnaire	oui

Tableau 3.1 : Types mutables et non mutables

Comme `mon_score` et `ton_score` sont des entiers, ils ne sont pas mutables et ne pourront pas être modifiés. Cela ne produira pas de message d'erreur, mais les deux scores resteront à 0.

Le paragraphe suivant montre une façon d'éliminer les effets de bord en utilisant des variables locales.

3.4.3. Fonction

La différence entre une fonction et une procédure, c'est qu'une fonction renvoie une (parfois plusieurs) valeurs. La valeur retournée est toujours placée après l'instruction `return`. Une fois cette instruction exécutée, le programme quitte immédiatement la fonction.

Transformons la procédure `augmenter_scores` en une fonction :

```
def augmenter_scores(mon_coup, ton_coup, mon_score, ton_score):
    mon_nouveau_score = mon_score
    ton_nouveau_score = ton_score
    if mon_coup == 1 and ton_coup == 2:
        ton_nouveau_score += 1
    elif mon_coup == 2 and ton_coup == 1:
        mon_nouveau_score += 1
    elif mon_coup == 1 and ton_coup == 3:
        mon_nouveau_score += 1
    elif mon_coup == 3 and ton_coup == 1:
        ton_nouveau_score += 1
    elif mon_coup == 3 and ton_coup == 2:
        mon_nouveau_score += 1
    elif mon_coup == 2 and ton_coup == 3:
        ton_nouveau_score += 1
    return mon_nouveau_score, ton_nouveau_score
```



ppc2.py

<http://ow.ly/bWDSM>

Cette fonction va renvoyer deux valeurs (`mon_nouveau_score` et `ton_nouveau_score`) qu'il faudra stocker dans des variables (dans notre exemple `mon_score` et `ton_score`). On appellera donc cette fonction par cette ligne :

```
mon_score, ton_score = augmenter_scores(mon_coup, ton_coup, mon_score, ton_score)
```



Comme les variables `mon_nouveau_score` et `ton_nouveau_score` sont déclarées dans la fonction `augmenter_scores`, elles seront **locales** à cette fonction et ne pourront pas être appelées depuis l'extérieur, contrairement aux variables **globales**. Si d'aventure il y a dans le code des variables de même nom, que ce soit dans le programme principal ou dans d'autres fonctions, il s'agira de variables **différentes** ! Pour reprendre l'image du chapitre 2, il y aura deux boîtes de même nom mais différentes, ayant probablement des contenus différents.

La durée de vie des variables locales est limitée. Elles n'existent que pendant l'exécution du sous-programme dans lequel elles se trouvent. Leur emplacement en mémoire n'est pas fixé à l'avance comme pour les variables globales mais un nouvel espace mémoire leur est alloué à chaque exécution du sous-programme. Cela a pour conséquence qu'elles ne conservent pas leur contenu d'une exécution à l'autre du sous-programme.



Exercice 3.1

Voici un exercice assez difficile, mais qui résume toutes les subtilités concernant les variables locales, les variables globales, les paramètres et les effets de bord.

Que vont écrire les programmes suivants ?



Avertissement ! Les programmes ci-dessous sont mal écrits et ne sont surtout pas à prendre en exemple...

Vous remarquerez qu'ils se ressemblent beaucoup. Faites bien attention à tous les détails !

Programme 1

```
def truc():
    n=10
    p=p+1

n, p = 5, 20    # on peut initialiser plusieurs variables sur une seule ligne
truc()
print(n,p)
```

Programme 2

```
def truc():
    global p
    n=10
    p=p+1

n, p = 5, 20
truc()
print(n,p)
```

Programme 3

```
def truc():
    global n, p
    n=10
    p=p+1

n, p = 5, 20
truc()
print(n,p)
```

Devine mon nombre !

Programme 4

```
def truc(p) :  
    n=10  
    p=p+1  
  
n, p = 5, 20  
truc(p)  
print(n,p)
```

Programme 5

```
def truc(p) :  
    global p  
    n=10  
    p=p+1  
  
n, p = 5, 20  
truc(p)  
print(n,p)
```

Programme 6

```
def truc(p) :  
    global n  
    n=10  
    p=p+1  
  
n, p = 5, 20  
truc(p)  
print(n,p)
```

Programme 7

```
def truc(n) :  
    global p  
    n=10  
    p=p+1  
  
n, p = 5, 20  
truc(p)  
print(n,p)
```



Exercice 3.2

Réécrivez le programme de l'exercice 2.3 (partie 2), où il était question d'écrire les tables de multiplication de 2 à 12, mais en utilisant une procédure `table(n)` qui écrira la table de multiplication de `n`.



Exercice 3.3

Écrivez une fonction `maximum(a,b,c)` qui renvoie le plus grand des trois nombres `a`, `b`, `c` fournis en arguments.



Exercice 3.4

Écrivez un programme qui convertit des degrés Celsius en degrés Fahrenheit, et vice-versa. Définissez deux fonctions qui effectuent les conversions.

Formules : si F est la température en °F et C la température en °C, alors

$$F = \frac{9}{5}C + 32 \quad \text{et} \quad C = \frac{5}{9}(F - 32)$$



Exercice 3.5

D'après un sondage effectué auprès de 2000 personnes, un humain joue *pierre* dans 41% des cas, *papier* dans 30% des cas, et *ciseaux* dans 29% des cas. Sachant cela, on va tenter de rendre l'ordinateur plus « malin ».

Modifiez le code du § 3.3. Il faudra remplacer la ligne

```
mon_coup = randint(1,3)
```

par

```
mon_coup = coup_ordi()
```

et programmer la fonction `coup_ordi()` qui retournera *papier* dans 41% des cas, *ciseaux* dans 30% des cas et *pierre* dans 29% des cas.



Exercice 3.6

Écrivez une fonction $f(x)$ qui retourne la valeur de $f(x) = 2x^3 - 3x - 1$.

Écrivez une procédure `tabuler` avec quatre paramètres : `f`, `borneInf`, `borneSup` et `pas`. Cette procédure affichera les valeurs de la fonction `f`, pour x compris entre `borneInf` et `borneSup`, avec un incrément de `pas`.

Exemple : `borneInf = -2.0`, `borneSup = 2.0` et `pas = 0.5`.

-2.0	-11.0
-1.5	-3.25
-1.0	0.0
-0.5	0.25
0.0	-1.0
0.5	-2.25
1.0	-2.0
1.5	1.25
2.0	9.0



3.5. Ce que vous avez appris dans ce chapitre

- Les procédures et les fonctions permettent de découper le programme en sous-programmes. Cela permet d'augmenter la lisibilité et d'éviter d'écrire plusieurs fois la même chose.
- La différence entre une fonction et une procédure, c'est qu'une fonction renvoie une ou plusieurs valeurs (§ 3.4.3).
- Il existe des variables globales et des variables locales (§ 3.4.3).
- Les effets de bord sont à éviter, si possible (§ 3.4.2).
- Il existe des types mutables et d'autres non mutables (tableau 3.1). On ne peut pas modifier les valeurs de type non mutable.



Chapitre 4

Pierre, papier, ciseaux

(version graphique)

4.1. Thèmes abordés dans ce chapitre

- Le module tkinter : Label, Button
- Fenêtre
- Événements
- Réceptionnaire d'événements

4.2. Règles du jeu

Voir § 3.2.

Exemple de partie

Nous allons dans ce chapitre programmer une version graphique du jeu. Cela ressemblera à ceci :



4.3. Programmes pilotés par des événements

Ce paragraphe est largement inspiré du livre de Swinnen « Apprendre à programmer avec Python », chapitre 8.

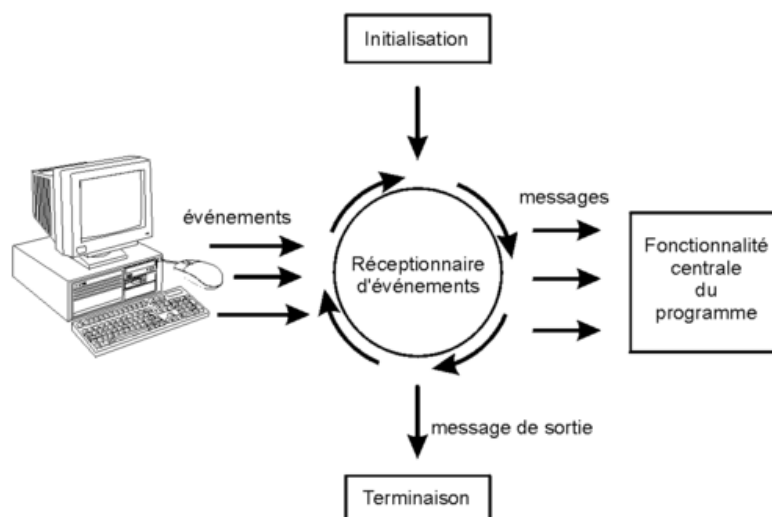
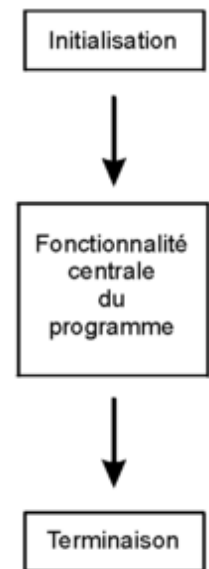
Tous les programmes d'ordinateur comportent *grosso modo* trois phases principales : une **phase d'initialisation**, laquelle contient les instructions qui préparent le travail à effectuer (appel des modules externes nécessaires, ouverture de fichiers, connexion à un serveur de bases de données ou à Internet, etc.), **une phase centrale** où l'on trouve la véritable fonctionnalité du programme (c'est-à-dire tout ce qu'il est censé faire : afficher des données à l'écran, effectuer des calculs, écrire dans un fichier, etc.), et enfin une **phase de terminaison** qui sert à stopper « proprement » les opérations (par exemple fermer les fichiers restés ouverts, fermer les fenêtres, etc.).

Dans un programme « en mode texte », ces trois phases sont simplement organisées suivant un schéma linéaire comme dans l'illustration ci-contre. En conséquence, ces programmes se caractérisent par une interactivité très limitée avec l'utilisateur. Celui-ci ne dispose pratiquement d'aucune liberté : il lui est demandé de temps à autre d'entrer des données au clavier, mais toujours dans un ordre prédéterminé correspondant à la séquence d'instructions du programme.

Dans le cas d'un programme qui utilise une interface graphique, par contre, l'organisation interne est différente. On dit que le programme est **piloté par les événements**. Après sa phase d'initialisation, un programme de ce type se met en quelque sorte « en attente », et passe la main à un autre logiciel, lequel est plus ou moins intimement intégré au système d'exploitation de l'ordinateur et « tourne » en permanence.

Ce **réceptionnaire d'événements**, comme on l'appelle, scrute sans cesse tous les périphériques (clavier, souris, horloge, modem, etc.) et réagit immédiatement lorsqu'un événement y est détecté. Lorsqu'il détecte un événement, le réceptionnaire envoie un message spécifique au programme, lequel doit être conçu pour réagir en conséquence.

La phase d'initialisation d'un programme utilisant une interface graphique comporte un ensemble d'instructions qui mettent en place les divers composants interactifs de cette interface (fenêtres, boutons, labels, etc.). D'autres instructions définissent les messages d'événements qui devront être pris en charge : on peut en effet décider que le programme ne réagira qu'à certains événements en ignorant tous les autres.



Alors que dans un programme « en mode texte », la phase centrale est constituée d'une suite d'instructions qui décrivent à l'avance l'ordre dans lequel la machine devra exécuter ses différentes tâches, on ne trouve dans la phase centrale d'un programme avec interface graphique qu'un ensemble de fonctions indépendantes. Chacune de ces fonctions est appelée spécifiquement lorsqu'un événement particulier est détecté par le système d'exploitation : elle effectue alors le travail que l'on

attend du programme en réponse à cet événement, et rien d'autre.

Il est important de bien comprendre ici que pendant tout ce temps, le réceptionnaire continue à « tourner » et à guetter l'apparition d'autres événements éventuels. S'il se produit d'autres événements, il peut donc arriver qu'une deuxième fonction (ou une troisième, une quatrième, ...) soit activée et commence à effectuer son travail « en parallèle » avec la première qui n'a pas encore terminé le sien. Les systèmes d'exploitation et les langages modernes permettent en effet ce **parallélisme** (que l'on appelle aussi **multitâche**).

4.4. Code du programme



ppc-graphique.py

<http://ow.ly/M7Gt305PEqq>

```
# jeu pierre, papier, ciseaux
# l'ordinateur joue au hasard

from random import randint
from tkinter import *

def augmenter_scores(mon_coup,ton_coup):
    global mon_score, ton_score
    if mon_coup == 1 and ton_coup == 2:
        ton_score += 1
    elif mon_coup == 2 and ton_coup == 1:
        mon_score += 1
    elif mon_coup == 1 and ton_coup == 3:
        mon_score += 1
    elif mon_coup == 3 and ton_coup == 1:
        ton_score += 1
    elif mon_coup == 3 and ton_coup == 2:
        mon_score += 1
    elif mon_coup == 2 and ton_coup == 3:
        ton_score += 1

def jouer(ton_coup):
    global mon_score, ton_score, score1, score2
    mon_coup = randint(1,3)
    if mon_coup==1:
        lab3.configure(image=pierre)
    elif mon_coup==2:
        lab3.configure(image=papier)
    else:
        lab3.configure(image=ciseaux)
    augmenter_scores(mon_coup,ton_coup)
    score1.configure(text=str(ton_score))
    score2.configure(text=str(mon_score))

def jouer_pierre():
    jouer(1)
    lab1.configure(image=pierre)

def jouer_papier():
    jouer(2)
    lab1.configure(image=papier)

def jouer_ciseaux():
    jouer(3)
    lab1.configure(image=ciseaux)

def reinit():
    global mon_score, ton_score, score1, score2, lab1, lab3
    ton_score = 0
    mon_score = 0
    score1.configure(text=str(ton_score))
    score2.configure(text=str(mon_score))
    lab1.configure(image=rien)
    lab3.configure(image=rien)

# variables globales
ton_score = 0
mon_score = 0
```

```
# fenetre graphique
fenetre = Tk()
fenetre.title("Pierre, papier, ciseaux")

# images
rien = PhotoImage(file='rien.gif')
versus = PhotoImage(file='versus.gif')
pierre = PhotoImage(file='pierre.gif')
papier = PhotoImage(file='papier.gif')
ciseaux = PhotoImage(file='ciseaux.gif')

# labels
textel = Label(fenetre, text="Humain :", font=("Helvetica", 16))
textel.grid(row=0, column=0)

texte2 = Label(fenetre, text="Machine :", font=("Helvetica", 16))
texte2.grid(row=0, column=2)

texte3 = Label(fenetre, text="Pour jouer, cliquez sur une des icones ci-dessous.")
texte3.grid(row=3, columnspan=3, pady=5)

score1 = Label(fenetre, text="0", font=("Helvetica", 16))
score1.grid(row=1, column=0)

score2 = Label(fenetre, text="0", font=("Helvetica", 16))
score2.grid(row=1, column=2)

lab1 = Label(fenetre, image=rien)
lab1.grid(row=2, column=0)

lab2 = Label(fenetre, image=versus)
lab2.grid(row=2, column=1)

lab3 = Label(fenetre, image=rien)
lab3.grid(row=2, column=2)

# boutons
bouton1 = Button(fenetre, command=jouer_pierre)
bouton1.configure(image=pierre)
bouton1.grid(row=4, column=0)

bouton2 = Button(fenetre, command=jouer_papier)
bouton2.configure(image=papier)
bouton2.grid(row=4, column=1)

bouton3 = Button(fenetre, command=jouer_ciseaux)
bouton3.configure(image=ciseaux)
bouton3.grid(row=4, column=2)

bouton4 = Button(fenetre, text='Recommencer', command=reinit)
bouton4.grid(row=5, column=0, pady=10, sticky=E)

bouton5 = Button(fenetre, text='Quitter', command=fenetre.destroy)
bouton5.grid(row=5, column=2, pady=10, sticky=W)

# demarrage :
fenetre.mainloop()
```

4.5. Analyse du programme

Reprenons ce programme pour l'expliquer en détails.

```
# jeu pierre, papier, ciseaux
# l'ordinateur joue au hasard

from random import randint
from tkinter import *
```

C'est le module externe `tkinter` qui va nous permettre de faire des graphiques.

Il faudra gérer des **événements** (*events*) provenant des périphériques (le clavier ou la souris). Par

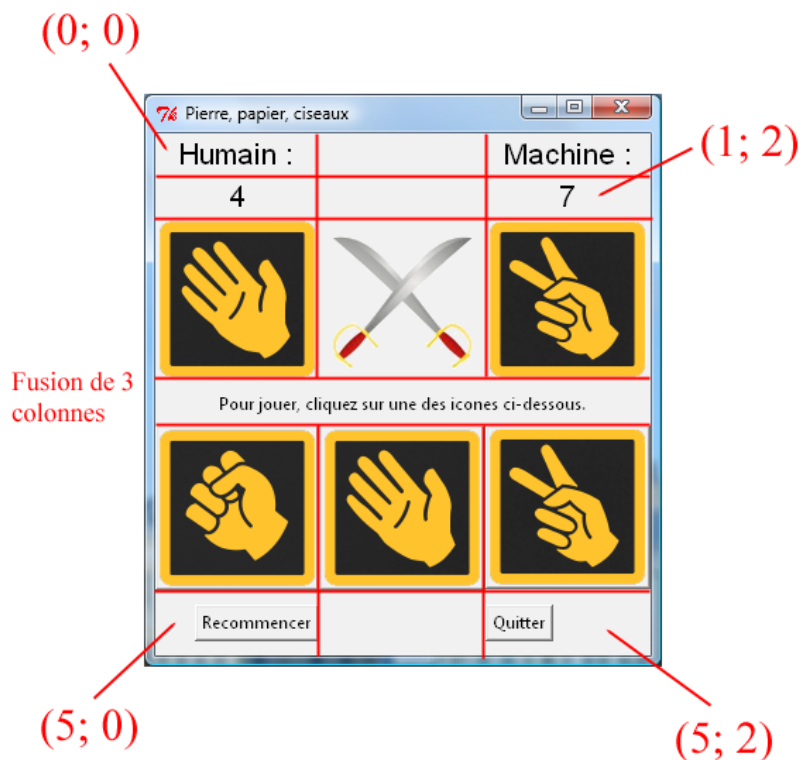
exemple, un événement pourrait être que la touche « espace » a été pressée, ou que la souris a été déplacée. Dans notre jeu, le seul événement qui nous intéressera sera le clic du bouton gauche de la souris. C'est en effet via la souris que le joueur va choisir le coup qu'il va jouer.

4.5.1. Les fenêtres

Il faudra ouvrir une fenêtre graphique dans laquelle on va placer divers éléments (des boutons, des textes, des étiquettes, etc.).

```
# fenetre graphique
fenetre = Tk()
fenetre.title("Pierre, papier, ciseaux")
```

Pour notre jeu, la fenêtre a été divisée virtuellement en plusieurs zones correspondant aux cases d'une grille. Chaque case est repérée par sa ligne et sa colonne. **Attention !** La numérotation commence à 0. Ainsi, la case en haut à gauche a pour coordonnées (0; 0). La case placée sur la deuxième ligne et la troisième colonne a pour coordonnées (1; 2).



On voit que ces zones n'ont pas la même taille. Elles s'adaptent automatiquement aux dimensions des objets qu'elles contiennent.

4.5.2. Importation des images

Nous aurons besoin de cinq images : les symboles pierre, papier, ciseaux, une case noire pour le début du jeu (« rien ») et les deux épées croisées pour faire joli. Toutes ces images, au format gif, doivent se trouver dans le même répertoire que le programme.

C'est la fonction `PhotoImage` du module `tkinter` qui importe les images.

```
# images
rien = PhotoImage(file='rien.gif')
versus = PhotoImage(file='versus.gif')
pierre = PhotoImage(file='pierre.gif')
papier = PhotoImage(file='papier.gif')
ciseaux = PhotoImage(file='ciseaux.gif')
```

4.5.3. Les labels

Les labels peuvent être du texte ou des images. Sur la première ligne de la grille, on a écrit les noms des joueurs (Humain et Machine). On a indiqué aussi la police utilisée (Helvetica) et la taille (16).

```
# labels
texte1 = Label(fenetre, text="Humain :", font=("Helvetica", 16))
texte1.grid(row=0, column=0)

texte2 = Label(fenetre, text="Machine :", font=("Helvetica", 16))
texte2.grid(row=0, column=2)
```

La quatrième ligne (row=3) est spéciale : elle est composée d'une seule case, qui s'étend sur la largeur de 3 colonnes (columnspan=3). Pour que le texte ne soit pas trop serré, on a mis une marge de 5 pixels en dessus et en dessous (pady=5).

```
texte3 = Label(fenetre, text="Pour jouer, cliquez sur une des icônes ci-dessous.")
texte3.grid(row=3, columnspan=3, pady=5)
```

Le contenu des labels peut varier durant l'exécution du programme. Par exemple, les scores vont évidemment changer en cours de partie. Au début, ils sont initialisés à 0 (text="0").

Les scores des deux joueurs sont placés sur la deuxième ligne (row=1) et sur la première (column=0) et troisième colonne (column=2). On a indiqué aussi la police utilisée (Helvetica) et la taille (16).

```
score1 = Label(fenetre, text="0", font=("Helvetica", 16))
score1.grid(row=1, column=0)

score2 = Label(fenetre, text="0", font=("Helvetica", 16))
score2.grid(row=1, column=2)
```

La troisième ligne (row=2) est composée de 3 labels : le coup joué par l'humain (rien au départ), les deux épées et le coup joué par la machine (rien au départ).

```
lab1 = Label(fenetre, image=rien)
lab1.grid(row=2, column=0)

lab2 = Label(fenetre, image=versus)
lab2.grid(row=2, column=1)

lab3 = Label(fenetre, image=rien)
lab3.grid(row=2, column=2)
```

4.5.4. Les boutons

Les boutons peuvent être du texte ou des images. Dans notre fenêtre, il y a 5 boutons : les symboles pierre, papier et ciseaux, ainsi que les boutons « Recommencer » et « Quitter ».

Commençons par les boutons images.

```
# boutons
bouton1 = Button(fenetre, command=jouer_pierre)
bouton1.configure(image=pierre)
bouton1.grid(row=4, column=0)

bouton2 = Button(fenetre, command=jouer_papier)
bouton2.configure(image=papier)
bouton2.grid(row=4, column=1)

bouton3 = Button(fenetre, command=jouer_ciseaux)
bouton3.configure(image=ciseaux)
bouton3.grid(row=4, column=2)
```

Pour chacun de ces boutons, la première ligne crée le bouton en indiquant dans quelle fenêtre il se trouve et la commande qui sera appelée quand on cliquera dessus. Cette commande est une procédure sans paramètres. La deuxième ligne indique quelle image sera associée au bouton. Quant à la troisième ligne, elle indique dans quelle case de la grille se trouve le bouton.

Passons maintenant aux boutons contenant du texte.

```
bouton4 = Button(fenetre, text='Recommencer', command=reinit)
bouton4.grid(row=5, column=0, pady=10, sticky=E)

bouton5 = Button(fenetre, text='Quitter', command=fenetre.destroy)
bouton5.grid(row=5, column=2, pady=10, sticky=W)
```

La création d'un bouton est presque identique à celle d'un bouton image, sauf que l'on précise le texte qui figurera dans le bouton. La deuxième ligne donne l'emplacement du bouton. On a précisé ici que le bouton « Recommencer » est calé à droite (`sticky=E`, avec E comme East) et le bouton « Quitter » à gauche (`sticky=W`, avec W comme West). Il existe aussi les lettres N pour North, donc calé en haut et S pour South, donc calé en bas.

4.5.5. Les procédures

On va retrouver des procédures écrites pour la version non graphique, parfois légèrement modifiées, ainsi que les procédures appelées par les boutons.

```
def augmenter_scores(mon_coup, ton_coup):
    global mon_score, ton_score
    if mon_coup == 1 and ton_coup == 2:
        ton_score += 1
    elif mon_coup == 2 and ton_coup == 1:
        mon_score += 1
    elif mon_coup == 1 and ton_coup == 3:
        mon_score += 1
    elif mon_coup == 3 and ton_coup == 1:
        ton_score += 1
    elif mon_coup == 3 and ton_coup == 2:
        mon_score += 1
    elif mon_coup == 2 and ton_coup == 3:
        ton_score += 1
```

Cette procédure est la même que dans la version non graphique.

```
def jouer(ton_coup):
    global mon_score, ton_score, score1, score2
    mon_coup = randint(1,3)
    if mon_coup==1:
        lab3.configure(image=pierre)
    elif mon_coup==2:
        lab3.configure(image=papier)
    else:
        lab3.configure(image=ciseaux)
    augmenter_scores(mon_coup, ton_coup)
    score1.configure(text=str(ton_score))
    score2.configure(text=str(mon_score))
```

Cette procédure `jouer` a été modifiée pour s'adapter au graphisme. Quand le joueur presse sur un des trois boutons, l'ordinateur tire au hasard un coup, puis modifie l'image du label correspondant au coup choisi (`lab3.configure(image=pierre)`). Les nouveaux scores sont ensuite calculés, puis les labels `score1` et `score2` sont mis à jour, grâce à la procédure `configure`.

```
def jouer_pierre():
    jouer(1)
    lab1.configure(image=pierre)

def jouer_papier():
    jouer(2)
    lab1.configure(image=papier)
```

```
def jouer_ciseaux():
    jouer(3)
    lab1.configure(image=ciseaux)
```

Ces trois procédures sont appelées par les boutons. Elles mettent à jour le label indiquant le coup choisi par le joueur.

```
def reinit():
    global mon_score, ton_score, score1, score2, lab1, lab3
    ton_score = 0
    mon_score = 0
    score1.configure(text=str(ton_score))
    score2.configure(text=str(mon_score))
    lab1.configure(image=rien)
    lab3.configure(image=rien)
```

Cette dernière procédure est appelée par le bouton « Recommencer ». Elle réinitialise les variables globales, ainsi que les labels scores et les labels montrant les coups joués.

4.5.6. Gestion des événements

Le récepteur d'événements est lancé par l'instruction :

```
fenetre.mainloop()
```

Dans notre programme de jeu, la phase de terminaison consistera simplement à fermer la fenêtre. C'est le bouton « Quitter » qui appellera la fonction `destroy` :

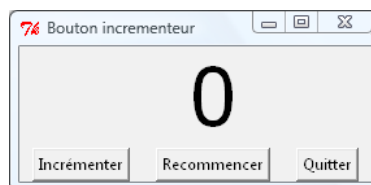
```
bouton5 = Button(fenetre, text='Quitter', command=fenetre.destroy)
```

Il est à noter que ce bouton n'était pas indispensable. En effet, cliquer le bouton rouge habituel en haut à droite de la fenêtre aurait eu le même effet.



Exercice 4.1

Reproduisez la fenêtre ci-dessous :

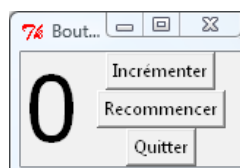


Le bouton « Incrémenter » augmentera de 1 le compteur. Le bouton « Recommencer » remettra le compteur à 0 et le bouton « Quitter » fermera la fenêtre.



Exercice 4.2

Reproduisez la fenêtre ci-dessous :



Les effets des boutons seront les mêmes que dans l'exercice 4.1.



Exercice 4.3

Reproduisez la frise ci-dessous, en utilisant une boucle (`while` ou `for`).



Exercice 4.4

Reproduisez une frise comme ci-dessous, en tirant les cartes au hasard. Chaque colonne représente une manche de pierre-papier-ciseaux. Comme pour l'exercice 4.3, utilisez une boucle.



Exercice 4.5

Modifiez le code du § 4.4.

Ajoutez une deuxième fenêtre pour que l'on puisse voir l'historique des 20 derniers coups joués dans la partie.

Attention ! Pour ouvrir une nouvelle fenêtre, il faudra utiliser l'instruction :



```
historique = Toplevel()
historique.title("Historique")
```

En effet, on ne peut pas utiliser l'instruction `Tk()` deux fois dans le même programme. Cette fenêtre sera une « fille » de la fenêtre principale, et elle disparaîtra en même temps qu'elle.

Cette deuxième fenêtre ressemblera à celle de l'exercice 4.4, mais en écrivant quelle est la ligne du joueur et celle de l'ordinateur. Quand 20 coups auront été joués, recommencez à dessiner les symboles depuis la première colonne, sans rien effacer.



Exercice 4.6

Modifiez le code du § 4.4.

Ajoutez un quatrième symbole : le puits. La pierre et les ciseaux tombent dans le puits (donc le puits gagne contre eux). Par contre, le papier recouvre le puits (le puits perd donc contre le papier).

Attention ! Les probabilités de gain changent, et la stratégie aussi !



Exercice 4.7

Let's Make A Deal ! est un show télévisé américain qui a commencé le 30 décembre 1963. À la fin du jeu, l'animateur, Monty Hall, vous offrait la possibilité de gagner ce qui se trouvait derrière une porte. Il y avait trois portes : derrière l'une d'entre elles se trouvait un prix magnifique (par exemple un voyage) et derrière les deux autres un prix moins intéressant (par exemple une chèvre ou une barre de chocolat). Vous choisissiez alors une porte.

Pour ménager le suspense, Monty Hall, avant de révéler ce qu'il y avait derrière la porte que vous aviez choisie, ouvrait une des deux autres portes (derrière laquelle ne se trouvait **jamais** le plus beau cadeau).

Il vous posait enfin la dernière question : « Conservez-vous votre premier choix, ou bien choisissez-vous l'autre porte encore fermée ? »

- Programmez ce jeu avec une interface graphique. Vous pourrez vous inspirer de cette page web : www.nymphomath.ch/random/monty/
- Quelle est la meilleure stratégie : changer de porte ou garder la porte choisie en premier ?



4.6. Ce que vous avez appris dans ce chapitre

- C'est dans le module `tkinter` que l'on trouve les procédures permettant de créer une interface graphique.
- Quand il y a une interface graphique, le programme est piloté par des événements (§ 4.3). C'est très différent des programmes « en mode texte » que l'on a vu dans les trois premiers chapitres.
- Dans le paragraphe 4.5 et les exercices, vous avez vu comment :
 - ouvrir et utiliser des fenêtres graphiques,
 - importer des images au format gif,
 - utiliser les labels,
 - définir des boutons,
 - gérer des événements.



Chapitre 5

Le jeu de Juniper Green

5.1. Nouveaux thèmes abordés dans ce chapitre

- objets
- classes
- méthodes
- listes
- opération sur les listes
- tuples

5.2. Règles du jeu

Le jeu a été créé par Richard **Porteous**, enseignant à l'école de Juniper Green, auquel il doit son nom. Il s'est réellement fait connaître grâce à Ian **Stewart**, qui en décrit les règles dans la revue *Pour la Science*, dans le numéro de juillet 1997.

Ce jeu comporte quatre règles :

1. Le joueur 1 choisit un nombre entre 1 et N_{max} .
2. À tour de rôle, chaque joueur doit choisir un nombre parmi les multiples ou les diviseurs du nombre choisi précédemment par son adversaire et inférieur à N_{max} .
3. Un nombre ne peut être joué qu'une seule fois.
4. Le premier nombre choisi doit être pair.

Le perdant est le joueur qui ne trouve plus de multiples ou de diviseurs communs au nombre choisi par l'adversaire.

Exemple de partie

```
Jouons avec des nombres entre 1 et 20.  
Je choisis comme nombre de départ 2  
Nombres valides: [1, 4, 6, 8, 10, 12, 14, 16, 18, 20]  
Que jouez-vous ? 4  
Nombres valides: [1, 8, 12, 16, 20]  
Je joue 8  
Nombres valides: [1, 16]  
Que jouez-vous ? 16  
Nombres valides: [1]  
Je joue 1  
Nombres valides: [3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20]  
Que jouez-vous ? 11  
Bravo!
```

Remarque : les nombres en gras ont été entrés au clavier par le joueur.

5.3. Les objets en Python (premier contact)

En Python, tout est objet ! On rencontre souvent cette phrase dans les livres sur Python et cela en dit long sur l'importance de ce concept...

Qu'est-ce qu'un **objet** ? Dans un premier temps, nous allons dire que c'est une structure de données qui contient des fonctions permettant de la manipuler. On appelle ces fonctions des **méthodes**.

Comme l'idée de cet ouvrage est de partir d'un programme pour découvrir de nouveaux concepts, nous allons prendre comme premier exemple les listes.

Les listes forment une **classe** (la classe 'list'). Un objet est issu d'une classe. Tous les objets de cette classe se ressembleront, mais ils auront leur existence propre. Par exemple, toutes les listes auront le même aspect (voir § 5.4), mais elles n'auront pas le même contenu.

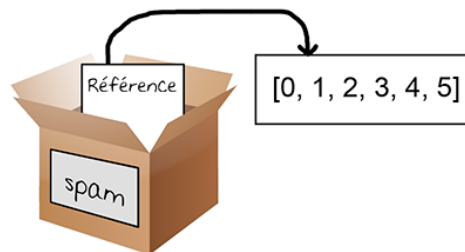
Autre exemple de classe venant de la vie courante : une Peugeot 3008 et une Ferrari Testarossa sont des objets de la classe 'voiture'.

5.4. Les listes

Comme son nom l'indique, une **liste** est une suite **ordonnée** de valeurs, qui peuvent être de types différents. Les éléments sont listés entre deux crochets et séparés par des virgules. On peut aussi voir une liste comme un tableau unidimensionnel où les éléments sont repérés par un **indice**. Ce dernier donne la position de l'élément dans la liste. Attention ! Comme dans beaucoup de langages, **l'indice du premier élément est 0** et non pas 1.



① spam = [0, 1, 2, 3, 4, 5]



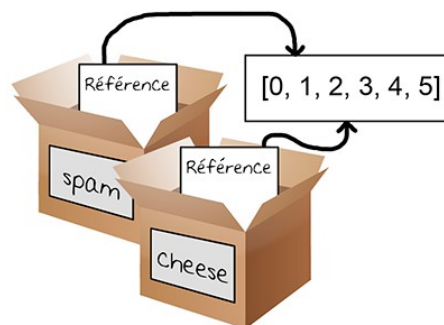
Dans le chapitre 2, nous avons représenté une variable par une boîte contenant une valeur. Dans le cas d'une liste, la boîte contient en fait une référence vers une liste, et non pas la liste elle-même. Cela a une grande importance lorsque l'on veut copier une liste. Si l'on se contente d'écrire



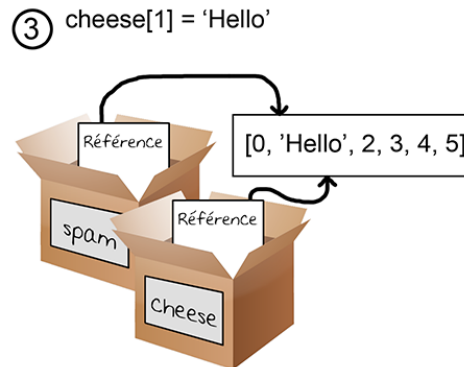
```
cheese = spam
```

on n'aura pas copié la liste, mais la référence vers cette liste, comme indiqué sur le schéma ci-dessous :

② cheese = spam



Cela implique que si l'on modifie une des deux listes, l'autre sera modifiée de la même façon (voir ci-dessous).



5.4.1. Création des listes

Si l'on tape
`type(spam)`, on
obtiendra la réponse
`<class 'list'>`

Pour définir une liste vide, l'instruction est logiquement :

```
spam = []
```

On peut évidemment directement créer une liste avec des éléments. Par exemple :

```
spam = [2, 3, 5, 7, 11]
```

On peut faire des
listes de listes, par
exemple pour
représenter un
tableau
bidimensionnel.

Les éléments de la liste peuvent être de types différents (ici, l'ordre, un entier, une chaîne de caractères, un réel, un caractère et une liste) :

```
spam = [3, "salut", 3.1415, "x", [1, 2, 3]]
```

Si tous les éléments de la liste doivent avoir la même valeur au départ, on peut multiplier la valeur désirée par le nombre d'éléments que l'on veut. Si on veut une liste de 10 zéros, on écrira :

```
spam = [0]*10
```

ce qui donnera la liste `[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]`.

La fonction `range()` génère par défaut une séquence de nombres entiers de valeurs croissantes, et différant d'une unité. Si vous appelez `range()` avec un seul argument, la liste contiendra un nombre de valeurs égal à l'argument fourni, en commençant à partir de zéro (c'est-à-dire que `range(n)` génère les nombres entiers de 0 à $n-1$). Ainsi,

```
spam = list(range(10))
```

créera la liste `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`.

On peut aussi utiliser `range()` avec deux, ou même trois arguments séparés par des virgules, afin de générer des séquences de nombres plus « sophistiquées » :

```
spam = list(range(3, 12))
```

produira la liste `[3, 4, 5, 6, 7, 8, 9, 10, 11]`. On voit que `range(a, b)` génère tous les nombres entiers de a jusqu'à $b-1$.

```
spam = list(range(3, 12, 2))
```

produira la liste `[3, 5, 7, 9, 11]`. Donc, `range(a, b, p)` génère des nombres entiers de a à $b-1$ avec un pas de p .

5.4.2. Manipulation des listes

La fonction `len` s'applique aussi aux chaînes de caractères, aux tuples, ... Ce n'est pas une méthode spécifique aux listes.

La fonction intégrée `len` permet de connaître la longueur (*length* en anglais) de la liste :

```
spam = [2, 3, 5, 7, 11]
len(spam)
```

donnera comme résultat 5. Les indices de la liste `spam` vont donc de 0 à 4.

Pour ajouter un élément en fin de liste, on a la méthode `append()` :

```
spam.append(13)
```

qui donnera la liste `[2, 3, 5, 7, 11, 13]`. Remarquez la forme caractéristique d'une méthode : `objet.méthode()`.

On peut aussi enlever un élément de la liste avec la méthode `remove()` :

```
spam.remove(5)
```

donnera comme résultat `[2, 3, 7, 11, 13]`.

Il ne faut pas confondre cette instruction avec `del(spam[5])`, qui permet d'enlever l'élément d'indice 5, donc le 6^{ème} élément de la liste. Par exemple, si `spam=[2, 3, 5, 7, 11, 13]`, alors

```
del(spam[5])
```

donnera comme résultat `[2, 3, 5, 7, 11]`.

On peut aussi facilement concaténer deux listes avec un simple `+` :

```
spam = [2, 3, 5, 7, 11]
cheese = [9, 8, 7, 6, 5]
fusion = cheese + spam
```

donnera la liste `[9, 8, 7, 6, 5, 2, 3, 5, 7, 11]`.

On peut trier la liste avec la méthode `sort()` :

```
fusion.sort()
```

donnera la liste `[2, 3, 5, 5, 6, 7, 7, 8, 9, 11]`.

Il arrive parfois que l'on doive inverser la liste :

```
fusion.reverse()
```

donnera la liste `[11, 9, 8, 7, 7, 6, 5, 5, 3, 2]`.

5.4.3. Accès aux éléments d'une liste

On peut accéder à un élément d'une liste en précisant son indice, mais on peut aussi accéder à plusieurs éléments en donnant un intervalle ; le résultat sera alors une liste. Reprenons notre liste `spam` et observons quelques exemples :

```
spam = [2, 3, 5, 7, 11, 13]
print(spam[2])
```

donnera comme résultat 5. C'est un nombre entier.

```
print(spam[1:3])
```

donnera comme résultat `[3, 5]`. C'est une liste composée des éléments `spam[1]` et `spam[2]`.

```
print(spam[2:3])
```

donnera comme résultat `[5]`. C'est une **liste** contenant un seul élément.

```
print(spam[2:])
```

donnera comme résultat `[5, 7, 11, 13]`.

```
print(spam[:2])
```

donnera comme résultat `[2, 3]`.

```
print(spam[-2])
```

donnera comme résultat `11`. C'est le deuxième élément depuis la fin de la liste.

```
print(spam[-1])
```

donnera comme résultat `13`. C'est le dernier élément de la liste.

On peut savoir si un élément appartient à une liste avec l'instruction `in`.

```
5 in spam
```

donnera comme résultat `True`. On peut aussi connaître l'indice de cet élément :

```
spam.index(5)
```

donnera comme résultat `2`. L'élément `5` est bien en 3^{ème} position de `[2, 3, 5, 7, 11, 13]`.

On peut modifier un élément de la liste. Par exemple,

```
spam[0]=1
```

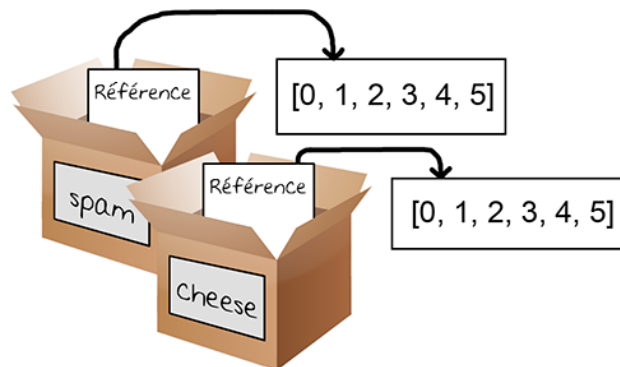
modifiera la liste `spam` ainsi :

```
[1, 3, 5, 7, 11, 13]
```

5.4.4. Copie d'une liste et insertion dans une liste

Il y a plusieurs façons de copier une liste. La plus courte est :

```
cheese = spam[:]
```



Rappelons encore une fois, car c'est important, que `cheese = spam` ne copie pas la liste, mais crée une nouvelle référence sur la liste `spam`.

Insertion d'un ou de plusieurs éléments n'importe où dans une liste

La technique du « slicing » (découpage en tranches) permet de modifier une liste à l'aide du seul opérateur []. On peut insérer un ou des éléments dans une liste. Par exemple,

```
capitales = ['Paris', 'Rome', 'Berne', 'Vienne']
capitales[2:2] = ['Berlin']
```

donnera comme résultat :

```
['Paris', 'Rome', 'Berlin', 'Berne', 'Vienne']
```

et

```
capitales[5:5] = ['Oslo', 'Londres']
```

donnera comme résultat :

```
['Paris', 'Rome', 'Berlin', 'Berne', 'Vienne', 'Oslo', 'Londres']
```

Attention aux particularités suivantes :

- Si vous utilisez l'opérateur [] **à la gauche** du signe égal pour effectuer une insertion ou une suppression d'élément(s) dans une liste, vous devez obligatoirement y indiquer une « tranche » dans la liste cible (c'est-à-dire deux index réunis par le symbole :), et non un élément isolé dans cette liste.
- L'élément que vous fournissez **à la droite** du signe égal *doit lui-même être une liste*. Si vous n'insérez qu'un seul élément, il vous faut donc le mettre entre crochets pour le transformer d'abord en une liste d'un seul élément. Notez bien que l'élément `capitales[1]` n'est pas une liste (c'est la chaîne 'Rome'), alors que l'élément `capitales[1:3]` en est une.



Suppression / remplacement d'éléments

On peut aussi supprimer et/ou remplacer des éléments par « slicing ». Partons de cette liste :

```
['Paris', 'Rome', 'Berlin', 'Berne', 'Vienne', 'Oslo', 'Londres']
```

Remplaçons la tranche [2:5] par une liste vide, ce qui correspond à un effacement.

```
capitales[2:5] = []
```

produira la liste :

```
['Paris', 'Rome', 'Oslo', 'Londres']
```

Remplaçons une tranche par un seul élément. Notez encore une fois que cet élément doit lui-même être une liste.

```
capitales[1:3] = ['Madrid']
```

produira la liste :

```
['Paris', 'Madrid', 'Londres']
```

Remplaçons une tranche de deux éléments par une autre qui en compte trois.

```
capitales[1:] = ['Lisbonne', 'Prague', 'Moscou']
```

produira la liste :

```
['Paris', 'Lisbonne', 'Prague', 'Moscou']
```

5.4.5. Les boucles for

En Python, une boucle `for` parcourt simplement une liste :

```
liste = ['a','b','c']
for lettre in liste:
    print(lettre,end=' ')
```

écrira a b c.

On peut aussi faire de simples compteurs avec une boucle `for` :

```
for chiffre in range(5):
    print(chiffre,end=' ')
```

écrira 0 1 2 3 4.



Exercice 5.1

Créez une liste de 15 nombres entiers générés au hasard entre 1 et 10. Certains nombres pourront y figurer à plusieurs exemplaires. Écrivez un programme qui recopie cette liste dans une autre, en omettant les doublons. La liste finale devra être triée.



Exercice 5.2

Soient les listes suivantes :

```
liste1 = [31,28,31,30,31,30,31,31,30,31,30,31]
```

```
liste2 = ['Janvier','Février','Mars','Avril','Mai','Juin','Juillet','Août',
          'Septembre','Octobre','Novembre','Décembre']
```

Écrivez un programme qui *insère* dans la seconde liste tous les éléments de la première, de telle sorte que chaque nom de mois soit suivi du nombre de jours correspondant :

```
liste2 = ['Janvier',31,'Février',28,'Mars',31, ...]
```

Insérez ensuite le nombre 29 entre 28 et 'Mars'.

```
liste2 = ['Janvier',31,'Février',28,29,'Mars',31, ...]
```



Exercice 5.3 : le crible d'Ératosthène

Définition : un nombre **premier** est un nombre qui n'est divisible que par un et par lui-même (par convention, 1 n'est pas un nombre premier).

On désigne sous le nom de *crible d'Ératosthène* (env. 276-194 av. J.-C.), une méthode de recherche des nombres premiers plus petits qu'un entier naturel n donné.

Algorithme

1. On écrit la liste de tous les nombres jusqu'à n .
2. On élimine 1.
3. On entoure 2 et on élimine tous les multiples de 2.
4. Puis on fait de même avec 3.
5. On choisit alors le plus petit nombre non entouré et non éliminé, et on élimine tous ses multiples.
6. On réitère le procédé jusqu'à la partie entière de la racine de n . Les nombres non éliminés sont les nombres premiers jusqu'à n .

Le jeu de Juniper Green



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Écrivez un programme qui établit la liste de tous les nombres premiers compris entre 1 et n , en utilisant la méthode du *crible d'Ératosthène* :

1. Créer une liste de $n+1$ éléments (disons $n=1000$), chacun initialisé à la valeur 1.
2. Pour les indices i allant de 2 à $\sqrt{n}$ faire :
Si l'élément d'indice i vaut 1, mettre à 0 tous les autres éléments de la liste dont les indices sont des multiples de i .
3. Parcourir toute la liste et écrire les indices des éléments qui sont restés à 1. Ce sont des nombres premiers.

5.5. Code du programme

```
# Jeu de Juniper-Green

from random import randint

def multiples(n):
    #renvoie la liste des multiples de n <= Nmax
    mult=[]
    i=2
    while i*n <= Nmax :
        if i*n in possibles:      # on l'ajoute seulement s'il n'a pas été joué
            mult.append(i*n)
        i += 1
    return mult

def diviseurs(n):
    #renvoie la liste des diviseurs de n
    div = []
    i=n
    while i >= 1:
        if n%i == 0 and n//i in possibles:      # on l'ajoute s'il n'a pas été joué
            div.append(n//i)
        i-=1
    return div

Nmax = 20
possibles = list(range(1,Nmax+1))      # liste des nombres pas encore utilisés
mon_nombre=2*randint(1,Nmax/2)        # l'ordinateur choisit un nombre pair
possibles.remove(mon_nombre)          # on enlève de la liste "possibles" tous les
                                     # nombres joués

# Début du jeu

print("Jouons avec des nombres entre 1 et",Nmax)
print("Je choisis comme nombre de départ",mon_nombre)
valides = diviseurs(mon_nombre) + multiples(mon_nombre)
while valides != []:
    print("Nombres valides:",valides)
    ton_nombre=int(input("Que jouez-vous ? "))
    while ton_nombre not in valides:
        ton_nombre=int(input("Incorrect. Que jouez-vous ? "))
```



juniper-green.py

<http://ow.ly/fkeUw>

```
possibles.remove(ton_nombre)
valides = diviseurs(ton_nombre) + multiples(ton_nombre)
if valides == []:
    print("Bravo!")
else:
    mon_nombre = valides[randint(0, len(valides)-1)]
    print("Nombres valides :", valides)
    print("Je joue", mon_nombre)
    possibles.remove(mon_nombre)
    valides = diviseurs(mon_nombre) + multiples(mon_nombre)
    if valides == []:
        print("Vous avez perdu!")
```

5.6. Analyse du programme

Comme les informations concernant les listes ont été données avant de montrer le code, nous ne verrons ici que le fonctionnement général du programme. Le lecteur devrait pouvoir comprendre le code assez aisément.

Une liste, nommée `possibles`, contient tous les nombres qui n'ont pas encore été joués. Au début, elle contient tous les nombres de 1 à `Nmax`. Chaque fois qu'un nombre a été joué, il est retiré de cette liste.

Le programme utilise la liste `valides` pour savoir quels nombres on peut jouer. Le jeu s'arrêtera quand cette liste sera vide. Au début du jeu, elle contient tous les diviseurs et les multiples du nombre de départ choisi par l'ordinateur (`mon_nombre`).

Ces deux listes sont des variables globales.

À tour de rôle, chaque joueur (la machine ou le joueur humain) choisit un nombre dans la liste `valides`, tant qu'elle n'est pas vide. Ce nombre est retiré de la liste `possibles`. La liste `valides` est mise à jour, et on recommence jusqu'à la victoire d'un des deux joueurs.



Exercice 5.4

Modifiez le code du § 5.5.

Avant de commencer le jeu, le programme demandera au joueur d'entrer la valeur de `Nmax`. Il demandera aussi si le joueur veut jouer en premier ; il faudra évidemment adapter le programme en conséquence.



Exercice 5.5

Modifiez le code de l'exercice 5.4.

Dans la version de l'exercice 5.4, l'ordinateur tire simplement au hasard un nombre parmi ceux qui sont valides. Trouvez une meilleure stratégie et programmez-la !



Exercice 5.6

Une variante pour jouer seul au jeu de Juniper Green consiste à trouver la plus longue partie possible pour un `Nmax` donné.

Programmez une méthode *probabiliste* : faites des milliers de parties en jouant des coups (légaux) au hasard et gardez en mémoire la partie la plus longue, que vous afficherez à la fin.

Note : la partie trouvée ne sera pas forcément la plus longue, mais plus vous ferez de parties, plus vous vous en approcherez.

Record actuel (11.12.2014) :

Il semble que la longueur maximale soit de 41 coups.

Je joue 5000000 parties avec des nombres entre 1 et 50
 Plus longue partie: 38 coups
 22, 1, 17, 34, 2, 26, 13, 39, 3, 33, 11, 44, 4, 24, 6, 30, 15, 45, 9, 18, 36, 12, 48, 16, 32, 8, 40, 20, 10, 50, 25, 5, 35, 7, 28, 14, 42, 21



Exercice 5.7 : Le Verger

Le **Verger** (Obstgarten) est un jeu de société coopératif créé par Anneliese Farkaschovsky. Il a été édité la première fois en 1986 par la société Haba.

Le jeu contient :

- un plan de jeu présentant 4 arbres fruitiers (prunier, pommier, poirier, cerisier) ainsi qu'un corbeau au centre.
- 40 fruits (10 pour chaque arbre du plan de jeu) en bois à disposer sur chacun des arbres.
- 4 paniers pour les joueurs (1 panier par joueur).
- un puzzle corbeau de 9 pièces que l'on reconstituera au fil du jeu.
- un gros dé.



But du jeu

Le but du jeu est de récupérer tous les fruits avant d'avoir reconstitué le puzzle corbeau.

Règles du jeu

Le plus jeune joueur commence en lançant le dé. Ce dé a quatre faces de couleur (jaune pour la poire, vert pour la pomme, bleu pour la prune et rouge pour la cerise), ainsi qu'une face « panier », et une face « corbeau ».

- S'il tombe sur une face couleur, il prend le fruit correspondant à la couleur, et le met dans son panier. S'il n'y a plus de fruit sur l'arbre, le joueur passe son tour.
- S'il tombe sur « panier », il prend deux fruits de son choix.
- S'il tombe sur « corbeau », il place une des pièces de puzzle corbeau sur le plateau de jeu.

Le jeu continue avec le joueur suivant.

Le gagnant

Les joueurs gagnent tous ensemble s'ils ont réussi à cueillir tous les fruits des arbres.

Les joueurs perdent ensemble si le puzzle corbeau de 9 pièces est reconstitué entièrement avant qu'ils n'aient pu récupérer tous les fruits.

Programme demandé

Écrivez un programme qui simulera 100'000 parties de ce jeu. Vous indiquerez comme résultat le pourcentage de victoires des joueurs, ainsi que le nombre moyen de coups dans une partie.



Exercice 5.8

Réécrivez le code de l'exercice 2.11 (calcul des probabilités au jeu « Risk ») en utilisant des boucles `for` et des listes.



Exercice 5.9 : Les cartes de loto

0	1	2	3	4	5	6	7	8
2		23		43	51			81
	16	24		48		61	77	
4	19		38			64		88

Écrivez un programme produisant une carte de loto (voir l'exemple ci-dessus, avec en rouge les numéros des colonnes). Il faudra tout d'abord trouver un algorithme permettant de créer une carte en respectant ces contraintes :

- Une carte contient 9 colonnes et 3 lignes.
- Il y a sur la carte 15 numéros différents choisis parmi les nombres de 1 à 90.
- Chaque ligne contient 5 numéros (et donc 4 espaces vides).
- Il y a toujours au moins un numéro par colonne.
- Il peut y avoir 3 numéros dans une colonne, mais seulement dans la colonne 8.
- La colonne 0 contient les numéros de 1 à 9.
- La colonne 1 contient les numéros de 10 à 19.
- La colonne 2 contient les numéros de 20 à 29.
- ...
- La colonne 7 contient les numéros de 70 à 79.
- La colonne 8 contient les numéros de 80 à 90.

5.7. Tuples

Python propose un type de données appelé **tuple**, qui est une liste non modifiable.

Du point de vue syntaxique, un tuple est une collection d'éléments séparés par des virgules :

```
alpha = 'a', 'b', 'c', 'd', 'e'
print(alpha)
```

donnera comme résultat :

```
('a', 'b', 'c', 'd', 'e')
```

Bien que cela ne soit pas nécessaire, il est fortement conseillé de mettre le tuple en évidence entre parenthèses pour améliorer la lisibilité du code. Comme ceci :

```
alpha = ('a', 'b', 'c', 'd', 'e')
```



Les opérations que l'on peut effectuer sur des tuples sont les mêmes que celles que pour les listes, si ce n'est que les tuples ne sont **pas mutables**. Il est donc impossible d'ajouter ou de supprimer des éléments d'un tuple.

5.7.1. Utilité des tuples

Les tuples trouvent principalement leur utilité dans deux cas :

- ils sont utilisés quand une fonction doit renvoyer plusieurs valeurs ;
- comme ils ne sont pas mutables, ils peuvent servir de clés pour un dictionnaire (voir chapitre 6), contrairement aux listes.

Le code ci-dessous définit une fonction appelée `cercle` qui prend en paramètre le rayon `r` du cercle et qui renvoie deux valeurs : la circonférence et l'aire du cercle. En utilisant un tuple comme type de la valeur de retour, une fonction peut retourner plusieurs valeurs.

```
import math

def cercle(r):
    #Retourne le couple (p,a) : périmètre et aire d'un cercle de rayon r
    return (math.pi*2*r, math.pi*r*r)

(p,a) = cercle(4)
print("périmètre =", p)
print("aire =", a)
```

Répetons que les parenthèses des tuples ne sont pas indispensables. Voici la version sans parenthèses :

```
import math

def cercle(r):
    #Retourne le couple (p,a) : périmètre et aire d'un cercle de rayon r
    return math.pi*2*r, math.pi*r*r

p,a = cercle(4)
print("périmètre =", p)
print("aire =", a)
```



Exercice 5.10

Réécrivez le code de l'exercice 4.4 du chapitre précédent, en utilisant un tuple pour désigner les trois coups possibles (pierre, papier ou ciseaux), à la place d'une fonction.



Exercice 5.11

Dans un article publié dans le *Journal of Recreational Mathematics*, Vol. 22, No. 4, 1990, p. 280, Mike **Keith** (www.cadaeic.net) propose un algorithme pour déterminer le jour de la semaine correspondant à une date donnée :

Entrée

d : jour ($d = 1, \dots, 31$)

m : mois ($m = 1, \dots, 12$; 1 = Janvier, 2 = Février, 3 = Mars, ..., 12 = Décembre)

y : année (> 1582)

Sortie

j : jour de la semaine ($j = 0$ à 6; 0 = Dimanche, 1 = Lundi, ..., 6 = Samedi)

On pose $z = y - 1$, si $m < 3$ et $z = y$ sinon.

$k = [(23 \cdot m)/9] + d + 4 + y + [z/4] - [z/100] + [z/400]$

$j = k \bmod 7$, si $m < 3$ et $j = (k-2) \bmod 7$ sinon

$[x]$ est le nombre entier résultant de la division euclidienne, le reste étant ignoré.

$k \bmod 7$: reste de la division par 7

- a. Programmez cet algorithme dans une fonction en Python.

Exemple : `jour_semaine(20,11,2014)` → 4

- b. Utilisez un tuple pour que la fonction retourne un nom de jour plutôt qu'un chiffre.

Exemple : `jour_semaine(20,11,2014)` → Jeudi

- c. Modifiez votre programme pour afficher tous les vendredis 13 du 21^{ème} siècle.



5.8. Ce que vous avez appris dans ce chapitre

- Vous avez pour la première fois rencontré la notion d'objet, qui est très importante en Python.
- Les listes sont des structures de données très utiles. Les principales instructions ont été présentées pour les créer et les manipuler.
- L'instruction `for` permet de parcourir une liste.
- Un tuple ressemble beaucoup à une liste, à part qu'il n'est pas modifiable.



Chapitre 6

Le dilemme du prisonnier

6.1. Nouveaux thèmes abordés dans ce chapitre

- dictionnaire
- fonction lambda

6.2. Les dictionnaires

Les dictionnaires sont des objets pouvant en contenir d'autres, à l'instar des listes. Cependant, au lieu de disposer ces informations dans un ordre précis, ils associent chaque objet contenu à une **clé** (la plupart du temps, une chaîne de caractères).

6.2.1. Création d'un dictionnaire

Un dictionnaire est un type **modifiable**. Nous pouvons donc d'abord créer un dictionnaire vide, puis le remplir. Contrairement aux listes, il n'y a pas de fonction spéciale (telle que `append()`) à appeler pour ajouter un objet.

```
dico = {}  
dico['red'] = 'rouge'  
dico['yellow'] = 'jaune'  
dico['blue'] = 'bleu'  
dico['green'] = 'vert'  
dico['pink'] = 'rose'
```

On pourra afficher ce dictionnaire à l'écran facilement :

```
print(dico)
```

```
{'blue': 'bleu', 'pink': 'rose', 'green': 'vert', 'yellow': 'jaune', 'red': 'rouge'}
```

Le dictionnaire apparaît comme une suite d'éléments séparés par des virgules, le tout entre accolades. Chaque élément est composé d'une paire d'objets (ici deux chaînes de caractères, mais on pourrait avoir d'autres types d'objets) séparés par un double point. Le premier objet est la **clé** et le second est une **valeur**.

On remarquera que les paires n'apparaissent pas dans l'ordre dans lequel elles ont été introduites. Rappelez-vous qu'un dictionnaire n'est pas ordonné.

Pour accéder à une valeur, il suffit de connaître sa clé :

```
print(dico['pink'])
```

affichera `rose`.

Les dictionnaires sont des objets. On peut donc leur appliquer des méthodes spécifiques. La méthode `keys()` renvoie les clés utilisées dans le dictionnaire :

```
print(dico.keys())
```

```
dict_keys(['blue', 'pink', 'green', 'yellow', 'red'])
```

Pour connaître les valeurs, la méthode à utiliser est `values()` :

```
print(dico.values())
```

```
dict_values(['bleu', 'rose', 'vert', 'jaune', 'rouge'])
```

On peut aussi extraire du dictionnaire une séquence équivalente de tuples avec la méthode `items()`. Cela nous sera utile quand nous voudrons parcourir un dictionnaire avec une boucle.

```
print(dico.items())
```

```
dict_items([('blue', 'bleu'), ('pink', 'rose'), ('green', 'vert'), ('yellow', 'jaune'), ('red', 'rouge')])
```

6.2.2. Manipuler un dictionnaire

La fonction intégrée `len()` permet de connaître le nombre d'entrées du dictionnaire :

```
len(dico)
```

donnera comme résultat `5`.

On a vu au § 6.2.1 comment ajouter des entrées. On peut aussi remplacer la valeur correspondant à une clé. Par exemple :

```
dico['blue'] = 'bleu ciel'
```

Pour résumer, si la clé existe, la valeur correspondante est remplacée ; si elle n'existe pas, une nouvelle entrée est créée.

On peut supprimer une entrée avec le mot-clé `del` :

```
del(dico['blue'])  
print(dico)
```

```
{'pink': 'rose', 'green': 'vert', 'yellow': 'jaune', 'red': 'rouge'}
```

La méthode `pop()` supprime également la clé précisée, mais elle renvoie en plus la valeur supprimée. Cela peut être utile parfois.

```
couleur = dico.pop('pink')  
print(couleur)
```

écrira `rose`.

D'une façon analogue à ce qui se passe avec les listes et les tuples, l'instruction `in` permet de savoir si un dictionnaire contient une clé déterminée :

```
'red' in dico
```

donnera comme résultat `True`.

Si l'on donne une clé qui n'est pas dans le dictionnaire, cela provoquera une erreur :

```
print(dico['cyan'])
```

provoquera l'erreur : **KeyError: 'cyan'**

Pour pallier ce problème, il existe la méthode `get()` :

```
print(dico.get('red', 'inconnu'))
```

écrira rouge, car red est dans le dictionnaire.

```
print(dico.get('cyan', 'inconnu'))
```

écrira inconnu, car cyan n'est pas dans le dictionnaire.

Le premier argument transmis à cette méthode est la clé de recherche, le second est la valeur que nous voulons obtenir en retour si la clé n'existe pas dans le dictionnaire.

6.2.3. Parcours d'un dictionnaire

Vous pouvez utiliser une boucle `for` pour traiter successivement tous les éléments contenus dans un dictionnaire, mais attention :

- au cours de l'itération, ce sont les **clés** du dictionnaire qui seront successivement affectées à la variable de travail, et non les valeurs ;
- l'ordre dans lequel les éléments seront parcourus est **imprévisible** (puisque'un dictionnaire n'est pas ordonné).

```
for i in dico:
    print(i, dico[i])
```

écrira :

```
blue bleu
pink rose
green vert
yellow jaune
red rouge
```

La manière de procéder suivante est plus élégante, pour un résultat identique :

```
for cle, valeur in dico.items():
    print(cle, valeur)
```

La méthode `items()` renvoie une suite de tuples (clé, valeur).

On peut aussi ne parcourir que les clés :

```
for cle in dico.keys():
    print(cle)
```

ou que les valeurs :

```
for valeur in dico.values():
    print(valeur)
```

Trier un dictionnaire



Il n'est pas possible de trier un dictionnaire, puisque, encore une fois, c'est une structure non ordonnée. Par contre, il est possible d'écrire un dictionnaire selon un certain ordre, mais il faut d'abord créer une liste de tuples (une liste, elle, peut être triée sans problèmes).

Voici la ligne de code pour créer une liste de tuples triée selon les clés :

On verra au § 6.3
ce que sont ces
lambda...

```
dico_trie = sorted(dico.items(), key=lambda x : x[0])
print(dico_trie)
```

Le résultat sera :

```
[('blue', 'bleu'), ('green', 'vert'), ('pink', 'rose'), ('red', 'rouge'), ('yellow', 'jaune')]
```

Et voici la ligne de code pour créer une liste de tuples triée selon les valeurs :

```
dico_trie = sorted(dico.items(), key=lambda x : x[1])
print(dico_trie)
```

Le résultat sera :

```
[('blue', 'bleu'), ('yellow', 'jaune'), ('pink', 'rose'), ('red', 'rouge'), ('green', 'vert')]
```

6.2.4. Copie d'un dictionnaire



La méthode `copy()` permet d'effectuer une vraie copie d'un dictionnaire. En effet, comme pour les listes (voir § 5.4.4), l'instruction `mon_dico = dico` **ne crée pas un nouveau dictionnaire**, mais une nouvelle **référence** vers le dictionnaire `dico`.

```
mon_dico = dico.copy()
```



Exercice 6.1

Construisez le dictionnaire des douze mois de l'année avec comme valeurs le nombre de jours respectif.

Utilisez la méthode `pprint()` du module `pprint` pour afficher ce dictionnaire à l'écran.



Exercice 6.2

Écrivez une fonction qui échange les clés et les valeurs d'un dictionnaire (ce qui permettra par exemple de transformer un dictionnaire anglais/français en un dictionnaire français/anglais). On suppose que le dictionnaire ne contient pas plusieurs valeurs identiques.



Exercice 6.3

En cryptographie, un chiffre de substitution simple consiste à remplacer chaque lettre d'un texte par une autre lettre (toujours la même). On utilise pour cela une table chiffante. Par exemple :

Clair	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Chiffré	W	B	H	A	Y	P	O	D	Q	Z	X	N	T	S	F	L	R	U	V	M	C	E	K	J	G	I

Ainsi, le mot «RENARD» sera chiffré « UYSWUA ».

1. Écrivez un programme qui permet de (dé)chiffrer un texte avec la table ci-dessus.
2. Utilisez votre programme pour déchiffrer le message suivant :

HYHQYVMCSHDQPPUYAYVCBVMQMCMQFS

P. S. On peut parcourir les lettres d'une chaîne de caractères avec l'instruction :

```
for lettre in chaine:
```



Exercice 6.4

Vous avez à votre disposition sur le site web compagnon un texte non accentué et non ponctué. Écrivez un programme qui compte les occurrences de chacune des lettres de l'alphabet dans ce texte, et qui en fait un histogramme comme l'exemple fictif ci-dessous :

```

: | | | | | | | | | | | | | | | |
a : | | | | | | | | | |
d : | | | |
e : | | | | | | | | | | | | | | | |
i : | | | | | | | |
l : | | | | | | | |
n : | | | | | | | | | |
o : | | | | | |
p : | |
r : | | | | | | | |
s : | | | | | | | |
t : | | | | | | | | | |
u : | | | | | |

```

6.3. Les fonctions lambda

Python permet une syntaxe intéressante pour définir des mini-fonctions d'une ligne. Empruntées au langage Lisp, ces fonctions dites *lambda* peuvent être employées partout où une fonction est nécessaire. Attention ! Les fonctions lambda sont limitées à **une seule** instruction.

La syntaxe d'une fonction lambda est la suivante :

```
ma_fonction = lambda arg1,arg2,... :instruction
```

où `arg1,arg2,...` est la liste des arguments de `ma_fonction`. Par exemple :

```
carre = lambda x: x*x
print(carre(9))
```

donnera comme résultat 81.

```
mult = lambda x,y=1: x*y
print(mult(6))
```

donnera comme résultat 6.

```
print(mult(6,3))
```

donnera comme résultat 18.

Il est évidemment possible que l'instruction soit une fonction définie ailleurs. Par exemple :

```
def signe(x):
    if x>0:
        return 1
    elif x<0:
        return -1
    else:
        return 0

f = lambda x : 2*signe(x)
print(f(-5))
```

donnera comme résultat -2.

6.4. Le dilemme du prisonnier - règles du jeu

Dans le jeu du « dilemme du prisonnier », deux détenus sont emprisonnés dans des cellules séparées. La police fait à chacun des deux le même marché :

« Tu as le choix entre dénoncer ton complice ou non. Si tu le dénonces et qu'il te dénonce aussi, vous aurez une remise de peine d'un an tous les deux. Si tu le dénonces et que ton complice te couvre, tu auras une remise de peine de 5 ans (et tu seras libéré), mais ton complice tirera le maximum. Mais si vous vous couvrez mutuellement, vous aurez tous les deux une remise de peine de 3 ans. »

		coopère	trahit
	coopère	 -3 -3	 0 -5
	trahit	 -5 0	 -1 -1

Dans cette situation, il est clair que si les détenus s'entendent, ils s'en tireront globalement mieux que si l'un trahit l'autre. Mais l'un peut être tenté de s'en tirer encore mieux en trahissant son complice. Craignant cela, l'autre risque aussi de trahir son complice pour ne pas être le dindon de la farce. Le dilemme est donc : « faut-il coopérer avec son complice (et donc le couvrir) ou non ? »

Le dilemme du prisonnier devient plus intéressant et plus réaliste lorsque la durée de l'interaction n'est pas connue. On peut alors envisager de se souvenir du comportement d'un joueur à son égard et développer une stratégie en rapport. Par exemple, si je sais que mon adversaire ne coopère jamais, mon intérêt de ne pas coopérer non plus, sous peine d'être systématiquement grugé. Par contre, si je sais que mon adversaire coopérera toujours quoi qu'il arrive, j'aurai intérêt à être vicieux et ne jamais coopérer pour maximiser mon gain.

Exemple de partie

Coups et score du joueur Donnant donnant
['C', 'T', 'T', 'T', 'T', 'C', 'T', 'C', 'T', 'T']
19

Coups et score du joueur Aléatoire
['T', 'T', 'T', 'T', 'C', 'T', 'C', 'T', 'T', 'C']
19

6.5. Code du premier programme (les duels)

Voyons tout d'abord un premier programme où deux joueurs s'affrontent en duel.

```
# dilemme du prisonnier itéré, version duel

from random import choice

choix = ['T','C'] # T : trahit, C : coopère

def gain(lui,moi):
    if lui=='C' and moi=='C':
        return 3
    elif lui=='C' and moi=='T':
        return 5
    elif lui=='T' and moi=='C':
        return 0
    elif lui=='T' and moi=='T':
        return 1
```

```

# Toujours seul
# ne coopère jamais
def toujours_seul(liste_lui, liste_moi):
    return 'T'

# Bonne poire
# coopère toujours
def bonne_poire(liste_lui, liste_moi):
    return 'C'

# Aléatoire
# joue avec une probabilité égale 'T' ou 'C'
def aleatoire(liste_lui, liste_moi):
    global choix
    return choice(choix)

# Donnant donnant
# coopère seulement si l'autre joueur a coopéré au coup précédent.
def donnant_donnant(liste_lui, liste_moi):
    if len(liste_lui) > 0:
        return liste_lui[-1]
    else: # premier coup
        return 'C'

# Majorité
# coopère seulement si l'autre joueur a coopéré en majorité.
def majorite(liste_lui, liste_moi):
    if len(liste_lui) > 0:
        if liste_lui.count('C') > len(liste_lui)//2:
            return 'C'
        else:
            return 'T'
    else: # premier coup
        return 'C'

# Une partie entre deux joueurs différents

liste = {}
score = {}

liste['Aléatoire'] = []
liste['Donnant donnant'] = []

for joueur in liste.keys():
    score[joueur] = 0

nb_coups = 0
nb_total_coups = 10 # à modifier

while nb_coups < nb_total_coups :
    coup_joueur1 = aleatoire(liste['Donnant donnant'], liste['Aléatoire'])
    coup_joueur2 = donnant_donnant(liste['Aléatoire'], liste['Donnant donnant'])
    liste['Aléatoire'].append(coup_joueur1)
    liste['Donnant donnant'].append(coup_joueur2)
    score['Aléatoire'] += gain(coup_joueur2, coup_joueur1)
    score['Donnant donnant'] += gain(coup_joueur1, coup_joueur2)
    nb_coups += 1

for joueur in liste.keys():
    print("Coups et score du joueur", joueur)
    print(liste[joueur])
    print(score[joueur])
    print()

```

Analyse du programme

La première partie du programme ne présente rien de nouveau : on définit les gains et les stratégies des joueurs (les noms des stratégies sont données par les noms des joueurs).

La grande nouveauté apparaît avec les dictionnaires :

```
liste = {}  
score = {}
```

On crée d'abord deux dictionnaires : `liste` donnant la liste des coups joués par les joueurs, et `score` donnant leur score respectif.

On initialise ensuite les deux listes de coups :

```
liste['Aléatoire'] = []  
liste['Donnant donnant'] = []
```

puis on met les scores à 0 :

```
for joueur in liste.keys():  
    score[joueur] = 0
```

Enfin, le duel commence et durera `nb_total_coups` :

```
nb_coups = 0  
nb_total_coups = 10    # à modifier  
while nb_coups < nb_total_coups :
```

Chaque joueur joue selon sa stratégie en tenant compte des coups précédents (le premier paramètre est la liste des coups de l'adversaire, le second la liste des coups du joueur) :

```
coup_joueur1 = aleatoire(liste['Donnant donnant'],liste['Aléatoire'])  
coup_joueur2 = donnant_donnant(liste['Aléatoire'],liste['Donnant donnant'])
```

On ajoute ce coup à la liste des coups :

```
liste['Aléatoire'].append(coup_joueur1)  
liste['Donnant donnant'].append(coup_joueur2)
```

puis on met à jour les scores et on recommence pour le coup suivant :

```
score['Aléatoire'] += gain(coup_joueur2,coup_joueur1)  
score['Donnant donnant'] += gain(coup_joueur1,coup_joueur2)  
nb_coups += 1
```

On affiche finalement les coups joués lors du duel et les scores des deux joueurs :

```
for joueur in liste.keys():  
    print("Coups et score du joueur",joueur)  
    print(liste[joueur])  
    print(score[joueur])  
    print()
```

6.6. Code partiel du second programme (le tournoi)

Dans ce second programme, on pourra faire s'affronter plus de deux joueurs. Chaque joueur affrontera en duel chacun des autres joueur, y compris lui-même !

```
# dilemme du prisonnier itéré, version tournoi
```

Le début du code est strictement identique à la version duel. Voir § 6.5.

```
# Le tournoi  
  
liste = {}  
strategie = {}  
score = {}
```

```

duel = {}

# ajouter des joueurs ci-dessous, selon les modèles des joueurs existants
# commencer ici
liste['Toujours seul'] = []
liste['Bonne poire'] = []
liste['Majorité'] = []
liste['Aléatoire'] = []
liste['Donnant donnant'] = []
# ...

strategie['Toujours seul'] = lambda lui, moi : toujours_seul(lui,moi)
strategie['Bonne poire'] = lambda lui, moi : bonne_poire(lui,moi)
strategie['Majorité'] = lambda lui, moi : majorite(lui,moi)
strategie['Aléatoire'] = lambda lui, moi : aleatoire(lui,moi)
strategie['Donnant donnant'] = lambda lui, moi : donnant_donnant(lui,moi)
# ...
# terminer là

nb_total_coups = 10 # à modifier

for joueur in liste.keys():
    score[joueur] = 0

for i in liste.keys(): # i et j sont les joueurs
    for j in liste.keys() :
        liste[i] = [] # on recommence une partie
        liste[j] = []
        if i>=j:
            nb_coups = 0
            score_joueur1 = 0
            score_joueur2 = 0
            while nb_coups < nb_total_coups :
                coup_joueur1 = strategie[i](liste[j],liste[i])
                coup_joueur2 = strategie[j](liste[i],liste[j])
                liste[i].append(coup_joueur1)
                if i!=j:
                    liste[j].append(coup_joueur2)
                score_joueur1 += gain(coup_joueur2,coup_joueur1)
                score_joueur2 += gain(coup_joueur1,coup_joueur2)
                nb_coups += 1
            duel[(i,j)] = score_joueur1
            if i!=j:
                duel[(j,i)] = score_joueur2
            score[i] += score_joueur1
            if i!=j:
                score[j] += score_joueur2

# affichage des résultats

def trie_par_valeur(d):
    #retourne une liste de tuples triée selon les valeurs
    return sorted(d.items(), key=lambda x: x[1])

def trie_par_cle(d):
    #retourne une liste de tuples triée selon les clés
    return sorted(d.items(), key=lambda x: x[0])

score_trie = trie_par_valeur(score)
score_trie.reverse()
for i in range(0,len(score_trie)):
    print(score_trie[i][0],":",score_trie[i][1])
print()
duel_trie = trie_par_cle(duel)
for i in range(0,len(duel_trie)):
    print(duel_trie[i][0][0],"contre",duel_trie[i][0][1],"gagne",duel_trie[i][1],"pts")

```

Analyse du programme

On aura besoin ici de quatre dictionnaires :

```
liste = {}
```

Le dilemme du prisonnier

```
strategie = {}
score = {}
duel = {}
```

Le dictionnaire `liste` mémorise la liste des coups durant un duel :

```
liste['Toujours seul'] = []
liste['Bonne poire'] = []
liste['Majorité'] = []
liste['Aléatoire'] = []
liste['Donnant donnant'] = []
```

Les stratégies sont « rangées » dans un dictionnaire. Remarquez l'utilisation des fonctions `lambda`. On est obligé de les utiliser parce que les fonctions-stratégies ont des paramètres.

```
strategie['Toujours seul'] = lambda lui, moi : toujours_seul(lui,moi)
strategie['Bonne poire'] = lambda lui, moi : bonne_poire(lui,moi)
strategie['Majorité'] = lambda lui, moi : majorite(lui,moi)
strategie['Aléatoire'] = lambda lui, moi : aleatoire(lui,moi)
strategie['Donnant donnant'] = lambda lui, moi : donnant_donnant(lui,moi)
```

Si la fonction n'a pas de paramètre, on peut se passer du `lambda`.

Par exemple :

```
def moi():
    print("Je m'appelle Jean")

fonct = {}
fonct['moi'] = moi()
fonct['moi']
```

écrira « Je m'appelle Jean ».

On met à zéro les scores des joueurs :

```
for joueur in liste.keys():
    score[joueur] = 0
```

Chaque joueur (*i*) affrontera une fois en duel chacun des autres joueurs (*j*), y compris lui-même. La condition `if i>=j` : est là pour n'autoriser qu'un duel et non pas deux.

Il faut aussi faire attention, quand *i* égale *j*, de ne pas compter les scores à double, de ne pas mettre tous les coups dans une seule liste et encore de mémoriser le bon score. Cela explique les trois `if i!=j` : du code ci-dessous.

```
for i in liste.keys(): # i et j sont les joueurs
    for j in liste.keys() :
        liste[i] = [] # on recommence une partie
        liste[j] = []
        if i>=j:
            nb_coups = 0
            score_joueur1 = 0
            score_joueur2 = 0
            while nb_coups < nb_total_coups :
                coup_joueur1 = strategie[i](liste[j],liste[i])
                coup_joueur2 = strategie[j](liste[i],liste[j])
                liste[i].append(coup_joueur1)
                if i!=j:
                    liste[j].append(coup_joueur2)
                score_joueur1 += gain(coup_joueur2,coup_joueur1)
                score_joueur2 += gain(coup_joueur1,coup_joueur2)
                nb_coups += 1
            duel[(i,j)] = score_joueur1
            if i!=j:
                duel[(j,i)] = score_joueur2
            score[i] += score_joueur1
            if i!=j:
```



```
score[j] += score_joueur2
```

Le dictionnaire `duel` va nous servir à détailler les scores à la fin de la simulation. On pourra ainsi mieux analyser les résultats du tournoi.

La fin du code permet d'afficher les résultats, triés par ordre décroissant pour les scores, et par ordre alphabétique pour les duels.

```
def trie_par_valeur(d):
    #retourne une liste de tuples triée selon les valeurs
    return sorted(d.items(), key=lambda x: x[1])

def trie_par_cle(d):
    #retourne une liste de tuples triée selon les clés
    return sorted(d.items(), key=lambda x: x[0])

score_trie = trie_par_valeur(score)
score_trie.reverse()
for i in range(0, len(score_trie)):
    print(score_trie[i][0], ":", score_trie[i][1])
print()
duel_trie = trie_par_cle(duel)
for i in range(0, len(duel_trie)):
    print(duel_trie[i][0][0], "contre", duel_trie[i][0][1], "gagne", duel_trie[i][1], "pts")
```



Exercice 6.5 – Tournoi

Le but de cet exercice est de définir une stratégie, c'est-à-dire de définir une règle pour savoir quand accepter et quand refuser la coopération avec un autre joueur. Les seules informations connues sont la liste des coups que l'adversaire a joués jusque-là, et la vôtre.

1. Programmez quelques stratégies dans le programme du § 6.5. N'oubliez pas le commentaire qui décrira succinctement votre stratégie.
2. Testez ces stratégies en modifiant le programme du § 6.6 et choisissez la meilleure.
3. Donnez votre stratégie préférée au professeur en vue du tournoi qui mettra en lice tous les élèves de la classe. Le nom de cette stratégie sera votre prénom.

Références

- Delahaye J.-P., *L'altruisme récompensé ?*, Pour la Science 181, novembre 1992, pp. 150-6
- Delahaye J.-P., Mathieu Ph., *L'altruisme perfectionné*, Pour la Science 187, mai 1993, pp. 102-7
- Delahaye J.-P., *Le dilemme du prisonnier et l'illusion de l'extorsion*, Pour la Science 435, janvier 2014, pp. 78-83
- Delahaye J.-P., Mathieu Ph., *Adoucir son comportement ou le durcir*, Pour la Science 462, avril 2016, pp. 80-5



Exercice 6.6 – Le dilemme du renvoi d'ascenseur

Situation ambiguë : je suis face à un inconnu au rez-de-chaussée de cette grande tour, devant ce petit ascenseur à une seule place dont la porte est ouverte. Il n'y a pas de bouton pour le rappel lorsqu'il sera monté, et je n'ai pas du tout, mais alors pas du tout, envie de gravir les huit étages à pied. L'inconnu me dit : « Laissez-moi passer en premier, car, arrivé en haut, je vous renverrai l'ascenseur. »

Dois-je accepter ? Pourquoi devrais-je le croire et accepter qu'il passe devant moi ? D'ailleurs tiendra-t-il sa promesse si je le laisse passer ? C'est le dilemme du renvoi d'ascenseur.

Lorsque nous imaginons que nous devons y jouer à de multiples reprises avec toutes sortes de gens, cette situation fait ressortir de multiples aspects inattendus. Cette variante du dilemme itéré du prisonnier (voir ex. 6.5) possède des propriétés remarquables.

Le dilemme du prisonnier

Refaites le tournoi de l'exercice 6.5 avec les gains ci-dessous :

```
def gain(lui,moi):
    if lui=='C' and moi=='C':
        return 3
    elif lui=='C' and moi=='T':
        return 8
    elif lui=='T' and moi=='C':
        return 0
    elif lui=='T' and moi=='T':
        return 1
```

Que constatez-vous ?

Pourrez-vous trouver une meilleure stratégie pour ce dilemme ?

Référence : Delahaye J.-P., Mathieu Ph., *Le dilemme du renvoi d'ascenseur*, Pour la Science 269, mars 2000, pp. 102-6

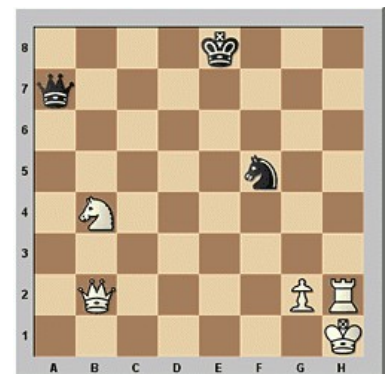


Exercice 6.7

Représentez la position du jeu d'échecs ci-contre en utilisant un dictionnaire.

La clé sera la position d'une case (un tuple, par exemple ('A', 7)), la valeur sera la pièce qui s'y trouve (par exemple « dame noire »).

Ne mettez dans le dictionnaire que les cases occupées.



Exercice 6.8 - Le jeu des échelles

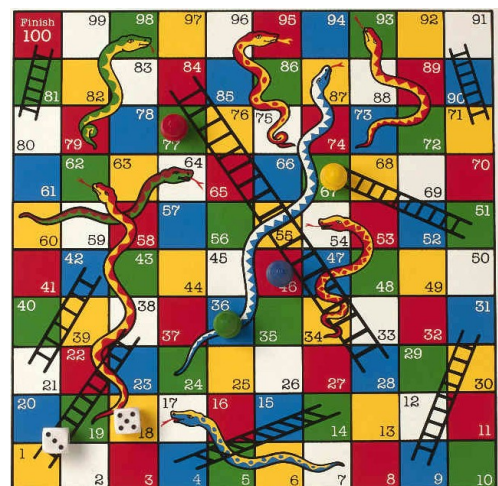
Règles du jeu

Le joueur lance un dé et avance d'autant de cases que de points sur le dé.

S'il tombe sur une case dans laquelle il y a le pied d'une échelle, il monte le long de celle-ci jusqu'en haut. S'il tombe sur une case dans laquelle il y a la tête d'un serpent, il doit redescendre jusqu'à la queue du serpent.

La partie se termine quand on arrive sur la case 100. Si le joueur tire un dé qui ne lui permet pas d'arriver exactement sur la case 100, il recule du nombre de points supplémentaires sur le dé.

Évidemment, une case ne peut être le départ ou l'arrivée que d'une seule échelle et un seul serpent.



Première question

La question à laquelle on voudrait répondre est la suivante : « Comment de fois faudra-t-il (en moyenne) lancer le dé pour terminer la partie ? » Simulez 100'000 parties sur le plateau ci-dessus.

Deuxième question (plus difficile)

Déplacez (ou pas) horizontalement les serpents et les échelles du plateau ci-dessus de manière à :

- maximiser le nombre de lancers de dés
- minimiser le nombre de lancers de dés.



Exercice 6.9

Première partie

Écrivez un programme qui code un message en Morse :

entrée : code morse

donnera comme résultat :

```
['-.-.', '----', '-.-.', '.', ' ', '--', '----', '-.-.', '...', '.']
```

Vous trouverez l'alphabet Morse à l'adresse : www.nymphomath.ch/crypto/morse/

Seconde partie

Utilisez ensuite la méthode `Beep` du module `winsound` pour transformer les traits et les points en signaux sonores.

Voici ce que dit la documentation Python sur la méthode `Beep` :

`winsound.Beep(frequency, duration)`

Beep the PC's speaker. The *frequency* parameter specifies frequency, in hertz, of the sound, and must be in the range 37 through 32,767. The *duration* parameter specifies the number of milliseconds the sound should last. If the system is not able to beep the speaker, **RuntimeError** is raised.



6.7. Ce que vous avez appris dans ce chapitre

- Vous avez vu comment créer un dictionnaire et le manipuler.
- Faites bien attention qu'un dictionnaire n'est pas ordonné. Cela peut parfois occasionner de mauvaises surprises...
- Vous avez aussi vu les fonctions `lambda`. Elles peuvent paraître superflues, mais sont utiles dans certains contextes, comme celui du jeu du dilemme du prisonnier, version Tournoi (§ 6.6). Généralement, on utilise plutôt le mot-clé `def`.
- Enfin, vous avez vu (et surtout entendu) une manière primitive de générer du son avec Python (exercice 6.9).



Chapitre 7

Jeux de lettres

7.1. Nouveaux thèmes abordés dans ce chapitre

- fichier
- chaîne de caractères (string)
- ensemble (set)

7.2. Fichier

On divise habituellement les fichiers en deux catégories.

Les **fichiers binaires** ne contiennent pas exclusivement du texte. Ils ne peuvent être « lus » que par des logiciels spécialisés. Une image JPEG, un fichier PDF, ou un MP3 sont des fichiers binaires.

Les **fichiers textes** sont « compréhensibles » par un humain : ils contiennent du texte (lettres, ponctuations, nombres, ...). Leur contenu est souvent organisé en lignes et on peut les lire avec des éditeurs de textes simples comme *NotePad++*, *WordPad*, etc.

Ici, nous allons exclusivement nous intéresser aux fichiers textes.

Pour jouer à des jeux de lettres, il nous faut d'abord constituer une liste de mots. On pourrait mettre tous ces mots dans une liste dans le corps du programme, mais il y a plusieurs inconvénients :

- on verra les mots en ouvrant le programme, ce qui gâchera le plaisir de les trouver ;
- si l'on a beaucoup de mots dans notre stock, cela nuira à la lecture du programme ;
- l'échange de données avec d'autres programmes (peut-être écrits dans d'autres langages) est tout simplement impossible, puisque ces données font partie du programme lui-même.

D'une manière générale, quand la quantité de données est importante, mieux vaut stocker ces dernières dans un **fichier**.

Un fichier est une **ressource**. L'accès aux ressources doit pouvoir être partagé entre les différents programmes qui peuvent en avoir besoin. Cela veut dire qu'il est nécessaire d'*acquérir* cette ressource avant de s'en servir, puis de la *libérer* après usage. Pour un fichier, cela se fait en l'ouvrant (`open`), puis en le fermant (`close`) :



Un fichier ouvert doit toujours être fermé après usage !

```
# Ouverture d'un fichier en lecture:
fichier = open("/etc/passwd", "r")
...
# Fermeture du fichier
fichier.close()
```

Le premier paramètre de la fonction `open` est le nom du fichier à manipuler, avec son chemin d'accès. Le second paramètre est le **mode** dans lequel on souhaite ouvrir le fichier. Le mode « `r` » (pour *read*) indique que nous allons ouvrir le fichier pour **lire** des données dedans. Tant qu'un fichier est ouvert, il ne peut pas être utilisé par un autre programme.

On peut aussi ouvrir un fichier pour **écrire** en utilisant le mode « `w` » (pour *write*) :



```
# Ouverture d'un fichier en écriture:
fichier = open("/etc/passwd", "w")
```

Mais attention ! Dans ce cas, si le fichier existe déjà, son contenu est **écrasé** ! Si vous voulez plutôt **ajouter** des données à la fin d'un fichier, il faut utiliser le mode « `a` » (pour *append*) :

```
# Ouverture d'un fichier en ajout:
fichier = open("/etc/passwd", "a")
```

7.2.1. Lire dans un fichier

La lecture dans un fichier texte se fait de manière séquentielle, une ligne après l'autre (c'est la raison pour laquelle on parle de fichier à accès **séquentiel**). Il n'est pas possible de lire une ligne au milieu du fichier avant d'avoir lu toutes celles placées avant.

Il est donc nécessaire de connaître la structure du fichier pour pouvoir le lire correctement. Par exemple, dans les fichiers que nous allons utiliser, chaque ligne du fichier comportera un seul mot.

La méthode `readlines` lit **toutes les lignes** du fichier et les met dans une liste. Elle est utile si on souhaite lire tout le fichier d'un coup :

```
toutesleslignes = fichier.readlines()
```

La méthode `readline` (au singulier) lit **la ligne suivante** dans le fichier.

```
ligne = fichier.readline()
```

Enfin, pour traiter l'une après l'autre toutes les lignes d'un fichier, on peut utiliser une boucle `for` :

```
for ligne in fichier:
    ...
```

7.2.2. Écrire dans un fichier

Écrire dans un fichier se fait très simplement en utilisant la méthode `write()`.

```
fichier.write("ce qu'il faut écrire\n")
```

Les symboles `\n` indiquent que l'on saute à la ligne suivante.

Ces données sont enregistrées dans le fichier les unes à la suite des autres. Chaque nouvel appel de `write()` continue l'écriture à la suite de ce qui est déjà enregistré.

7.3. Chaîne de caractères



Les chaînes de caractères forment la classe 'str' (pour *string*). Une chaîne de caractères est une suite de symboles placées entre guillemets ("salut") ou entre apostrophes ('salut'). L'intérêt d'avoir ces deux délimiteurs est de pouvoir écrire des chaînes contenant des apostrophes ("aujourd'hui") ou des guillemets ('Pressez "enter" pour continuer').

Les méthodes de la classe 'str' sont nombreuses. Elles sont détaillées dans la documentation officielle de Python à cette adresse :

<http://docs.python.org/dev/library/stdtypes.html#string-methods>

Nous ne verrons ici que les plus utiles.

Prenons comme exemple de chaîne de caractères :

```
chaîne = "ma Chaîne de Caractères"
```

7.3.1. Accès à des sous-chaînes

La fonction intégrée `len`, que l'on a déjà rencontrée, permet de connaître la longueur (*length* en anglais) d'une chaîne de caractères :

```
len(chaîne)
```



donnera comme résultat 23. Les positions de la string `chaîne` vont donc de 0 à 22.

On peut accéder à des lettres isolées de la chaîne ou à des sous-chaînes d'une manière analogue à celle utilisée avec les listes.

```
print(chaîne[0])
```

donnera le premier caractère de la chaîne ; dans notre exemple 'm'.

```
print(chaîne[3:])
```

donne la sous-chaîne commençant à la troisième position, à savoir 'Chaîne de Caractères'. (la numérotation des places commence à 0, comme pour les listes). Inversement,

```
print(chaîne[:9])
```

donne la sous-chaîne se terminant à la huitième (!) position, à savoir 'ma Chaîne'. En combinant les deux :

```
print(chaîne[3:9])
```

on obtiendra la sous-chaîne 'Chaîne'.

Il est possible de transformer une chaîne de caractères en une liste de « mots ». Il faut juste indiquer ce qui joue le rôle de séparateur. Par exemple, si le séparateur est l'espace,

```
chaîne.split(' ')
```

produira comme résultat : ['ma', 'Chaîne', 'de', 'Caractères'].

Pour avoir la liste des caractères de la chaîne, rien de plus facile :

```
list(chaîne)
```

retournera la liste ['m', 'a', ' ', 'C', 'h', 'a', 'î', 'n', 'e', ' ', 'd', 'e', ' ', 'C', 'a', 'r', 'a', 'c', 't', 'è', 'r', 'e', 's'].

On peut parcourir tous les caractères d'une chaîne avec l'instruction `for` :

```
for ch in chaîne:
    print(ch, end=" ")
```

produira comme résultat : m a C h a î n e d e C a r a c t è r e s

On peut savoir si une sous-chaîne se trouve dans une chaîne :

```
'de' in chaîne
```

donnera `True`. Il est aussi possible de connaître la position où commence une sous-chaîne :

```
print(chaîne.index('de'))
```

donnera 10, car « de » apparaît pour la première fois à la position numéro 10. Si la sous-chaîne

recherchée n'existe pas, le programme renvoie un message d'erreur :

```
print(chaine.index('u'))
```

produit **ValueError: substring not found**

7.3.2. Mises en forme

Pour tout écrire en majuscules :

```
print(chaine.upper())
```

donnera comme résultat : 'MA CHAÎNE DE CARACTÈRES'

Pour tout écrire en minuscules :

```
print(chaine.lower())
```

donnera comme résultat : 'ma chaîne de caractères'

Pour écrire la première lettre en majuscules et les autres en minuscules :

```
print(chaine.capitalize())
```

donnera comme résultat : 'Ma chaîne de caractères'

7.3.3. Manipulations des strings



Attention ! Contrairement aux listes, **les chaînes de caractères ne sont pas modifiables**. Ainsi, une affectation du type :

```
chaine[0]="T"
```

donnera le message d'erreur : **TypeError: 'str' object does not support item assignment**

On peut cependant faire des opérations sur des chaînes de caractères. On peut par exemple enlever des caractères qui se trouvent en début ou en fin de chaîne avec la méthode `strip()` :

```
print('www.nymphomath.ch'.strip('chw.'))
```

donnera comme résultat : 'nymphomat'

Si la fonction `strip()` n'a pas d'argument, on enlèvera par défaut des espaces. Si on veut enlever des caractères en début de chaîne, il existe la méthode `lstrip()` ; si on veut enlever des caractères en fin de chaîne on utilisera la méthode `rstrip()` :

```
print('www.nymphomath.ch'.lstrip('chw.'))
```

donnera comme résultat 'nymphomath.ch', tandis que

```
print('www.nymphomath.ch'.rstrip('chw.'))
```

donnera comme résultat 'www.nymphomat'

On peut concaténer deux chaînes avec l'opérateur `+` :

```
chaine1 = "ma Chaîne"  
chaine2 = " de Caractères"  
chaine = chaine1 + chaine2  
print(chaine)
```

donnera 'ma Chaîne de Caractères'.

On peut aussi répéter une chaîne de caractères plusieurs fois :

```
chaine = "salut "
```

```
chaine = chaine*4
print(chaine)
```

donnera : 'salut salut salut salut '

7.4. Le pendu - règles du jeu

Le jeu du pendu consiste à retrouver un mot le plus vite possible (avant que le dessin du pendu soit terminé) en proposant des lettres. Si la lettre appartient au mot, elle est écrite aux bons emplacements, sinon on continue de dessiner le pendu.

Nous allons commencer par une version non graphique où il s'agira de deviner un mot avec le moins d'essais possible. Nous verrons une version graphique au chapitre 8.

Exemple de partie

```
-----
Entrez une lettre ou '?' pour abandonner : E
-----
Entrez une lettre ou '?' pour abandonner : U
U---U-
Entrez une lettre ou '?' pour abandonner : A
U-A-U-
Entrez une lettre ou '?' pour abandonner : N
U-ANU-
Entrez une lettre ou '?' pour abandonner : S
U-ANUS
Entrez une lettre ou '?' pour abandonner : R
URANUS
Bravo ! Le mot URANUS a été trouvé en 6 coups
```

Remarque : les lettres en gras ont été entrées au clavier par le joueur.

7.5. Code du programme



pendu.py

<http://ow.ly/KEyn2>

```
# Le jeu du pendu
from random import choice

fichier = open("liste_mots.txt", "r")
liste_mots = fichier.readlines() # met tous les mots du fichiers dans une liste
mot = choice(liste_mots) # prend au hasard un mot dans la liste
mot = mot.rstrip() # supprime le caractère "saut à la ligne"
fichier.close()

mot_devine = "-" * len(mot)
print(mot_devine)
nbr_essais = 0

while mot_devine != mot:
    lettre = input("Entrez une lettre ou '?' pour abandonner : ")
    lettre = lettre[0] # évite des erreurs si un mot est entré au lieu d'une lettre
    if lettre == '?':
        print('Le mot était', mot)
        break
    lettre = lettre.upper()
    for i in range(len(mot)):
        if lettre == mot[i]:
            mot_devine = mot_devine[:i] + lettre + mot_devine[i+1:]
    print(mot_devine)
    nbr_essais += 1

if mot == mot_devine:
    print('Bravo ! Le mot', mot, 'a été trouvé en', nbr_essais, 'coups')
```


7.6. Analyse du programme

```
from random import choice
```

La fonction `choice` permet de choisir un élément au hasard dans une liste. Ici, il s'agira de choisir un mot.

```
fichier = open("liste_mots.txt", "r")
liste_mots = fichier.readlines() # met tous les mots du fichiers dans une liste
mot = choice(liste_mots)         # prend au hasard un mot dans la liste
mot = mot.rstrip()               # supprime le caractère "saut à la ligne"
fichier.close()
```

Tous les mots se trouvent dans le fichier `liste_mots.txt`. Ce fichier doit se trouver au même niveau que le programme.

```
mot_devine = "-" * len(mot)
print(mot_devine)
nbr_essais = 0
```

Au début, on écrit une suite de tirets pour indiquer la longueur du mot. Ces tirets seront remplacés par des lettres au fur et à mesure que le joueur les découvrira.

```
while mot_devine != mot:
```

La boucle principale se répète tant que l'on n'a pas découvert le mot.

```
    lettre = input("Entrez une lettre ou '?' pour abandonner : ")
    lettre = lettre[0] # évite des erreurs si un mot est entré au lieu d'une lettre
    if lettre == '?':
        print('Le mot était', mot)
        break
```

On demande une lettre au joueur. Il peut en fait entrer n'importe quoi, mais cela ne le fera pas avancer dans sa quête... S'il entre un point d'interrogation, alors on donnera le mot à trouver et on sortira de la boucle par l'instruction `break`.

```
        lettre = lettre.upper()
```

Comme dans le fichier contenant notre stock de mots, tous les mots sont écrits en majuscules, on s'assure que la lettre entrée est bien une majuscule.

```
        for i in range(len(mot)):
            if lettre == mot[i]:
                mot_devine = mot_devine[:i] + lettre + mot_devine[i+1:]
        print(mot_devine)
        nbr_essais += 1
```

La variable `i` parcourt toutes les positions du mot. Si la lettre proposée se trouve à la position `i`, alors on remplace le tiret par cette lettre (3^{ème} ligne du bout de code ci-dessus). On écrit ensuite le nouveau résultat intermédiaire et on incrémente le nombre d'essais.

```
    if mot == mot_devine:
        print('Bravo ! Le mot', mot, 'a été trouvé en', nbr_essais, 'coups')
```

Après être sorti de la boucle, on écrit un petit message de félicitation si le bon mot a été découvert.



Exercice 7.1

Imaginons que la fonction `choice` n'existe pas. Comment choisir un mot au hasard dans la liste `liste_mots` ?



Exercice 7.2

Réutilisez le fichier `liste_mots.txt` pour un autre jeu qui consistera à retrouver un mot, l'ordinateur ayant écrit ses lettres par ordre alphabétique. Par exemple :

```
AEIIMMNPRT
Entrez l'anagramme : imprimante
Bravo !
```

7.7. Ensemble

Les ensembles en Python sont les mêmes ensembles qu'en mathématiques. On peut leur appliquer les mêmes opérations : union, intersection, différence symétrique, différence. On peut aussi tester si un ensemble est inclus dans un autre.

Voici un programme qui résume toutes ces opérations.



ensembles.py

<http://ow.ly/KEyn2>

```
ens1 = set([1, 2, 3, 4, 5, 6])
ens2 = set([5, 6, 7, 8])
ens3 = set([2, 4, 6])

# union
print(ens1 | ens2)
# intersection
print(ens1 & ens2)
# différence symétrique
print(ens1 ^ ens2)
# différence
print(ens1 - ens2)

# inclusion
print(ens2.issubset(ens1))
print(ens3.issubset(ens1))

# transformation en une liste
listel = list(ens1)
print(listel)
```

Voici le résultat de ce programme :

```
{1, 2, 3, 4, 5, 6, 7, 8}
{5, 6}
{1, 2, 3, 4, 7, 8}
{1, 2, 3, 4}
False
True
[1, 2, 3, 4, 5, 6]
```

Remarquez que dans un ensemble, chaque élément n'apparaît qu'**une seule fois**.

Notons enfin que l'on peut facilement convertir une chaîne de caractères en un ensemble :

```
set('carnaval')
```

retournera l'ensemble : {'l', 'n', 'a', 'c', 'v', 'r'}



Exercice 7.3

Écrivez un programme qui remplace les voyelles d'un texte (éventuellement accentuées) par une étoile. Par exemple « Il était une fois » deviendra « *l *t***t *n* f**s ».



Exercice 7.4

Pour toutes les questions ci-dessous, utilisez le fichier dico.txt.



dico.txt

<http://ow.ly/KEyn2>

- Calculez le pourcentage de mots français où la seule voyelle est le « e » (il peut y en avoir plusieurs dans le mot).
Par exemple : exemple, telle, égrener.
- Calculez le pourcentage de mots français où deux lettres identiques se suivent.
Par exemple : femme, panne, créer.
- Affichez les mots de moins de 10 lettres ayant comme lettre centrale un « w ». Le mot doit avoir un nombre impair de lettres.
Par exemple : edelweiss, kiwis.
- Affichez la liste des mots contenant la suite de lettres « dile ».
Par exemple : crocodile, prédilection.
- Affichez les mots de moins de 5 lettres qui commencent et finissent par la même lettre.
Par exemple : aura, crocc, dardd.
- Affichez tous les mots palindromes.
Par exemple : serres, kayak, ressasser.



Exercice 7.5

Pour toutes les questions ci-dessous, utilisez le fichier dico.txt.

Dans le jeu du pendu, les mots les plus difficiles à trouver sont ceux qui ont peu de voyelles et des consonnes rares. Voici les lettres de la plus fréquente à la moins fréquente, en français :



dico.txt

<http://ow.ly/KEyn2>

Lettres courantes : E A I S T N R U L O D M P
Lettres rares : C V Q G B F J H Z X Y K W

- Affichez la liste des mots n'ayant que des voyelles et des lettres rares.
- Affichez la liste des mots de plus de 15 lettres n'ayant que des lettres courantes.
- Affichez la liste des mots n'ayant aucune des lettres E A I S T N R U.
- Affichez la liste des mots ayant exactement deux voyelles et plus de 9 caractères (tiret compris). Par exemple : transports, check-list.
- Affichez la liste des mots commençant par au moins 4 consonnes consécutives (tiret non autorisé).



Exercice 7.6

L'argot *javanais*, apparu en France dans la dernière moitié du 19^{ème} siècle, consiste à intercaler dans les mots la syllabe « av » entre une consonne et une voyelle. Nous n'utiliserons ici que les règles simplifiées (*) :

- On rajoute « av » après chaque consonne (ou groupe de consonnes comme par exemple ch, cl, ph, tr, ...) d'un mot.
- Si le mot commence par une voyelle, on ajoute « av » devant cette voyelle.
- On ne rajoute jamais « av » après la consonne finale d'un mot.

Écrivez un programme qui traduit un texte français en javanais. Par exemple,
Je vais acheter des allumettes au supermarché.

deviendra

Jave vavaïs avachavetaver daves avallavumavettaves avau savupavermavarchavé.

(*) Pour les règles complètes, voir [https://fr.wikipedia.org/wiki/Javanais_\(argot\)](https://fr.wikipedia.org/wiki/Javanais_(argot))

7.8. Le mot le plus long – règles du jeu

Les règles sont élémentaires : on entre une série de lettres et l'ordinateur essaie de trouver le plus long mot possible avec ces lettres. Il peut y avoir plusieurs fois la même lettre dans la série.

Exemple de partie

```
entrez votre tirage : urtefaaijg
10 lettres
Le script a mis 0.184 s pour trouver des mots de 9 lettres
['fatiguera', 'jaugerait']
```



Exercice 7.7

Nous allons utiliser les ensembles dans l'implémentation du jeu « Le mot le plus long ». Mais avant cela, nous aurons besoin d'un lexique ordonné, que nous allons élaborer à partir du fichier texte « dico.txt », qui contient une liste de 323'467 mots classés par ordre alphabétique.

Utilisez « dico.txt » pour faire un dictionnaire (au sens Python du terme, voir chapitre 6), où la clé sera la longueur du mot, et où la valeur sera la liste alphabétique des mots de cette longueur. Si des mots contiennent un ou des tirets, il faudra supprimer les tirets ! Ainsi, le mot « porte-avions » deviendra « porteavions ». Idem avec les apostrophes.

7.9. Code du programme



motlepluslong.py

<http://ow.ly/KEyn2>

```
# Le mot le plus long
from time import time

def dictionnaire_ordonne():
    # on lit le fichier et on range les mots alphabétiquement selon leur longueur
    fichier = open("dico.txt", "r")
    dict_ord = {}
    for longueur in range(26):
        dict_ord[longueur+1] = []
    mot = fichier.readline()
    while mot != '':
        mot = mot.strip('\n')
        if '-' in mot:
            mot = mot.replace('-', '')
        if "'" in mot:
            mot = mot.replace("'", '')
        dict_ord[len(mot)].append(mot)
        mot = fichier.readline()
    fichier.close()
    return dict_ord

def lettres_multiples_ok(mot, tirage):
    # teste si chaque lettre figure suffisamment de fois dans le tirage
    for lettre in mot:
        if lettre in tirage:
            tirage.remove(lettre)
        else:
            return False
    return True

def trouver_plus_long_mot(dico, tirage):
    longueur_mot = len(tirage)
```

```

solution= []
set tirage = set(tirage)
while longueur_mot > 0:
    for mot in dico[longueur_mot]:
        if set(mot).issubset(set_tirage):
            # les lettres du mot sont un sous-ensemble du tirage
            tirage_test = list(tirage)
            if lettres_multiples_ok(mot, tirage_test):
                solution.append(mot)
    if solution != [] or longueur_mot==1:
        return solution, longueur_mot
    longueur_mot -= 1

dico = dictionnaire_ordonne()
jouer = True
while jouer:
    tirage = input('Entrez votre tirage : ').lower()
    print(len(tirage), 'lettres')
    t0 = time()
    solution, longueur = trouver_plus_long_mot(dico, tirage)
    if solution == []:
        print('Pas de mot trouvé !')
    else:
        t1 = time()-t0
        print('Le script a mis', '{0:.3f}'.format(t1),
              's pour trouver des mots de', longueur, 'lettres')
        print(solution)
    rejouer = input('Rejouer ? (o/n) : ')
    jouer = (rejouer == "o")

```

7.10. Analyse du programme

```
from time import time
```

C'est dans le module `time` que se trouve la fonction... `time()`, qui renvoie un nombre réel correspondant au nombre de secondes écoulées depuis le 1^{er} janvier 1970 à 00:00:00. Nous allons utiliser cette fonction pour chronométrer la durée de la recherche du mot le plus long ; nous verrons comment un peu plus loin.

```

def dictionnaire_ordonne():
    # on lit le fichier et on range les mots alphabétiquement selon leur longueur
    fichier = open("dico.txt", "r")
    dict_ord = {}
    for longueur in range(25):
        dict_ord[longueur+1] = []

```

La fonction `dictionnaire_ordonne` était l'objet de l'exercice 7.5. Le dictionnaire est « découpé » en 25 listes, la liste k contenant tous les mots de k lettres classés alphabétiquement, avec k allant de 1 à 25. Rappelons que le plus long mot de la langue française comporte 25 lettres : « anticonstitutionnellement ».

```

mot = fichier.readline()
while mot != '':
    mot = mot.strip('\n')
    if '-' in mot:
        mot = mot.replace('-', '')
    if '"' in mot:
        mot = mot.replace('"', '')
    dict_ord[len(mot)].append(mot)
    mot = fichier.readline()

```

On lit tous les mots du fichier, ligne par ligne (il y a un mot par ligne dans le fichier) ; on supprime le caractère saut de ligne :

```
mot = mot.strip('\n')
```

et les tirets qui se trouveraient dans le mot (idem pour les apostrophes)

```
mot = mot.replace('-', '')
```

puis on ajoute à la liste correspondante ce mot.

```
dict_ord[len(mot)].append(mot)
```

Le fichier est finalement fermé et le dictionnaire ordonné est retourné.

```
fichier.close()
return dict_ord
```

La fonction `lettres_multiples_ok()` teste si chaque lettre figure suffisamment de fois dans le tirage. L'idée est simple : les lettres du mot sont enlevées une à une de la liste tirage. Si la lettre courante du mot n'est pas dans la liste tirage, le mot ne peut pas être écrit et la fonction renvoie `False`.

```
def lettres_multiples_ok(mot, tirage):
    # teste si chaque lettre figure suffisamment de fois dans le tirage
    for lettre in mot:
        if lettre in tirage:
            tirage.remove(lettre)
        else:
            return False
    return True
```

La fonction `trouver_plus_long_mot()` parcourt le dictionnaire ordonné, en commençant par la liste des plus longs mots possibles.

```
def trouver_plus_long_mot(dico, tirage):
    longueur_mot = len(tirage)
    solution = []
    set_tirage = set(tirage)
    while longueur_mot > 0:
        for mot in dico[longueur_mot]:
```

La ligne qui suit fait un prétraitement : avant de voir si les lettres sont en suffisance, on regarde d'abord si toutes les lettres du mot sont dans le tirage.

```
        if set(mot).issubset(set_tirage):
            # les lettres du mot sont un sous-ensemble du tirage
            tirage_test = list(tirage)
            if lettres_multiples_ok(mot, tirage_test):
                solution.append(mot)
```

Si une solution a été trouvée, on la retourne. Sinon, on essaie avec la liste des mots ayant une lettre de moins.

```
        if solution != [] or longueur_mot == 1:
            return solution, longueur_mot
        longueur_mot -= 1
```

Le programme principal ne présente pas de difficultés.

```
dico = dictionnaire_ordonne()
jouer = True
while jouer:
    tirage = input('Entrez votre tirage : ').lower()
    print(len(tirage), 'lettres')
    t0 = time()
    solution, longueur = trouver_plus_long_mot(dico, tirage)
    if solution == []:
        print('Pas de mot trouvé !')
    else:
        t1 = time() - t0
```

```
print('Le script a mis', '{0:.3f}'.format(t1),
      's pour trouver des mots de', longueur, 'lettres')
print(solution)
rejouer = input('Rejouer ? (o/n) : ')
jouer = (rejouer == "o")
```

Pour chronométrer le temps de calcul, on enregistre « l'heure » dans la variable `t0` avant de lancer la recherche du mot le plus long, puis, juste après la recherche, on enregistre la différence entre « la nouvelle heure » et `t0`.

```
t1 = time() - t0
```



Exercice 7.8

Est-il possible de se passer des ensembles dans le programme du § 7.9 ? Essayez de supprimer du programme la ligne :

```
if set(mot).issubset(set_tirage)
```

ainsi que les lignes ayant un rapport avec ce prétraitement.

Votre programme est-il plus rapide ? Pour le savoir, testez-le avec la fonction `time()`.



Exercice 7.9

Modifiez le programme du § 7.9 pour qu'il écrive toutes les anagrammes des mots d'une longueur donnée par l'utilisateur.

Par exemple :

```
Longueur des mots : 5
abats : ['basat', 'batas']
abeti : ['batie', 'beait']
abime : ['amibe', 'iambe']
...
zoner : ['ornez']
```



Exercice 7.10

Au Scrabble (version française), les valeurs des lettres sont les suivantes :

- | | |
|--------------------------------|-----------|
| • A, E, I, L, N, O, R, S, T, U | 1 point |
| • D, G, M | 2 points |
| • B, C, P | 3 points |
| • F, H, V | 4 points |
| • J, Q | 8 points |
| • K, W, X, Y, Z | 10 points |



Modifiez le programme du § 7.9 pour qu'il écrive les mots qui rapporteraient le plus de points au Scrabble, en utilisant les lettres données par l'utilisateur.

Par exemple :

```
Entrez votre tirage : gijquretsa
['gastrique', 'tragiques', 'jaquiers'] : 22 points au Scrabble
```



Exercice 7.11

Deux joueurs vont s'affronter en lançant plusieurs fois de suite une pièce de monnaie. Avant la partie, chacun des joueurs choisit une séquence de trois résultats (chaque résultat étant Pile ou Face). Puis, on lance la pièce de monnaie autant de fois qu'il le faut (en notant à chaque fois le résultat) jusqu'à ce qu'une des deux séquences apparaisse.

Par exemple, avant de commencer à lancer la pièce, le joueur 1 choisit la séquence *PFP* (Pile-Face-Pile) et le joueur 2 choisit la séquence *FPP*. On lance ensuite la pièce plusieurs fois de suite et voici les issues obtenues successivement :

F P F F P F F F P P

Cette partie a duré 10 lancers et la première des deux séquences sortie est celle du joueur 2, qui est alors déclaré vainqueur.

A priori, les deux joueurs devraient avoir la même chance de gagner... en tout cas, c'est ce que l'intuition nous laisse penser. Ce n'est pourtant pas le cas. Si vous connaissez la séquence choisie par l'adversaire, vous pourrez définir la vôtre et gagner plus souvent que votre adversaire.

En simulant des milliers de parties, trouvez la meilleure séquence du joueur 2 à opposer à chacune des 8 séquences possibles du joueur 1.

Déterminez ensuite la stratégie gagnante.



Exercice 7.12



Le jeu « Motus » repose sur la recherche de mots d'un nombre fixé de lettres (entre 7 et 10). Un candidat propose un mot qui doit contenir le bon nombre de lettres et être correctement orthographié, sinon il est refusé. Le mot apparaît alors sur une grille : les lettres bien placées sont colorées en rouge, les lettres présentes mais mal placées sont en jaune.

C	A	S	T	O	R
C	I	N	E	M	A
C	Y	P	R	E	S
C	I	T	R	O	N

Une lettre ne peut être colorée au maximum que le nombre de fois que cette lettre apparaît dans le mot. Par exemple, si le mot cherché est CABLES et que l'on propose CABANE, on aura la coloration :

C	A	B	A	N	E
---	---	---	---	---	---

et non pas

C	A	B	A	N	E
---	---	---	---	---	---

car il n'y a qu'un « A » dans CABLES.

Mots non valables

- les noms propres ;
- les mots mal orthographiés ;
- les mots composés (chewing-gum, week-end, ...) ;
- les verbes conjugués (seuls les infinitifs et les participes passés et présents sont acceptés) ;
- les mots qui ne commencent pas par la première lettre indiquée ;
- les mots qui ne respectent pas le nombre de lettres donné.

Votre programme devra :

- prendre un mot au hasard dans le fichier « dico.txt ». Ce mot devra avoir entre 7 et 10 lettres. Contrairement aux règles originales, on permettra les formes conjuguées ;
- indiquer le nombre de lettres du mot à trouver et donner la première lettre ;
- écrire le numéro de l'essai ;
- écrire sous la proposition du joueur :
 - une lettre majuscule si elle est bien placée ;
 - une lettre minuscule si elle est mal placée ;
 - un point si la lettre n'est pas dans le mot, ou si elle a moins d'occurrences ;
 - un message d'erreur si le mot est non valide.

Exemples de partie

Vous cherchez un mot de 7 lettres. Vous avez 6 essais.

```
O.....
1 orageux
OrA.e..
2 opaline
O.A.I.e
3 otaries
OTARIES
Bravo !
```

Vous cherchez un mot de 8 lettres. Vous avez 6 essais.

```
F.....
1 fouinant
FOUi.a.t
2 febriles
Fe..i..s
3 fouettes
FOUet..s
4 foutasse
Pas dans le dictionnaire
5 foutaise
FOUTAISE
Bravo !
```

Vous cherchez un mot de 7 lettres. Vous avez 6 essais.

```
T.....
1 trousse
T.o.s..
2 passeur
Première lettre non valide
3 traceur
T.aC...
4 tamisees
Longueur non valide
5 tapisse
TA..s..
6 tapoter
TA.o...
Désolé, la réponse était TANCONS
```



7.11. Le défi Turing

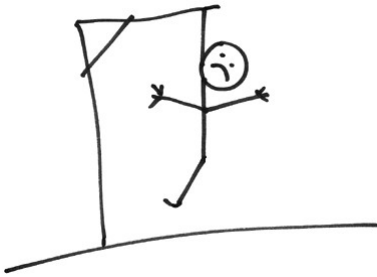
Avec vos connaissances actuelles en Python, vous pourrez résoudre plus ou moins facilement presque tous les exercices du défi Turing : www.nymphomath.ch/turing

Le défi Turing est une série d'énigmes mathématiques qui pourront difficilement être résolues sans un programme informatique. **Attention !** Votre programme devra trouver la réponse **en moins d'une minute !**



7.12. Ce que vous avez appris dans ce chapitre

- Vous avez vu comment stocker et lire des données dans un fichier texte.
- Une chaîne de caractères (*string*) est une suite ordonnée de caractères (lettres, chiffres, symboles). Elle n'est pas modifiable, mais on peut quand même la manipuler grâce aux nombreuses méthodes de la classe 'str'.
- Vous avez vu que l'on peut manipuler des ensembles (*set*) avec les mêmes opérations qu'en mathématiques. Les ensembles sont parfois pratiques, par exemple pour éliminer des doublons.



Chapitre 8

Le pendu

8.1. Nouveaux thèmes abordés dans ce chapitre

- code ASCII
- tkinter : canvas, pack, place, create_text, delete

8.2. Règles du jeu

Voir § 7.4.

8.3. Code du programme



pendu-
graphique.py

<http://ow.ly/HmGfN>

```
# Le jeu du pendu

from tkinter import *
from random import choice

fichier = open("liste_mots.txt", "r")
liste_mots = fichier.readlines() # met tous les mots du fichier dans une liste
fichier.close()

def lettre_dans_mot(lettre) :
    global partie_en_cours, mot_partiel, mot_choisi, nb_echecs, image_pendu
    if partie_en_cours :
        nouveau_mot_partiel = ""
        lettre_dans_mot = False
        i=0
        while i<len(mot_choisi):
            if mot_choisi[i]==lettre:
                nouveau_mot_partiel = nouveau_mot_partiel + lettre
                lettre_dans_mot = True
            else:
                nouveau_mot_partiel = nouveau_mot_partiel + mot_partiel[i]
            i+=1
        mot_partiel = nouveau_mot_partiel
        afficher_mot(mot_partiel)
    if not lettre_dans_mot : # lettre fausse. Changer le dessin.
        nb_echecs += 1
        nomFichier = "pendu_"+str(nb_echecs)+".gif"
        photo=PhotoImage(file=nomFichier)
        image_pendu.config(image=photo)
        image_pendu.image=photo
```

```
        if nb_echecs == 7: # trop d'erreurs. Fini.
            partie_en_cours = False
            afficher_mot(mot_choisi)
        elif mot_partiel == mot_choisi: # le mot a été trouvé !
            partie_en_cours = False

def afficher_mot(mot):
    global lettres
    mot_large = ""
    i=0
    while i<len(mot): # ajoute un espace entre les lettres
        mot_large = mot_large + mot[i] + " "
        i+=1
    canvas.delete(lettres)
    lettres = canvas.create_text(320,60,text=mot_large,fill='black',font='Courier 30')

def init_jeu():
    global mot_choisi, mot_partiel, image_pendu, lettres, nb_echecs
    global partie_en_cours, liste_mots
    nb_echecs = 0
    partie_en_cours = True
    mot_choisi = choice(liste_mots).rstrip()
    mot_choisi = mot_choisi.upper()
    mot_partiel = "-" * len(mot_choisi)
    canvas.delete(lettres)
    afficher_mot(mot_partiel)
    photo=PhotoImage(file="pendu_0.gif")
    image_pendu.config(image=photo)
    image_pendu.image=photo

# création du widget principal

fenetre = Tk()
fenetre.title("Le jeu du pendu")

canvas = Canvas(fenetre, bg='white', height=500, width=620)
canvas.pack(side=BOTTOM)

bouton = [0]*26
for i in range(26):
    bouton[i] = Button(fenetre,text=chr(i+65),command=lambda x=i+65:lettre_dans_mot(chr(x)))
    bouton[i].pack(side=LEFT)

bouton2 = Button(fenetre,text='Quitter',command=fenetre.quit)
bouton2.pack(side=RIGHT)
bouton1 = Button(fenetre,text='Recommencer',command=init_jeu)
bouton1.pack(side=RIGHT)

photo=PhotoImage(file="pendu_0.gif")
image_pendu = Label(canvas, image=photo, border=0)
image_pendu.place(x=120, y=140)
lettres = canvas.create_text(320,60,text="",fill='black',font='Courier 30')

init_jeu()

fenetre.mainloop()
fenetre.destroy()
```

8.4. Analyse du programme

Le programme est divisé en cinq parties.

Tout d'abord, on charge les mots lus dans un fichier externe dans la liste `liste_mots`, comme dans la version non graphique (voir chapitre 7).

```
fichier = open("liste_mots.txt", "r")
liste_mots = fichier.readlines() # met tous les mots du fichier dans une liste
fichier.close()
```

La procédure `lettre_dans_mot()` se charge de vérifier que la lettre proposée figure dans le mot choisi. Si c'est le cas, la lettre sera placée au bon endroit dans le mot qui sera affiché.

```
def lettre_dans_mot(lettre) :
    global partie_en_cours, mot_partiel, mot_choisi, nb_echecs, image_pendu
    if partie_en_cours :
        nouveau_mot_partiel = ""
        lettre_dans_mot = False
        i=0
        while i<len(mot_choisi):
            if mot_choisi[i]==lettre:
                nouveau_mot_partiel = nouveau_mot_partiel + lettre
                lettre_dans_mot = True
            else:
                nouveau_mot_partiel = nouveau_mot_partiel + mot_partiel[i]
            i+=1
        mot_partiel = nouveau_mot_partiel
        afficher_mot(mot_partiel)
```

Dans le cas contraire, le dessin du pendu sera continué. En fait, l'image du pendu sera remplacée par une autre plus complète.

```
if not lettre_dans_mot : # lettre fausse. Changer le dessin.
    nb_echecs += 1
    nomFichier = "pendu_"+str(nb_echecs)+".gif"
    photo=PhotoImage(file=nomFichier)
    image_pendu.config(image=photo)
    image_pendu.image=photo
```

Si le dernier dessin est affiché, la partie s'arrête et le mot complet est affiché.

```
if nb_echecs == 7: # trop d'erreurs. Fini.
    partie_en_cours = False
    afficher_mot(mot_choisi)
elif mot_partiel == mot_choisi: # le mot a été trouvé !
    partie_en_cours = False
```

Comme son nom l'indique, la procédure `afficher_mot()` écrit sur l'écran un mot partiel composé de tirets et des lettres qui ont été trouvées.

```
def afficher_mot(mot) :
    global lettres
    mot_large = ""
    i=0
    while i<len(mot): # ajoute un espace entre les lettres
        mot_large = mot_large + mot[i] + " "
        i+=1
    canvas.delete(lettres)
    lettres = canvas.create_text(320,60,text=mot_large,fill='black',font='Courier 30')
```

Nous reviendrons aux paragraphes 8.4.1 et 8.4.2 sur la dernière ligne de cette procédure. Nous verrons ce que signifie le `chr` et le `lambda`.

Remarquons que l'ancien mot partiel, qui a comme nom de variable `lettres`, est effacé par l'instruction `canvas.delete(lettres)`, avant que le nouveau soit réécrit au même endroit par l'instruction `canvas.create_text()`.

La procédure `init_jeu()` initialise toutes les variables globales avant de (re)commencer une partie, tire au hasard un nouveau mot dans la liste et écrit des tirets correspondant au nombre de lettres, et enfin affiche le premier dessin du pendu, qui se nomme `pendu_0.gif`.

```
def init_jeu() :
    global mot_choisi, mot_partiel, image_pendu, lettres, nb_echecs
    global partie_en_cours, liste_mots
    nb_echecs = 0
    partie_en_cours = True
    mot_choisi = choice(liste_mots).rstrip()
```

```
mot_choisi = mot_choisi.upper()
mot_partiel = "-" * len(mot_choisi)
afficher_mot(mot_partiel)
photo=PhotoImage(file="pendu_0.gif")
image_pendu.config(image=photo)
image_pendu.image=photo
```

Dans la dernière partie, on met en place les différentes composantes de l'interface graphique.

La fenêtre

La première ligne crée la fenêtre, la seconde lui donne un titre. Notez que l'on ne donne pas les dimensions de la fenêtre : elle est « élastique » et s'adaptera automatiquement au contenu.

```
fenetre = Tk()
fenetre.title("Le jeu du pendu")
```

Le canevas

Un canevas est une surface réservée aux éléments graphiques. On indique dans quelle fenêtre il devra se trouver, sa couleur de fond (blanc), et ses dimensions (500 pixels de haut et 620 de large). La seconde ligne indique que ce canevas sera placé en bas de la fenêtre. La fonction `pack` sera expliquée plus en détail au § 8.4.3.

```
canevas = Canvas(fenetre, bg='white', height=500, width=620)
canevas.pack(side=BOTTOM)
```

Les boutons

Les 26 boutons lettres seront placés dans la fenêtre, au-dessus du canevas (puisque l'on a dit que le canevas occupe le bas de la fenêtre) et tassés à gauche (la fonction `chr` sera expliquée au § 8.4.1),

```
bouton = [0]*26
for i in range(26):
    bouton[i] = Button(fenetre, text=chr(i+65), command=lambda x=i+65: lettre_dans_mot(chr(x)))
    bouton[i].pack(side=LEFT)
```

un bouton *Quitter*, tassé à droite

```
bouton2 = Button(fenetre, text='Quitter', command=fenetre.quit)
bouton2.pack(side=RIGHT)
```

et un bouton *Recommencer*, tassé à droite

```
bouton1 = Button(fenetre, text='Recommencer', command=init_jeu)
bouton1.pack(side=RIGHT)
```

Notez que le bouton le plus à droite (*Quitter*) doit être écrit en premier.

Le dessin du pendu

Le dessin du pendu, qui est un label, est placé dans le canevas aux coordonnées (120,140). La fonction `place` sera expliquée plus en détail au § 8.4.4.

```
photo=PhotoImage(file="pendu_0.gif")
image_pendu = Label(canevas, image=photo, border=0)
image_pendu.place(x=120, y=140)
```

Le mot à découvrir

Le mot à découvrir est lui aussi placé dans le canevas, aux coordonnées (320, 60). Il sera écrit en noir, avec la police « Courier 30 ».

```
lettres = canevas.create_text(320, 60, text="", fill='black', font='Courier 30')
```

Remarquez bien que le dessin du pendu et le mot à découvrir sont placés dans le canevas,

contrairement aux boutons.

On prononce
phonétiquement
« aski »

8.4.1. Le code ASCII

Le code ASCII (American Standard Code for Information Interchange) est une norme d'encodage informatique des caractères alphanumériques de l'alphabet latin. La norme **ASCII** établit une correspondance entre une représentation numérique des caractères de l'alphabet latin ainsi que les symboles et les signes. Par exemple, le caractère « A » est associé à 65 et « a » à 97. Les nombres du code ASCII vont de 0 à 127.

Le codage **ASCII** (voir tableau ci-dessous) est souvent complété par des correspondances supplémentaires afin de permettre l'encodage informatique d'autres caractères, comme les caractères accentués par exemple. Cette norme s'appelle **ISO-8859** et se décline par exemple en **ISO-8859-1** lorsqu'elle étend l'**ASCII** avec les caractères accentués d'Europe occidentale.

La colonne jaune
est le code ASCII
en base 16, aussi
appelée base
hexadécimale.
Elle ne nous sert à
rien pour l'instant.

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	Null	32	20	Space	64	40	@	96	60	`
1	01	Start of heading	33	21	!	65	41	A	97	61	a
2	02	Start of text	34	22	"	66	42	B	98	62	b
3	03	End of text	35	23	#	67	43	C	99	63	c
4	04	End of transmit	36	24	\$	68	44	D	100	64	d
5	05	Enquiry	37	25	%	69	45	E	101	65	e
6	06	Acknowledge	38	26	&	70	46	F	102	66	f
7	07	Audible bell	39	27	'	71	47	G	103	67	g
8	08	Backspace	40	28	(	72	48	H	104	68	h
9	09	Horizontal tab	41	29	)	73	49	I	105	69	i
10	0A	Line feed	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage return	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	47	2F	/	79	4F	O	111	6F	o
16	10	Data link escape	48	30	0	80	50	P	112	70	p
17	11	Device control 1	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	50	32	2	82	52	R	114	72	r
19	13	Device control 3	51	33	3	83	53	S	115	73	s
20	14	Device control 4	52	34	4	84	54	T	116	74	t
21	15	Neg. acknowledge	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	54	36	6	86	56	V	118	76	v
23	17	End trans. block	55	37	7	87	57	W	119	77	w
24	18	Cancel	56	38	8	88	58	X	120	78	x
25	19	End of medium	57	39	9	89	59	Y	121	79	y
26	1A	Substitution	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	60	3C	<	92	5C	\	124	7C	
29	1D	Group separator	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	63	3F	?	95	5F	_	127	7F	□

En Python, la fonction qui donne de code ASCII d'un caractère s'appelle `ord`. Ainsi,

```
ord('A')
```

donnera comme résultat 65.

Inversement, pour écrire le caractère correspondant à un code ASCII, on utilisera la fonction `chr` :

```
chr(65)
```

donnera comme résultat 'A'. On comprend maintenant mieux notre ligne mystérieuse :

```
bouton[i] = Button(fenetre, text=chr(i+65), command=lambda x=i+65: lettre_dans_mot(chr(x)))
```

Sur le bouton 0 sera écrit la lettre 'A' (`chr(65)`), sur le bouton 1 la lettre 'B' (`chr(66)`), etc.



Exercice 8.1

Écrivez une fonction qui construit un dictionnaire ayant comme clé un symbole (lettre, chiffres, etc.) et comme valeur son code ASCII.



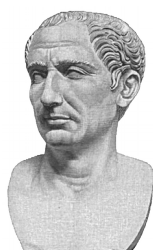
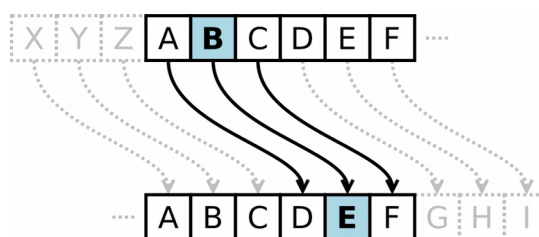
Exercice 8.2

La fonction `upper()` permet d'écrire une phrase avec uniquement des majuscules. Programmez cette fonction, que vous nommerez `majuscules(texte)`, après avoir remarqué que chaque lettre majuscule est éloignée de 32 rangs de sa correspondante minuscule. Les autres caractères (espace, caractères de ponctuation, ...) resteront inchangés. On supposera que le texte est en anglais, et donc non accentué.



Exercice 8.3 : chiffre de César

Le *chiffre de César* est une méthode de chiffrement consistant à décaler les lettres de l'alphabet de quelques crans vers la gauche. Par exemple, si on décale les lettres de 3 rangs (comme le faisait Jules César), on obtient la table de chiffrement suivante :



En utilisant cette table, le message *Ave Caesar morituri te salutant* devient DYH FDHVDU PRULWXUL WH VDOXWDQW.

Cette méthode a été utilisée par **Jules César** dans ses correspondances secrètes (ce qui explique le nom « chiffre de César »).

Écrivez un programme Python qui implémente le chiffre de César. Utilisez pour cela les fonctions `ord()` et `chr()`. On doit pouvoir choisir le décalage des lettres entre 1 et 25. L'entrée sera un texte quelconque sans accents et sans ponctuation ; par contre, le texte d'entrée sera composé de majuscules et de minuscules. Le texte de sortie sera écrit tout en majuscules. Les espaces du texte d'entrée se retrouveront aux mêmes endroits dans le texte chiffré.

Faites en sorte que l'on puisse crypter, mais aussi décrypter un message.



Exercice 8.4 : chiffre de Vigenère

Le *chiffre de Vigenère* est une amélioration décisive du chiffre de César. Sa force réside dans l'utilisation non pas d'un, mais de 26 alphabets décalés pour chiffrer un message. Ce chiffre utilise une clé qui définit le décalage pour chaque lettre du message (A: décalage de 0 cran, B: 1 cran, C: 2 crans, ..., Z: 25 crans).

Exemple: chiffrons le texte « CHIFFRE DE VIGENERE » avec la clef « BACHELIER » (cette clé est éventuellement répétée plusieurs fois pour être aussi longue que le texte clair).

Clair	c	h	i	f	f	r	e	d	e	v	i	g	e	n	e	r	e
Clé	B	A	C	H	E	L	I	E	R	B	A	C	H	E	L	I	E
Décalage	1	0	2	7	4	11	8	4	17	1	0	2	7	4	11	8	4
Chiffré	D	H	K	M	J	C	M	H	V	W	I	I	L	R	P	Z	I



Blaise de Vigenère
(1523-1596)

Modifiez le programme de l'exercice 8.3 pour crypter et décrypter des messages avec le chiffre de Vigenère.

Nous avons déjà vu les fonctions lambda au § 6.4

8.4.2. Les fonctions lambda

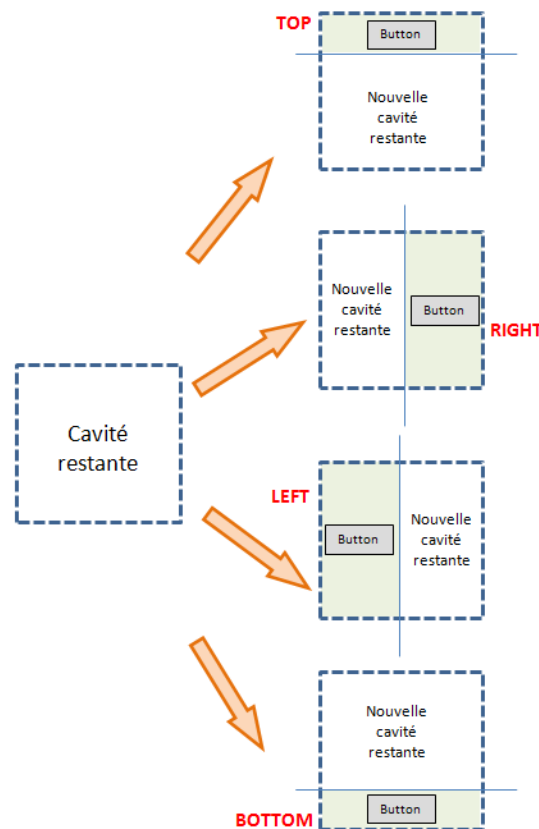
Par défaut, le paramètre `command` d'un `Button` nécessite une fonction **sans argument** (voir programme du chapitre 6). Vous pouvez cependant utiliser les fonctions `lambda` pour passer des arguments supplémentaires, ce qui peut être utile si vous avez plusieurs `Button` qui pointent sur la même fonction.

8.4.3. L'instruction pack de tkinter

Le module `tkinter` contient trois fonctions permettant de positionner les éléments d'une interface graphique : `grid` (que nous avons déjà vue au chapitre 4), `pack` et `place`. (voir § 8.4.4).

Le terme **widget** désigne toute entité susceptible d'être placée dans une fenêtre (un bouton, une image, etc)

Lorsqu'on fait appel à `pack()`, `tkinter` réduit la fenêtre pour épouser au mieux les éléments qu'elle contient. Chaque ajout d'un **widget** se fera de l'une des quatre manières ci-dessous, selon le côté choisi (`LEFT`, `RIGHT`, `TOP` ou `BOTTOM`).



Nous utiliserons le terme de « cavité » pour désigner l'espace qui sera prochainement occupé par un widget. C'est Calogero Gigante qui a inventé ce terme (www.gigante.be) et fait le schéma ci-dessus.

On divise la cavité restante en deux, pour créer une nouvelle cavité restante, à laquelle on appliquera le même schéma lors de l'ajout d'un nouveau widget.

Dans notre programme, on a d'abord placé un canevas en bas de la fenêtre. La cavité restante est donc la partie supérieure de la fenêtre. Ensuite, on a ajouté l'un après l'autre les 26 boutons lettres à gauche ; la cavité restante était donc la partie supérieure droite de la fenêtre. Puis on a ajouté à droite le bouton *Quitter*. La cavité restante se trouve donc toujours dans la partie supérieure de la fenêtre, coincée entre le bouton *Z* et le bouton *Quitter*. C'est donc là que sera ajouté le dernier bouton *Recommencer*. À noter qu'il reste une cavité entre *Z* et *Recommencer*.

8.4.4. L'instruction `place` de `tkinter`

La commande `place` permet de placer un widget à la position (x,y) . Pour être précis, c'est le coin supérieur gauche du widget qui est positionné en (x,y) .

```
image_pendu.place(x=120, y=140)
```



Attention ! Les coordonnées $(0,0)$ se trouvent **en haut à gauche** du canevas (et non en bas à gauche comme on pourrait s'y attendre).

Il est possible que l'image sorte du canevas : cela ne provoquera pas d'erreur, mais votre image sera tronquée.



Exercice 8.5

Modifiez le code du § 8.3.

Quand un bouton lettre aura été pressé, la lettre y figurant disparaîtra. Cela permettra de savoir quelles lettres ont déjà été proposées.

Ajoutez aussi un message de félicitation qui apparaîtra quand le mot aura été trouvé.



Exercice 8.6



Créez une interface graphique pour le jeu de l'exercice 7.11.



8.5. Ce que vous avez appris dans ce chapitre

- Les caractères et autres symboles sont codés grâce au code ASCII.
- Un canevas est une surface réservée aux éléments graphiques.
- Il existe trois manières de positionner les éléments d'une interface graphique : `grid`, `pack` et `place`.
- Une chaîne de caractères peut être transformée en un élément graphique avec la fonction `create_text()`.



Chapitre 9

Memory

9.1. Nouveaux thèmes abordés dans ce chapitre

- Menus
- tkinter : find_closest(), frame

9.2. Memory

Memory est un jeu de société basé sur la mémoire édité pour la première fois par *Ravensburger* en 1959. Il se joue avec un certain nombre de paires de cartes identiques. Les cartes sont toutes retournées sur le plateau. Ensuite, à son tour, chaque joueur retourne deux cartes de son choix. Si les cartes sont identiques, il les ramasse et rejoue. Sinon, les cartes sont retournées face cachée à leur emplacement initial. Le jeu se termine quand toutes les cartes ont été ramassées et le gagnant est celui qui a ramassé le plus de cartes.

9.3. Code du programme



memory.py

<http://ow.ly/jtKrs>

```
# Memory pour 2 joueurs humains

from tkinter import *
from random import randint, shuffle

# ----- variables globales -----
images = []          # contient les liens aux fichiers images
cartes = []          # contient le lien vers l'image des différentes cartes
cartes_jouees = []   # contient les cartes jouées
nb_lignes, nb_colonnes = 5, 4
joueur_actuel = 0
score = [0,0]
fini = False
peut_jouer = True

# ----- Images -----
def charger_images():
    del images[:]      # vide la liste
    nb_images = 21     # l'image no 0 est le dos des cartes
    choixCartes = []
```

```

choixCartes.append(0)
i=0
while i < nb_images-1:           # tirage au sort des cartes à utiliser
    x = randint(1, nb_images-1)
    if x not in choixCartes:
        choixCartes.append(x)
        i += 1
for i in range(nb_images):       # importation des images
    nom = 'carte-' + str(choixCartes[i]) + '.gif'
    image = PhotoImage(file = nom)
    images.append(image)

# ----- Melange des cartes -----
def melanger_cartes():
    global nb_colonnes, nb_lignes, cartes
    nb_cartes = nb_colonnes * nb_lignes
    cartes=list(range(1,nb_cartes//2+1))*2
    shuffle(cartes)

# ----- Retourne les deux cartes à la fin de la sélection -----
def gerer_tirage():
    global nb_colonnes, nb_lignes, cartes_jouees
    global joueur_actuel, fini, peut_jouer
    if cartes[cartes_jouees[0]-1] == cartes[cartes_jouees[1]-1]:
        # enlève les cartes identiques. Le joueur actuel reste le même
        canvas.delete(cartes_jouees[0])
        canvas.delete(cartes_jouees[1])
        score[joueur_actuel] += 1
    else:
        # retourne les cartes différentes. Le joueur actuel change
        canvas.itemconfig(cartes_jouees[0], image = images[0])
        canvas.itemconfig(cartes_jouees[1], image = images[0])
        joueur_actuel = (joueur_actuel+1)%2      # la main passe à l'autre joueur
    cartes_jouees = []
    text1 = 'Joueur 1 : ' + str(score[0]*2)
    text2 = 'Joueur 2 : ' + str(score[1]*2)
    points_joueur1.config(text = text1)
    points_joueur2.config(text = text2)
    peut_jouer = True      # réactive l'effet du clic de la souris
    if joueur_actuel == 0:  # celui qui joue est en orange
        points_joueur1.config(bg = 'orange')
        points_joueur2.config(bg = 'white')
    else:
        points_joueur2.config(bg = 'orange')
        points_joueur1.config(bg = 'white')
    if score[0] + score[1] == (nb_colonnes*nb_lignes)//2:
        fini = True      # afficher le résultat de la partie
        if score[0] > score[1]:
            texte = "Le joueur 1 a gagné !"
        elif score[0] < score[1]:
            texte = "Le joueur 2 a gagné !"
        else:
            texte = "Egalité !"
        canvas.create_rectangle(0,0,(110*nb_colonnes)+20,(110*nb_lignes)+20,
                                fill='white')
        canvas.create_text((55*nb_colonnes)+10,(55*nb_lignes)+10,
                            text=texte,font='Calibri 24',fill='black')

# ----- Retourne la carte sélectionnée -----
def cliquer_carte(event):
    global fini, plateau, cartes_jouees, peut_jouer
    if len(cartes_jouees) < 2:
        carteSel = canvas.find_closest(event.x, event.y)
        carteID = carteSel[0]
        if fini:
            fini = False
            reinit()
        else:
            canvas.itemconfig(carteID, image = images[cartes[carteID-1]])
            if len(cartes_jouees) == 0:
                cartes_jouees.append(carteID)      # enregistre la carte jouée
            elif carteID != cartes_jouees[0]:      # ne pas cliquer 2x sur la même carte

```

```

        cartes_jouees.append(carteID)
    if peut_jouer and len(cartes_jouees) == 2:
        peut_jouer = False # désactive l'effet du clic de la souris
        plateau.after(1500,gerer_tirage) # patiente 1,5 secondes

# ----- Change la taille du plateau de jeu -----
def jeu5x4():
    global nb_colonnes
    nb_colonnes = 4
    reinit()

def jeu5x6():
    global nb_colonnes
    nb_colonnes = 6
    reinit()

def jeu5x8():
    global nb_colonnes
    nb_colonnes = 8
    reinit()

# ----- création des menus et sous-menus -----
def creer_menus(fen):
    top = Menu(fen)
    fen.config(menu=top)

    jeu = Menu(top, tearoff=False)
    top.add_cascade(label='Jeu', menu=jeu)
    jeu.add_command(label='Nouvelle partie', command=reinit)

    submenu=Menu(jeu, tearoff=False)
    jeu.add_cascade(label='Dimensions', menu=submenu)
    submenu.add_command(label='5 x 4', command=jeu5x4)
    submenu.add_command(label='5 x 6', command=jeu5x6)
    submenu.add_command(label='5 x 8', command=jeu5x8)

    jeu.add_command(label='Quitter', command=fen.destroy)

# ----- Création du canvas -----
def creer_canevas(fen, col, lig):
    return Canvas(fen, width=(110*col)+10, height=(110*lig)+10, bg='white')

# ----- Modifier le canvas -----
# Redémarre une partie et change éventuellement la difficulté
def reinit():
    global canvas, joueur_actuel, score, nb_lignes, nb_colonnes
    joueur_actuel = 0
    score = [0,0]
    del cartes[:]
    del cartes_jouees[:]
    canvas.destroy()
    canvas = creer_canevas(plateau, nb_colonnes, nb_lignes)
    canvas.pack(side = TOP, padx = 5, pady = 5)
    canvas.bind("<Button-1>", cliquer_carte) # permet le clic sur les cartes
    melanger_cartes()
    for i in range(nb_colonnes):
        for j in range(nb_lignes):
            canvas.create_image((110*i)+60, (110*j)+60, image=images[0])
    text1 = 'Joueur 1 : ' + str(score[0]*2)
    text2 = 'Joueur 2 : ' + str(score[1]*2)
    points_joueur1.config(text = text1, bg = 'orange')
    points_joueur2.config(text = text2, bg = 'white')

# ----- Programme principal -----
fenetre = Tk()
fenetre.title("Memory")
creer_menus(fenetre)
# création du canvas dont la taille dépend du nombre de cartes
plateau = Frame(fenetre)

```

```

plateau.pack()
canvas=creer_canevas(plateau, nb_colonnes, nb_lignes)
canvas.pack(side = TOP, padx = 2, pady = 2)
points_joueur1 = Label(fenetre, text = "Joueur 1 : 0",
                        bg="orange", font="Helvetica 16")
points_joueur1.pack(pady = 2, side = LEFT)
points_joueur2 = Label(fenetre, text = "Joueur 2 : 0", font="Helvetica 16")
points_joueur2.pack(pady = 2, side = RIGHT)
charger_images()
melanger_cartes()
for i in range(nb_colonnes):
    for j in range(nb_lignes):
        canvas.create_image((110*i)+60, (110*j)+60, image = images[0])
canvas.bind("<Button-1>", cliquer_carte) # permet le clic sur les cartes
fenetre.mainloop()

```

9.4. Représentation des données

Voyons la manière dont on a représenté les cartes, les emplacements et les cartes choisies par les joueurs.

9.4.1. Les cartes et leur emplacement

Dans ce programme, il ne faut pas confondre une carte et l'image qui y est représentée. Les cartes peuvent être considérées comme les emplacements indiqués sur la figure de gauche ci-dessous.



À ces emplacements se trouvent les images (figure de droite ci-dessus).

Le jeu peut être représenté par une liste. Pour l'exemple ci-dessus, la liste serait :

`cartes = [5, 7, 7, 10, 4, 1, 3, 9, 2, 6, 5, 1, 8, 3, 9, 4, 10, 6, 8, 2].`

N'oubliez pas que les rangs d'une liste sont numérotés à partir de 0. Donc, pour savoir quelle image se cache sous la carte no 4, il faut afficher `cartes[3]`. Le résultat est 10.

9.4.2. Mélange des cartes

Pour mélanger les cartes on procède ainsi :

1. les images, numérotées de 1 à `nb_cartes/2` sont placées deux fois dans la liste `cartes`
2. on utilise la fonction `shuffle` du module `random` pour les mélanger.

Exemple de mélange

Avant : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Après : [5, 7, 7, 10, 4, 1, 3, 9, 2, 6, 5, 1, 8, 3, 9, 4, 10, 6, 8, 2]

```
# ----- Melange des cartes -----
def melanger_cartes():
    global nb_colonnes, nb_lignes, cartes
    nb_cartes = nb_colonnes * nb_lignes
    cartes=list(range(1,nb_cartes//2+1))*2
    shuffle(cartes)
```

9.4.3. Cartes jouées

Les cartes choisies alternativement par les joueurs (humain ou cybernétique) seront stockées dans la liste `cartes_jouees`, qui contiendra toujours 2 nombres entiers.

Ces nombres indiquent les emplacements des cartes. Pour connaître l'image de la première carte choisie, il faut afficher `cartes[cartes_jouees[0]]`.

9.5. Analyse du programme

Voyons maintenant plus en détails les nouveautés de ce programme.

9.5.1. Quelle carte a été cliquée ?

```
# ----- Retourne la carte sélectionnée -----
def cliquer_carte(event):
    global fini, plateau, cartes_jouees, peut_jouer
    if len(cartes_jouees) < 2:
        carteSel = canvas.find_closest(event.x, event.y)
        carteID = carteSel[0]
    ...
```

La méthode `find_closest()` renvoie le numéro de l'objet le plus proche des coordonnées de la souris au moment du clic. Le résultat sera un tuple. Par exemple, le résultat (2,) indiquera la carte no 2. L'identifiant de la carte sera donc le premier élément du tuple. Voilà l'explication de la ligne

```
carteID = carteSel[0]
```

Il est donc possible de cliquer à côté d'une carte. Ce sera la carte la plus proche qui sera retournée.

9.5.2. Menus

```
# ----- création des menus et sous-menus -----
def creer_menus(fen):
    top = Menu(fen)
    fen.config(menu=top)

    jeu = Menu(top, tearoff=False)
    top.add_cascade(label='Jeu', menu=jeu)
    jeu.add_command(label='Nouvelle partie', command=reinit)

    submenu=Menu(jeu, tearoff=False)
    jeu.add_cascade(label='Dimensions', menu=submenu)
    submenu.add_command(label='5 x 4', command=jeu5x4)
    submenu.add_command(label='5 x 6', command=jeu5x6)
    submenu.add_command(label='5 x 8', command=jeu5x8)

    jeu.add_command(label='Quitter', command=fen.destroy)
```

Voici à quoi ressemblera la barre de menus :



Le widget `Menu` permet de créer une barre de menus dans une fenêtre. Pour cela, vous devez tout d'abord définir un premier menu dans une fenêtre :

```
top = Menu(fen)
```

puis associer ce menu à la fenêtre :

```
fen.config(menu=top)
```

Pour ajouter des menus à la barre de menus, vous créez de nouveaux menus ayant pour parent le premier menu créé. Ils seront insérés dans la barre par la fonction `add_cascade()` :

```
jeu = Menu(top, tearoff=False)
top.add_cascade(label='Jeu', menu=jeu)
```

Le widget `Menu` admet un paramètre booléen `tearoff` qui indique si le menu en question peut être détaché de son parent ou non. Par défaut, `tearoff=True` et le menu est détachable. Un menu détachable est caractérisé par la présence d'une ligne pointillée en son début.

La fonction `add_cascade()` peut recevoir une liste de paramètres dont les principaux sont `label` pour indiquer le titre du menu fils et `menu` représentant l'identifiant du menu fils en question.

Pour ajouter des options à un menu fils, vous utilisez la fonction `add_command()` dont les principaux paramètres sont `label` pour le nom du sous-menu et `command` qui représente la fonction associée au clic sur l'option.

```
jeu.add_command(label='Nouvelle partie', command=reinit)
...
jeu.add_command(label='Quitter', command=fen.destroy)
```

Pour ajouter des sous-menus, on utilisera à nouveau la fonction `add_cascade()` :

```
submenu=Menu(jeu, tearoff=False)
jeu.add_cascade(label='Dimensions', menu=submenu)
submenu.add_command(label='5 x 4', command=jeu5x4)
submenu.add_command(label='5 x 6', command=jeu5x6)
submenu.add_command(label='5 x 8', command=jeu5x8)
```

9.5.3. Fenêtre, frame, canvas et label

```
# ----- Création du canvas -----
def creer_canevas(fen, col, lig):
    return Canvas(fen, width=(110*col)+10, height=(110*lig)+10, bg='white')

...

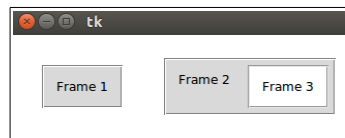
# ----- Programme principal -----
fenetre = Tk()
fenetre.title("Memory")
creer_menus(fenetre)
```


Ces lignes suivantes vont nous permettre de préciser certains éléments.

Un *frame* (cadre en français) est une surface rectangulaire dans la fenêtre, où l'on peut disposer d'autres widgets. C'est le conteneur par excellence. Cette surface peut être colorée et aussi décorée d'une bordure.

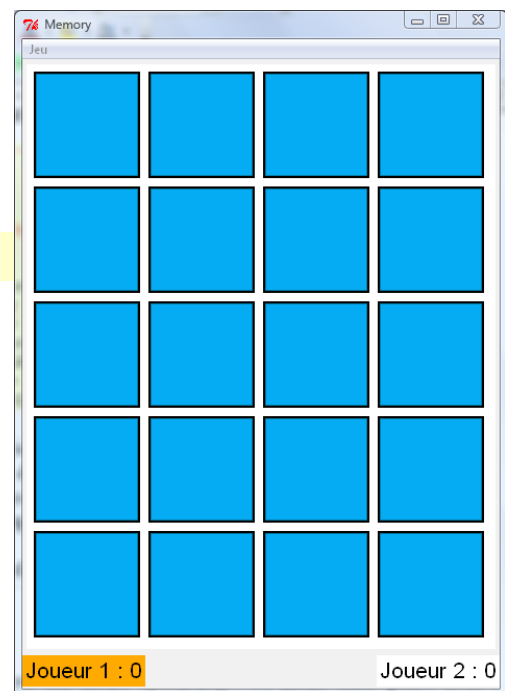
```
plateau = Frame(fenetre)
plateau.pack()
```

Il peut y avoir plusieurs *frames* dans une fenêtre, comme nous le verrons plus tard.



Le widget *canvas* (toile, tableau en français) est un espace pour disposer divers éléments graphiques. Ce widget peut être utilisé pour dessiner (voir chapitre 10).

```
canvas=creer_canevas(plateau, nb_colonnes,
nb_lignes)
canvas.pack(side = TOP, padx = 2, pady = 2)
```



On voit que le *canvas* a été créé dans le *frame* « plateau ». Il a été poussé en haut de la fenêtre (*side = TOP*). On lui a imposé une marge de 2 pixels en hauteur et en largeur.

Un *label* est un espace prévu pour un texte quelconque (ou éventuellement une image).

Puisque le haut de la fenêtre est occupé par le *canvas*, les textes indiquant les scores des joueurs sont automatiquement placés en bas, à gauche et à droite (*side = LEFT, side = RIGHT*).

```
points_joueur1 = Label(fenetre, text = "Joueur 1 : 0",
                        bg="orange", font="Helvetica 16")
points_joueur1.pack(pady = 2, side = LEFT)
points_joueur2 = Label(fenetre, text = "Joueur 2 : 0", font="Helvetica 16")
points_joueur2.pack(pady = 2, side = RIGHT)
```



Exercice 9.1

Ajoutez à droite du menu « Jeu » un nouveau menu « Joueurs ». Ce menu aura quatre sous-menus : « Joueur contre joueur », « Joueur contre ordinateur », « Ordinateur contre joueur » et « Ordinateur contre ordinateur ».

Ajoutez à droite du menu « Joueurs » un nouveau menu « Niveaux ». Ce menu aura trois sous-menus : « Facile », « Moyen », « Difficile ».

Dans un premier temps, tous ces sous-menus n'auront pas d'effet.



Exercice 9.2

Intégrez dans le programme du § 9.3 une section qui permettra de jouer contre l'ordinateur. Programmez au moins trois niveaux :

Au niveau « facile », l'ordinateur retourne des cartes au hasard, sans se souvenir des cartes qui ont déjà été retournées.

Au niveau « difficile », l'ordinateur se souvient de toutes les cartes retournées.

Au niveau « moyen », il se souvient des cartes, mais peut se tromper avec une certaine probabilité (à déterminer).

Quelques astuces pour déboguer votre programme



1. Commencez par programmer le plus simple. Par exemple, dans cet exercice, commencez par programmer le niveau « facile ».
2. Chaque fois que vous modifiez votre programme, testez-le. S'il y a un bug, il sera plus facile à trouver.
3. N'hésitez pas à afficher les valeurs des variables à l'écran. Elles n'ont pas toujours les valeurs auxquelles on s'attend. Alors, en phase de test, des `print` bien placés sont d'une grande aide.
4. Attention à « l'effet tunnel » ! On se focalise parfois sur une partie du code, alors que l'erreur se trouve ailleurs.
5. Pour éviter de perdre du temps, vous pouvez réduire le nombre de cartes du jeu pour vos tests. Quatre cartes peuvent suffire à vérifier certaines choses.
6. L'ordinateur peut aussi jouer plus vite. Il suffit de prendre une valeur inférieure à 1500 :
`plateau.after(1500, gerer_tirage).`
7. Avoir toujours la même configuration des cartes au départ, c'est possible. Il suffit :
 - soit de décrire la liste des cartes au lieu de la générer aléatoirement :
`cartes = [5,7,7,10,4,1,3,9,2,6,5,1,8,3,9,4,10,6,8,2]`
 - soit de fixer le germe du générateur de nombres pseudo-aléatoires avec la fonction `seed` (voir § 1.4.1).



Exercice 9.3

Activez les menus de l'ex. 9.1.



Exercice 9.4

Ajoutez une version solo, où l'ordinateur se contentera d'évaluer la performance du joueur (pourcentage d'erreurs, rapidité, etc.).



9.6. Ce que vous avez appris dans ce chapitre

- Vous savez maintenant comment créer des menus et des sous-menus.
- La méthode `find_closest()` renvoie l'objet le plus proche des coordonnées de la souris au moment du clic.
- Vous faites la différence entre un *frame*, un *canvas*, un *label* et une fenêtre.



Chapitre 10

Blackjack (vingt et un)

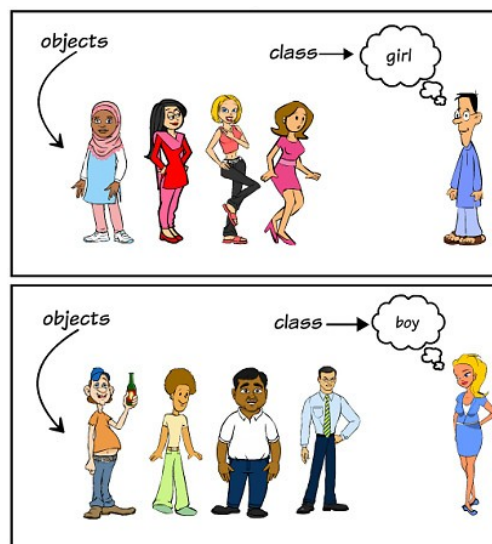
10.1. Nouveaux thèmes abordés dans ce chapitre

- Programmation orientée objet
- Sons

10.2. Programmation orientée objet (POO)

Un *paradigme* est une manière de programmer.

La **programmation orientée objet (POO)** est un paradigme de programmation informatique élaboré au début des années 1960. Il consiste en la définition et l'interaction de « briques logicielles » appelées *objets*. Un objet représente un concept, une entité du monde physique, comme une voiture, une personne, etc.



Python est un langage orienté objet, mais on peut très bien programmer en Python sans utiliser ce paradigme. D'ailleurs, c'est ce que l'on a fait jusqu'à présent ! L'intérêt d'une approche par objets est que chaque objet peut être développé séparément, ce qui est très pratique pour travailler sur un projet en équipe.

Nous présentons dans ce paragraphe les notions essentielles de la POO (classe, objet, attribut, méthode) et comment les programmer en Python. Des notions plus évoluées telle que l'héritage et le polymorphisme seront présentées plus tard dans ce chapitre.

Classe

Une classe est un **ensemble d'items qui ont une structure commune et un comportement commun**, par exemple une date, un étudiant, un arbre, une carte à jouer, etc.

Par convention, les noms de classe commencent par une majuscule.

```
class Carte:
    "Définition d'une carte"
```

Objet / instance

Un objet est **une instance d'une classe**.

Un objet est unique et identifiable avec un rôle bien défini. Il est persistant¹ et change d'état.

```
ma_carte = Carte()
```

Attribut

Un attribut est **une propriété observable des objets** d'une classe. Dans notre exemple, une carte est définie par une valeur et une couleur :

```
ma_carte.valeur = « roi »
ma_carte.couleur = « pique »
```

Méthode

Une méthode est **une opération qui peut changer la valeur des certains attributs d'un objet**. Dans l'exemple ci-dessous, il y a deux méthodes : la méthode spéciale `__init__()`, appelée *constructeur*, permet d'initialiser la valeur d'une carte (par défaut l'as de carreau) ; la méthode `afficher()` permet d'écrire une carte, par exemple « as de carreau ».

```
class Carte(object):
    "Définition d'une carte"

    def __init__(self, val='as', coul='carreau'):
        self.valeur = val
        self.couleur = coul

    def afficher(self):
        print(self.valeur+" de "+self.couleur)

ma_carte=Carte('roi','pique')
ma_carte.afficher()
```

Le **constructeur** est une méthode de notre objet se chargeant de créer nos attributs. En vérité, c'est même la méthode qui sera appelée quand on voudra créer notre objet.

Quand une méthode est appelée, une référence vers l'objet principal est passée en tant que premier argument. Par convention le premier argument appelé par les méthodes est toujours `self`. Voilà pourquoi la fonction `__init__(self, val='as', coul='carreau')` a trois arguments, mais elle est appelée avec seulement deux arguments : `Carte('roi', 'pique')`.

10.3. Exemple de classe : un paquet de cartes

Voici un exemple plus complet de classes : un paquet de 52 cartes. Le but du programme est simple : on va mélanger 52 cartes et les faire défiler à l'écran. On aura besoin de 52 images des cartes, que l'on va mettre dans le répertoire « cartes » et, pour améliorer le résultat à l'écran, deux

¹ **Persistant** : Un objet est créé à un certain moment, subit des changements d'états, et il est seulement détruit suite à une requête directe d'un utilisateur ou via un objet autorisé.

sons au format wave, que l'on mettra dans le répertoire « sons ».



paquet.py

<http://ow.ly/jtL7P>

```
# Affichage de cartes mélangées

from random import randrange
from tkinter import *
from winsound import PlaySound

couleur = ('pique', 'trefle', 'carreau', 'coeur')
valeur = ('as', '2', '3', '4', '5', '6', '7', '8', '9', '10', 'valet', 'dame', 'roi')

class Carte(object):

    def __init__(self, val='as', coul='carreau'):
        self.valeur = val
        self.couleur = coul

    def dessin_carte(self):
        "Renvoi du nom du fichier image de la carte"
        # les cartes sont dans le répertoire "cartes"
        nom = "cartes/"+self.valeur+"_"+self.couleur+".gif"
        return PhotoImage(file=nom)

class Paquet_de_cartes(object):

    def __init__(self):
        "Construction de la pile des 52 cartes"
        self.cartes = []
        for coul in range(4):
            for val in range(13):
                nouvelle_carte = Carte(valeur[val], couleur[coul])
                self.cartes.append(nouvelle_carte)

    def battre(self):
        "Mélanger les cartes"
        PlaySound("sons/distrib.wav", 2)
        t = len(self.cartes)
        for i in range(t):
            h1, h2 = randrange(t), randrange(t)
            self.cartes[h1], self.cartes[h2] = self.cartes[h2], self.cartes[h1]

    def tirer(self):
        "Tirer la première carte de la pile"
        PlaySound("sons/ramass.wav", 1)
        t = len(self.cartes)
        if t>0:
            carte = self.cartes[0] # choisir la première carte du jeu
            del(self.cartes[0])     # et la supprimer du jeu
            return carte
        else:
            return None

##### Programme de test #####

def jouer():
    global cartel, lab1
    c = jeu.tirer()
    if c != None:
        cartel = c.dessin_carte()
        lab1.configure(image=cartel)

def reinit():
    global jeu
    jeu = Paquet_de_cartes()
    jeu.battre()
    jouer()

# fenetre graphique
fenetre = Tk()
fenetre.title("Cartes")
jeu = Paquet_de_cartes()
```

```

jeu.battre()
c = jeu.tirer()
cartel = c.dessin_carte()
lab1 = Label(fenetre, image=cartel)
lab1.grid(row=0, column=0)
bouton1 = Button(fenetre, text='Tirer', command=jouer)
bouton1.grid(row=1, column=0, sticky=W)
bouton2 = Button(fenetre, text='Recommencer', command=reinit)
bouton2.grid(row=1, column=0, sticky=E)

# demarrage :
fenetre.mainloop()

```

10.4. Blackjack (règles simplifiées)

Les règles complètes sont sur Wikipédia :

<http://ow.ly/jtKfh>

La partie oppose tous les joueurs contre le croupier (pour simplifier, il n'y aura ici qu'un seul joueur). Le but est de faire plus de points que le croupier sans dépasser 21. Dès qu'un joueur fait plus que 21, on dit qu'il « saute » et il perd sa mise initiale. La valeur des cartes est établie comme suit :

- de 2 à 10 → valeur nominale de la carte
- une figure → 10 points
- un as → 1 ou 11 (au choix)

Un blackjack est composé d'un as et d'une « bûche » (carte ayant pour valeur 10, donc 10, valet, dame ou roi). Cependant, si le joueur atteint le point 21 en 3 cartes ou plus on compte le point 21 et non pas blackjack.

Au début de la partie le croupier distribue une carte face visible à chaque joueur et tire une carte face visible également pour lui. Il tire ensuite pour chacun des joueurs une seconde carte face visible et tire une seconde carte face cachée pour lui au blackjack américain. Au blackjack européen, le croupier tire sa seconde carte après le tour de jeu des joueurs.

Puis il demande au premier joueur de la table (joueur situé à sa gauche) l'option qu'il désire choisir. Si le joueur veut une carte, il doit l'annoncer en disant « Carte ! ». Le joueur peut demander autant de cartes qu'il le souhaite pour approcher 21 sans dépasser. Si après le tirage d'une carte, il a dépassé 21, il perd sa mise et le croupier passe au joueur suivant. S'il décide de s'arrêter, en disant « Je reste », le croupier passe également au joueur suivant.

Le croupier répète cette opération jusqu'à ce que tous les joueurs soient servis.

Ensuite, il joue pour lui selon une règle simple et codifiée « la banque tire à 16, reste à 17 ». Ainsi, le croupier tire des cartes jusqu'à atteindre un nombre compris entre 17 et 21 que l'on appelle un point. S'il fait plus de 21, tous les joueurs restants gagnent mais s'il fait son point, seuls gagnent ceux ayant un point supérieur au sien (sans avoir sauté). Dans cette situation, le joueur remporte l'équivalent de sa mise. En cas d'égalité le joueur garde sa mise mais n'empêche rien en plus. À noter que le blackjack (une bûche et un as en deux cartes) est plus fort que 21 fait en ayant tiré plus de deux cartes.

Nous allons ici suivre les règles européennes.

10.5. Code du programme



blackjack.py

<http://ow.ly/jtL7P>

```

# Blackjack
# règles : http://fr.wikipedia.org/wiki/Blackjack_(jeu)

from random import randrange
from tkinter import *
from winsound import PlaySound

couleur = ('pique', 'trefle', 'carreau', 'coeur')
valeur = ('as', '2', '3', '4', '5', '6', '7', '8', '9', '10', 'valet', 'dame', 'roi')

def calculer_main(cartes):
    # Les cartes sont des numéros allant de 1 (as) à 13 (roi)
    val = 0
    for carte in cartes:
        if 1 < carte <= 10:
            val += carte
        elif carte > 10:      # bûche
            val += 10
    # valeur de l'as ou des as

```

Ce programme
n'est pas tout à
fait terminé...

En effet, vous
constaterez que
ce programme
« bugue » quand
on clique à des
moments
inopportuns.

Que pourrait-on
faire pour éviter
ces problèmes ?

```

nb_as = cartes.count(1) # Attention s'il y a plusieurs as dans la main
while nb_as>1:
    val += 1 # un seul as peut valoir 11 pts. Les autres valent 1.
    nb_as -= 1
if nb_as == 1 and val + 11 <= 21:
    return val + 11
elif 1 in cartes:
    return val + 1
else:
    return val

class Carte(object):

    def __init__(self, val='as', coul='carreau'):
        self.valeur = val
        self.couleur = coul

    def dessin_carte(self):
        "Renvoi du nom du fichier image de la carte"
        # les cartes sont dans le répertoire "cartes"
        nom = "cartes/"+self.valeur+"_"+self.couleur+".gif"
        return PhotoImage(file=nom)

class Joueur(object):
    """Main du joueur"""

    def __init__(self):
        "Construction de la liste des 52 cartes"
        self.main = []

    def ajouter(self, c):
        "Ajoute la carte c"
        # 1:as, 2...10, 11:valet, 12:dame, 13:roi
        self.main.append(valeur.index(c.valeur)+1)

    def total(self):
        "Calcule le total des points de la main"
        return calculer_main(self.main)

    def nb_cartes(self):
        return len(self.main)

    def reinit(self):
        self.main = []

class Paquet_de_cartes(object):
    """Paquet de cartes"""

    def __init__(self):
        "Construction de la liste des 52 cartes"
        self.cartes = []
        for coul in range(4):
            for val in range(13):
                nouvelle_carte = Carte(valeur[val], couleur[coul])
                self.cartes.append(nouvelle_carte)

    def battre(self):
        "Mélanger les cartes"
        PlaySound("sons/distrib.wav", 2)
        t = len(self.cartes)
        for i in range(t):
            h1, h2 = randrange(t), randrange(t)
            self.cartes[h1], self.cartes[h2] = self.cartes[h2], self.cartes[h1]

    def tirer(self):
        "Tirer la première carte de la pile"
        PlaySound("sons/ramass.wav", 1)
        t = len(self.cartes)
        if t>0:
            carte = self.cartes[0] # choisir la première carte du jeu

```

Blackjack

```
        del(self.cartes[0])        # et la supprimer du jeu
        return carte
    else:
        return None

def reinit():
    global mes_cartes, ses_cartes, mon_score, son_score, moi, croupier
    global jeu, can

    can.delete(ALL)
    can.create_text(60,30,text="Croupier",fill='black',font='Arial 18')
    can.create_text(60,470,text="Joueur",fill='black',font='Arial 18')
    can.update()
    croupier.reinit()
    ses_cartes = []
    jeu = Paquet_de_cartes()
    jeu.battre()        # mélange des cartes
    sleep(1)            # pause d'une seconde
    c = jeu.tirer()
    ses_cartes.append(c.dessin_carte())
    can.create_image(100, 150, image=ses_cartes[0])
    croupier.ajouter(c)
    son_score = can.create_text(150,30,text=str(croupier.total()),fill='black',font='Arial 18')
    can.update()
    sleep(1)

    moi.reinit()        # joueur humain
    mes_cartes = []
    c = jeu.tirer()
    mes_cartes.append(c.dessin_carte())
    can.create_image(100, 350, image=mes_cartes[0])
    can.update()
    sleep(1)
    moi.ajouter(c)
    c = jeu.tirer()
    mes_cartes.append(c.dessin_carte())
    can.create_image(150, 350, image=mes_cartes[1])
    can.update()
    moi.ajouter(c)
    mon_score = can.create_text(150,470,text=str(moi.total()),fill='black',font='Arial 18')
    can.update()
    if moi.total()==21 :
        can.create_text(250,470,text="Blackjack",fill='red',font='Arial 18')
        can.update()
        sleep(3)
        reinit()

def hit(): # le joueur tire une carte
    global mes_cartes, moi, mon_score, son_score
    c = jeu.tirer()
    mes_cartes.append(c.dessin_carte())
    moi.ajouter(c)
    n = moi.nb_cartes()
    can.create_image(150+50*(n-2), 350, image=mes_cartes[n-1])
    can.delete(mon_score)
    mon_score = can.create_text(150,470,text=str(moi.total()),fill='black',font='Arial 18')
    can.update()
    if moi.total()>21:
        can.create_text(250,30,text="Gagné",fill='red',font='Arial 18')
        can.update()
        sleep(3)
        reinit()

def stay(): # le joueur s'arrête et le croupier joue
    global ses_cartes, croupier, moi, son_score, mon_score
    c = jeu.tirer()
    ses_cartes.append(c.dessin_carte())
    croupier.ajouter(c)
    n = croupier.nb_cartes()
    can.create_image(100+50*(n-1), 150, image=ses_cartes[n-1])
    can.delete(son_score)
```



```

son_score = can.create_text(150,30,text=str(croupier.total()),fill='black',font='Arial 18')
can.update()
if croupier.total()>21:
    can.create_text(250,470,text="Gagné",fill='red',font='Arial 18')
    can.update()
    sleep(3)
    reinit()
elif croupier.total()==21 and n==2:
    can.create_text(250,30,text="Blackjack",fill='red',font='Arial 18')
    can.update()
    sleep(3)
    reinit()
elif croupier.total()<17:
    sleep(2)
    stay()
else:
    if croupier.total()>moi.total():
        can.create_text(250,30,text="Gagné",fill='red',font='Arial 18')
    elif croupier.total()<moi.total():
        can.create_text(250,470,text="Gagné",fill='red',font='Arial 18')
    can.update()
    sleep(3)
    reinit()

# fenetre graphique
moi = Joueur()
mes_cartes = []
croupier = Joueur()
ses_cartes = []
jeu = Paquet_de_cartes()
fenetre = Tk()
fenetre.title("Blackjack")
can = Canvas(fenetre, width=600, height=500, bg='white')
can.pack(side=TOP, padx=5, pady=5)
b2 = Button(fenetre, text='Quitter', width=15, command=fenetre.quit)
b2.pack(side=RIGHT)
b1 = Button(fenetre, text='Carte !', width=15, command=hit)
b1.pack(side=LEFT)
b1 = Button(fenetre, text='Je reste', width=15, command=stay)
b1.pack(side=LEFT)
reinit()

# demarrage :
fenetre.mainloop()
fenetre.destroy()

```

Il y a dans ce programme trois classes. Par rapport à notre premier programme, on a rajouté la classe `joueur`, qui est utilisée pour gérer les cartes qu'a en main le joueur.

```

class Joueur(object):
    """Main du joueur"""

    def __init__(self):
        """Construction de la liste des 52 cartes"""
        self.main = []

    def ajouter(self, c):
        """Ajoute la carte c"""
        # 1:as, 2...10, 11:valet, 12:dame, 13:roi
        self.main.append(valeur.index(c.valeur)+1)

    def total(self):
        """Calcule le total des points de la main"""
        return calculer_main(self.main)

    def nb_cartes(self):
        return len(self.main)

    def reinit(self):
        self.main = []

```



Exercice 10.1

Écrivez une classe `Cercle()`. Les attributs seront son centre et son rayon. Les méthodes seront `perimetre` qui renverra le périmètre du cercle, `aire` qui donnera son aire et `contient` qui indiquera si oui un non un point est dans le cercle.

Par défaut, ce sera un cercle centré à l'origine de rayon 1.

Exemple d'utilisation de cette classe :

```
c = Cercle((0,0),5)
p = (2,5)
print(c.perimetre())
print(c.aire())
print(c.contient(p))

31.41592653589793
78.53981633974483
False
```



Exercice 10.2

Définissez une classe `Piece()` qui permette d'instancier des pièces de monnaie truquées. On donnera en paramètre la probabilité d'obtenir pile (un nombre réel entre 0 et 1). La valeur par défaut est 0.5.

Une méthode `lancer()` simuera le jet de la pièce et renverra le résultat (pile ou face).

Exemple d'utilisation de cette classe :

```
p1 = Piece(0.1)
print(p1.lancer())
p2 = Piece()
print(p2.lancer())

face
face
```



Exercice 10.3

Définissez une classe `De()` qui permette d'instancier des objets simulant des dés. Un dé pourra avoir un nombre de faces quelconque, avec un nombre quelconque écrit dessus. Une méthode `lancer()` simuera le jet du dé et renverra le résultat d'une face au hasard.

Par défaut, on définira un dé standard : 6 faces numérotées de 1 à 6.



Exercice 10.4

Modifiez le programme blackjack pour implémenter les règles complètes, que vous trouverez sur Wikipédia : [http://fr.wikipedia.org/wiki/Blackjack_\(jeu\)](http://fr.wikipedia.org/wiki/Blackjack_(jeu))

Contentez-vous dans un premier temps de programmer un seul joueur contre le croupier.

10.6. POO: notions avancées

10.6.1. Encapsulation

L'encapsulation est un mécanisme consistant à rassembler les données et les méthodes au sein

d'une structure en cachant l'implémentation de l'objet, c'est-à-dire en empêchant l'accès aux données par un autre moyen que les services proposés. L'encapsulation permet donc de garantir l'intégrité des données contenues dans l'objet.

Exemple

Dans le programme de blackjack que nous avons implémenté ci-dessus, on ne manipule jamais les cartes directement : on passe par les méthodes `battre` et `tirer` pour respectivement mélanger le jeu et tirer la première carte de la pile. L'usage de ces méthodes garantit le bon fonctionnement de ces deux actions.

10.6.2. Héritage

L'héritage est un principe propre à la programmation orientée objet, permettant de créer une nouvelle classe à partir d'une classe existante. Le nom d'« héritage » vient du fait que la classe dérivée contient les attributs et les méthodes de sa superclasse. L'intérêt majeur de l'héritage est de pouvoir définir de nouveaux attributs et de nouvelles méthodes pour la classe dérivée, qui viennent s'ajouter à ceux et celles héritées.

Par ce moyen, on crée une hiérarchie de classes de plus en plus spécialisées. Cela a comme avantage majeur de ne pas avoir à repartir de zéro lorsque l'on veut spécialiser une classe existante. De cette manière il est possible d'acheter dans le commerce des librairies de classes, qui constituent une base, pouvant être spécialisées à loisir.



```
class Personnage(object):
    def __init__(self, nom, force, vie):
        self._nom = nom
        self._force = force
        self._vie = vie

    def getNom(self) : return self._nom
    def getForce(self) : return self._force
    def getVie(self) : return self._vie

class Guerrier(Personnage):
    def __init__(self, nom, force, vie):
        Personnage.__init__(self, nom, force, vie) # héritage

class Mage(Personnage):
    def __init__(self, nom, force, vie, mana):
        Personnage.__init__(self, nom, force, vie) # héritage
        self._mana = mana

    def getMana(self) : return self._mana
```

Héritage multiple

Certains langages orientés objet, tels que le C++ ou Python, permettent de faire de l'héritage multiple, c'est-à-dire regrouper au sein d'une seule et même classe les attributs et méthodes de plusieurs classes.

Par exemple, on peut imaginer une classe « centaure », qui hériterait à la fois des caractéristiques des classes « homme » et « cheval ».

```
class Personnage(object):
    def __init__(self, nom, force, vie):
        self._nom = nom
        self._force = force
        self._vie = vie

    def getNom(self) : return self._nom
    def getForce(self) : return self._force
    def getVie(self) : return self._vie

class Homme(Personnage):
    def __init__(self, nom, force, vie, intelligence):
```



```

Personnage.__init__(self,nom,force,vie) # héritage
self._intelligence = intelligence

def getIntelligence(self) : return self._intelligence

class Cheval(Personnage):
    def __init__(self,nom,force,vie,vitesse):
        Personnage.__init__(self,nom,force,vie) # héritage
        self._vitesse = vitesse

    def getVitesse(self) : return self._vitesse

class Centaure(homme,cheval): # héritage multiple
    def __init__(self,nom,force,vie,intelligence,vitesse):
        Homme.__init__(self,nom,force,vie,intelligence)
        self._vitesse = vitesse

chiron = Centaure("Chiron",10,6,5,15)
print(chiron.getVitesse())

```

Dans le design objet de tous les jours, l'héritage multiple est peu utilisé, car il introduit des dépendances généralement plus rigides que souhaitées.

10.6.3. Polymorphisme

Le mot *polymorphisme* vient du grec et signifie « qui peut prendre plusieurs formes ». Cette caractéristique est un des concepts essentiels de la programmation orientée objet. Alors que l'héritage concerne les classes (et leur hiérarchie), le polymorphisme est relatif aux méthodes des objets.

Ces trois types de polymorphisme existent en Python.

On distingue généralement trois types de polymorphisme :

- le polymorphisme *ad hoc* (également appelé **surcharge**),
- le polymorphisme paramétrique (également appelé **généricité**),
- le polymorphisme d'héritage (également appelé **redéfinition**).

Le polymorphisme *ad hoc*

Le polymorphisme *ad hoc* permet de définir des **opérateurs** dont l'utilisation sera différente selon le type des paramètres qui leur sont passés. Il est donc possible par exemple de surcharger l'opérateur « + » et de lui faire réaliser des actions différentes selon qu'il s'agit d'une opération entre deux entiers (addition : 3+4 = 7) ou entre deux chaînes de caractères (concaténation : 'parle'+ 'ment' = 'parlement').

Le polymorphisme paramétrique

Le polymorphisme paramétrique représente la possibilité de définir plusieurs **fonctions** de même nom mais possédant des paramètres différents (en nombre et/ou en type). Le polymorphisme paramétrique rend ainsi possible le choix automatique de la bonne méthode à adopter en fonction du type de donnée passée en paramètre.

Ainsi, on peut par exemple définir plusieurs méthodes homonymes `add(a,b)` effectuant une « somme » de valeurs `a` et `b`.

Par exemple, en Python 3 :

```

def add(a,b):
    return a + b

print(add(12, 5))           # somme de deux entiers
print(add('parle', 'ment')) # concaténation de deux strings
print(add(3.8, 9))          # somme d'un réel et d'un entier
print(add([4, 9], [3.2, 6, 'a'])) # concaténation de deux listes

```

produira comme résultat :

```
17
parlement
12.8
[4, 9, 3.2, 6, 'a']
```

Le polymorphisme d'héritage

La possibilité de redéfinir une méthode dans des classes héritant d'une classe de base s'appelle la spécialisation. Il est alors possible d'appeler la méthode d'un objet sans se soucier de son type intrinsèque : il s'agit du polymorphisme d'héritage. Ceci permet de faire abstraction des détails des classes spécialisées d'une famille d'objets, en les masquant par une interface commune (qui est la classe de base).

Reprenons notre exemple des personnages virtuels et ajoutons une méthode « attaquer ».

```
class Personnage(object):
    def __init__(self, nom, force, vie):
        self._nom = nom
        self._force = force
        self._vie = vie

    def getNom(self) : return self._nom
    def getForce(self) : return self._force
    def getVie(self) : return self._vie

class Guerrier(Personnage):
    def __init__(self, nom, force, vie):
        Personnage.__init__(self, nom, force, vie) # héritage
    def attaquer(self, qui):
        print(self._nom, "pourfend", qui.getNom(), "de son épée")

class Mage(Personnage):
    def __init__(self, nom, force, vie, mana):
        Personnage.__init__(self, nom, force, vie) # héritage
        self._mana = mana

    def getMana(self) : return self._mana
    def attaquer(self, qui):
        print(self._nom, "ensorcelle", qui.getNom())

merlin = Mage("Merlin", 3, 10, 1)
duroc = Guerrier("Duroc", 9, 10)
merlin.attaquer(duroc)
duroc.attaquer(merlin)
```

Le résultat affiché sera :
Merlin ensorcelle Duroc
Duroc pourfend Merlin de son épée



Exercice 10.5

Créez une classe « Nain » où un personnage de ce type porte une attaque avec sa hache.

Créez un personnage de cette classe et nommez-le « Gimli ».



Exercice 10.6

Imaginez d'autres races et sous-races (par exemple les elfes, les trolls, les trolls-cyclopes, etc.) et implémentez un système pour simuler des combats entre ces personnages.



Blackjack

Les attributs vie, force, mana (et éventuellement d'autres que vous pourrez ajouter) seront modifiés selon les blessures infligées, dont la gravité sera déterminée aléatoirement. À vous de définir les règles ! Un personnage aura perdu le combat quand ses points de vie seront à zéro.

Organisez ensuite un tournoi où chaque personnage se battra en duel avec chacun des autres. Il y aura 10 personnages différents, pas forcément tous de race différente. Le tournoi sera donc composé de 45 combats.

Quelques idées de personnages fantastiques...



Sorcier



Harpie



Sirène

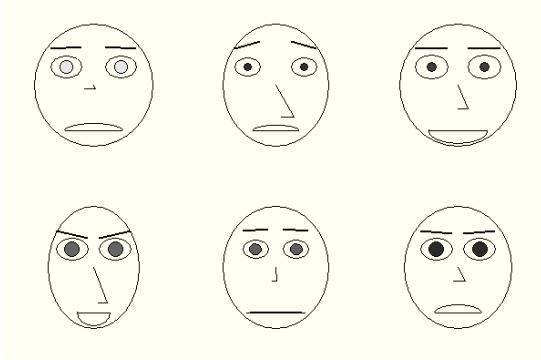


Troll-cyclope



10.7. Ce que vous avez appris dans ce chapitre

- Vous avez vu les notions fondamentales de la programmation orientée objet (POO) : classe, objet, attribut, méthode, encapsulation, héritage, polymorphisme.
- Python est fait pour ce paradigme, qui est particulièrement intéressant quand on développe un projet en groupe.
- Vous avez pour la première fois incorporé des sons à vos programmes.



Chapitre 11

Figures de Chernoff

11.1. Nouveaux thèmes abordés dans ce chapitre

- tkinter : segments, rectangles, polygones, cercles, ovales, secteurs
- couleurs

11.2. Code de l'exemple

Pour expliquer comment dessiner des formes, le plus simple est de regarder un exemple :



exemple.py

<http://ow.ly/fh92f>

```
# Exemples de dessins
from tkinter import *
from random import choice

def change_couleur():
    global coul
    palette=['snow', 'ghost white', 'white smoke', 'gainsboro', 'floral white', 'old
lace', 'linen', 'antique white', 'papaya whip', 'blanched almond', 'bisque', 'peach
puff', 'navajo white', 'lemon chiffon', 'mint cream', 'azure', 'alice blue',
'lavender', 'lavender blush', 'misty rose', 'dark slate gray', 'dim gray', 'slate
gray', 'light slate gray', 'gray', 'light grey', 'midnight blue', 'navy',
'cornflower blue', 'dark slate blue', 'slate blue', 'medium slate blue', 'light
slate blue', 'medium blue', 'royal blue', 'blue', 'dodger blue', 'deep sky blue',
'sky blue', 'light sky blue', 'steel blue', 'light steel blue', 'light blue',
'powder blue', 'pale turquoise', 'dark turquoise', 'medium turquoise', 'turquoise',
'cyan', 'light cyan', 'cadet blue', 'medium aquamarine', 'aquamarine', 'dark green',
'dark olive green', 'dark sea green', 'sea green', 'medium sea green', 'light sea
green', 'pale green', 'spring green', 'lawn green', 'medium spring green', 'green
yellow', 'lime green', 'yellow green', 'forest green', 'olive drab', 'dark khaki',
'khaki', 'pale goldenrod', 'light goldenrod yellow', 'light yellow', 'yellow',
'gold', 'light goldenrod', 'goldenrod', 'dark goldenrod', 'rosy brown', 'indian
red', 'saddle brown', 'sandy brown', 'dark salmon', 'salmon', 'light salmon',
'orange', 'dark orange', 'coral', 'light coral', 'tomato', 'orange red', 'red', 'hot
pink', 'deep pink', 'pink', 'light pink', 'pale violet red', 'maroon', 'medium
violet red', 'violet red', 'medium orchid', 'dark orchid', 'dark violet', 'blue
violet', 'purple', 'medium purple', 'thistle', 'snow2', 'snow3', 'snow4',
'seashell12', 'seashell13', 'seashell14', 'AntiqueWhite1', 'AntiqueWhite2',
'AntiqueWhite3', 'AntiqueWhite4', 'bisque2', 'bisque3', 'bisque4', 'PeachPuff2',
'PeachPuff3', 'PeachPuff4', 'NavajoWhite2', 'NavajoWhite3', 'NavajoWhite4',
'LemonChiffon2', 'LemonChiffon3', 'LemonChiffon4', 'cornsilk2', 'cornsilk3',
```

Figures de Chernoff

```
'cornsilk4', 'ivory2', 'ivory3', 'ivory4', 'honeydew2', 'honeydew3', 'honeydew4',
'LavenderBlush2', 'LavenderBlush3', 'LavenderBlush4', 'MistyRose2', 'MistyRose3',
'MistyRose4', 'azure2', 'azure3', 'azure4', 'SlateBlue1', 'SlateBlue2',
'SlateBlue3', 'SlateBlue4', 'RoyalBlue1', 'RoyalBlue2', 'RoyalBlue3', 'RoyalBlue4',
'blue2', 'blue4', 'DodgerBlue2', 'DodgerBlue3', 'DodgerBlue4', 'SteelBlue1',
'SteelBlue2', 'SteelBlue3', 'SteelBlue4', 'DeepSkyBlue2', 'DeepSkyBlue3',
'DeepSkyBlue4', 'SkyBlue1', 'SkyBlue2', 'SkyBlue3', 'SkyBlue4', 'LightSkyBlue1',
'LightSkyBlue2', 'LightSkyBlue3', 'LightSkyBlue4', 'SlateGray1', 'SlateGray2',
'SlateGray3', 'SlateGray4', 'LightSteelBlue1', 'LightSteelBlue2', 'LightSteelBlue3',
'LightSteelBlue4', 'LightBlue1', 'LightBlue2', 'LightBlue3', 'LightBlue4',
'LightCyan2', 'LightCyan3', 'LightCyan4', 'PaleTurquoise1', 'PaleTurquoise2',
'PaleTurquoise3', 'PaleTurquoise4', 'CadetBlue1', 'CadetBlue2', 'CadetBlue3',
'CadetBlue4', 'turquoise1', 'turquoise2', 'turquoise3', 'turquoise4', 'cyan2',
'cyan3', 'cyan4', 'DarkSlateGray1', 'DarkSlateGray2', 'DarkSlateGray3',
'DarkSlateGray4', 'aquamarine2', 'aquamarine4', 'DarkSeaGreen1', 'DarkSeaGreen2',
'DarkSeaGreen3', 'DarkSeaGreen4', 'SeaGreen1', 'SeaGreen2', 'SeaGreen3',
'PaleGreen1', 'PaleGreen2', 'PaleGreen3', 'PaleGreen4', 'SpringGreen2',
'SpringGreen3', 'SpringGreen4', 'green2', 'green3', 'green4', 'chartreuse2',
'chartreuse3', 'chartreuse4', 'OliveDrab1', 'OliveDrab2', 'OliveDrab4',
'DarkOliveGreen1', 'DarkOliveGreen2', 'DarkOliveGreen3', 'DarkOliveGreen4',
'khaki1', 'khaki2', 'khaki3', 'khaki4', 'LightGoldenrod1', 'LightGoldenrod2',
'LightGoldenrod3', 'LightGoldenrod4', 'LightYellow2', 'LightYellow3',
'LightYellow4', 'yellow2', 'yellow3', 'yellow4', 'gold2', 'gold3', 'gold4',
'goldenrod1', 'goldenrod2', 'goldenrod3', 'goldenrod4', 'DarkGoldenrod1',
'DarkGoldenrod2', 'DarkGoldenrod3', 'DarkGoldenrod4', 'RosyBrown1', 'RosyBrown2',
'RosyBrown3', 'RosyBrown4', 'IndianRed1', 'IndianRed2', 'IndianRed3', 'IndianRed4',
'sienna1', 'sienna2', 'sienna3', 'sienna4', 'burlywood1', 'burlywood2',
'burlywood3', 'burlywood4', 'wheat1', 'wheat2', 'wheat3', 'wheat4', 'tan1', 'tan2',
'tan4', 'chocolate1', 'chocolate2', 'chocolate3', 'firebrick1', 'firebrick2',
'firebrick3', 'firebrick4', 'brown1', 'brown2', 'brown3', 'brown4', 'salmon1',
'salmon2', 'salmon3', 'salmon4', 'LightSalmon2', 'LightSalmon3', 'LightSalmon4',
'orange2', 'orange3', 'orange4', 'DarkOrange1', 'DarkOrange2', 'DarkOrange3',
'DarkOrange4', 'coral1', 'coral2', 'coral3', 'coral4', 'tomato2', 'tomato3',
'tomato4', 'OrangeRed2', 'OrangeRed3', 'OrangeRed4', 'red2', 'red3', 'red4',
'DeepPink2', 'DeepPink3', 'DeepPink4', 'HotPink1', 'HotPink2', 'HotPink3',
'HotPink4', 'pink1', 'pink2', 'pink3', 'pink4', 'LightPink1', 'LightPink2',
'LightPink3', 'LightPink4', 'PaleVioletRed1', 'PaleVioletRed2', 'PaleVioletRed3',
'PaleVioletRed4', 'maroon1', 'maroon2', 'maroon3', 'maroon4', 'VioletRed1',
'VioletRed2', 'VioletRed3', 'VioletRed4', 'magenta2', 'magenta3', 'magenta4',
'orchid1', 'orchid2', 'orchid3', 'orchid4', 'plum1', 'plum2', 'plum3', 'plum4',
'MediumOrchid1', 'MediumOrchid2', 'MediumOrchid3', 'MediumOrchid4', 'DarkOrchid1',
'DarkOrchid2', 'DarkOrchid3', 'DarkOrchid4', 'purple1', 'purple2', 'purple3',
'purple4', 'MediumPurple1', 'MediumPurple2', 'MediumPurple3', 'MediumPurple4',
'thistle1', 'thistle2', 'thistle3', 'thistle4', 'gray1', 'gray2', 'gray3', 'gray4',
'gray5', 'gray6', 'gray7', 'gray8', 'gray9', 'gray10', 'gray11', 'gray12', 'gray13',
'gray14', 'gray15', 'gray16', 'gray17', 'gray18', 'gray19', 'gray20', 'gray21',
'gray22', 'gray23', 'gray24', 'gray25', 'gray26', 'gray27', 'gray28', 'gray29',
'gray30', 'gray31', 'gray32', 'gray33', 'gray34', 'gray35', 'gray36', 'gray37',
'gray38', 'gray39', 'gray40', 'gray42', 'gray43', 'gray44', 'gray45', 'gray46',
'gray47', 'gray48', 'gray49', 'gray50', 'gray51', 'gray52', 'gray53', 'gray54',
'gray55', 'gray56', 'gray57', 'gray58', 'gray59', 'gray60', 'gray61', 'gray62',
'gray63', 'gray64', 'gray65', 'gray66', 'gray67', 'gray68', 'gray69', 'gray70',
'gray71', 'gray72', 'gray73', 'gray74', 'gray75', 'gray76', 'gray77', 'gray78',
'gray79', 'gray80', 'gray81', 'gray82', 'gray83', 'gray84', 'gray85', 'gray86',
'gray87', 'gray88', 'gray89', 'gray90', 'gray91', 'gray92', 'gray93', 'gray94',
'gray95', 'gray97', 'gray98', 'gray99']
coul = choice(palette)

def dessiner_ligne():
    global coul
    can.create_line(10, 10, 100, 200, width=2, fill=coul)

def dessiner_rectangle():
    global coul
    can.create_rectangle(10, 10, 100, 200, fill=coul)

def dessiner_polygone():
    global coul
    sommets = [150, 100, 200, 120, 250, 30, 360, 20, 300, 210, 150, 150]
    can.create_polygon(sommets, outline='red', fill=coul)

def dessiner_disque():
    global coul
    can.create_oval(200, 300, 300, 400, fill=coul)
```



```
def dessiner_oval():
    global coul
    can.create_oval(100, 250, 150, 350, fill=coul)

def dessiner_secteur():
    global coul
    can.create_oval(100, 250, 150, 350, start=45, extent=90, fill=coul)

#----- Programme principal -----
coul = 'cyan' # couleur initiale
fen = Tk()
can = Canvas(fen,bg='white',height=400,width=400)
can.pack(side=LEFT)
# Boutons
bou1 = Button(fen,text='Quitter',width=10,command=fen.quit)
bou1.pack(side=BOTTOM)
bou2 = Button(fen,text='Ligne',width=10,command=dessiner_ligne)
bou2.pack()
bou3 = Button(fen,text='Rectangle',width=10,command=dessiner_rectangle)
bou3.pack()
bou4 = Button(fen,text='Polygone',width=10,command=dessiner_polygone)
bou4.pack()
bou5 = Button(fen,text='Disque',width=10,command=dessiner_disque)
bou5.pack()
bou6 = Button(fen,text='Ovale',width=10,command=dessiner_oval)
bou6.pack()
bou7 = Button(fen,text='Secteur',width=10,command=dessiner_secteur)
bou7.pack()
bou8 = Button(fen,text='Autre couleur',width=10,command=change_couleur)
bou8.pack()
fen.mainloop()
fen.destroy()
```

11.3. Analyse du programme

On a utilisé dans ce programme des objets classiques en dessin : segments, rectangles, polygones et ovales.

11.3.1. Les couleurs

Les couleurs peuvent se définir par leur nom. Nous avons profité de ce petit script pour les énumérer toutes. C'est un peu idiot, mais comme ces noms ne sont pas faciles à trouver dans la documentation Python, autant le faire une fois pour toutes.

```
def change_couleur():
    global coul
    palette=['snow', 'ghost white', 'white smoke', 'gainsboro', ...]

    (voir le code du §11.2 pour avoir la liste complète)
```

Une autre manière, plus pratique, de décrire une couleur est de donner le codage **RVB** (rouge, vert, bleu) ou, en anglais, **RGB** (red, green, blue). La combinaison de ces trois couleurs donnera $16'777'216$ (256^3) couleurs.

Attention ! On parle ici de combinaisons de lumières, et non pas de peintures. Ainsi, du rouge avec du bleu donnera du magenta, du bleu avec du vert donnera du cyan, et du rouge avec du vert donnera du jaune. La combinaison des trois couleurs donnera du blanc.

En Python, la couleur aura le format suivant : `#rrggbb`, où `rr` sera un nombre, exprimé en base 16, compris entre 0 (00) et 256 (ff), qui donnera la valeur du rouge. Idem pour le vert (gg) et le bleu (bb). Par exemple, `#ff0000` est du rouge, `#ffff00` du jaune, `#000000` du noir, `#ffffff` du blanc, etc. On aurait ainsi pu écrire :

```
coul = '#00ffff' # cyan
```



11.3.2. Segment

Un segment de droite est défini par les coordonnées de ses deux extrémités.

```
def dessiner_ligne():
    global coul
    can.create_line(10, 10, 100, 200, width=2, fill=coul)
```

Dans l'exemple, le segment va du point (10, 10) au point (100, 200). Il a une largeur de 2 pixels et une couleur définie par coul.

Attention ! L'origine (0, 0) est dans le coin supérieur gauche de la fenêtre. Le coin inférieur droit aura comme coordonnées (larg, haut), dans notre exemple (400, 400) :

```
can = Canvas(fen, bg='white', height=400, width=400)
```

Notons enfin que l'on peut prolonger le segment en une ligne polygonale : il suffira d'ajouter des sommets à la liste. Par exemple :

```
can.create_line(10, 10, 100, 200, 150, 50, width=2, fill=coul)
```

11.3.3. Rectangle

Un rectangle est défini par les coordonnées des extrémités d'une de ses diagonales. Ses côtés seront parallèles aux axes.

```
def dessiner_rectangle():
    global coul
    can.create_rectangle(10, 10, 100, 200, fill=coul)
```

Dans l'exemple, on a donné comme extrémités de la diagonale le coin supérieur gauche (10, 10) et le coin inférieur droit (100, 200). Le rectangle a un contour (outline) noir et une couleur de remplissage (fill) définie par coul.

11.3.4. Polygone

Un polygone est défini par la suite des coordonnées de ses sommets. Le dernier sommet sera relié au premier.

```
def dessiner_polygone():
    global coul
    sommets = [150, 100, 200, 120, 250, 30, 360, 20, 300, 210, 150, 150]
    can.create_polygon(sommets, outline='red', fill=coul)
```

Dans l'exemple, la suite des sommets est (150, 100), (200, 120), (250, 30), (360, 20), (300, 210) et (150, 150).

11.3.5. Disque et ovale

Un ovale est défini par le rectangle (voir § 11.3.3) dans lequel il est inscrit. Si le rectangle est un carré, on obtient bien évidemment un disque.

```
def dessiner_disque():
    global coul
    can.create_oval(200, 300, 300, 400, fill=coul)

def dessiner_ovale():
    global coul
    can.create_oval(250, 250, 400, 350, fill=coul)
```

11.3.6. Secteur

On tourne donc dans le sens trigonométrique positif.

Un secteur est une « part de gâteau » d'un disque ou d'un ovale. La syntaxe est la presque même que pour un ovale, mais on ajoute deux paramètres : l'inclinaison de départ (`start`) et l'angle du secteur (`extent`). Ces angles sont donnés en degrés et on tourne dans le sens inverse des aiguilles d'une montre.

L'exemple ci-dessous dessinera un quart d'ovale, incliné de 45°, inscrit dans le rectangle défini par les sommets (100, 250) et (150, 350).

```
def dessiner_secteur():
    global coul
    can.create_arc(100, 250, 150, 350, start=45, extent=90, fill=coul)
```



Exercice 11.1

Modifiez les valeurs des paramètres dans le code du § 11.2 et observez quelles sont les répercussions sur les formes dessinées.



Exercice 11.2 : la marche de l'ivrogne

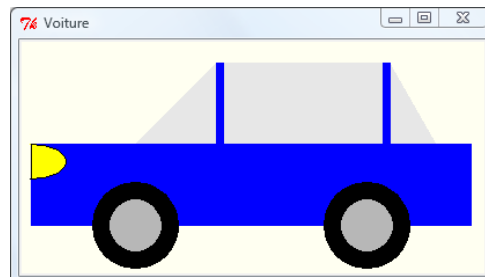
Un ivrogne se déplace en titubant. À chaque pas, il a le choix entre quatre directions : en haut, en bas, à gauche, à droite. Il prend une de ces quatre directions au hasard.

En partant du centre d'un carré, dessinez la trajectoire de l'ivrogne après 1000 pas.



Exercice 11.3

Reproduisez ce dessin en utilisant un polygone, trois rectangles, deux disques et un secteur.



Vous pouvez aussi dessiner un engin plus à votre goût...



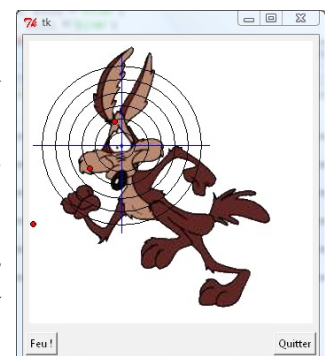
Exercice 11.4

Il existe aussi une méthode appelée `create_image` qui place une image dans un canevas.

```
photo = PhotoImage(file='coyote.gif')
can.create_image(156, 156, image=photo)
```

Les coordonnées (156, 156) permettent de placer l'image dans le canevas. Le centre de l'image sera positionné sur ces coordonnées.

1. Insérez une image dans une fenêtre. Vous trouverez l'image à l'adresse www.nymphomath.ch/pj/chernoff/
2. Ses dimensions sont de 312 pixels sur 312 pixels.
3. Dessinez une cible centrée sur l'œil gauche du coyote (voir image ci-dessus)
4. Insérez un bouton « Quitter » et un bouton « Feu! »
5. Quand on pressera le bouton « Feu! » trois points rouges apparaîtront au hasard dans le plus petit carré englobant la cible, donnant l'illusion que l'on tire sur le coyote.

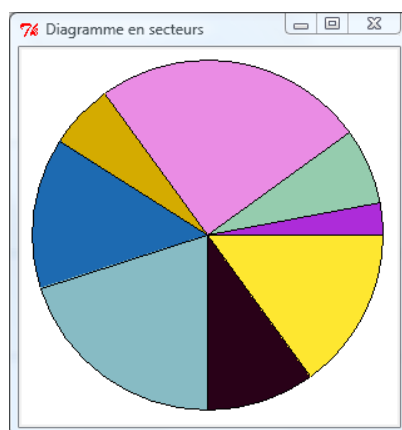




Exercice 11.5

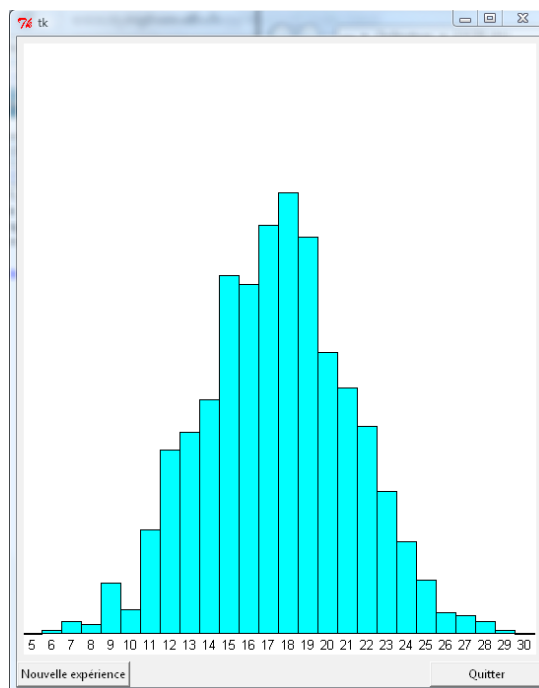
Écrivez un programme Python qui dessine un diagramme à secteurs (un camembert) d'après un tuple de pourcentages donnés. Chaque part de gâteau aura une couleur aléatoire. Le camembert ci-dessous a été obtenu à partir du tuple (3,7,25,6,14,20,10,15).

La somme des huit
nombres fait bien
100...



Exercice 11.6

Écrivez un programme Python qui dessine un histogramme (voir exemple ci-dessous). Les nombres en bas de l'image sont les sommes possibles quand on lance 5 dés : ces sommes vont de $5=1+1+1+1+1$ à $30=6+6+6+6+6$. La hauteur est proportionnelle au nombre de fois que la somme correspondante est survenue. On appelle cela une distribution.



Les nombres seront tirés au sort dans le programme, si bien que l'on pourra faire plusieurs séries de lancers, en cliquant sur le bouton « Nouvelle expérience ». Chaque série comportera plusieurs centaines de lancers.

Avant de démarrer la partie graphique, le programme demandera le nombre de dés à utiliser (entre 1 et 10). Il faudra donc adapter les sommes...



Exercice 11.7

Modifiez le programme de l'exercice 11.6 afin de représenter la distribution obtenue en suivant le protocole de l'exercice 1.9.

11.4. Les figures de Chernoff

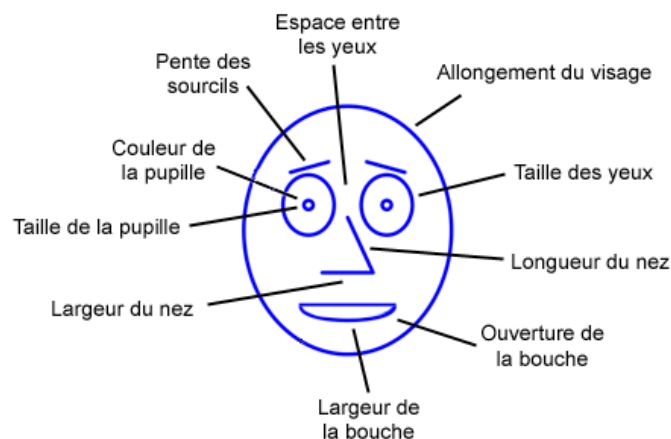


Cette méthode a été inventée par Herman **Chernoff**, mathématicien et physicien américain.

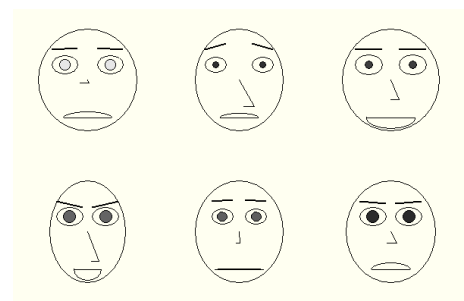
Chernoff, H. (1973). *The use of faces to represent points in k-dimensional space graphically*, Journal of the American Statistical Association 68 (342) : 361-368

Trouver une représentation graphique intuitive et facilement compréhensible de données à n dimensions lorsque n est supérieur à 3 n'est pas simple. Les représentations les plus courantes utilisent habituellement une, deux ou les trois dimensions spatiales et parfois quelques dimensions supplémentaires via un coloriage adéquat. Les **figures de Chernoff** permettent d'augmenter le nombre de dimensions à représenter. Elles tirent profit de notre capacité à détecter de très légers changements dans les expressions faciales.

Une figure de Chernoff s'obtient en associant à chaque composante du vecteur représentant les données un trait d'une expression faciale. La première composante permettra par exemple de préciser la forme de la tête, la seconde fixera la taille des yeux, la troisième leur aspect, la quatrième la distance entre ceux-ci, etc. En utilisant ainsi les traits les plus frappants d'un visage, il est possible de représenter un nombre de dimensions largement supérieur à trois.



Voici un exemple utilisant les caractéristiques ci-dessus. On peut ainsi comparer des entités comportant 10 paramètres, par exemple des élèves d'une classe (les paramètres seraient les notes dans différentes disciplines), des pays (PIB, population, chômage, etc.), des aliments en fonction de la composition, etc.



11.5. Code du programme



chernoff.py

<http://ow.ly/fh92f>

```
# figures de Chernoff
from random import random
from math import *
from tkinter import *

def gris(c):
    # choisit un niveau de gris (gray1 à gray99)
    c = int(c*100)
    if c==100:
        return "white"
```

```

elif c==0:
    return "black"
else:
    return "gray"+str(c)

def visage(a):
    # a : allongement du visage. 0=rond
    a = 1-0.25*a
    # 0.75 <= a <= 1
    can.create_oval(centre-a*cote/2.6, centre-cote/2.6/a,
                    centre+a*cote/2.6, centre+cote/2.6/a)

def nez(l, g):
    # l : longueur du nez
    # g : profondeur du nez
    # 0 <= l <= 1
    l = 38*(l+0.1)
    k = 2+2*g
    # 2 <= k <= 4 pour calculer l'angle du nez
    can.create_line(centre, centre, centre+l*cos(pi/k), centre+l*sin(pi/k),
                    centre+l*cos(pi/k)-20*g-5, centre+l*sin(pi/k))

def yeux(e, p, o, f, c):
    # e : espacement entre les yeux
    # p : rayon de la pupille
    # o : rayon de l'oeil
    # f : froncement des sourcils
    # c : couleur de la pupille
    e = cote/10+cote/18*e
    p = 5*p+cote/60
    o = cote/40*o+cote/15
    # pupilles
    can.create_oval(centre-e-p, centre-0.1*cote-p,
                    centre-e+p, centre-0.1*cote+p, fill=gris(c))
    can.create_oval(centre+e-p, centre-0.1*cote-p,
                    centre+e+p, centre-0.1*cote+p, fill=gris(c))
    # oeil
    can.create_oval(centre-e-o, centre-0.1*cote-0.7*o,
                    centre-e+o, centre-0.1*cote+0.7*o)
    can.create_oval(centre+e-o, centre-0.1*cote-0.7*o,
                    centre+e+o, centre-0.1*cote+0.7*o)
    # sourcils
    f = f - 0.5
    can.create_line(centre-e-o, centre-0.2*cote,
                    centre-e+o, centre-0.2*cote+30*f, width=2)
    can.create_line(centre+e-o, centre-0.2*cote,
                    centre+e+o, centre-0.2*cote+30*f, width=2)

def bouche(m, j):
    # m : largeur
    # j : hauteur
    j = j-0.5
    m = m+0.02
    if j<0:
        can.create_arc(centre-cote/6*m, centre+0.25*cote+cote/6*j,
                        centre+cote/6*m, centre+0.25*cote-cote/6*j,
                        start=0, extent=180)
    else:
        can.create_arc(centre-cote/6*m, centre+0.25*cote+cote/6*j,
                        centre+cote/6*m, centre+0.25*cote-cote/6*j,
                        start=180, extent=180)

def chernoff(a, l, g, e, p, o, f, c, m, j):
    # coefficients entre 0 et 1
    visage(a)
    nez(l,g)
    yeux(e,p,o,f,c)
    bouche(m,j)

def reinit():
    can.delete(ALL)
    a = random()
    l = random()
    g = random()

```

```

e = random()
p = random()
o = random()
f = random()
c = random()
j = random()
m = random()
chernoff(a,l,g,e,p,o,f,c,m,j)

cote = 300
centre = 1.2*cote/2

root = Tk()
root.title("Figures de Chernoff")
can = Canvas(root, height=1.2*cote, width=1.2*cote, background="ivory")
can.pack()
b = Button(root, text='Une autre!', command=reinit)
b.pack()
chernoff(1,1,1,1,1,1,1,1,1,1)
root.mainloop()

```

Comme ce programme ne contient rien de nouveau, nous laissons le lecteur l'analyser seul. Nous noterons simplement que la variable `cote` définit à la fois la taille de la fenêtre ($1.2*cote$) et celle du visage, qui est placé au centre de la fenêtre ($centre = 1.2*cote/2$).



Exercice 11.8

Modifiez le programme du § 11.5. Ajoutez un 11^{ème} paramètre b influant sur la forme des yeux. Inspirez-vous de la fonction `visage()`. Pour $b = 0.5$, l'œil sera parfaitement rond. Pour $0 \leq b < 0.5$, l'œil sera allongé horizontalement et pour $0.5 < b \leq 1$, l'œil sera allongé verticalement.



Exercice 11.9

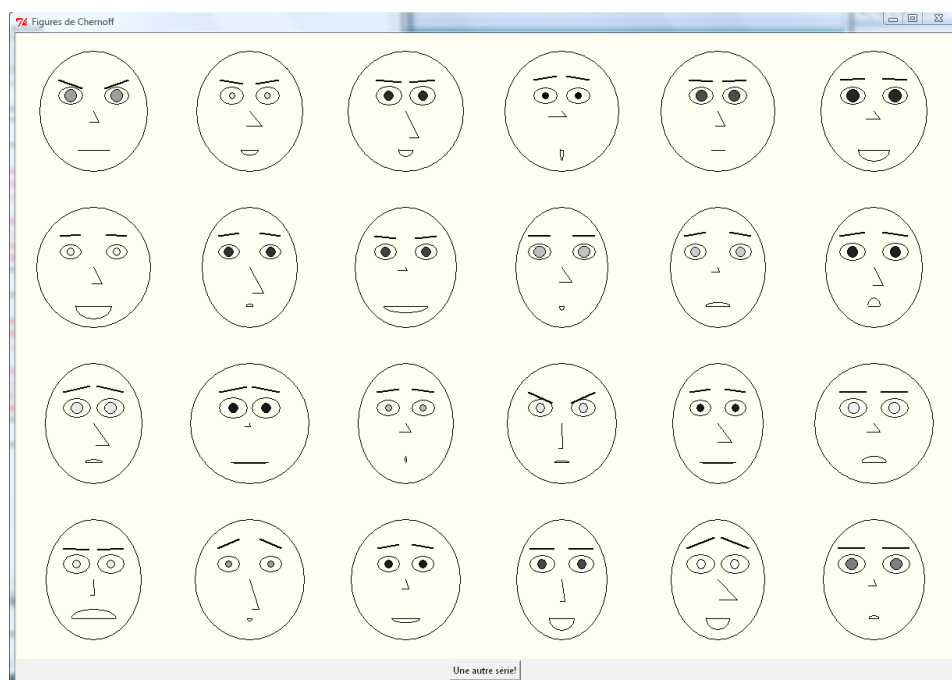
Reprenez le programme `chernoff.py` du chapitre 9 et créez une classe `chernoff` qui permettra de définir et dessiner une figure de Chernoff.

Disposez ensuite 24 figures aléatoires sur une feuille, comme ci-après :



`chernoff.py`

<http://ow.ly/fh92f>





Exercice 11.10

Modifiez le programme de l'exercice 11.9 pour lire et visualiser le fichier suivant, qui contient les notes de 12 élèves :

```
Lucien 6.0 6.0 6.0 6.0 6.0 6.0 6.0 6.0 6.0 6.0
Paul 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0
Malo 4.0 4.0 4.0 4.0 4.0 4.0 4.0 4.0 4.0 4.0
Julie 3.5 5.5 5.0 4.5 4.5 4.5 3.5 4.0 5.5 6.0
Marie-France 4.0 2.5 3.5 5.5 5.5 6.0 5.0 4.5 4.0 5.0
Géraldine 6.0 5.5 4.5 3.5 2.5 4.0 2.0 3.5 4.0 5.0
Octavia 6.0 2.5 5.5 5.5 3.5 6.0 5.0 4.5 4.0 5.0
Jules 3.5 2.5 3.5 5.5 3.5 6.0 5.0 4.5 3.0 5.0
René 6.0 2.5 3.5 3.5 3.5 1.5 5.0 4.5 4.5 6.0
Yvan 5.0 2.5 3.5 5.5 4.5 6.0 5.0 4.5 5.0 2.0
Agapanthe 2.0 2.5 3.0 5.5 5.5 5.0 5.0 5.5 4.0 3.0
Véronique 4.0 2.5 4.5 5.5 5.0 6.0 5.0 4.5 4.0 5.0
```

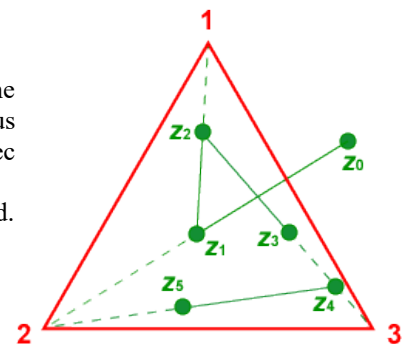
Écrivez le nom de l'élève sous chaque tête, puis comparez visuellement ces élèves.



Exercice 11.11

Dessinez les trois sommets d'un triangle équilatéral sur une feuille. Placez ensuite un point z_0 au hasard sur cette feuille. Vous allez maintenant construire et dessiner une suite de points avec les règles suivantes :

1. prendre l'un des trois sommets du triangle au hasard. Appelons-le s .
2. dessiner le point z_{i+1} qui est au milieu du segment $z_i s$
3. incrémenter i et retourner en 1.



- a) Avant de programmer cela, essayez d'imaginer quel sera le résultat.
- b) Programmez cet algorithme. Générez une suite de 50'000 points dans une fenêtre de 800 pixels de large. Qu'obtenez-vous ?



Exercice 11.12

Essayez les règles de l'exercice 11.11 avec une autre forme initiale que le triangle et/ou en modifiant l'endroit du segment où se trouvera le nouveau point (par exemple en prenant le tiers du segment plutôt que le milieu).



Exercice 11.13 : la tortue de Logo

Logo est un langage de programmation essentiellement connu pour sa fameuse tortue graphique (qui l'a cantonné à une image faussement puérile). Logo est en fait un langage complet de haut niveau, proche du Lisp, apparu en 1967 au MIT. Pour en savoir plus sur ce langage, voir [http://fr.wikipedia.org/wiki/Logo_\(langage\)](http://fr.wikipedia.org/wiki/Logo_(langage)).

Logo permet de faire des dessins en donnant des ordres à un instrument appelé *tortue*. La tortue laisse une trace (ou pas) de ses déplacements. On peut imaginer qu'elle tient un crayon dans sa bouche.

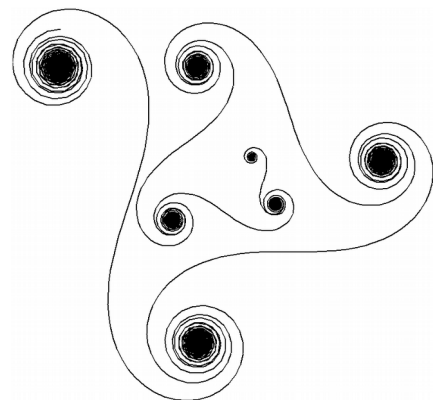
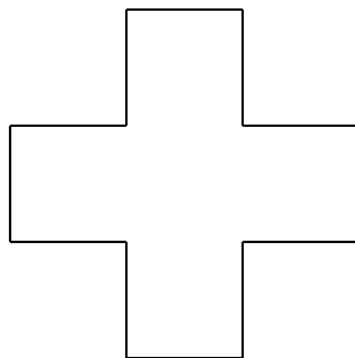
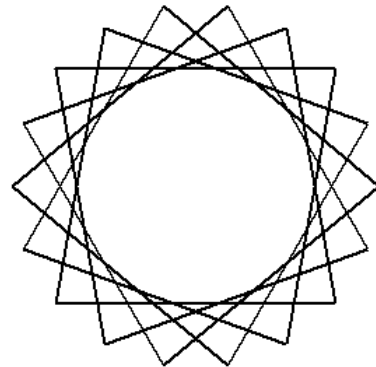
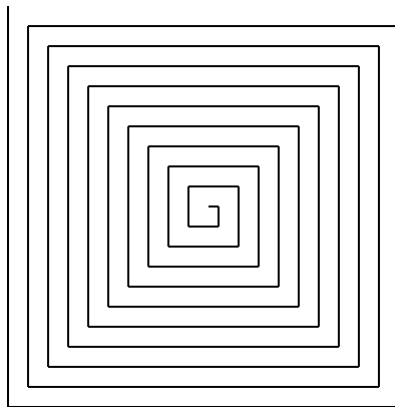
Dans cet exercice, vous allez programmer en Python certaines commandes de la tortue. Le « pré » de la tortue sera un carré de 800 pixels sur 800 pixels. Vous les utiliserez ensuite pour réaliser des figures géométriques.

Commandes à programmer

<code>origine</code>	la tortue est placée au centre du pré, aux coordonnées (400; 400)
<code>fpos(x0,y0)</code>	la tortue est positionnée en (x0; y0)
<code>fcap(angle0)</code>	la tortue s'oriente dans une direction, en degrés. 0° représente l'Est, 90° le Nord, 180° l'Ouest et 270° le Sud.
<code>av(d)</code>	la tortue avance de d pixels
<code>re(d)</code>	la tortue recule de d pixels
<code>td(a)</code>	la tortue tourne sur elle-même de a degrés vers la droite
<code>tg(a)</code>	la tortue tourne sur elle-même de a degrés vers la gauche
<code>lc</code>	lever le crayon (la tortue ne dessinera rien en se déplaçant)
<code>bc</code>	abaisser le crayon (la tortue dessinera en se déplaçant)
<code>cc(nom)</code>	choisir la couleur du crayon, donnée par son nom

Figures à reproduire avec la tortue

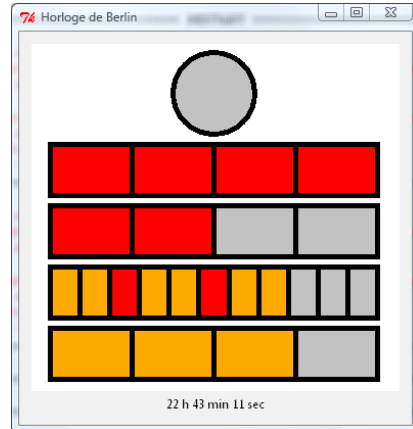
Utilisez les commandes ci-dessus pour reproduire les figures suivantes. Vous pouvez les combiner à des instructions Python, par exemple pour faire des boucles.



La dernière figure est très difficile à reproduire, et pourtant le code n'a que 11 lignes !

**Exercice 11.14 : l'horloge de Berlin**

La « Mengenlehreuhr » ou « horloge de Berlin » est la première horloge publique au monde qui a donné l'heure au moyen de rectangles lumineux. Elle a été inventée par un certain Dieter Binniger et elle fut installée en 1975 à Berlin. Elle se trouve au Europa Center (Wittenbergplatz).



Chaque rectangle allumé indique qu'un certain temps s'est écoulé. Plus précisément :

- chaque lumière de la première ligne (celle du haut) représente 5 heures
- chaque lumière de la deuxième ligne représente 1 heure
- chaque lumière de la troisième ligne représente 5 minutes (les lumières rouges indiquent les quarts d'heure)
- chaque lumière de la dernière ligne représente 1 minute
- le disque au-dessus de l'horloge clignote : il est allumé les secondes paires et éteint les secondes impaires.

Programmez cette horloge de sorte qu'elle indique l'heure actuelle. On la verra donc fonctionner en direct.

Indications

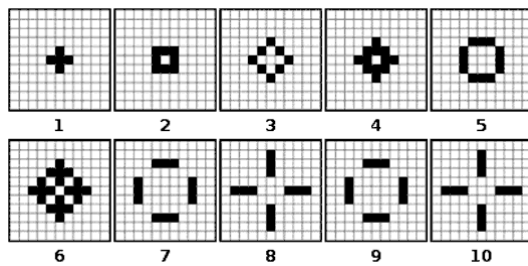
Il faut importer le module `time`.

L'instruction `h,m,s = time.localtime()[3:6]` récupère l'heure, les minutes et les secondes actuelles.

11.6. Ce que vous avez appris dans ce chapitre



- Les couleurs peuvent être désignées par un nom (la liste est longue...), ou par le système RVB.
- Il existe dans le module externe `tkinter` des procédures pour dessiner des segments, des rectangles, des polygones, des ovales et des secteurs. On peut modifier leurs caractéristiques (position, couleur, ...) en cours d'exécution du programme.
- On peut importer des images.



Chapitre 12

Automates cellulaires

12.1. Nouveaux thèmes abordés dans ce chapitre

- animation automatique
- tableaux bidimensionnels
- récursivité

Source :
Wikipédia
[fr.wikipedia.org/
wiki/Jeu_de_la_vie](https://fr.wikipedia.org/wiki/Jeu_de_la_vie)

12.2. Le jeu de la vie

En préambule, il faut préciser que le jeu de la vie n'est pas vraiment un jeu au sens ludique, puisqu'il ne nécessite aucun joueur ; il s'agit d'un **automate cellulaire**, un modèle où chaque état conduit mécaniquement à l'état suivant à partir de règles pré-établies.

Le jeu de la vie fut inventé par John Horton **Conway** en 1970, alors qu'il était professeur de mathématiques à l'université de Cambridge, au Royaume-Uni.

Le premier contact que le grand public eut avec ces travaux se fit en 1970 à travers une publication dans *Scientific American* (et sa traduction française *Pour la Science*) dans la rubrique de Martin **Gardner** : « *Mathematical Games* » ou « *Récréations mathématiques* ».

Gardner écrivait dans ces colonnes que « le Jeu de la Vie rendit Conway rapidement célèbre mais il ouvrit aussi un nouveau champ de recherche mathématique, celui des automates cellulaires. En effet, les analogies du Jeu de la Vie avec le développement, le déclin et les altérations d'une colonie de micro-organismes, le rapprochent des jeux de simulation qui miment les processus de la vie réelle. »

D'après Gardner, Conway expérimenta plusieurs jeux de règles concernant la naissance, la mort et la survie d'une cellule avant d'en choisir un où la population des cellules n'explose pas (ce qui arrive souvent lorsque les conditions de naissances sont moins strictes) mais où des structures intéressantes apparaissent cependant facilement.

Plusieurs structures intéressantes furent découvertes, comme le « **planeur** », ou divers « **canons** » qui génèrent un flux sans fin de planeurs. Ces possibilités augmentèrent l'intérêt pour le jeu de la vie. De plus, arrivant à une époque où une nouvelle génération de mini-ordinateurs meilleur marché fut commercialisée, ce qui permettait de tester des structures pendant la nuit, lorsque personne d'autre ne les utilisait, sa popularité augmenta d'autant.

Vers la fin des années 1980, la puissance des ordinateurs fut suffisante pour permettre la création de programmes de recherche de structures automatiques efficaces ; couplés au développement massif d'Internet, ils conduisirent à un renouveau dans la production de structures intéressantes.



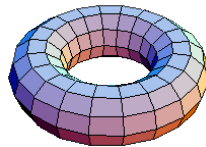
John **Conway**
en 1987

Voir exercices
12.2 et 12.3

Règles

Le jeu se déroule sur une grille à deux dimensions, théoriquement infinie (mais de longueur et de largeur finies et plus ou moins grandes dans la pratique), dont les cases — qu'on appelle des « cellules », par analogie avec les cellules vivantes — peuvent prendre deux états distincts : « vivantes » ou « mortes ».

Ici, nous travaillerons sur un tore : il n'y aura pas de bords. Sur notre grille, la première colonne sera voisine de la dernière et la première ligne sera voisine de la dernière.



À chaque étape, l'évolution d'une cellule (la case bleue ci-dessous) est entièrement déterminée par l'état de ses huit voisins de la façon suivante (les cellules vivantes sont les cases noires) :

	Si une cellule a moins de deux voisines vivantes, elle mourra à l'étape suivante.
	Si une cellule a plus de trois voisines vivantes, elle mourra à l'étape suivante.
	Si une cellule morte a exactement trois cellules vivantes comme voisines, elle renaîtra à l'étape suivante.
	Si une cellule (morte ou vivante) a exactement deux voisines vivantes, elle restera dans son état actuel à l'étape suivante.

12.3. Code du programme



jeudelavie.py

<http://ow.ly/cnx4x>

```
# Jeu de la vie de Conway
from tkinter import *
from random import randrange

haut = 30 # hauteur du tableau
larg = 30 # largeur du tableau
cote = 15 # côté d'une cellule
vivant = 1
mort = 0

# Créer les matrices
cell = [[0 for row in range(haut)] for col in range(larg)]
etat = [[mort for row in range(haut)] for col in range(larg)]
temp = [[mort for row in range(haut)] for col in range(larg)]
```

```

# Calculer et dessiner le prochain tableau
def tableau():
    calculer()
    dessiner()
    fenetre.after(100, tableau)

# Données initiales
def init():
    for y in range(haut):
        for x in range(larg):
            etat[x][y] = mort
            temp[x][y] = mort
            cell[x][y] = canvas.create_rectangle((x*cote, y*cote,
                                                    (x+1)*cote, (y+1)*cote), outline="gray", fill="white")
    # placer au hasard environ 25% de cellules vivantes
    for i in range(larg*haut//4):
        etat[randrange(larg)][randrange(haut)] = vivant

# Appliquer les 4 règles
def calculer():
    for y in range(haut):
        for x in range(larg):
            # Règle 1 - Mort de solitude
            if etat[x][y] == vivant and nb_voisins < 2:
                temp[x][y] = mort
            # Règle 2 - Toute cellule avec 2 ou 3 voisins survit.
            if etat[x][y] == vivant and (nb_voisins == 2 or nb_voisins == 3):
                temp[x][y] = vivant
            # Règle 3 - Mort par asphyxie
            if etat[x][y] == vivant and nb_voisins > 3:
                temp[x][y] = mort
            # Règle 4 - Naissance
            if etat[x][y] == mort and nb_voisins == 3:
                temp[x][y] = vivant
    for y in range(haut):
        for x in range(larg):
            etat[x][y] = temp[x][y]

# Compter les voisins vivants - tableau torique
def voisins_vivants_tore(a,b):
    nb_voisins = 0
    if etat[(a-1)%larg][(b+1)%haut] == 1:
        nb_voisins += 1
    if etat[a][(b+1)%haut] == 1:
        nb_voisins += 1
    if etat[(a+1)%larg][(b+1)%haut] == 1:
        nb_voisins += 1
    if etat[(a-1)%larg][b] == 1:
        nb_voisins += 1
    if etat[(a+1)%larg][b] == 1:
        nb_voisins += 1
    if etat[(a-1)%larg][(b-1)%haut] == 1:
        nb_voisins += 1
    if etat[a][(b-1)%haut] == 1:
        nb_voisins += 1
    if etat[(a+1)%larg][(b-1)%haut] == 1:
        nb_voisins += 1
    return nb_voisins

# Dessiner toutes les cellules
def dessiner():
    for y in range(haut):
        for x in range(larg):
            if etat[x][y]==0:
                coul = "white"
            else:
                coul = "blue"
            canvas.itemconfig(cell[x][y], fill=coul)

# Lancement du programme
fenetre = Tk()
fenetre.title("Le jeu de la vie de Conway")
canvas = Canvas(fenetre, width=cote*larg, height=cote*haut, highlightthickness=0)
canvas.pack()

```

```
init()
tableau()
fenetre.mainloop()
```

12.4. Analyse du programme

```
from random import randrange
```

La fonction `randrange()` correspond à un tirage au hasard dans la liste d'entiers qui serait générée par la fonction `range` avec les mêmes paramètres. Par exemple `range(4)` génère la liste `[0,1,2,3]` et `randrange(4)` tire un élément au hasard dans la liste `[0,1,2,3]`.

Attention ! Les méthodes `randrange()` et `randint()` ont un paramétrage différent :

```
randrange(8) = randint(0, 7)
```



```
# Créer les matrices
cell = [[0 for row in range(haut)] for col in range(larg)]
etat = [[mort for row in range(haut)] for col in range(larg)]
temp = [[mort for row in range(haut)] for col in range(larg)]
```

Trois matrices (tableaux bidimensionnels) seront utilisées : `cell` mémorisera les carrés qui représentent les cellules, `etat` mémorisera les statuts des cellules (mort ou vivant), `temp` contiendra la nouvelle matrice d'états.

Pourquoi faut-il deux matrices pour calculer les changements d'états ? Parce que tous les changements sont **simultanés**. Si l'on utilisait que la matrice `etat`, les éventuels changements des cellules seraient influencés par les changements des cellules précédentes : l'ordre des changements d'états aurait de l'importance. Avec deux matrices, on n'a plus ce problème : on calcule la nouvelle grille en regardant l'ancienne, sans modifier cette dernière. Une fois que tous les statuts ont été mis à jour, on recopie la matrice `temp` dans la matrice `etat` (voir procédure `calculer()`).

```
# Calculer et dessiner le prochain tableau
def tableau():
    calculer()
    dessiner()
    fenetre.after(100, tableau)
```

Une nouveauté se trouve tout à la fin de la fonction `tableau()` : vous y noterez l'utilisation de la méthode `after()`. Elle déclenche l'appel d'une fonction après qu'un certain laps de temps se soit écoulé. Ainsi par exemple, `fenetre.after(100, qqc)` déclenche pour le widget `fenetre` un appel de la fonction `qqc()` après une pause de 100 millisecondes.

Dans notre script, la fonction qui est appelée par la méthode `after()` est la fonction `tableau()` elle-même. Nous utilisons donc ici pour la première fois une technique de programmation très puissante, que l'on appelle **récurtivité**. Pour faire simple, nous dirons que la récursivité est ce qui se passe lorsqu'une fonction s'appelle elle-même. On obtient bien évidemment ainsi un bouclage, qui peut se perpétuer indéfiniment si l'on ne prévoit pas aussi un moyen pour l'interrompre.

Voyons comment cela fonctionne dans notre exemple.

La fonction `tableau()` est invoquée une première fois à l'avant-dernière ligne du programme. Elle effectue son travail. Ensuite, par l'intermédiaire de la méthode `after()`, elle s'invoque elle-même après une petite pause. Elle repart donc pour un second tour, puis s'invoque elle-même à nouveau, et ainsi de suite, indéfiniment... Nous aurons donc une boucle infinie que l'on ne pourra stopper qu'en fermant la fenêtre.

La suite du programme ne présente pas de difficultés.



Exercice 12.1

Ajoutez au programme du § 12.3 quatre boutons permettant de :

- démarrer la simulation
- stopper la simulation
- suivre l'animation pas à pas (il faudra presser sur ce bouton pour que l'animation passe à

- l'étape suivante)
• quitter le programme

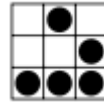
Indication

Pour stopper ou recommencer la simulation, il faudra utiliser une variable globale `flag` qui vaudra 0 pour arrêter la simulation, 1 pour la démarrer, et 2 pour ne faire qu'une étape.



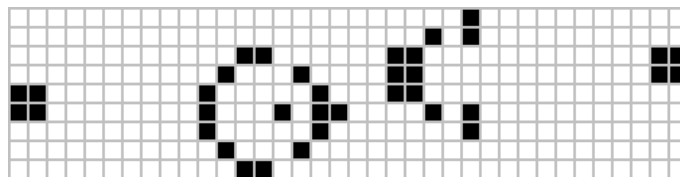
Exercice 12.2

Modifiez le programme de l'exercice 12.1 pour observer le comportement de ce « vaisseau » :



Exercice 12.3

Modifiez le programme de l'exercice 12.1 pour observer le comportement de ce « canon » :



Prenez une grille assez grande (60 x 60 par exemple) et mettez une « marge » de deux carrés autour du canon.



Exercice 12.4

Le petit script ci-dessous vous montre comment détecter un clic sur le bouton gauche de la souris <Button-1>, et comment récupérer les coordonnées *x* et *y* du pointeur.

```
from tkinter import *

def rond(event):
    x, y = event.x, event.y
    can.create_oval(x-3,y-3,x+3,y+3, fill='red')

fen = Tk()
can = Canvas(fen, width=200, height=200, bg="white")
can.bind("<Button-1>", rond)
can.pack()
fen.mainloop()
```



clic.py

<http://ow.ly/enx4x>

Modifiez le programme de l'exercice 12.1 pour que l'on puisse placer les cellules vivantes grâce à la souris, en vous inspirant du script ci-dessus. Plus précisément, quand on cliquera sur une case du tableau, on veut qu'elle change d'état : de morte elle deviendra vivante et vice-versa.



Exercice 12.5

Modifiez l'exercice 12.4 pour observer le « *Replicator* ». Les règles de cet automate sont : une cellule naît ou reste vivante si, parmi ses huit cellules voisines, un nombre impair sont vivantes. Autrement, elle ne naît pas, ou meurt.

Afin de bien voir ce qui se passe, prenez un tableau 75 x 75 avec des côtés de cellule de longueur 10. Prévoyez aussi un nouveau bouton pour tout effacer.

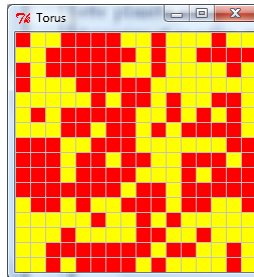
Dessinez un motif au milieu du tableau et lancez la simulation.

12.5. Évolution des opinions sur la planète Torus

Cet automate est décrit dans un article de A. Dewdney : **Le hasard simulé**, Pour la Science - Dossier Hors Série, Avril 1996, pp. 88-90

Sur la planète *Torus*, planète qui, comme son nom l'indique, est en forme de tore, 256 personnes occupent chacune un territoire. Les habitants sont partagés entre deux courants politiques, le parti rouge et le parti jaune. Au début, les opinions politiques sont réparties au hasard sur la planète.

Une particularité des habitants de cette planète est qu'ils sont très influençables. Ainsi, chaque jour, un habitant va trouver un de ses huit voisins et se rallie à ses convictions politiques.



Carte géopolitique de la planète Torus

12.6. Code du programme



torus.py

<http://ow.ly/cnx4x>

```
# Evolution des opinions politiques sur la planète Torus

from tkinter import *
from random import randrange

haut = 16 # hauteur du tableau
larg = 16 # largeur du tableau
cote = 15 # côté d'une cellule
rouge = 0
jaune = 1
nb_rouges = haut*larg
nb_jaunes = 0

# Créer les matrices
cell = [[0 for row in range(haut)] for col in range(larg)]
parti = [[rouge for row in range(haut)] for col in range(larg)]

# Dessiner toutes les cellules
def dessiner():
    for y in range(haut):
        for x in range(larg):
            if parti[x][y]==rouge:
                coul = "red"
            else:
                coul = "yellow"
            canvas.itemconfig(cell[x][y], fill=coul)

# Données initiales
def init():
    global nb_rouges, nb_jaunes
    for y in range(haut):
        for x in range(larg):
            parti[x][y] = rouge # tout le monde est rouge au départ
            cell[x][y] = canvas.create_rectangle((x*cote, y*cote,
                                                    (x+1)*cote, (y+1)*cote), outline="gray", fill="red")

    # placer au hasard 50% d'opinions jaunes
    while nb_jaunes<nb_rouges:
        x = randrange(larg)
        y = randrange(haut)
        if parti[x][y] == rouge:
            parti[x][y] = jaune
            nb_jaunes += 1
            nb_rouges -= 1
```



```

        nb_jaunes += 1
        nb_rouges -= 1
    dessiner()

# Choisir l'opinion d'un des 8 voisins de la cellule (a,b) - tableau torique
def opinion_voisin(a,b):
    voisin = randrange(8)
    if voisin==1:
        opinion = parti[(a-1)%larg][(b+1)%haut]
    elif voisin==2:
        opinion = parti[a][(b+1)%haut]
    elif voisin==3:
        opinion = parti[(a+1)%larg][(b+1)%haut]
    elif voisin==4:
        opinion = parti[(a-1)%larg][b]
    elif voisin==5:
        opinion = parti[(a+1)%larg][b]
    elif voisin==6:
        opinion = parti[(a-1)%larg][(b-1)%haut]
    elif voisin==7:
        opinion = parti[a][(b-1)%haut]
    else:
        opinion = parti[(a+1)%larg][(b-1)%haut]
    return opinion

# Appliquer la règle
def calculer():
    global nb_rouges, nb_jaunes
    x = randrange(larg)
    y = randrange(haut)
    nouvelle_opinion = opinion_voisin(x,y)
    if parti[x][y] != nouvelle_opinion:
        if nouvelle_opinion == rouge:
            nb_rouges += 1
            nb_jaunes -= 1
        else :
            nb_rouges -= 1
            nb_jaunes += 1
        parti[x][y] = nouvelle_opinion
        if parti[x][y] == rouge:
            coul = "red"
        else:
            coul = "yellow"
        canvas.itemconfig(cell[x][y], fill=coul)

# Calculer et dessiner le prochain tableau
def tableau():
    calculer()
    fenetre.after(1, tableau)

# Lancement du programme
fenetre = Tk()
fenetre.title("Torus")
canvas = Canvas(fenetre, width=cote*larg, height=cote*haut, highlightthickness=0)
canvas.pack()
init()
tableau()
fenetre.mainloop()

```

12.7. Analyse du programme

Ce programme est directement inspiré du programme du § 12.3. Nous allons seulement nous attarder sur la procédure `init()`.

```

def init():
    global nb_rouges, nb_jaunes
    for y in range(haut):
        for x in range(larg):
            parti[x][y] = rouge # tout le monde est rouge au départ
            cell[x][y] = canvas.create_rectangle((x*cote, y*cote,

```

```

(x+1)*cote, (y+1)*cote), outline="gray", fill="red")
# placer au hasard 50% d'opinions jaunes
while nb_jaunes < nb_rouges:
    x = randrange(larg)
    y = randrange(haut)
    if parti[x][y] == rouge:
        parti[x][y] = jaune
        nb_jaunes += 1
        nb_rouges -= 1
dessiner()

```

Pour avoir au début de la simulation autant d'habitants rouges que de jaunes, nous avons d'abord attribué la couleur rouge à tout le monde, puis nous avons peint en jaune certains rouges pris au hasard, jusqu'à ce qu'il y ait autant de jaunes que de rouges (à un près si le nombre de cases est impair).

Si l'on n'a vraiment pas de chance, cette méthode peut durer un certain temps... En effet, si l'on veut peindre en jaune un habitant jaune, il ne se passera rien.



Exercice 12.6

Imaginez et programmez une autre manière de faire pour que la population de Torus soit partagée en deux partis politiques avec une répartition moitié rouge moitié jaune.



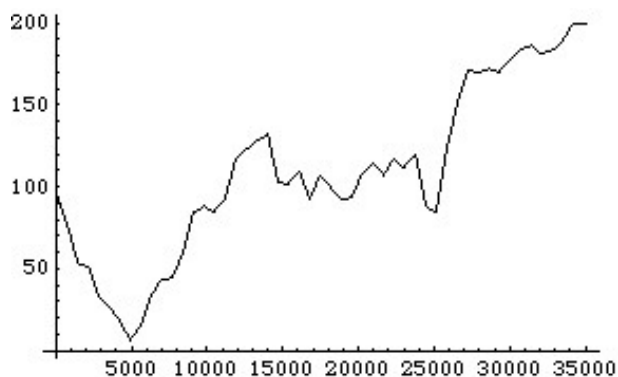
Exercice 12.7

Modifiez le programme du § 12.6 afin de pouvoir dessiner un graphe représentant l'évolution de la population rouge dans le temps. Pour cela, sauvegarder l'évolution au jour le jour dans un fichier.

Vous pourrez ensuite :

- soit utiliser un tableur pour lire ce fichier et faire un graphique
- soit écrire en Python un programme qui fera un graphique à partir des données lues dans ce fichier.

Le graphe devra ressembler à ceci :



On aura en abscisse les jours et en ordonnée la population des rouges.



Exercice 12.8

Faites des expériences pour réfléchir aux questions suivantes. Que se passe-t-il si...

- on change les valeurs de `larg` et `haut` (largeur et hauteur du tableau) ?
- on change le pourcentage de rouges dans l'état initial ?
- on modifie la répartition géographique des couleurs au départ ?
- on prend plus de deux couleurs au départ ?
- on change la fonction de voisinage ?
- on change la forme de la planète (rectangle avec bords, cylindre, ruban de Möbius, ...) ?



Exercice 12.9 - Modèle de Schelling pour la ségrégation urbaine

Dans une ville composée de plusieurs groupes sociaux ou communautés (ethniques, religieuses, économiques...), le modèle de **Schelling** montre comment un comportement de ségrégation spatiale peut apparaître sans être forcément le résultat de comportements individuels délibérément ségrégatifs ou racistes. En effet, il montre que même si tous les individus ont un seuil élevé de tolérance concernant la présence « d'étrangers à leur groupe » dans leur voisinage, on voit néanmoins émerger, avec le temps, une séparation, une ségrégation socio-spatiale, qui se traduit par l'apparition de quartiers beaucoup plus homogènes que la tolérance individuelle ne le laissait supposer.

C'est donc un exemple à la fois simple et significatif de la notion d'émergence dans un système complexe. Même si la réalité est très différente, ce modèle montre quand même, sans forcément tomber dans un fatalisme malsain, que le tout (c'est-à-dire le comportement collectif) ne se déduit pas simplement des règles du comportement individuel.



Thomas C.
Schelling (1921-)

Prix Nobel
d'économie en
2005

Description

Les cellules (au nombre de 10'000) représentent chacune une habitation de la ville, elle-même représentée par un damier 100 x 100. La ville est composée de trois communautés notées *A* (en rouge), *B* (en bleu) et *C* (en vert). On a donc trois états possibles pour une cellule habitée : *A*, *B* ou *C*. Lorsqu'une maison n'est pas habitée, la cellule est dans l'état *L* (libre) de couleur grise.

La configuration initiale résulte d'un tirage aléatoire pour chaque cellule d'un état parmi les quatre états possibles (*A*, *B*, *C*, ou *L*) qui donne donc un poids quasi égal entre les communautés.

Les règles de transition sont très simples et au nombre de deux.

- **Règle n°1** : si une cellule est libre, une famille appartenant à n'importe laquelle des trois communautés, *A*, *B* ou *C* peut s'y installer, avec une chance égale. L'installation d'une famille n'est pas liée à la libération d'une autre cellule, elle est considérée comme venant de l'extérieur. Néanmoins le modèle possède un seuil maximum de 99% de remplissage de manière à garder un certain dynamisme au système (donc 100 cases libres au minimum).
- **Règle n°2** : si une famille habitant une cellule donnée est entourée de plus de 70% d'étrangers à son groupe, alors elle déménage et libère la cellule. Suivant son emplacement sur la grille, une case peut avoir 3, 5 ou 8 voisins. Le déménagement n'est pas lié à un emménagement immédiat dans une autre cellule (on peut donc considérer que la famille va à l'extérieur).

Remarques

Ce modèle est légèrement différent du modèle de Schelling original. En effet le modèle de Schelling ne gère que deux types d'individus (noirs et blancs), de plus, il procède de manière synchrone (toutes les cellules changent en même temps) et enfin, chaque déménagement est immédiatement suivi d'un relogement ailleurs. Ici la simulation est asynchrone aléatoire, c'est-à-dire que les cellules changent d'état les unes après les autres par un choix totalement aléatoire et indépendant des tirages précédents.

Travail

Programmez ce modèle.

Étudiez ensuite ce modèle en faisant varier le taux de tolérance d'étrangers autour de chez soi.



Exercice 12.10 – Le gasp

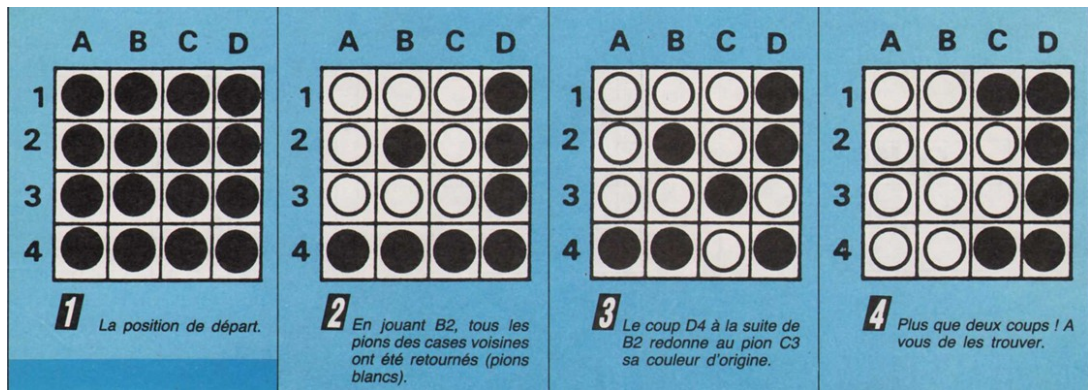
Le Gasp est un casse-tête apparu dans la revue *Jeux & Stratégie* n°38 (avril/mai 1986).

Sur un damier $n \times n$, on place n^2 pions bicolores. Au départ, tous les pions sont noirs. Le but est de retourner ces pions pour qu'ils deviennent tous blancs.

Les cases centrales ont 8 voisines, celles du bord 5 voisines, tandis que celles des coins n'en ont que 3.

Pour retourner des pions, on désigne une case. Ses voisines changent de couleur (mais pas la case désignée).

Le schéma ci-dessous, tiré de *Jeux & Stratégie*, montre un début de partie.



Programmez ce casse-tête en Python. Inspirez-vous de l'exercice 12.4. On veut pouvoir désigner les cases avec le bouton gauche de la souris.

Essayez ensuite de résoudre un Gasp 4 x 4, 6 x 6, 8 x 8, 10 x 10, 12 x 12 ! Et pourquoi ne pas écrire un programme qui cherche ces solutions ?



Une solution possible pour un Gasp 4 x 4.
L'ordre des coups n'a pas d'importance.



12.8. Ce que vous avez appris dans ce chapitre

- Vous avez vu pour la première fois une fonction récursive (§ 12.4). Nous nous attarderons un peu plus sur ce sujet au chapitre suivant. La récursivité, associée à la méthode `after()`, permet de créer une animation automatique.
- Bien que mus par des règles très simples, les automates cellulaires peuvent avoir des comportements complexes, et même plus ou moins modéliser la vie réelle : on peut les utiliser pour simuler un feu de forêt ou la formation d'un tas de sable, par exemple.
- Vous avez vu comment utiliser le bouton droit de la souris et récupérer les coordonnées de l'emplacement du curseur.



Chapitre 13

Démineur

13.1. Nouveaux thèmes abordés dans ce chapitre

- Récursivité (première approche)
- Frames multiples
- Boutons radio

13.2. Exploration d'un labyrinthe

Pour introduire la notion de récursivité, nous allons, avant de nous occuper du jeu du démineur, voir comment explorer un labyrinthe.

Voici ci-dessous un labyrinthe (les « X » sont des murs), avec une case d'entrée notée « E » et une sortie notée « S ».

```
XXXXXX
X X S
X X
XEX X
X X X
XXXXXX
```

Le programme ci-dessous lit un fichier contenant le plan d'un labyrinthe. Il parcourt toutes les cases vides à partir de la position courante en les marquant avec un « . », afin d'éviter de tourner en rond. Il essaie successivement les directions *droite*, *gauche*, *bas* puis *haut*. C'est ce que fait la procédure **récursive** `thesee` (l'adjectif récursif signifie qu'elle s'appelle elle-même).



`labyrinthe.py`

<http://ow.ly/QBK1n>

```
# Exploration récursive d'un labyrinthe

def lire_labyrinthe():
    # met en mémoire la labyrinthe
    # et renvoie les coordonnées du point de départ
    global case, haut, larg
    fichier = open("dedale.txt", "r")
    lignes = fichier.readlines()
    fichier.close()
    haut = len(lignes)
    larg = len(lignes[0]) - 1
    print(haut, "lignes", larg, "colonnes")
    case = [[0 for col in range(larg)] for row in range(haut)]
```

```

    for i in range(haut):
        for j in range(larg):
            case[i][j]=lignes[i][j]
            if lignes[i][j]=='E':
                i0=i
                j0=j
                case[i0][j0]="E"
    return i0,j0

def imprimer_labyrinthe():
    global case, haut, larg
    for i in range(haut):
        for j in range(larg):
            print(case[i][j],end=" ")
        print()
    print()

def thesee(i,j):
    global case
    if case[i][j] == "S":
        imprimer_labyrinthe()
    elif case[i][j] == " " or case[i][j] == "E":
        case[i][j] = "."
        thesee(i,j+1)
        thesee(i,j-1)
        thesee(i+1,j)
        thesee(i-1,j)
        case[i][j] = " "

# programme principal

i0, j0 = lire_labyrinthe()
imprimer_labyrinthe()
print("Solution(s)")
thesee(i0, j0)

```

La récursivité permet de résoudre certains problèmes d'une manière très rapide, alors que si on devait les résoudre de manière itérative, il nous faudrait beaucoup plus de temps et de structures de données intermédiaires.

La récursivité utilise toujours la pile du programme en cours. On appelle **pile** une zone mémoire réservée à chaque programme. Son rôle est de stocker les variables locales et les paramètres d'une procédure. Supposons que nous sommes dans une procédure `proc1` dans laquelle nous avons des variables locales. Faisons ensuite appel à une procédure `proc2`; comme le microprocesseur va commencer à exécuter `proc2` mais qu'ensuite il reviendra continuer l'exécution de `proc1`, il faut bien stocker quelque part les variables de la procédure en cours `proc1`; c'est le rôle de la pile. Tout ceci est géré de façon transparente pour l'utilisateur. Dans une procédure récursive, toutes les variables locales sont stockées dans la pile, et empilées autant de fois qu'il y a d'appels récursifs. Donc la pile se remplit progressivement, et si on ne fait pas attention on peut arriver à un *débordement de pile* (*stack overflow* en anglais). Ensuite, les variables sont désempilées.

Dans notre exemple du labyrinthe, les variables seront les coordonnées des cases du labyrinthe. Chaque fois qu'une case sera marquée, elle sera mise dans la pile. Quand la marque sera effacée, la case sera désempilée. C'est l'idée-clé pour bien comprendre la récursivité.

Pour voir en détails ce que fait le programme, mettons des coordonnées aux cases et affichons toutes les cases parcourues, dans l'ordre :

01234	
0XXXXX	
1X X S	S se trouve aux coordonnées (1 ; 4)
2X X	
3XEX X	E se trouve aux coordonnées (3 ; 1)
4X X X	
5XXXXX	
 3 1	case de départ ; on marque la case avec un « . »
3 2	on essaie à droite : c'est un mur
3 0	on essaie à gauche : c'est un mur

```

4 1      on se déplace en bas ; on marque la case (4 ; 1)
4 2      on essaie à droite : c'est un mur
4 0      on essaie à gauche : c'est un mur
5 1      on essaie en bas : c'est un mur
3 1      on a tout essayé pour la case (4; 1) ; on remonte et on efface la marque de (4 ; 1),
          car on arrive sur une case marquée
2 1      la case (3 ; 1) est marquée, donc on reprend où on en était : on monte et on marque
          la case (2 ; 1)
2 2      on essaie à droite : ça passe ; on marque la case (2 ; 2)
2 3      on essaie à droite : ça passe ; on marque la case (2 ; 3)
2 4      on essaie à droite : c'est un mur
2 2      on essaie à gauche : la case est marquée
3 3      on se déplace en bas ; on marque la case (3 ; 3)
3 4      on essaie on essaie à droite : c'est un mur
3 2      on essaie on essaie à gauche : c'est un mur
4 3      on se déplace en bas ; on marque la case (4 ; 3)
4 4      on essaie à droite : c'est un mur
4 2      on essaie à gauche : c'est un mur
5 3      on essaie en bas : c'est un mur
3 3      on a tout essayé pour la case (4; 3) ; on efface la marque et on retourne à la case
          du sommet de la pile : (3 ; 3)
2 3      la case (3 ; 3) est marquée, donc on reprend où on en était : on monte, et on efface
          la marque de (3 ; 3), car on arrive sur une case marquée
1 3      la case (2 ; 3) est marquée, donc on reprend où on en était : on monte. On marque
          la case (1 ; 3)
1 4 !    on essaie à droite : c'est la sortie !

```

Le programme a trouvé un chemin. Mais cela n'est pas fini... On n'a pas exploré entièrement le labyrinthe. Il y a peut-être d'autres chemins...

```

XXXXXX
X X . S
X . . . X
X . X X
X X X
XXXXXX

```

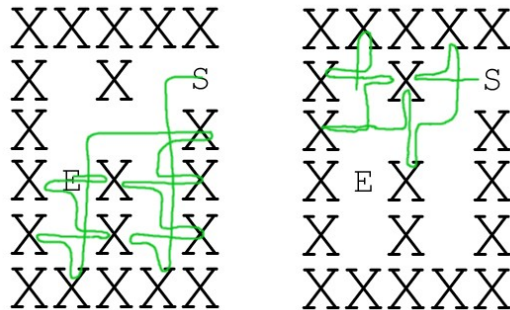
```

1 2      on essaie à gauche : c'est un mur
2 3      on essaie en bas : la case est marquée
0 3      on essaie en haut : c'est un mur
2 1      on a tout essayé pour la case (1; 3) ; on efface la marque et on retourne à la case
          du haut de la pile : (2 ; 3). Mais cette case est marquée, donc on reprend où on
          en était. Or, on a tout essayé pour la case (2; 3) ; on efface la marque et on retourne
          à la case du haut de la pile : (2 ; 2). Mais cette case est marquée, donc on
          reprend où on en était : on essaie à gauche : la case est marquée
3 2      on essaie en bas : c'est un mur
1 2      on essaie en haut : c'est un mur
2 0      on a tout essayé pour la case (2; 2) ; on efface la marque et on retourne à la case
          du sommet de la pile : (2 ; 1). Mais cette case est marquée, donc on reprend où on
          en était. On essaie à gauche : c'est un mur
3 1      on essaie en bas : la case est marquée
1 1      on se déplace en haut ; on marque cette case
1 2      on essaie à droite : c'est un mur
1 0      on essaie à gauche : c'est un mur
2 1      on essaie en bas : la case est marquée
0 1      on essaie en haut: c'est un mur. on a tout essayé pour la case (1; 1) ; on efface la
          marque et on retourne à la case du haut de la pile : (2 ; 1). Mais cette case est

```

marquée, donc on reprend où on en était. Or, on a tout essayé pour la case (2; 1) ; on efface la marque et on retourne à la case du haut de la pile : (3 ; 1). Mais cette case est marquée, donc on reprend où on en était. Or, on a tout essayé pour la case (3; 1).

STOP. La pile est vide. On est donc passé par toutes les cases vides et on a essayé toutes les directions à partir d'elles. Le schéma ci-dessous résume les chemins parcourus, avant et après avoir trouvé la sortie.



Voici un exemple plus complexe avec trois chemins possibles de E à S :



dedale2.txt

<http://ow.ly/QBK1n>

Donnée

```

XXXXXXXXXXXXXXXXXXXXXXXXX
XEX X X XX X
X X X XXXXX XXXXX XXXX X
X X X X X X XXX S
X X X X XXX XXX XXX X
XXXXX X X XXX X XX XX X
X X X X X
XXX XX XXXXXX XXXX XXXX X
X X X X
XXXXXXXXXXXXXXXXXXXXXXXXX
    
```

Solutions

```

XXXXXXXXXXXXXXXXXXXXXXXXX
X.X X X XX X
X.X X XXXXX XXXXX XXXX X
X.X X X X X XXX S
X....X X X XXX XXX XXX X
XXXXX.X X...XXX X XX XX X
X X X X X
XXX XX XXXXXX.XXXX.XXXX X
X X X X
XXXXXXXXXXXXXXXXXXXXXXXXX
    
```

```

XXXXXXXXXXXXXXXXXXXXXXXXX
X.X X X .....XX X
X.X X XXXXX.XXXXXX.XXXX X
X.X X X X X.X .....XXX S
X....X X X.XXX.XXX XXX X
XXXXX.X X...XXX.X XX XX X
X X X X
XXX XX XXXXXX.XXXX.XXXX X
X X X X
XXXXXXXXXXXXXXXXXXXXXXXXX
    
```

```

XXXXXXXXXXXXXXXXXXXXXXXXX
X.X X X XX X
X.X X XXXXX XXXXX XXXX X
X.X X X X X XXX S
X....X X X XXX XXX XXX X
XXXXX.X X XXX X XX XX X
X X X X
XXX XX XXXXXX.XXXX.XXXX X
X X X X
XXXXXXXXXXXXXXXXXXXXXXXXX
    
```




Exercice 13.1

Essayez de comprendre le fonctionnement de ce programme en l'exécutant « à la main » sur ce petit labyrinthe :

```
XXXXXX
XE  X
X X S
X  X
XXXXXX
```



Exercice 13.2

Modifiez le programme Python présenté ci-dessus de telle manière que l'on puisse suivre le cheminement dans le labyrinthe : « droite », « gauche », « bas », « haut ».

Affichez aussi le labyrinthe chaque fois que la procédure `Thesee` est appelée, afin de suivre l'évolution des marques.

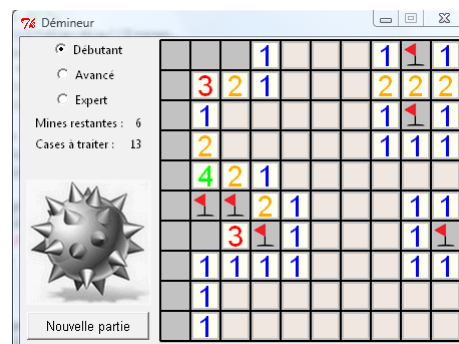
Explorez ensuite le labyrinthe de l'ex. 13.1. et vérifiez que vous avez bien compris le fonctionnement du programme.

13.3. Règles du jeu du démineur

Le *démineur* est un jeu de réflexion. Il faut trouver par déduction l'emplacement de mines dans un quadrillage. En cliquant sur une case, il y a deux possibilités : soit c'est une mine (et on a perdu la partie), soit c'est du terrain déminé. Si la case choisie est bordée par une mine, un chiffre entre 1 et 8 apparaît. Ce chiffre indique le nombre de mines adjacentes à cette case. Par adjacent, on entend qui touche une case soit par un bord, soit par un coin.

Si ce n'est pas une mine et qu'il n'y a aucune mine dans les cases adjacentes, le chiffre 0 n'apparaît pas, mais on découvre toute la zone sans mine délimitée par des mines ou le bord de la grille. **C'est cette partie qui est la plus difficile à programmer, et qui nous intéressera particulièrement dans ce chapitre.**

Avec le bouton droit de la souris, on peut marquer une case avec un drapeau ou un point d'interrogation. Cela sert d'aide mémoire et n'a pas d'influence sur la fin de la partie.



13.4. Code du programme



demineur.py

<http://ow.ly/QBK1n>

```
# Démineur
from tkinter import *
import random, sys

# Dessine la grille de jeu
def grille(nb_col, nb_lignes, dim, origine):
    xl= origine
```

```

y1= origine
# Détermine la largeur de la grille
y2 = y1 + (dim*nb_lignes)
# Détermine la hauteur de la grille
x2 = x1 + (dim*nb_col)
colonne = 0
while colonne <= nb_col:
    colonne=colonne+1
    # Création de la ligne verticale
    can.create_line(x1,y1,x1,y2,width=2,fill="black")
    # Décalage de la ligne vers la droite
    x1 = x1 + dim
x1 = origine
ligne = 0
while ligne <= nb_lignes:
    ligne=ligne+1
    # Création de la ligne horizontale
    can.create_line(x1,y1,x2,y1,width=2,fill="black")
    # Décalage de la ligne vers le bas
    y1 = y1 + dim

# Initialise le niveau de jeu
def init_niveau():
    global nb_col, nb_lig, nb_mines
    niveau = choix.get()
    # niveau débutant
    if niveau == 1:
        nb_col, nb_lig, nb_mines = 10, 10, 12
    # niveau avancé
    elif niveau == 2:
        nb_col, nb_lig, nb_mines = 15, 15, 30
    # niveau expert
    else:
        nb_col, nb_lig, nb_mines = 20, 20, 50
    # taille du canevas pour chaque niveau
    can.configure(width=(nb_col*dim)+gap, height=(nb_lig*dim)+gap)
    init_jeu()

# Initialisation des paramètres du jeu
def init_jeu():
    global nb_mines_cachees, nb_cases_vues, on_joue
    on_joue = True
    nb_cases_vues = 0
    can.delete(ALL)
    nb_mines_cachees = nb_mines
    affiche_compteurs()
    # Initialisation des 2 tableaux avec des chaines vides
    y = 0
    while y < nb_lig:
        x = 1
        y += 1
        while x <= nb_col:
            tab_m[x,y]= 0 # Initialisation dans le tableau des mines
            tab_j[x,y]= "" # Initialisation dans le tableau de jeu
            can.create_rectangle((x-1)*dim+gap, (y-1)*dim+gap,
                                x*dim+gap, y*dim+gap, width=0, fill="grey")
            x += 1
    grille(nb_col, nb_lig, dim, gap) # Dessine la grille
    # place les mines aléatoirement dans la grille de jeu
    nb_mines_voisines = 0
    while nb_mines_voisines < nb_mines:
        col = random.randint(1, nb_col)
        lig = random.randint(1, nb_lig)
        if tab_m[col, lig] != 9: # Vérifie si la cellule contient déjà une mine
            tab_m[col, lig] = 9
            nb_mines_voisines = nb_mines_voisines + 1

# calcule le nombre de mines qu'il reste à trouver
def affiche_compteurs():
    decomp_mines.configure(text=str(nb_mines_cachees))
    # Décompte des cases à traiter pour la fonction gagnée
    decomp_cases.configure(text=str((nb_col*nb_lig)-nb_cases_vues))

```

```

# affiche le nombre de mines restantes
def affiche_nb_mines(nb_mines_voisines, col, lig):
    global nb_mines_cachees, nb_cases_vues
    # si la case est vide
    if tab_j[col, lig] == "":
        nb_cases_vues = nb_cases_vues + 1
        # S'il y a un drapeau : modification du compteur de mines
        if (tab_j[col, lig] == "d"):
            # Ajout d'une mine
            nb_mines_cachees = nb_mines_cachees + 1
            # le drapeau est considéré comme une case vue
            nb_cases_vues = nb_cases_vues - 1
            affiche_compteurs()
        tab_j[col, lig] = nb_mines_voisines
        # Dessine un carré "ivoire"
        can.create_rectangle((col-1)*dim+gap+3, (lig-1)*dim+gap+3,
                             col*dim+gap-3, lig*dim+gap-3, width=0, fill="ivory")
        # Affichage du nombre de mines avec les couleurs correspondantes
        coul = ['blue', 'orange', 'red', 'green', 'cyan', 'skyblue', 'pink']
        can.create_text(col*dim-dim//2+gap, lig*dim-dim//2+gap,
                        text=str(nb_mines_voisines),
                        fill=coul[nb_mines_voisines-1], font='Arial 22')

# calcule le nombre de mines qui touchent la case
def nb_mines_adj(col, ligne):
    if col > 1:
        min_col = col - 1
    else:
        min_col = 1
    if col < nb_col:
        max_col = col + 1
    else:
        max_col = col
    if ligne > 1:
        min_lig = ligne - 1
    else:
        min_lig = 1
    if ligne < nb_lig:
        max_lig = ligne + 1
    else:
        max_lig = nb_lig
    txtinfo = ""
    nb_mines = 0
    indice_lig = min_lig
    while indice_lig <= max_lig:
        indice_col = min_col
        while indice_col <= max_col:
            if tab_m[indice_col, indice_lig] == 9:
                nb_mines += 1
            indice_col = indice_col + 1
        indice_lig = indice_lig + 1
    return nb_mines

# Affiche toutes les cases zéro adjacentes et leur bordure
def vide_plage_zero(col, ligne):
    global nb_mines_cachees, nb_cases_vues
    # si on a déjà la cellule n'est pas vide
    if tab_j[col, ligne] != 0:
        # S'il y a un drapeau on modifie le compteur de mines et de cases traitées
        if (tab_j[col, ligne] == "d"):
            nb_mines_cachees = nb_mines_cachees + 1
            nb_cases_vues = nb_cases_vues - 1
        # Affichage du fond
        can.create_rectangle((col-1)*dim+gap+3, (ligne-1)*dim+gap+3,
                             col*dim+gap-3, ligne*dim+gap-3,
                             width=0, fill="seashell2")
        # Stockage des 0 dans le tableau de jeu
        tab_j[col, ligne] = 0
        nb_cases_vues = nb_cases_vues + 1
        # Vérifie les cases voisines en croix

```

```

if col > 1:
    nb_mines_voisines = nb_mines_adj(col-1, ligne)
    if nb_mines_voisines == 0:
        vide_plage_zero(col-1, ligne)
    else:
        affiche_nb_mines(nb_mines_voisines, col-1, ligne)
if col < nb_col:
    nb_mines_voisines = nb_mines_adj(col+1, ligne)
    if nb_mines_voisines == 0:
        vide_plage_zero(col+1, ligne)
    else:
        affiche_nb_mines(nb_mines_voisines, col+1, ligne)
if ligne > 1:
    nb_mines_voisines = nb_mines_adj(col, ligne-1)
    if nb_mines_voisines == 0:
        vide_plage_zero(col, ligne-1)
    else:
        affiche_nb_mines(nb_mines_voisines, col, ligne-1)
if ligne < nb_lig:
    nb_mines_voisines = nb_mines_adj(col, ligne+1)
    if nb_mines_voisines == 0:
        vide_plage_zero(col, ligne+1)
    else:
        affiche_nb_mines(nb_mines_voisines, col, ligne+1)
# Vérification des diagonales pour afficher les bords de la plage zéro
if col > 1 and ligne > 1:
    nb_mines_voisines = nb_mines_adj(col-1, ligne-1)
    if nb_mines_voisines == 0:
        vide_plage_zero(col-1, ligne-1)
    else:
        affiche_nb_mines(nb_mines_voisines, col-1, ligne-1)
if col > 1 and ligne < nb_lig:
    nb_mines_voisines = nb_mines_adj(col-1, ligne+1)
    if nb_mines_voisines == 0:
        vide_plage_zero(col-1, ligne+1)
    else:
        affiche_nb_mines(nb_mines_voisines, col-1, ligne+1)
if col < nb_col and ligne > 1:
    nb_mines_voisines = nb_mines_adj(col+1, ligne-1)
    if nb_mines_voisines == 0:
        vide_plage_zero(col+1, ligne-1)
    else:
        affiche_nb_mines(nb_mines_voisines, col+1, ligne-1)
if col < nb_col and ligne < nb_lig:
    nb_mines_voisines = nb_mines_adj(col+1, ligne+1)
    if nb_mines_voisines == 0:
        vide_plage_zero(col+1, ligne+1)
    else:
        affiche_nb_mines(nb_mines_voisines, col+1, ligne+1)
affiche_compteurs()

def perdu():
    global on_joue
    on_joue = False
    # Parcours du tableau pour afficher toutes les mines
    nLig = 0
    while nLig < nb_lig:
        nCol = 1
        nLig = nLig + 1
        while nCol <= nb_col:
            # affichage de grille exacte
            if tab_m[nCol, nLig] == 9:
                if tab_j[nCol, nLig] == "?":
                    can.create_image(nCol*dim-dim//2+gap,
                                    nLig*dim-dim//2+gap, image = im_mine)
                elif tab_j[nCol, nLig] == "":
                    can.create_image(nCol*dim-dim//2+gap,
                                    nLig*dim-dim//2+gap, image = im_mine)
            else:
                if tab_j[nCol, nLig] == "d":
                    can.create_image(nCol*dim-dim//2+gap,
                                    nLig*dim-dim//2+gap, image = im_erreur)
            nCol = nCol+1

```

```

can.create_text((nb_col/2)*dim-15+gap, (nb_lig/2)*dim-5+gap,
               text='Perdu !', fill='black', font='Arial 50')
fen.update_idletasks() # Raffraichit la fenêtre avant le bruitage
winsound.PlaySound('explosion.wav', winsound.SND_FILENAME)

def gagne():
    can.create_text((nb_col/2)*dim-15+gap, (nb_lig/2)*dim-5+gap,
                   text='Bravo !', fill='black', font='Arial 50')
    fen.update_idletasks()
    winsound.PlaySound('gagne.wav', winsound.SND_FILENAME)

# gère le clic gauche de la souris
def pointeurG(event):
    global nb_cases_vues
    # si la partie n'est pas en cours (quand on a perdu), blocage du jeu
    if on_joue :
        nCol = (event.x - gap) // dim + 1
        nLig = (event.y - gap) // dim + 1
        # si la cellule est vide
        if tab_j[nCol, nLig] == "":
            # Vérifie si on est bien dans le tableau
            if nCol>=1 and nCol<=nb_col and nLig>=1 and nLig<=nb_lig:
                # Vérifie si la cellule contient une mine
                if (tab_m[nCol, nLig] == 9):
                    perdu()
                else:
                    nb_mines_voisines = nb_mines_adj(nCol, nLig)
                    if nb_mines_voisines >= 1:
                        affiche_nb_mines(nb_mines_voisines, nCol, nLig)
                        affiche_compteurs()
                    else: # Traitement des cases vides
                        vide_plage_zero(nCol, nLig)
            # Vérification des compteurs
            if ((nb_col*nb_lig) == nb_cases_vues and nb_mines_cachees == 0):
                gagne()

# gère le clic droit de la souris
def pointeurD(event):
    global nb_mines_cachees, nb_cases_vues
    # si la partie n'est pas en cours (quand on a perdu), blocage du jeu
    if on_joue :
        nCol = (event.x - gap) // dim + 1
        nLig = (event.y - gap) // dim + 1
        # si la cellule est vide
        if tab_j[nCol, nLig] == "":
            # Affiche le drapeau
            can.create_image(nCol*dim-dim//2+gap, nLig*dim-dim//2+gap,
                           image = im_flag)
            tab_j[nCol, nLig] = "d"
            nb_cases_vues = nb_cases_vues + 1
            nb_mines_cachees = nb_mines_cachees - 1
        # si la cellule contient un drapeau
        elif tab_j[nCol, nLig] == "d":
            # Remise à blanc
            can.create_rectangle((nCol-1)*dim+gap+3, (nLig-1)*dim+gap+3,
                                nCol*dim+gap-3, nLig*dim+gap-3, width=0, fill="grey")
            # Affiche le ?
            can.create_text(nCol*dim-dim//2+gap, nLig*dim-dim//2+gap,
                           text="?", fill='black', font='Arial 20')
            tab_j[nCol, nLig] = "?"
            # le ? n'est pas considéré comme une case traitée
            nb_cases_vues = nb_cases_vues - 1
            # Ajoute une mine car le ? ne désigne pas une mine
            nb_mines_cachees = nb_mines_cachees + 1
        # si la cellule contient un ?
        elif tab_j[nCol, nLig] == "?":
            # Remise à blanc
            can.create_rectangle((nCol-1)*dim+gap+3, (nLig-1)*dim+gap+3,
                                nCol*dim+gap-3, nLig*dim+gap-3,
                                width=0, fill="grey")
            # Stocke du vide dans le tableau de jeu

```

Démineur

```
        tab_j[nCol, nLig] = ""
        affiche_compteurs()
        # Vérification des compteurs
        if ((nb_col*nb_lig) == nb_cases_vues and nb_mines_cachees == 0):
            gagne()

# -----
# Début du programme
# -----

fen=Tk()
fen.title("Démineur")
fen.resizable(width=False, height=False)

# Déclarations des variables lorsqu'on ouvre la fenêtre principale
# niveau débutant par défaut
nb_col, nb_lig, nb_mines = 0,0,0
dim, gap, nb_cases_vues = 30, 3, 0
on_joue = True
# Chargement des images
im_mine = PhotoImage(file = "minej.gif")
im_erreur = PhotoImage(file = "croixj.gif")
im_flag = PhotoImage(file = "drapeauj.gif")
tab_m = {} # tableau des mines
tab_j = {} # tableau des cases modifiées par le joueur

can=Canvas(fen, width=(nb_col*dim)+gap, height=(nb_lig*dim)+gap, bg="grey")
can.bind("<Button-1>", pointeurG)
can.bind("<Button-3>", pointeurD)
can.pack(side=RIGHT)

# Frame à gauche de la grille de jeu pour disposer les boutons radios
f2 = Frame(fen)
# Création de cases à cocher pour le niveau
choix=IntVar()
choix.set(1)
case1=Radiobutton(f2)
case1.configure(text='Débutant', command=init_niveau, variable=choix, value=1)
case1.pack(anchor= NW ,padx=30)
case2=Radiobutton(f2)
case2.configure(text='Avancé', padx=3, command=init_niveau, variable=choix, value=2)
case2.pack(anchor= NW, padx=30)
case3=Radiobutton(f2)
case3.configure(text='Expert', padx=3, command=init_niveau, variable=choix, value=3)
case3.pack(anchor= NW, padx=30)
f2.pack()

# Frame à gauche de la grille de jeu pour les compteurs
f3 = Frame(fen)
# Champ pour l'affichage du décompte des mines
texte_mines = Label (f3, text = "Mines restantes :")
decompte_mines = Label (f3, text = "100")
texte_mines.grid(row=4,column=1,sticky='NW')
decompte_mines.grid(row=4,column=2,sticky='NE')
# Champ pour l'affichage du décompte des cases
texte_cases = Label (f3, text = "Cases à traiter :")
decompte_cases = Label (f3, text = "10")
texte_cases.grid(row=5,column=1,sticky='NW')
decompte_cases.grid(row=5,column=2,sticky='NE')
f3.pack()

# Frame à gauche de la grille de jeu pour disposer les boutons
f1 = Frame(fen)
boul = Button(f1, width=14, text="Nouvelle partie", font="Arial 10",
command=init_jeu)
boul.pack(side=BOTTOM, padx=5, pady=5)
f1.pack(side=BOTTOM)

# Frame à gauche de la grille de jeu pour afficher l'image
f4 = Frame(fen)
photo=PhotoImage(file="mine1.gif")
labl = Label(f4, image=photo)
labl.pack(side=BOTTOM)
```

```
f4.pack(side=BOTTOM)

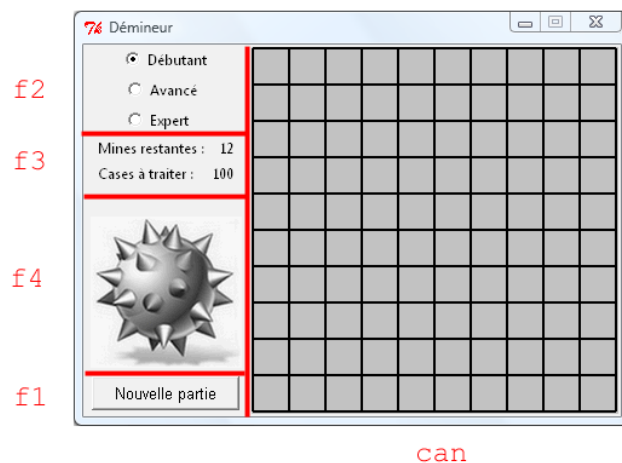
init_niveau()
init_jeu()
fen.mainloop()
```

13.5. Analyse du programme

Comme le programme est long (presque 400 lignes) et suffisamment commenté, nous allons nous concentrer sur les parties contenant des nouveautés.

13.5.1. Frames multiples

La fenêtre est découpée en 4 frames et un canevas (voir la dernière page du programme) :



Elles ont été placées à l'aide de l'instruction pack (voir § 8.4.3).

13.5.2. Boutons radio

Un **bouton radio** est un élément visuel des interfaces graphiques (un widget). Les boutons radio sont toujours utilisés en groupe (donc deux boutons radio au minimum), puisque leur objectif est de permettre à l'utilisateur de choisir une, et une seule, option parmi plusieurs possibles. Graphiquement (voir image ci-dessus, dans la frame f2), un bouton radio est représenté par un cercle et est accompagné d'une étiquette, c'est-à-dire un court texte qui décrit le choix qui lui est associé. Si l'utilisateur choisit cette option, un point apparaît à l'intérieur du cercle pour symboliser le choix, sinon le cercle reste vide.

```
# Frame à gauche de la grille de jeu pour disposer les boutons radios
f2 = Frame(fen)
# Création de cases à cocher pour le niveau
choix=IntVar()
choix.set(1)
case1=Radiobutton(f2)
case1.configure(text='Débutant', command=init_niveau, variable=choix, value=1)
case1.pack(anchor= NW ,padx=30)
case2=Radiobutton(f2)
case2.configure(text='Avancé', padx=3, command=init_niveau, variable=choix, value=2)
case2.pack(anchor= NW, padx=30)
case3=Radiobutton(f2)
case3.configure(text='Expert', padx=3, command=init_niveau, variable=choix, value=3)
case3.pack(anchor= NW, padx=30)
f2.pack()
```

Les boutons radio ont été placés dans la frame f2.

Ils serviront à donner une valeur à la variable `choix`. On précise que cette valeur sera du type entier :

```
choix=IntVar()
```

Par défaut, la valeur de `choix` est 1. Cela correspond au niveau « débutant » :

```
choix.set(1)
```

C'est l'instruction :

```
case1=Radiobutton(f2)
```

qui crée le bouton dans la frame `f2`. Il doit ensuite être configuré : l'étiquette sera « Débutant » et quand on « pressera » sur ce bouton, on appellera la procédure `init_niveau` et on mettra la valeur 1 dans la variable `choix`.

```
case1.configure(text='Débutant', command=init_niveau, variable=choix, value=1)
```

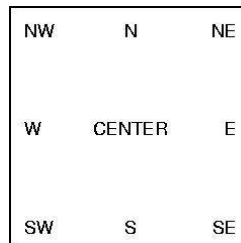
Dans la procédure `init_niveau`, on voit la ligne

```
niveau = choix.get()
```

Le terme de *cavité restante* a été introduit au § 8.4.3

qui récupère la valeur dans la variable `choix`.

Le bouton doit finalement être placé dans la frame. L'instruction `anchor=NW` indique qu'il sera placé en haut à gauche (au nord-ouest), dans la *cavité restante*. Les valeurs possibles pour `anchor` sont :



L'instruction `padx=30` définit l'espacement avec les autres boutons.

```
case1.pack(anchor= NW, padx=30)
```

13.5.3. Récursivité

La procédure récursive s'appelle ici `vide_plage_zero`. Quand on la regarde de plus près, elle voit qu'elle ressemble beaucoup, dans l'idée du moins, à la procédure `thesee` qui nous a permis d'explorer un labyrinthe (voir § 13.2). La grande différence, c'est qu'il faut aussi regarder les cases voisines en diagonale.

```
# Affiche toutes les cases zéro adjacentes et leur bordure
def vide_plage_zero(col, ligne):
    global nb_mines_cachees, nb_cases_vues
    # si on a déjà la cellule n'est pas vide
    if tab_j[col, ligne] != 0:
        # S'il y a un drapeau on modifie le compteur de mines et de cases traitées
        if (tab_j[col, ligne] == "d"):
            nb_mines_cachees = nb_mines_cachees + 1
            nb_cases_vues = nb_cases_vues - 1
        # Affichage du fond
        can.create_rectangle((col-1)*dim+gap+3, (ligne-1)*dim+gap+3,
                             col*dim+gap-3, ligne*dim+gap-3,
                             width=0, fill="seashell2")
    # Stockage des 0 dans le tableau de jeu
```



```

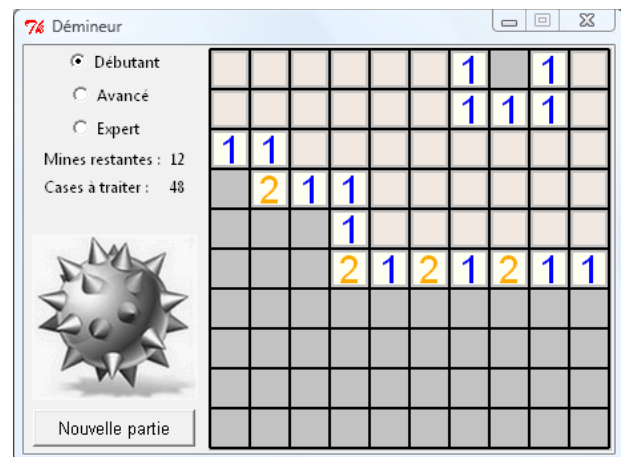
tab_j[col, ligne] = 0
nb_cases_vues = nb_cases_vues + 1
# Vérifie les cases voisines en croix
if col > 1:
    nb_mines_voisines = nb_mines_adj(col-1, ligne)
    if nb_mines_voisines == 0:
        vide_plage_zero(col-1, ligne)
    else:
        affiche_nb_mines(nb_mines_voisines, col-1, ligne)
if col < nb_col:
    nb_mines_voisines = nb_mines_adj(col+1, ligne)
    if nb_mines_voisines == 0:
        vide_plage_zero(col+1, ligne)
    else:
        affiche_nb_mines(nb_mines_voisines, col+1, ligne)
if ligne > 1:
    nb_mines_voisines = nb_mines_adj(col, ligne-1)
    if nb_mines_voisines == 0:
        vide_plage_zero(col, ligne-1)
    else:
        affiche_nb_mines(nb_mines_voisines, col, ligne-1)
if ligne < nb_lig:
    nb_mines_voisines = nb_mines_adj(col, ligne+1)
    if nb_mines_voisines == 0:
        vide_plage_zero(col, ligne+1)
    else:
        affiche_nb_mines(nb_mines_voisines, col, ligne+1)
# Vérification des diagonales pour afficher les bords de la plage zéro
if col > 1 and ligne > 1:
    nb_mines_voisines = nb_mines_adj(col-1, ligne-1)
    if nb_mines_voisines == 0:
        vide_plage_zero(col-1, ligne-1)
    else:
        affiche_nb_mines(nb_mines_voisines, col-1, ligne-1)
if col > 1 and ligne < nb_lig:
    nb_mines_voisines = nb_mines_adj(col-1, ligne+1)
    if nb_mines_voisines == 0:
        vide_plage_zero(col-1, ligne+1)
    else:
        affiche_nb_mines(nb_mines_voisines, col-1, ligne+1)
if col < nb_col and ligne > 1:
    nb_mines_voisines = nb_mines_adj(col+1, ligne-1)
    if nb_mines_voisines == 0:
        vide_plage_zero(col+1, ligne-1)
    else:
        affiche_nb_mines(nb_mines_voisines, col+1, ligne-1)
if col < nb_col and ligne < nb_lig:
    nb_mines_voisines = nb_mines_adj(col+1, ligne+1)
    if nb_mines_voisines == 0:
        vide_plage_zero(col+1, ligne+1)
    else:
        affiche_nb_mines(nb_mines_voisines, col+1, ligne+1)
affiche_compteurs()

```

Elle permet ici de déterminer la **zone sûre** : si une case sans mine n'a aucun voisin avec une mine, on peut automatiquement découvrir les cases voisines, etc. La partie complexe du démineur est la propagation des cases sûres. C'est ceci qui permet d'obtenir la configuration ci-contre juste en cliquant sur le coin supérieur gauche de la grille.

On découvre toutes les cases qui ne contiennent rien (ni mine, ni entier strictement supérieur à 0), en s'arrêtant aux cases qui contiennent un entier strictement supérieur à 0.

Le calcul de la zone sûre se fait de la



manière suivante :

Pour une case donnée (sans mine et non déjà découverte),

1. on compte le nombre de mines parmi les cases adjacentes : on appelle le résultat n
2. on met à jour cette case
3. on teste :
 - si $n > 0$, on s'arrête,
 - si $n = 0$, on recommence l'opération récursivement avec toutes les cases adjacentes.



Exercice 13.3

Ajoutez une cinquième frame, où sera affiché le temps qui passe. Si le joueur dépasse une certaine limite, il aura perdu !



13.6. Ce que vous avez appris dans ce chapitre

- Vous avez vu pour la première fois en détails une procédure récursive. Elles sont assez délicates à utiliser, mais peuvent être très élégantes. Il faut savoir que toute procédure récursive peut aussi s'écrire sous forme itérative (et vice-versa). Il faut aussi faire attention au fait que les procédures récursives peuvent provoquer des débordements de mémoire et qu'elles ne sont pas toujours recommandées. Mais elles existent et simplifient parfois la vie (comme dans les deux exemples que nous avons vus), à condition de les maîtriser.
- Les boutons radio sont pratiques pour choisir une valeur parmi plusieurs possibles.
- On peut découper une fenêtre en plusieurs frames.



Chapitre 14

Le loup et les moutons

14.1. Nouveau thème abordé dans ce chapitre

- glisser-déposer

Il n'y a qu'une nouvelle notion dans ce chapitre, mais comme beaucoup de jeux se jouent sur un damier (le terme générique est plutôt *tablier*), il est bon de consacrer un chapitre à ce sujet. Cela permettra de renforcer les notions vues aux chapitres précédents.

14.2. Damiers avec des lignes



lignes.py

<http://ow.ly/43Gd30fdwPg>

```
# Dessin d'un damier en traçant des lignes
from tkinter import *

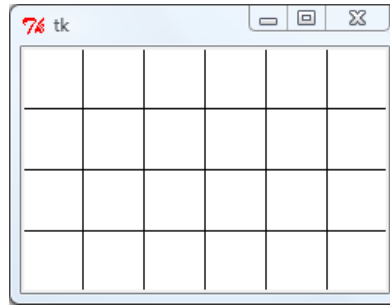
def lignes_verticales(c,nc,nl):
    x=0
    while x < nc*c:
        canvas.create_line(x,0,x,nl*c,width=1,fill='black')
        x += c

def lignes_horizontales(c,nc,nl):
    y=0
    while y < nl*c:
        canvas.create_line(0,y,nc*c,y,width=1,fill='black')
        y += c

def damier(c, nl, nc):
    lignes_verticales(c,nc,nl)
    lignes_horizontales(c,nc,nl)

c = 40          # côté d'un carré
nl = 4          # nombre de lignes du damier
nc = 6          # nombre de colonnes
fen = Tk()
canvas = Canvas(fen, width=c*nc-3, height=c*nl-3, bg='white')
canvas.pack(side=TOP)
damier(c, nl, nc)
fen.mainloop()
```

Voici la grille obtenue :



14.3. Damiers avec des carrés



carres.py

<http://ow.ly/43Gd3>
[0fdwPg](#)

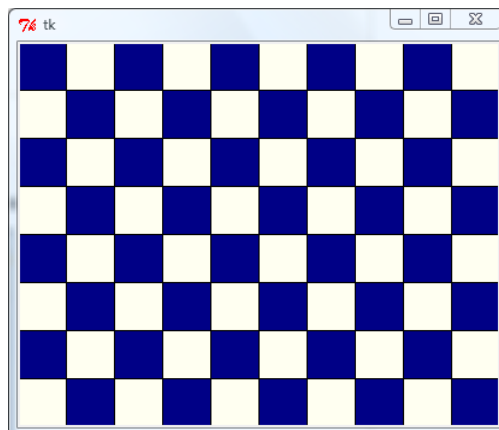
```
# Dessin d'un damier avec des carrés
from tkinter import *

def ligne_de_carres(x, y, c, n):
    # dessiner une ligne de n carrés
    i = 0
    while i < n:
        can.create_rectangle(x, y, x+c, y+c, fill='navy')
        i += 1
        x += c*2                # espacer les carrés

def damier(c, nl, nc):
    # dessiner nl lignes de nc carrés de taille c avec décalage alterné
    y = 0
    while y < nl:
        if y % 2 == 0:          # une ligne sur deux, on
            x = 0               # commencera la ligne de
        else:                   # carrés avec un décalage
            x = 1               # de la taille d'un carré
        ligne_de_carres(x*c, y*c, c, nc)
        y += 1

c = 40                        # côté d'un carré
nl = 8                        # nombre de lignes du damier
nc = 10                       # nombre de colonnes
fen = Tk()
can = Canvas(fen, width=c*nc-3, height=c*nl-3, bg='ivory')
can.pack(side=TOP)
damier(c, nl, nc)
fen.mainloop()
```

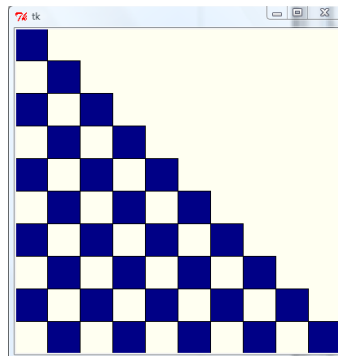
Voici le damier obtenu :





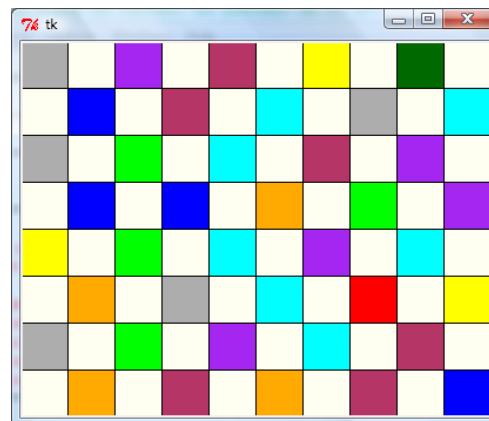
Exercice 14.1

Modifiez le programme du § 14.3 pour obtenir le damier ci-dessous :



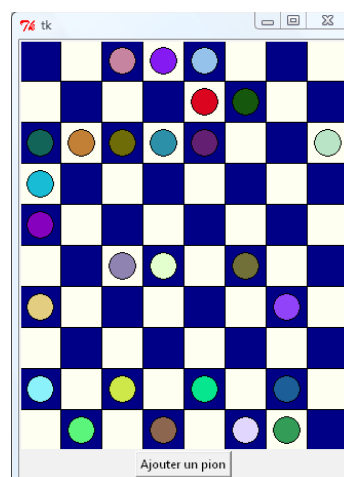
Exercice 14.2

Modifiez le programme du § 14.3 pour que les cases de couleur « navy » du damier d'origine ait une couleur aléatoire choisie parmi une liste de 10 (à vous de choisir lesquelles).



Exercice 14.3

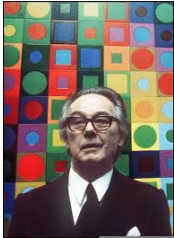
Modifiez le programme du § 14.3. Ajoutez un bouton « Ajouter un pion ». Quand vous cliquerez sur ce bouton, un pion d'une couleur aléatoire apparaîtra sur une case choisie au hasard.



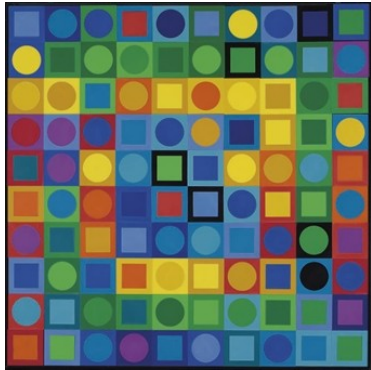


Exercice 14.4

Victor **Vasarely** (1908-1997) est un plasticien hongrois, naturalisé français en 1961, reconnu comme étant le père de l'*art optique* ou *Op art*. Voici une de ces œuvres (à gauche) :



Victor Vasarely



Vasarely Planetary Folklore Participations No. 1 (1969)



Faux généré par ordinateur

Créez un programme qui générera avec des couleurs aléatoires un tableau ressemblant à « un Vasarely ». Les petits ronds et les petits carrés seront eux aussi placés aléatoirement.



Exercice 14.5

Écrivez un programme qui dessine un quadrillage composé de 16 rectangles sur 30, chaque rectangle étant coloré d'une des 479 couleurs définies dans le code du § 11.2, avec le nom de la couleur écrit en noir à l'intérieur du rectangle correspondant. Un rectangle sera trois fois plus long que large.

14.4. Damiers avec des hexagones



hexagones.py

<http://ow.ly/43Gd30fdwPg>

```
# Dessin d'un damier hexagonal

from tkinter import *
from random import choice

def couleur():
    coul = "#"
    liste = ['0','1','2','3','4','5','6','7','8','9','a','b','c','d','e','f']
    for i in range(6):
        coul += choice(liste)
    return coul

def hexagone(origine, c):
    un_seg = c/4
    deux_seg = c/2
    x, y = origine[0], origine[1]
    # hexagone
    can.create_polygon(x, y + deux_seg, x + un_seg, y,
        x + un_seg + deux_seg, y, x + c, y + deux_seg,
        x + un_seg + deux_seg, y + c, x + un_seg, y + c,
        x, y + deux_seg, outline='black', width=1, fill = couleur())

def ligne_d_hexagones(x, y, c, n):
    # dessiner une ligne d'hexagones
    i = 0
    un_seg = c/4
    while i < n:
        hexagone((x+2, y+2), c) # +2 : décalage pour que tout soit bien dessiné
        i += 1
```

```

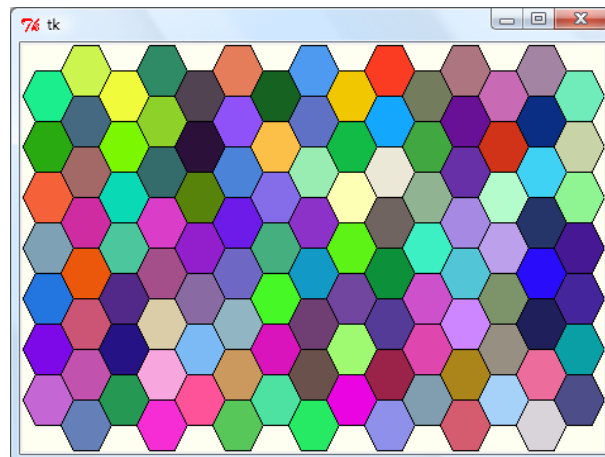
        x += 6*un_seg

def damier(c, nl, nc):
    # dessiner nl lignes d'hexagones de taille c avec décalage alterné
    y = 0
    no_ligne = 0
    while no_ligne < nl:
        if no_ligne%2 == 1:          # une ligne sur deux, on
            x = 0                    # commencera la ligne
        else:                        # avec un décalage
            x = 0.75
        if nc%2 == 1 :
            if no_ligne%2 == 0:
                ligne_d_hexagones(x*c, y*c, c, nc//2)
            else:
                ligne_d_hexagones(x*c, y*c, c, nc//2+1)
        else:
            ligne_d_hexagones(x*c, y*c, c, nc//2)
        y += 0.5
        no_ligne += 1

c = 40          # taille des carrés circonscrit à l'hexagone
nl = 15         # nombre de lignes
nc = 15         # nombre de colonnes
fen = Tk()
can = Canvas(fen, width=c+(nc-1)*0.75*c, height=c+(nl-1)*c/2+1, bg='ivory')
can.pack(side=TOP)
damier(c,nl,nc)
fen.mainloop()

```

Voici le résultat obtenu :



14.5. Analyse du programme

```

# Dessin d'un damier hexagonal

from tkinter import *
from random import choice

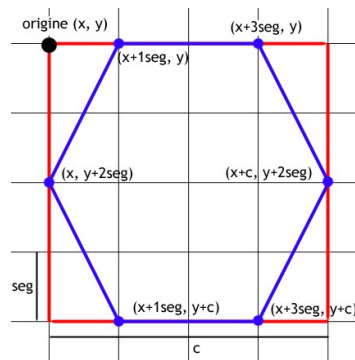
def couleur():
    coul = "#"
    liste = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'a', 'b', 'c', 'd', 'e', 'f']
    for i in range(6):
        coul += choice(liste)
    return coul

```

Vous remarquerez que nous avons utilisé le format RGB pour définir une couleur aléatoire. La couleur aura pour format un # suivi de 6 symboles choisis dans `liste`.

```
def hexagone(origine, c):
    seg = c/4
    x, y = origine[0], origine[1]
    # hexagone
    can.create_polygon(x, y+2*seg, x+seg, y, x+3*seg, y, x+c, y+2*seg,
        x+3*seg, y+c, x+seg, y+c, x, y+2*seg,
        outline = 'black', width = 1, fill = couleur())
```

Le schéma ci-dessous explique comment est construit un hexagone :

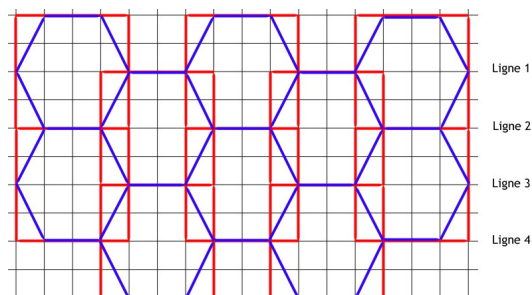


Notez au passage que ce n'est pas un hexagone régulier, puisque ses côtés n'ont pas tous la même longueur. L'avantage de cet hexagone est qu'il est simple à dessiner et à calculer.

```
def ligne_d_hexagones(x, y, c, n):
    # dessiner une ligne d'hexagones
    i = 0
    seg = c/4
    while i < n:
        hexagone((x+2, y+2), c) # +2 : décalage pour que tout soit bien dessiné
        i += 1
        x += 6*seg

def damier(c, nl, nc):
    # dessiner nl lignes d'hexagones de taille c avec décalage alterné
    y = 0
    no_ligne = 0
    while no_ligne < nl:
        if no_ligne%2 == 1:
            x = 0
        else:
            x = 0.75
        if nc%2 == 1:
            if no_ligne%2 == 0:
                ligne_d_hexagones(x*c, y*c, c, nc//2)
            else:
                ligne_d_hexagones(x*c, y*c, c, nc//2+1)
        else:
            ligne_d_hexagones(x*c, y*c, c, nc//2)
        y += 0.5
        no_ligne += 1
```

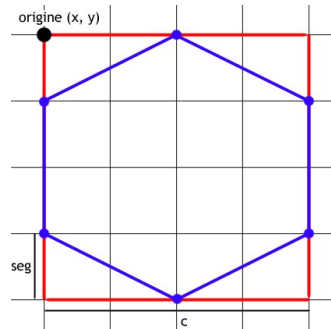
Le schéma ci-dessous explique comment sont alignés les hexagones pour former un damier :



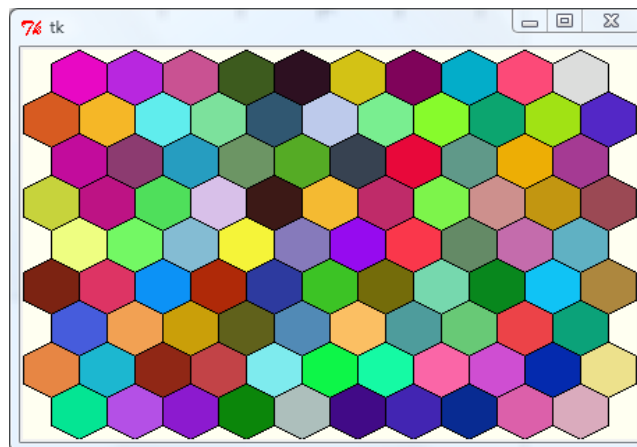


Exercice 14.6

Modifiez le programme du § 14.4. Construisez les hexagones comme expliqué sur le schéma ci-dessous.



Utilisez-les pour dessiner un damier à trame hexagonale ressemblant à celui ci-dessous.



14.6. Glisser-déposer

Le **glisser-déposer** (traduction recommandée de l'anglais *drag and drop*) consiste à utiliser une souris pour déplacer d'un endroit à un autre un élément graphique présent sur l'écran d'un ordinateur. Pour ce faire, il faut cliquer sur cet élément graphique, maintenir le bouton droit de la souris enfoncé, et ne le relâcher que lorsque le pointeur a atteint l'endroit souhaité.

Le programme ci-dessous montre comment glisser-déposer un pion.



dragdrop.py

<http://ow.ly/43Gd3>
[0fdwPg](#)

```
# exemple de glisser-déposer (drag and drop)
from tkinter import *

def glisser(evt):
    global x, y
    if x == -1:
        if x_pion-50 < evt.x < x_pion+50 and y_pion-100 < evt.y < y_pion+100 :
            x, y = evt.x, evt.y
    else :
        delta_x, delta_y = evt.x - x, evt.y - y
        fond.coords(pion, x_pion + delta_x, y_pion + delta_y)

def deposer(evt):
    global x_pion, y_pion, x
    if x != -1 :
        x_pion, y_pion = x_pion + evt.x - x, y_pion + evt.y - y
```

```

x = -1

def dessus(evt) :
    # modifie la forme du curseur
    forme = "arrow"
    if x_pion-50 < evt.x < x_pion+50 and y_pion-100 < evt.y < y_pion+100 :
        forme = "fleur"
    fond.config(cursor = forme)

fenetre=Tk()
fond=Canvas(fenetre,width=800,height=600,bg="white")

x, y = -1, -1 # coordonnées initiales de la souris
x_pion, y_pion = 400, 400 # coordonnées initiales du pion
f_pion = PhotoImage(file="pion.gif") # dimensions de l'image : 107 x 200
pion = fond.create_image(x_pion,y_pion,image=f_pion)

fond.grid()
fond.bind('<B1-Motion>',glisser)
fond.bind('<ButtonRelease-1>',deposer)
fond.bind('<Motion>',dessus)

fenetre.mainloop()

```

14.7. Analyse du programme

Avant toute chose, il faut remarquer que l'image du pion que l'on utilise est inscrite dans un rectangle de 107 pixels de large et de 200 pixels de haut. On va repérer le pion avec son centre (initialement $x_pion = y_pion = 400$). La souris sera sur le pion quand les coordonnées de la souris seront dans le rectangle englobant le pion.

La première fois que l'événement `<B1-Motion>` survient (c'est-à-dire quand la souris se déplace avec le bouton gauche enfoncé), on appelle la procédure `glisser`. On stocke dans les variables x, y les coordonnées de la souris. Celles-ci ont été initialisées à $(-1; -1)$ pour signifier qu'aucun mouvement n'est en cours.

À chaque fois que l'événement `<B1-Motion>` survient à nouveau, on calcule le vecteur de déplacement entre les coordonnées actuelles de la souris ($evt.x; evt.y$) et le point $(x; y)$. On fait subir à l'image le même déplacement sans mettre à jour les coordonnées du centre du pion pour le moment.

Enfin, lorsqu'on relâche le bouton (`<ButtonRelease-1>`), on appelle la procédure `deposer`. Les coordonnées x_pion, y_pion sont mises à jour, et on donne à x la valeur -1 pour indiquer que le déplacement est terminé.

Formes du curseur

Pour améliorer le rendu, on transforme le curseur lors du survol du pion pour signifier au joueur que cette pièce peut être déplacée. On exécutera la procédure `dessus` sur l'événement `<Motion>`.

Le module `tkinter` contient différentes formes de curseur, dont le motif peut varier selon le système d'exploitation. Voici une liste des noms les plus intéressants : `arrow`, `circle`, `clock`, `cross`, `dotbox`, `exchange`, `fleur`, `heart`, `man`, `mouse`, `pirate`, `plus`, `shuttle`, `sizing`, `spider`, `spraycan`, `star`, `target`, `tcross`, `trek`, `watch`.



Exercice 14.7

Modifiez le programme du § 14.6 afin de pouvoir glisser-déposer trois pions au lieu d'un.



Exercice 14.8

Modifiez le programme de l'exercice 14.7 afin de pouvoir glisser-déposer un pion noir et un pion blanc sur un damier 8×8 . Ils devront toujours être déposés bien au centre d'une case. Procédez de la

manière suivante :

1. Combinez les programmes du § 14.3 et de l'exercice 14.7.
2. Utilisez une liste pour stocker les deux pions.
3. Centrez les pions sur les cases après le glisser-déposer.
4. Pensez au fait que les pions ne doivent pas sortir du damier.

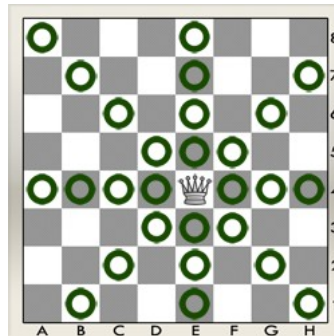


Exercice 14.9

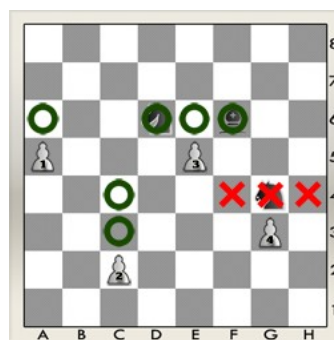
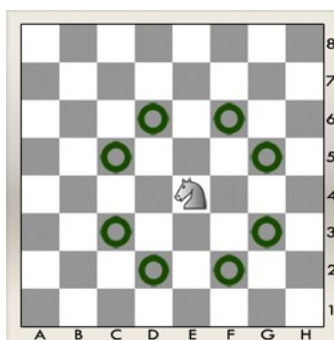
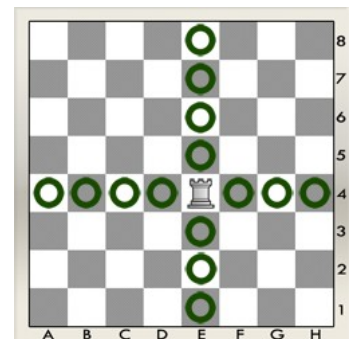
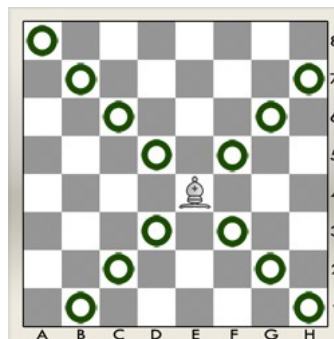
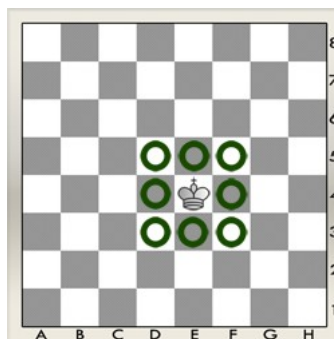
En vous inspirant de l'exercice 14.8, écrivez un programme qui permette de déplacer une dame sur un échiquier en respectant les règles du jeu (voir schéma ci-dessous).

Source des images :

<http://www.chess-mind.com/fr/echecs/regles>



Faites ensuite de même pour les autres pièces du jeu (un programme par pièce).



Comme case de départ, choisissez pour le pion une case de la seconde ligne.

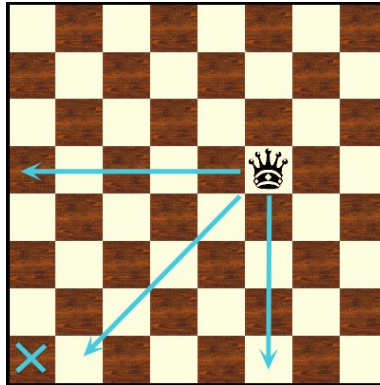


Exercice 14.10

On déplace une dame sur un échiquier. Comme la dame des échecs, elle peut se déplacer horizontalement, verticalement et en diagonale, mais seulement vers la gauche et vers le bas.

Deux joueurs s'affrontent. Le premier pose la dame sur l'échiquier, le second la déplace, puis chaque joueur la déplace à tour de rôle. Le vainqueur est le joueur qui réussit à atteindre la case

inférieure gauche de l'échiquier.



Programmez ce jeu de sorte que l'on puisse jouer contre l'ordinateur. Il faudra d'abord trouver une stratégie avant de la programmer...

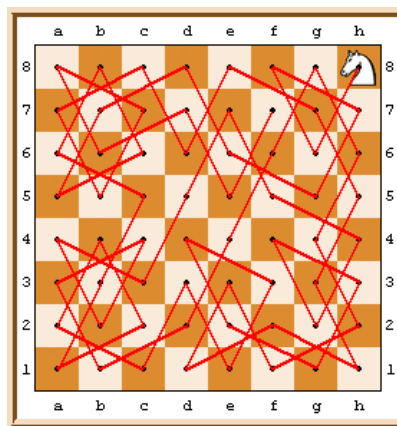
Vous déplacerez la dame avec un glisser-déposer et vérifierez que les mouvements sont valides.



Exercice 14.11 – Le parcours du cavalier

Le *parcours du cavalier* sur un échiquier est un problème ancien : il s'agit de déterminer un trajet parcouru par un cavalier qui part d'une case quelconque de l'échiquier et qui visite toutes les cases une et une seule fois. On exige parfois le retour sur la première case, on parle alors de parcours fermé.

Voici un exemple de parcours (départ en h8, arrivée en f7) :



Modifiez le programme de l'exercice 14.9 pour qu'un joueur puisse tenter de trouver un parcours du cavalier. Vous tracerez le chemin du cavalier comme ci-dessus.



Exercice 14.12 – L'heuristique de Warnsdorff

Une *heuristique* est un algorithme qui fournit rapidement une solution (pas forcément optimale) pour un problème d'optimisation. L'heuristique proposée en 1823 par le mathématicien allemand *Warnsdorff* permet de trouver une solution au problème du cavalier d'Euler en un temps linéaire. Cependant, il est possible dans certains cas que cette heuristique ne marche pas.

Le principe de cette heuristique est le suivant :

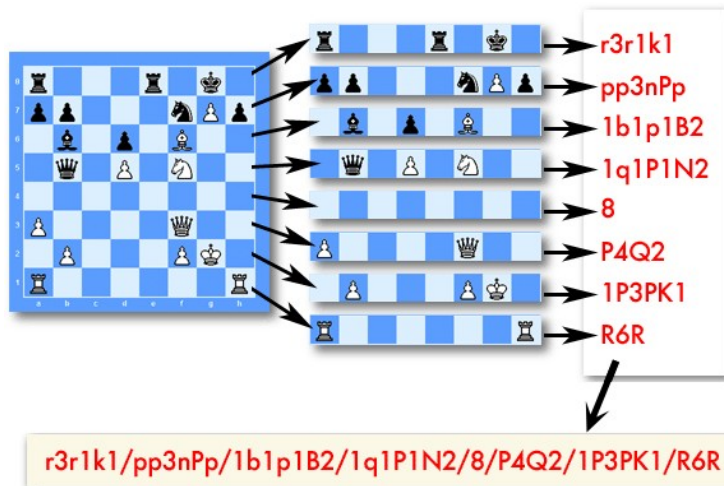
- à chaque case c de l'échiquier, on affecte un poids égal au nombre de cases accessibles à partir de c ;
- lorsque l'on parcourt l'échiquier, on choisit les cases de poids minimal.

Modifiez le programme de l'exercice 14.11 et programmez cette heuristique.



Exercice 14.13 – La notation FEN

FEN est l'abréviation de Forsyth-Edwards Notation. Ce format permet de décrire la position des pièces lors d'une partie d'échecs.



En partant de la case A8 en haut à gauche, chaque rangée de l'échiquier est décrite de gauche à droite. Elles sont séparées par le symbole /.

Une lettre indique la présence d'une pièce, un chiffre indique le nombre de cases vides jusqu'à la prochaine pièce ou la fin de la rangée (le total d'une rangée doit faire 8, en disant qu'une pièce est égale à 1). Les lettres minuscules désignent les pièces noires, les lettres majuscules les pièces blanches.



Vous trouverez des images sur le site web compagnon.

<http://ow.ly/43Gd30fdwPg>

K = roi blanc
R = tour blanche
N = cavalier blanc

k = roi noir
r = tour noire
n = cavalier noir

Q = dame blanche
B = fou blanc
P = pion blanc

q = dame noire
b = fou noir
p = pion noir

Écrivez un programme qui affiche successivement à l'écran les trois positions suivantes :

```
rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR
rnbqkbnr/pp1ppppp/8/2p5/4P3/5N2/PPPP1PPP/RNBQKB1R
2kr3r/1pp2ppp/1n2b3/p3P3/PnP2B2/1K2P3/1P4PP/R4BNR
```



Exercice 14.14

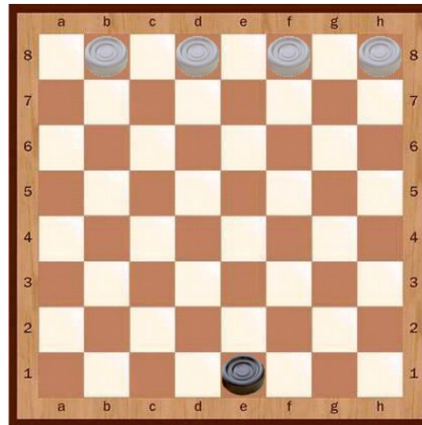
Exercice inverse de l'exercice 14.13 : écrivez un programme qui permette de sauvegarder dans un fichier une position d'une partie d'échecs avec la notation FEN.

Il faudra programmer une interface graphique agréable qui permettra à l'utilisateur de choisir sa pièce et de la placer sur la case souhaitée. Il faudra aussi penser à pouvoir corriger une erreur lors de la mise en place.

14.8. Le loup et les moutons - Règles

Ce jeu se joue sur les cases noires d'un échiquier. Un joueur prend un pion noir (le loup), l'autre joueur quatre pions blancs (les moutons). Les pions se déplacent en diagonale pour une case vide adjacente. Les moutons ne se déplacent que vers l'avant, mais le loup peut reculer. Au début de la partie, les quatre moutons occupent la dernière ligne de l'échiquier et le loup sur la première (voir figure ci-dessous). Le loup bouge le premier.

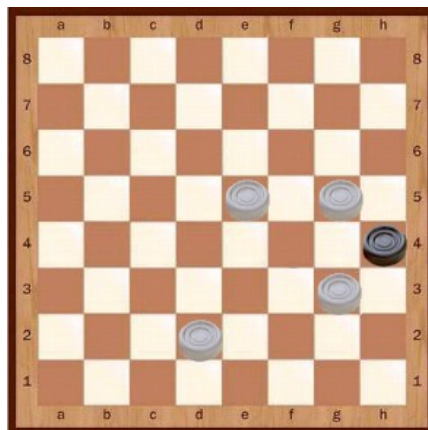
Le loup et les moutons



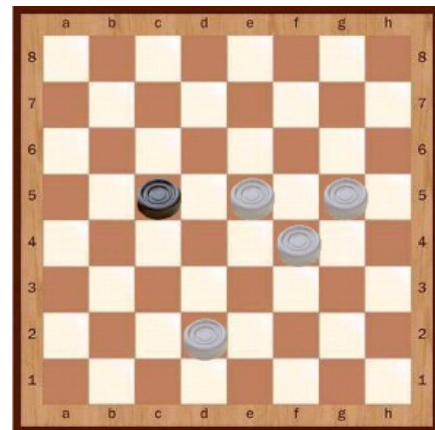
Le jeu est *asymétrique*, c'est-à-dire que les deux joueurs n'ont pas le même objectif.

Les moutons gagnent s'ils arrivent à faire en sorte que le loup ne puisse plus bouger.

Le loup gagne s'il atteint la dernière ligne occupée par des moutons, car il ne pourra plus être coincé.



Les moutons gagnent



Le loup gagne



Exercice 14.15

Programmez le jeu du loup et des moutons pour deux joueurs humains. Les pions devront être déplacés selon les règles. Évidemment, un pion ne pourra être déposé que sur une case vide. Il faudra indiquer le vainqueur de la partie, quand elle s'arrêtera.



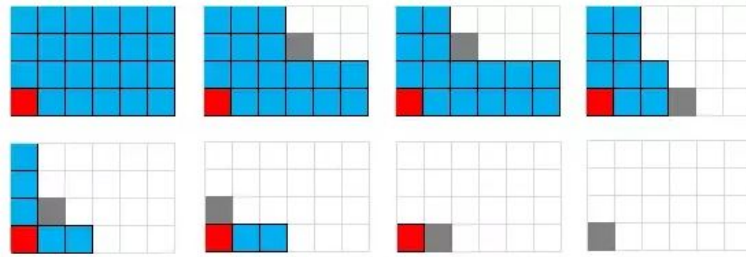
Exercice 14.16

Trouvez la stratégie gagnante (si elle existe) pour le loup et pour les moutons. Programmez ensuite cette stratégie pour qu'un humain puisse jouer contre l'ordinateur, avec le loup ou les moutons. On peut imaginer plusieurs niveaux de difficulté.

14.9. Jeu de Chomp – Règles

Chomp est un jeu mathématique à deux joueurs, joué avec une « tablette de chocolat », c'est-à-dire un rectangle composé de blocs carrés. Les joueurs choisissent un carré à tour de rôle, et le

« mangent », ainsi que tous les carrés situés à sa droite ou plus haut. Le carré en bas à gauche est empoisonné et celui qui le mange perd la partie.



Le jeu a été inventé en 1952 par Frederik **Schuh**, en termes de choix de diviseurs à partir d'un entier donné, puis ré-inventé indépendamment en 1974 par David **Gale** sous sa formulation actuelle.



Exercice 14.17

Programmez le jeu de Chomp pour deux joueurs humains. Il faudra indiquer le vainqueur de la partie, quand elle s'arrêtera.



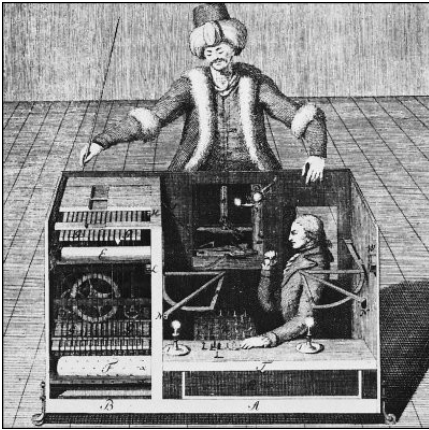
Exercice 14.18

Trouvez la stratégie gagnante (si elle existe) pour le jeu de Chomp. Programmez ensuite cette stratégie pour qu'un humain puisse jouer contre l'ordinateur.



14.10. Ce que vous avez appris dans ce chapitre

- Vous avez vu comment dessiner plusieurs types de damiers et les utiliser.
- Vous pouvez déplacer des objets par la méthode du glisser-déposer.
- Vous avez rencontré plusieurs exercices concernant le jeu d'échecs, qui sera le sujet du chapitre suivant.



Chapitre 15

Programmer un jeu d'échecs

15.1. Nouveaux thèmes abordés dans ce chapitre

- Arbre de jeux
- Algorithme minimax et élagage alpha-bêta

15.2. Jeu d'échecs

Nous avons déjà vu le mouvement des pièces du jeu d'échecs au chapitre 14. Nous y reviendrons au § 15.3.3. Pour les règles complètes du jeu, nous renvoyons le lecteur par exemple à la page « Échecs » de Wikipédia : <https://fr.wikipedia.org/wiki/Échecs>.

Un des objectifs des premiers informaticiens a été de mettre au point des machines capables de jouer aux échecs, mais ce n'est qu'en 1997 que Deep Blue est devenu le premier ordinateur à battre un champion du monde en titre, dans un match qui l'opposait à Garry **Kasparov**. Cependant, ce rêve d'intelligence artificielle date de bien avant...

Le **Turc mécanique** (voir l'image en haut de cette page) est un canular célèbre construit à la fin du 18^{ème} siècle : ce prétendu automate jouait aux échecs. Construit et dévoilé pour la première fois en 1770 par Johann Wolfgang **von Kempelen**, le mécanisme semble être en mesure de jouer contre un adversaire humain, ainsi que de résoudre le problème du cavalier (voir chapitre 14).

En réalité, le mécanisme n'était qu'une illusion permettant de masquer la profondeur réelle du meuble. Celui-ci possédait un autre compartiment secret dans lequel un vrai joueur pouvait se glisser, et manipuler le mannequin sans être vu de quiconque. L'automate était alors capable de jouer une vraie partie d'échecs contre un adversaire humain. Grâce au talent de ses joueurs cachés, le Turc mécanique remporta la plupart des parties d'échecs auxquelles il participa en Europe et en Amérique durant près de 84 ans, y compris contre certains hommes d'état tels que Napoléon **Bonaparte**, Catherine II de Russie et Benjamin **Franklin**.

15.2.1. Source du programme étudié dans ce chapitre

Programmer un jeu d'échecs de A à Z serait trop long et trop difficile pour des programmeurs peu expérimentés. Aussi, nous allons plutôt analyser un programme écrit par Jean-François **Gazet** : *Jepychess*, qui signifie à peu de chose près « JeffProd Simple Python Chess Program ». Ce programme a l'avantage d'être écrit en Python 3 et d'être disponible gratuitement et intégralement sur le site de partage « GitHub » : <https://github.com/Tazeg/JePyChess>.

15.2.2. Licence GNU GPL

Ce programme est en outre placé sous la *Licence publique générale GNU, version 2*. L'objectif de la licence GNU GPL est de garantir à l'utilisateur les droits suivants (appelés libertés) sur un programme informatique :

1. la liberté d'exécuter le logiciel, pour n'importe quel usage ;
2. la liberté d'étudier le fonctionnement d'un programme et de l'adapter à ses besoins, ce qui passe par l'accès aux codes sources ;
3. la liberté de redistribuer des copies ;
4. l'obligation de faire bénéficier des versions modifiées à la communauté.

La quatrième liberté passe par un choix : la deuxième autorisant de modifier un programme, il n'est pas tenu de publier une version modifiée tant qu'elle est pour un usage personnel ; par contre, en cas de distribution d'une version modifiée, la quatrième liberté amène l'obligation à ce que les modifications soient retournées à la communauté sous la même licence. Pour en savoir plus :

https://fr.wikipedia.org/wiki/Licence_publice_générale_GNU

15.3. Représentation du jeu

Les explications ci-dessous proviennent en grande partie du blog de l'auteur du programme : <http://fr.jeffprod.com/blog/2014/comment-programmer-un-jeu-dechecs.html>

Avant de passer au programme, nous allons voir les différentes composantes d'un jeu d'échecs et comment les implémenter.

On peut décomposer un logiciel d'échecs en quatre parties :

- un échiquier ;
- des pièces d'échecs ;
- un moteur de jeu ;
- une interface graphique.

Ainsi, le projet sera composé de quatre fichiers :

- **piece.py** : les attributs et les méthodes des pièces d'échecs ;
- **board.py** : les attributs et les méthodes de l'échiquier ;
- **engine.py** : l'intelligence artificielle du jeu, capable d'analyser une position de l'échiquier et déterminer le meilleur coup ;
- **main.py** : le programme principal qui affiche l'échiquier et répond aux sollicitations de l'utilisateur.

15.3.1. L'échiquier

Il existe plusieurs façons de représenter l'échiquier. Étant composé de 64 cases, le plus simple en programmation est d'utiliser un tableau de 64 éléments. Les indices de ce tableau vont de 0 à 63. Ainsi la case a8 correspond à 0, et h1 à 63, comme le montre le dessin ci-dessous.

8	0	1	2	3	4	5	6	7
7	8	9	10	11	12	13	14	15
6	16	17	18	19	20	21	22	23
5	24	25	26	27	28	29	30	31
4	32	33	34	35	36	37	38	39
3	40	41	42	43	44	45	46	47
2	48	49	50	51	52	53	54	55
1	56	57	58	59	60	61	62	63
	a	b	c	d	e	f	g	h

```
coord=[
    'a8','b8','c8','d8','e8','f8','g8','h8',
    'a7','b7','c7','d7','e7','f7','g7','h7',
    'a6','b6','c6','d6','e6','f6','g6','h6',
    'a5','b5','c5','d5','e5','f5','g5','h5',
    'a4','b4','c4','d4','e4','f4','g4','h4',
    'a3','b3','c3','d3','e3','f3','g3','h3',
    'a2','b2','c2','d2','e2','f2','g2','h2',
    'a1','b1','c1','d1','e1','f1','g1','h1'
]
```

Avec la définition du tableau ci-dessus, `coord[3]` retournera `'d8'`.

Sur une case de l'échiquier sera éventuellement posée une pièce. Pour des raisons pratiques, on mettra toujours un objet-pièce sur chaque case de l'échiquier mais cet objet pourra être vide, ce qui se traduira par `Piece()`.

```
self.cases = [
    Piece('TOUR', 'noir'), Piece('CAVALIER', 'noir'), Piece('FOU', 'noir'),
    Piece('DAME', 'noir'), Piece('ROI', 'noir'), Piece('FOU', 'noir'),
    Piece('CAVALIER', 'noir'), Piece('TOUR', 'noir'),

    Piece('PION', 'noir'), Piece('PION', 'noir'), Piece('PION', 'noir'),
    Piece('PION', 'noir'), Piece('PION', 'noir'), Piece('PION', 'noir'),
    Piece('PION', 'noir'), Piece('PION', 'noir'), Piece('PION', 'noir'),

    Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(),

    Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(),

    Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(),

    Piece('PION', 'blanc'), Piece('PION', 'blanc'), Piece('PION', 'blanc'),
    Piece('PION', 'blanc'), Piece('PION', 'blanc'), Piece('PION', 'blanc'),
    Piece('PION', 'blanc'), Piece('PION', 'blanc'),

    Piece('TOUR', 'blanc'), Piece('CAVALIER', 'blanc'), Piece('FOU', 'blanc'),
    Piece('DAME', 'blanc'), Piece('ROI', 'blanc'), Piece('FOU', 'blanc'),
    Piece('CAVALIER', 'blanc'), Piece('TOUR', 'blanc')
]
```

Ainsi, en parcourant les indices (de 0 à 63) du tableau ci-dessus :

- `self.cases[1]` est un cavalier noir ;
- `self.cases[35]` est vide ;
- `self.cases[60]` est le roi blanc.

Outre ses cases, l'échiquier possède les attributs suivants :

- le camp qui a le trait (à qui est-ce de jouer ?) ;
- la case d'une prise en passant ;
- le nombre de coups joués depuis le début de la partie ;
- un historique des coups joués ;
- les droits au roque.

On va créer les méthodes suivantes dans l'objet `Echiquier` :

- générer la liste de tous les coups pour un camp donné ;
- possibilité de définir une position des pièces au format FEN (voir ex. 14.13) ;
- possibilité d'exporter cette position avec la notation FEN ;
- déplacer une pièce selon les règles du jeu ;
- annuler le dernier coup joué ;
- changer le joueur qui a le trait ;
- savoir si un roi est en échec ;
- savoir si une pièce est attaquée ;
- afficher l'échiquier en mode console ;
- afficher l'historique des coups joués ;
- donner une note à une position.

15.3.2. Les pièces

Comme vous le savez, les pièces sont : roi, dame, tour, cavalier, fou, pion. Certaines sont plus importantes que d'autres et il peut être grave de se les faire prendre. On leur attribue donc une valeur : traditionnellement 9 points pour la dame, 5 pour la tour, 3 pour le cavalier, 3 pour le fou, et 1 pour le pion. Comme le roi ne peut pas être pris, il n'est pas utile de lui donner une valeur.

```
nomPiece=(VIDE, 'ROI', 'DAME', 'TOUR', 'CAVALIER', 'FOU', 'PION')
valeurPiece=(0,0,9,5,3,3,1)
```

Une pièce a les attributs suivants :

- un nom (ROI, DAME, TOUR, ...);
- une couleur (blanc, noir);
- une valeur (définie en points ci-dessus).

15.3.3. Les déplacements des pièces

Les pièces peuvent être déplacées d'une case à une autre selon les règles propres à chacune. On va donc créer les fonctions permettant de trouver l'ensemble des cases de destination d'une pièce d'après sa case de départ.

Prenons un cas concret : une tour est placée sur la case **a4** (indice 32). Si on veut la déplacer d'une case vers la droite, pas de problème : il suffit d'ajouter 1. L'indice 33 correspond bien à la case **b4**. Par contre, si on essaie de la déplacer vers la gauche, ce qui n'est pas possible, elle se retrouverait sur la case $32-1=31$ soit **h5** !

Pour éviter ce débordement, nous utiliserons la méthode de Robert **Hyatt** appelée « mailbox ». Il s'agit de deux tableaux de 64 et 120 éléments, ressemblant à une boîte aux lettres selon son créateur, d'où son nom :



Robert **Hyatt** est un informaticien américain, spécialiste de la programmation des jeux d'échecs.

```
tab120 = (
-1, -1, -1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1, -1, -1,
-1, 0, 1, 2, 3, 4, 5, 6, 7, -1,
-1, 8, 9, 10, 11, 12, 13, 14, 15, -1,
-1, 16, 17, 18, 19, 20, 21, 22, 23, -1,
-1, 24, 25, 26, 27, 28, 29, 30, 31, -1,
-1, 32, 33, 34, 35, 36, 37, 38, 39, -1,
-1, 40, 41, 42, 43, 44, 45, 46, 47, -1,
-1, 48, 49, 50, 51, 52, 53, 54, 55, -1,
-1, 56, 57, 58, 59, 60, 61, 62, 63, -1,
-1, -1, -1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1, -1, -1
)
```

```
tab64 = (
21, 22, 23, 24, 25, 26, 27, 28,
31, 32, 33, 34, 35, 36, 37, 38,
41, 42, 43, 44, 45, 46, 47, 48,
51, 52, 53, 54, 55, 56, 57, 58,
61, 62, 63, 64, 65, 66, 67, 68,
71, 72, 73, 74, 75, 76, 77, 78,
81, 82, 83, 84, 85, 86, 87, 88,
91, 92, 93, 94, 95, 96, 97, 98
)
```

-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	0	1	2	3	4	5	6	7	-1
-1	8	9	10	11	12	13	14	15	-1
-1	16	17	18	19	20	21	22	23	-1
-1	24	25	26	27	28	29	30	31	-1
-1	32	33	34	35	36	37	38	39	-1
-1	40	41	42	43	44	45	46	47	-1
-1	48	49	50	51	52	53	54	55	-1
-1	56	57	58	59	60	61	62	63	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

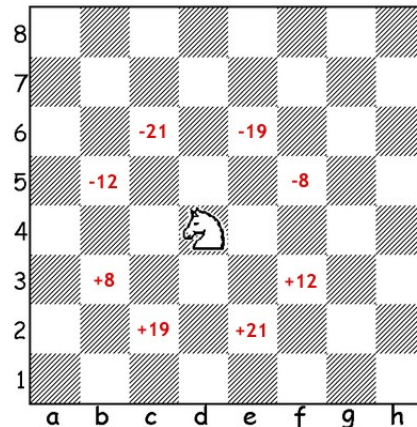
8	21	22	23	24	25	26	27	28
7	31	32	33	34	35	36	37	38
6	41	42	43	44	45	46	47	48
5	51	52	53	54	55	56	57	58
4	61	62	63	64	65	66	67	68
3	71	72	73	74	75	76	77	78
2	81	82	83	84	85	86	87	88
1	91	92	93	94	95	96	97	98
	a	b	c	d	e	f	g	h

Dans le tableau « tab64 », la case **a4** correspond la valeur 61.

On déplace une tour d'une case à gauche en soustrayant 1, on obtient 60. Regardons le 60^{ème} élément dans le tableau « tab120 » : c'est -1. Cette valeur indique une sortie de plateau.

Toujours d'après le tableau « tab64 », on peut définir les directions des déplacements possibles des pièces :

```
deplacements_tour = (-10,10,-1,1) # depl. vertical et horizontal
deplacements_fou = (-11,-9,11,9) # depl. en diagonal
deplacements_cavalier = (-12,-21,-19,-8,12,21,19,8) # 8 depl. du cavalier
```



Déplacements du cavalier (sur le tableau tab64)

Inutile de spécifier des vecteurs pour la dame et le roi car ceux-ci se déplacent à la fois comme le fou et la tour, mais d'une seule case à la fois pour le roi.

15.3.3.1. Les déplacements du roi

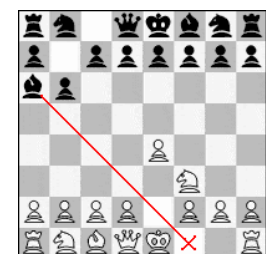
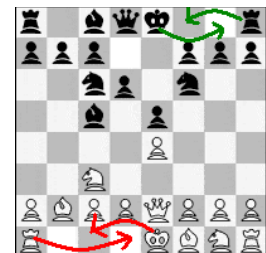
(*Roque* = *Castle*
moves en anglais)

Le roi est simple à déplacer (mouvement d'une case dans les huit directions possibles), sauf pour la règle du *roque*. Ce coup consiste à déplacer horizontalement le roi de deux cases vers l'une des deux tours de son camp et de placer cette tour sur la dernière case que le roi a traversée. On parle de *petit roque* pour la tour de la colonne **h** et de *grand roque*, pour la tour de la colonne **a**. Ci-contre, les blancs vont faire un grand roque et les noirs un petit roque.

Le petit roque se note 0-0 et le grand roque 0-0-0.

Le roque nécessite trois conditions :

- toutes les cases entre le roi et la tour doivent être vides ;
- ni le roi, ni la tour ne doivent avoir bougé avant le roque. Par conséquent, chaque camp ne peut roquer qu'une seule fois par partie.
- aucune des trois cases (de départ, de passage ou d'arrivée) traversée par le roi lors du roque ne doit être menacée par une pièce adverse. En particulier, le roque ne permet pas d'esquiver un échec. Ci-contre, les blancs ne peuvent pas roquer puisque le fou noir en **a6** menace la case **f1** (symbolisée par la croix rouge) par laquelle le roi devrait transiter pour faire son roque. En revanche, la tour, ainsi que sa case adjacente dans le cas du grand roque, peuvent être menacées.



```
def pos2_roi(self, pos1, cAd, echiquier, dontCallIsAttacked=False):

    """Returns the list of moves for the king :
    - which is at square 'pos1'
    - which opponent color is 'cAd' (blanc, noir)
    - dontCallIsAttacked is set to avoid recursive calls
    between is_attacked() and gen_moves_list().
    """

    liste=[]
```

```

for i in (self.deplacements_tour+self.deplacements_fou):
    n=self.tab120[self.tab64[pos1]+i]
    if n!=-1:
        if(echiquier.cases[n].isEmpty() or echiquier.cases[n].couleur==cAd):
            liste.append((pos1,n,''))

if dontCallIsAttacked:
    return liste # we just wanted moves that can attack

# The current side to move is the opposite of cAd
c=echiquier.oppColor(cAd)

# Castle moves
if c=='blanc':
    if echiquier.white_can_castle_63):
        # If a rook is at square 63
        # And if squares between KING and ROOK are empty
        # And if squares on which KING walks are not attacked
        # And if KING is not in check
        # Then we can add this castle move
        if echiquier.cases[63].nom=='TOUR' and \
echiquier.cases[63].couleur=='blanc' and \
echiquier.cases[61].isEmpty() and \
echiquier.cases[62].isEmpty() and \
echiquier.is_attacked(61,'noir')==False and \
echiquier.is_attacked(62,'noir')==False and \
echiquier.is_attacked(pos1,'noir')==False:
            liste.append((pos1,62,''))
    if echiquier.white_can_castle_56):
        # If a rook is at square 56
        if echiquier.cases[56].nom=='TOUR' and \
echiquier.cases[56].couleur=='blanc' and \
echiquier.cases[57].isEmpty() and \
echiquier.cases[58].isEmpty() and \
echiquier.cases[59].isEmpty() and \
echiquier.is_attacked(58,cAd)==False and \
echiquier.is_attacked(59,cAd)==False and \
echiquier.is_attacked(pos1,cAd)==False:
            liste.append((pos1,58,''))
elif c=='noir':
    if echiquier.black_can_castle_7):
        if echiquier.cases[7].nom=='TOUR' and \
echiquier.cases[7].couleur=='noir' and \
echiquier.cases[5].isEmpty() and \
echiquier.cases[6].isEmpty() and \
echiquier.is_attacked(5,cAd)==False and \
echiquier.is_attacked(6,cAd)==False and \
echiquier.is_attacked(pos1,cAd)==False:
            liste.append((pos1,6,''))
    if echiquier.black_can_castle_0):
        if echiquier.cases[0].nom=='TOUR' and \
echiquier.cases[0].couleur=='noir' and \
echiquier.cases[1].isEmpty() and \
echiquier.cases[2].isEmpty() and \
echiquier.cases[3].isEmpty() and \
echiquier.is_attacked(2,cAd)==False and \
echiquier.is_attacked(3,cAd)==False and \
echiquier.is_attacked(pos1,cAd)==False:
            liste.append((pos1,2,''))

return liste

```

15.3.3.2. Les déplacements de la tour

La tour se déplace verticalement et horizontalement. Elle peut se déplacer tant qu'elle n'est pas bloquée par une autre pièce (amie ou ennemie).

```

def pos2_tour(self,pos1,cAd,echiquier):

    """Returns the list of moves for a ROOK :
    - at square number 'pos1' (0 to 63)

```

```

- opponent color is cAd (blanc,noir)"""

liste=[]

for k in self.deplacements_tour:
    j=1
    while True:
        n=self.tab120[self.tab64[pos1] + (k * j)]
        if n!=-1: # as we are not out of the board
            if(echiquier.cases[n].isEmpty() or echiquier.cases[n].couleur==cAd):
                liste.append((pos1,n,''))
                # append the move if square is empty of opponent color
            else:
                break # stop if outside of the board
        if not echiquier.cases[n].isEmpty():
            break # destination square is not empty (opponent or not)
                # then the rook won't pass through
        j=j+1

return liste

```

15.3.3.3. Les déplacements du cavalier

Paradoxalement, le cavalier, qui a les mouvements les plus « bizarres », est la pièce la plus simple à déplacer informatiquement parlant, car c'est la seule pièce qui peut passer par-dessus d'autres pièces.

```

def pos2_cavalier(self,pos1,cAd,echiquier):

    """Returns the list of moves for a KNIGHT :
    - at square number 'pos1' (0 to 63)
    - opponent color is cAd (blanc,noir)"""

    liste=[]

    for i in self.deplacements_cavalier:
        n=self.tab120[self.tab64[pos1]+i]
        if n!=-1:
            if(echiquier.cases[n].isEmpty() or echiquier.cases[n].couleur==cAd):
                liste.append((pos1,n,''))

    return liste

```

15.3.3.4. Les déplacements du fou

Le fou se déplace en diagonale. Il peut se déplacer tant qu'il n'est pas bloqué par une autre pièce (amie ou ennemie).

```

def pos2_fou(self,pos1,cAd,echiquier):

    """Returns the list of moves for a BISHOP :
    - at square number 'pos1' (0 to 63)
    - opponent color is cAd (blanc,noir)"""

    liste=[]

    for k in self.deplacements_fou:
        j=1
        while True:
            n=self.tab120[self.tab64[pos1] + (k * j)]
            if(n!=-1): # as we are not out of the board
                if(echiquier.cases[n].isEmpty() or echiquier.cases[n].couleur==cAd):
                    liste.append((pos1,n,''))
                    # append the move if square is empty of opponent color
                else:
                    break # stop if outside of the board
            if not echiquier.cases[n].isEmpty():
                break # destination square is not empty (opponent or not)
                    # then the bishop won't pass through
            j=j+1

    return liste

```

```
j=j+1
return liste
```

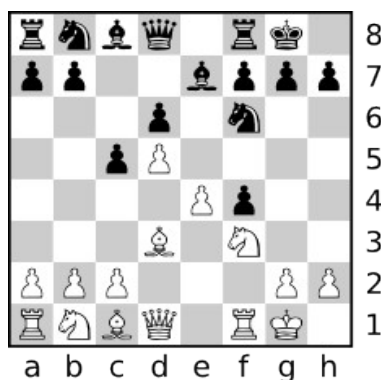
15.3.3.5. Les déplacements de la dame

On n'a pas besoin de programmer une fonction qui donne les coups possibles de la dame. Il suffit de combiner les cases atteignables par un fou et celles atteignables par une tour.

15.3.3.6. Les déplacements du pion

Le pion est assez complexe à gérer. Il peut seulement avancer devant lui et jamais reculer, en général d'une case (vide), mais éventuellement de deux cases lors de son premier déplacement.

Cependant, la prise d'une pièce adverse se fait en diagonale, et d'une case. Il existe en outre une prise spéciale, la *prise en passant*, qui obéit à des règles strictes : lorsqu'un pion se trouve sur la cinquième rangée et que l'adversaire avance un pion d'une colonne voisine de deux cases (les deux pions se retrouvent alors côte à côte sur la même rangée), le premier pion peut prendre le second. Pour effectuer la prise en passant, le joueur avance son pion en diagonale, derrière le pion à capturer (sur la sixième rangée) et ôte le pion adverse de l'échiquier. C'est le seul cas où on ne remplace une pièce adverse par la sienne. La *prise en passant* doit se faire immédiatement après le double pas du pion adverse ; elle n'est plus possible par la suite.



Exemple : les noirs viennent de jouer c7-c5.

Les blancs peuvent prendre le pion en c5 en jouant d5-c6 e.p., mais seulement ce coup-ci.
(e.p. est l'abréviation de « en passant »).

Pour finir, lorsqu'un pion atteint la dernière rangée, il est promu et peut se transformer en n'importe quelle pièce, sauf le roi. Après la promotion d'un pion blanc, il peut très bien y avoir deux dames blanches sur l'échiquier (ou plus).

```
def pos2_pion(self, pos1, couleur, echiquier):

    """Returns the list of moves for a PAWN :
    - at square number 'pos1' (0 to 63)
    - opponent color is cAd (blanc, noir)"""

    liste=[]

    # White PAWN -----
    if couleur=='blanc':

        # Upper square
        n=self.tab120[self.tab64[pos1]-10]
        if n!=-1:
            if echiquier.cases[n].isEmpty():
                # If the PAWN has arrived to rank 8 (square 0 to 7)
                if n<8:
                    # it will be promoted
                    liste.append((pos1,n,'q'))
                    liste.append((pos1,n,'r'))
                    liste.append((pos1,n,'n'))
```

```

        liste.append((pos1,n,'b'))
    else:
        liste.append((pos1,n,''))

    # 2nd square if PAWN is at starting square
    if echiquier.ROW(pos1)==6:
        # If the 2 upper squares are empty
        if echiquier.cases[pos1-8].isEmpty() and echiquier.cases[pos1-16].isEmpty():
            liste.append((pos1,pos1-16,''))

    # Capture upper left
    n=self.tab120[self.tab64[pos1]-11]
    if n!=-1:
        if echiquier.cases[n].couleur=='noir' or echiquier.ep==n:
            if n<8 # Capture + promote
                liste.append((pos1,n,'q'))
                liste.append((pos1,n,'r'))
                liste.append((pos1,n,'n'))
                liste.append((pos1,n,'b'))
            else:
                liste.append((pos1,n,''))

    # Capture upper right
    n=self.tab120[self.tab64[pos1]-9]
    if n!=-1:
        if echiquier.cases[n].couleur=='noir' or echiquier.ep==n:
            if n<8:
                liste.append((pos1,n,'q'))
                liste.append((pos1,n,'r'))
                liste.append((pos1,n,'n'))
                liste.append((pos1,n,'b'))
            else:
                liste.append((pos1,n,''))

    # Black PAWN -----
    else:

        # Upper square
        n=self.tab120[self.tab64[pos1]+10]
        if n!=-1:
            if echiquier.cases[n].isEmpty():
                # PAWN has arrived to 8th rank (square 56 to 63),
                # it will be promoted
                if n>55 :
                    liste.append((pos1,n,'q'))
                    liste.append((pos1,n,'r'))
                    liste.append((pos1,n,'n'))
                    liste.append((pos1,n,'b'))
                else:
                    liste.append((pos1,n,''))

        # 2nd square if PAWN is at starting square
        if echiquier.ROW(pos1)==1:
            # If the 2 upper squares are empty
            if echiquier.cases[pos1+8].isEmpty() and echiquier.cases[pos1+16].isEmpty():
                liste.append((pos1,pos1+16,''))

        # Capture bottom left
        n=self.tab120[self.tab64[pos1]+9]
        if n!=-1:
            if echiquier.cases[n].couleur=='blanc' or echiquier.ep==n:
                if n>55:
                    liste.append((pos1,n,'q'))
                    liste.append((pos1,n,'r'))
                    liste.append((pos1,n,'n'))
                    liste.append((pos1,n,'b'))
                else:
                    liste.append((pos1,n,''))

        # Capture bottom right
        n=self.tab120[self.tab64[pos1]+11]
        if n!=-1:
            if echiquier.cases[n].couleur=='blanc' or echiquier.ep==n:

```



```
if n>55:
    liste.append((pos1,n,'q'))
    liste.append((pos1,n,'r'))
    liste.append((pos1,n,'n'))
    liste.append((pos1,n,'b'))
else:
    liste.append((pos1,n,''))

return liste
```

Toutes ces fonctions retournent la liste des coups possibles pour chaque pièce. Il s'agira ensuite de choisir parmi tous ces coups le meilleur. C'est le sujet du paragraphe suivant.



John McCarthy
(1927-2011)



Marvin Lee
Minsky
(1927-2016)

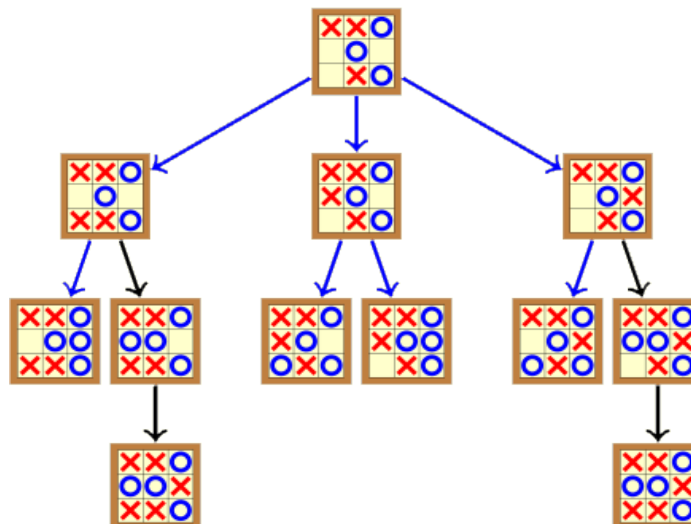
15.4. Intelligence artificielle

Le terme « intelligence artificielle », créé par John McCarthy, est souvent abrégé par le sigle « I. A. » (ou « A. I. » en anglais, pour *Artificial Intelligence*). Il est défini par l'un de ses créateurs, Marvin Lee Minsky, comme « la construction de programmes informatiques qui s'adonnent à des tâches qui sont, pour l'instant, accomplies de façon plus satisfaisante par des êtres humains, car elles demandent des processus mentaux de haut niveau tels que : l'apprentissage perceptuel, l'organisation de la mémoire et le raisonnement critique ».

On y trouve donc le côté « artificiel » atteint par l'usage des ordinateurs ou de processus électroniques élaborés et le côté « intelligence » associé à son but d'imiter le comportement. Cette imitation peut se faire dans le raisonnement, par exemple dans les jeux ou la pratique des mathématiques, dans la compréhension des langues naturelles, dans la perception : visuelle (interprétation des images et des scènes), auditive (compréhension du langage parlé) ou par d'autres capteurs, dans la commande d'un robot dans un milieu inconnu ou hostile.

15.4.1. Arbre de jeu

Un arbre de jeu est une représentation imagée de toutes les positions que l'on peut atteindre à partir de celles du coup précédent. Voici l'arbre de jeu d'une fin de partie de jeu de morpion.



Selon le jeu, cet arbre devient gigantesque à partir de peu d'étages déjà. Aux échecs par exemple, les blancs ont le choix entre 20 coups (16 coups de pions et 4 coups de cavaliers) pour débiter la partie. Cela fait donc déjà 20 branches qui partent du sommet. Ensuite, les noirs ont aussi 20 coups à disposition. Cela fait déjà 400 positions possibles après 2 coups, puisque 20 branches partent des 20 positions précédentes ! Autant dire, que, sauf pour des jeux très simples, il est impossible de dessiner l'arbre de jeu complet. Aussi doit-on élaguer l'arbre pour éliminer les positions manifestement mauvaises pour le joueur. Nous verrons plus tard comment s'y prendre.

Voici ci-dessous un arbre partiel pour une fin de partie d'échecs, sur un échiquier 4x4.



Insistons sur le fait que cet arbre ne devra pas être implémenté tel quel. Il est utilisé par l'homme pour se représenter toutes les suites de coups possibles, mais l'ordinateur n'en a pas besoin.

15.4.2. L'évaluation

Pour « évaluer » une position, c'est-à-dire savoir si elle est bonne ou mauvaise, il faut lui attribuer une valeur. Chaque position de l'échiquier est analysée selon plusieurs critères. Une note est calculée

en fonction des pièces présentes et de nombreux bonus/malus. Voici la plus simple des fonctions d'évaluation, car elle ne prend en compte que le matériel :

```
def evaluer(self):  
  
    WhiteScore=0  
    BlackScore=0  
  
    # Parsing the board squares from 0 to 63  
    for pos1,piece in enumerate(self.cases):  
  
        # Material score  
        if piece.couleur=='blanc':  
            WhiteScore+=piece.valeur  
        else:  
            # NB : here is for black piece or empty square  
            BlackScore+=piece.valeur  
  
    if self.side2move=='blanc':  
        return WhiteScore-BlackScore  
    else:  
        return BlackScore-WhiteScore
```

On pourrait compléter cette fonction d'évaluation avec les bonus et malus suivants :

- pion passé : pion qui ne rencontrera plus de pion adverse et qui peut donc aller en promotion ;
- tour en 7^{ème} rangée ;
- roi dans une colonne ouverte ;
- pions du roque avancés ;
- le roi a été déplacé et ne peut plus roquer ;
- pions doublés (deux pions dans la même colonne) ;
- pion isolé : pion qui ne peut plus être protégé par un autre pion.

On peut également accorder des bonus/malus selon l'avancement de la partie :

- en début de partie, il est préférable de roquer pour protéger le roi. Au contraire, en fin de partie, il faut placer le roi au centre pour éviter un mat sur un bord ou dans un coin de l'échiquier ;
- un cavalier peut atteindre un plus grand nombre de cases s'il est centré ;
- les pions prennent de la valeur s'ils approchent la 8^{ème} rangée.

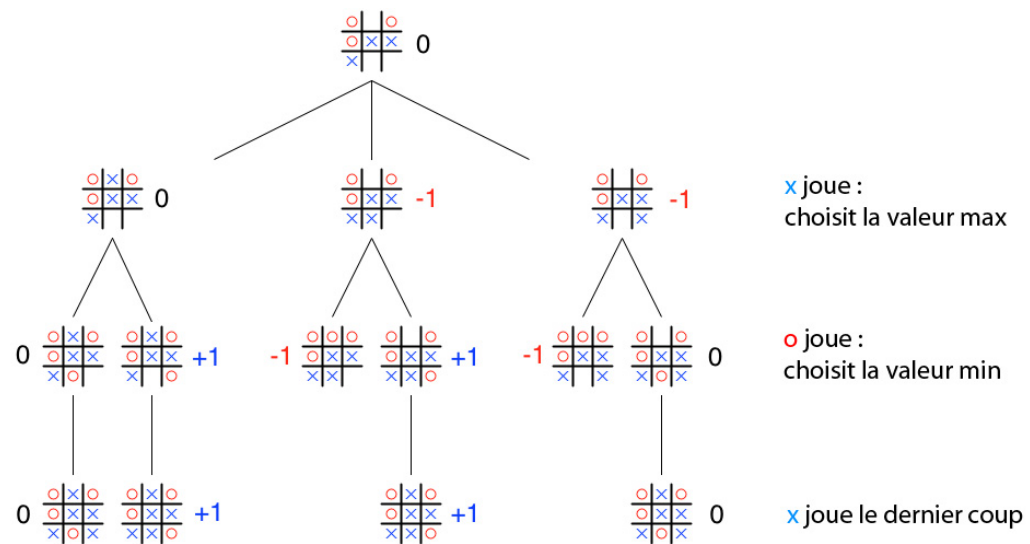
15.4.3. La recherche du meilleur coup

C'est à l'ordinateur de jouer. Il génère la liste de tous les coups possibles. Il déplace une pièce. Il attribue une note à la position. Il se met à la place de l'adversaire et il fait exactement la même chose, et ainsi de suite. Il faut donc préciser au moteur de recherche une profondeur fixe (par exemple 5 déplacements) ou un temps donné de réflexion. Le programme va donc élaborer un arbre de recherche. Il construit une arborescence de positions de l'échiquier où chacune est notée.

Cet arbre fait penser à un arbre généalogique : le père est en haut, il a un certain nombre de fils, qui ont eux-mêmes des fils, etc.

Prenons comme exemple un arbre de jeu de morpion. On descend dans l'arbre le plus possible, jusqu'à trouver une fin de partie : soit un match nul (auquel on attribue la valeur 0), une victoire des croix (valeur de +1) ou une victoire des ronds (valeur de -1).

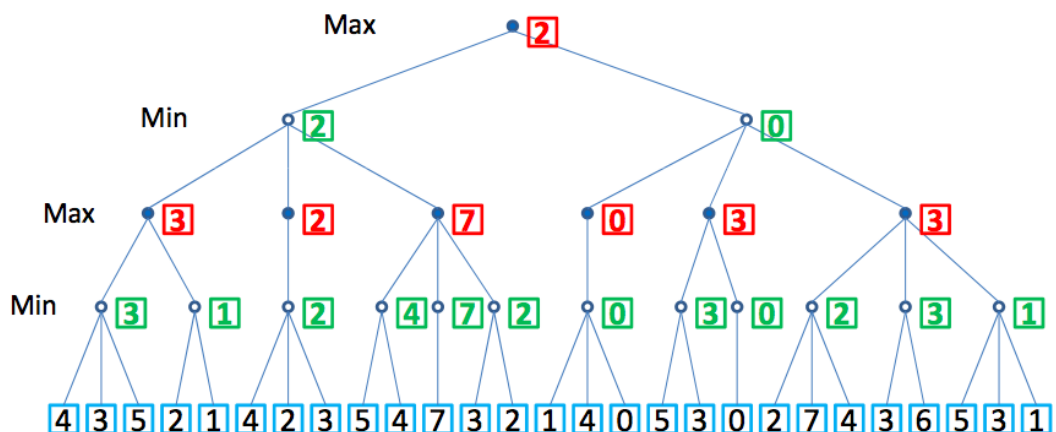
On « remonte » ensuite ces valeurs vers le haut de l'arbre en donnant au père la valeur minimale de ses fils si c'est les ronds qui jouent et la valeur maximale si c'est les croix qui jouent (d'où le nom d'**algorithme minimax**).



La valeur obtenue en haut de l'arbre est 0. Cela signifie que si les deux joueurs jouent parfaitement, l'issue sera un match nul (la position finale sera celle tout à gauche, en bas de l'arbre).

Aux échecs, contrairement au morpion, il est impossible d'explorer toutes les branches. L'élagage **alpha-bêta** permet de « couper » des branches.

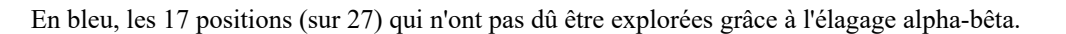
Prenons un arbre de jeu quelconque dont les valeurs ont été calculées selon l'algorithme minimax.



On pourrait se demander s'il est vraiment nécessaire d'évaluer toutes les positions. La réponse est non. Certaines branches peuvent être élaguées, car elles ne changeront pas la valeur attribuée au père. Les positions finales sont examinées de gauche à droite.

Observons la 5^{ème} position, évaluée à 1. Le père des 3 premières positions est évalué à 3 (le minimum de ses 3 fils). Pour l'instant le grand-père est évalué à 3 et le père des positions 4 et 5 vaut 2. Comme ce père va choisir la valeur minimale de ses fils, sa valeur ne pourra pas dépasser 2. Et comme le grand-père prend la valeur maximale de ses deux fils, sa valeur de 3 ne pourra pas changer, quelle que soit l'évaluation de la position 5. Il est donc inutile d'explorer cette branche.

Autre cas. La 10^{ème} position a été explorée. On a remonté la valeur 4 ($\min(5,4)$) vers le père et le grand-père. La valeur du grand-père ne pourra pas descendre en dessous de 4, puisque l'on va prendre la valeur maximale des fils. Mais comme l'arrière-grand-père va prendre la valeur minimale de ses fils, il est inutile d'explorer d'avantage cette branche.



```
def alphabeta(self, depth, alpha, beta, b):

    # We arrived at the end of the search : return the board score
    if depth==0:
        return b.evaluator()
        # TOTO : return quiesce(alpha,beta)

    self.nodes+=1
    self.pv_length[b.ply] = b.ply

    # Do not go too deep
    if b.ply >= self.MAX_PLY-1:
        return b.evaluator()

    # Extensions
    # If king is in check, let's go deeper
    chk=b.in_check(b.side2move) # 'chk' used at the end of func too
    if chk:
        depth+=1

    # Generate all moves for the side to move. Those who
    # let king in check will be processed in domove()
    mList=b.gen_moves_list()

    f=False # flag to know if at least one move will be done
    for i,m in enumerate(mList):

        # Do the move 'm'.
        # If it lets king in check, undo it and ignore it
        # remind : a move is defined with (pos1,pos2,promote)
        # i.e. : 'e7e8q' is (12,4,'q')
        if not b.domove(m[0],m[1],m[2]):
            continue

        f=True # a move has passed

        score=-self.alphabeta(depth-1,-beta,-alpha,b)

        # Unmake move
        b.undomove()

        if score>alpha:
```

```

        # this move caused a cutoff,
        if(score>=beta):
            return beta
        alpha = score

    # Updating the triangular PV-Table
    self.pv[b.ply][b.ply] = m
    j = b.ply + 1
    while j<self.pv_length[b.ply+1]:
        self.pv[b.ply][j] = self.pv[b.ply+1][j]
        self.pv_length[b.ply] = self.pv_length[b.ply + 1]
        j+=1

    # If no move has been done : it is DRAW or MAT
    if not f:
        if chk:
            return -self.INFINITY + b.ply # MAT
        else:
            return 0 # draw

    return alpha

```

15.5. Améliorations possibles

Le code a été écrit au plus simple. Il est possible par exemple de l'optimiser pour augmenter la vitesse de calcul.

Il reste également à implémenter :

- move ordering : suivre la variation principale et les captures de pièces en premier pour la recherche des meilleurs coups ;
- la règle des 50 coups sans prise (partie nulle) ;
- la règle des 3 répétitions de position (partie nulle) ;
- la gestion du temps (partie en blitz, recherche par temps fixe...) ;
- détecter les parties nulles pour matériel insuffisant ;
- bibliothèque d'ouvertures ;
- base de données de fins de partie ;
- recherche « quiescent » : au lieu de stopper net sur la note d'une position, si une pièce vient d'être prise, il peut s'avérer utile de continuer la recherche des prises uniquement, ceci afin de limiter l'effet d'horizon.



Exercice 15.1

Pouvez-vous répondre aux questions suivantes ?

1. Qu'est-ce que la « mailbox » ? À quoi sert-elle ?
2. Qu'est-ce qu'un arbre de jeu ?
3. Qu'est-ce que l'algorithme minimax ?
4. À quoi sert l'élagage alpha-bêta et comment cela fonctionne-t-il ?
5. Qu'est-ce qu'une fonction d'évaluation ? Quelle est la fonction utilisée dans le programme étudié ?

15.6. Interface graphique

L'interface du programme créé par Jean-François **Gazet** est des plus rudimentaires : tous les déplacements se font par l'intermédiaire du clavier et l'échiquier est affiché à l'écran sous forme de chaînes de caractères.

Résumons les fonctions qui existent dans les différents modules avant de nous lancer dans la réalisation d'une interface graphique.

15.6.1. Le module « piece »

Le fichier **piece.py** ne contient que la classe `piece`, qui gère la création et les déplacements des pièces.

Les variables :

- `VIDE='.'`
- `nomPiece=(VIDE, 'ROI', 'DAME', 'TOUR', 'CAVALIER', 'FOU', 'PION')`
- `valeurPiece=(0,0,9,5,3,3,1)`
- les déplacements des pièces selon la description du § 15.3.3
 - `tab120` et `tab64`
 - `deplacements_tour=(-10,10,-1,1)`
 - `deplacements_fou=(-11,-9,11,9)`
 - `deplacements_cavalier=(-12,-21,-19,-8,12,21,19,8)`

Les fonctions :

```
def __init__(self,nom=VIDE,couleur=''):

    """Creating a piece object, with its attributes :
    - 'nom' as name (ROI, DAME...);
    - 'couleur' as color (blanc,noir);
    - 'valeur' as its value"""

def isEmpty(self):

    """Returns TRUE or FALSE if this piece object is defined,
    As any square on board can have a piece on it, or not,
    we can set a null piece on a square."""

def pos2_roi(self,pos1,cAd,echiquier,dontCallIsAttacked=False):

    """Returns the list of moves for the king :
    - which is at square 'pos1'
    - which opponent color is 'cAd' (blanc,noir)
    - dontCallIsAttacked is set to avoid recursive calls
    between is_attacked() and gen_moves_list()."""

def pos2_tour(self,pos1,cAd,echiquier):

    """Returns the list of moves for a ROOK :
    - at square number 'pos1' (0 to 63)
    - opponent color is cAd (blanc,noir)"""

def pos2_cavalier(self,pos1,cAd,echiquier):

    """Returns the list of moves for a KNIGHT :
    - at square number 'pos1' (0 to 63)
    - opponent color is cAd (blanc,noir)"""

def pos2_fou(self,pos1,cAd,echiquier):

    """Returns the list of moves for a BISHOP :
    - at square number 'pos1' (0 to 63)
    - opponent color is cAd (blanc,noir)"""

def pos2_pion(self,pos1,couleur,echiquier):

    """Returns the list of moves for a PAWN :
    - at square number 'pos1' (0 to 63)
    - opponent color is cAd (blanc,noir)"""
```

15.6.2. Le module « board »

Le fichier **board.py** ne contient que la classe `board`, qui gère l'échiquier : mouvements possibles des pièces, promotion, prise en passant, alternance des joueurs, etc.

La variable :

```
# Names of the 64 squares
coord=[
    'a8','b8','c8','d8','e8','f8','g8','h8',
    'a7','b7','c7','d7','e7','f7','g7','h7',
    'a6','b6','c6','d6','e6','f6','g6','h6',
    'a5','b5','c5','d5','e5','f5','g5','h5',
    'a4','b4','c4','d4','e4','f4','g4','h4',
    'a3','b3','c3','d3','e3','f3','g3','h3',
    'a2','b2','c2','d2','e2','f2','g2','h2',
    'a1','b1','c1','d1','e1','f1','g1','h1',
]
```

Les fonctions :

```
def init(self):

    "Init the chess board at starting position"

    # Chessboard has 64 squares, numbered from 0 to 63 (a8 to h1)
    # Placing pieces ('cases' is 'square' in french :)
```

```
def gen_moves_list(self,color=' ',dontCallIsAttacked=False):

    """Returns all possible moves for the requested color.
    If color is not given, it is considered as the side to move.
    dontCallIsAttacked is a boolean flag to avoid recursive calls,
    due to the actually wrotten is_attacked() function calling
    this gen_moves_list() function.
    A move is defined as it :
    - the number of the starting square (pos1)
    - the number of the destination square (pos2)
    - the name of the piece to promote '','q','r','b','n'
      (queen, rook, bishop, knight)
    """
```

```
def setboard(self,fen):

    """Set the board to the FEN position given. i.e. :
    rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - - 0
    Returns TRUE or FALSE if done or not.
    If not : print errors.
    """
```

```
def getboard(self):

    """Returns the FEN notation of the current board. i.e. :
    rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - - 0
    """
```

```
def domove(self,depart,arrivee,promote):

    """Move a piece on the board from the square numbers
    'depart' to 'arrivee' (0..63) respecting rules :
    - prise en passant
    - promote and under-promote
    - castle rights
    Returns :
    - TRUE if the move do not let king in check
    - FALSE otherwise and undomove is done.
    """
```

```
def undomove(self):

    "Undo the last move in history"
```

```
def changeTrait(self):

    "Change the side to move"
```



```
def oppColor(self,c):
    "Returns the opposite color of the 'c' color given"

def in_check(self,couleur):
    """Returns TRUE or FALSE
    if the KING of the given 'color' is in check"""

def is_attacked(self,pos,couleur):
    """Returns TRUE or FALSE if the square number 'pos' is a
    destination square for the color 'couleur'.
    If so we can say that 'pos' is attacked by this side.
    This function is used for 'in check' and for castle moves."""

def render(self):
    "Display the chessboard in console mode"

def caseStr2Int(self,c):
    """'c' given in argument is a square name like 'e2'
    This functino returns a square number like 52"""

def caseInt2Str(self,i):
    """Given in argument : an integer between 0 and 63
    Returns a string like 'e2'"""

def showHistory(self):
    "Displays the history of the moves played"

def evaluer(self):
    """A wonderful evaluate() function
    returning actually only the material score"""
```

15.6.3. Le module « engine »

Le fichier **engine.py** ne contient que la classe **engine**, qui implémente l'intelligence artificielle du programme. C'est notamment là que l'on parcourt l'arbre de jeu avec l'algorithme alpha-bêta.

```
def __init__(self):
    self.MAX_PLY=32
    self.pv_length=[0 for x in range(self.MAX_PLY)]
    self.INFINITY=32000
    self.init()

def usermove(self,b,c):
    """Move a piece for the side to move, asked in command line.
    The command 'c' in argument is like 'e2e4' or 'b7b8q'.
    Argument 'b' is the chessboard.
    """

def chkCmd(self,c):
    """Check if the command 'c' typed by user is like a move,
    i.e. 'e2e4', 'b7b8n'...
    Returns '' if correct.
    Returns a string error if not.
    """

def search(self,b):
    """Search the best move for the side to move,
    according to the given chessboard 'b'
    """
```

```

def alphabeta(self,depth,alpha,beta,b):

def print_result(self,b):
    "Check if the game is over and print the result"

def clear_pv(self):
    "Clear the triangular PV table containing best moves lines"

def setboard(self,b,c):
    """Set the chessboard to the FEN position given by user with
    the command line 'setboard ...'.
    'c' in argument is for example :
    'setboard 8/5k2/5P2/8/8/5K2/8/8 w - - 0 0'
    """

def setDepth(self,c):
    """'c' is the user command line, i.e. 'sd [x]'
    to set the search depth.
    """

def perft(self,c,b):
    """PERFformance Test :
    This is a debugging function through the move generation tree
    for the current board until depth [x].
    'c' is the command line written by user : perft [x]
    """

def legalmoves(self,b):
    "Show legal moves for side to move"

def getboard(self,b):
    """The user requests the current FEN position
    with the command 'getboard'"""

def newgame(self,b):

def bench(self,b):
    """Test to calculate the number of nodes a second.
    The position used is the 17th move of the game :
    Bobby Fischer vs. J. Sherwin, New Jersey State
    Open Championship, 9/2/1957 :
    1rb2rk1/p4ppp/1p1qp1n1/3n2N1/2pP4/2P3P1/PPQ2PBP/R1B1R1K1 w - - 0 1
    The engine searches to a given depth, 3 following times.
    The kilonodes/s is calculated with the best time.
    """

def undomove(self,b):
    "The user requested a 'undomove' in command line"

def get_ms(self):

def init(self):

```



Exercice 15.2

Créez une interface graphique du jeu d'échecs proposé dans ce chapitre. Il faudra principalement modifier le fichier **main.py**. Il faudra avoir bien compris les trois autres modules pour pouvoir les réutiliser, peut-être en les modifiant légèrement : **piece.py**, **board.py** et **engine.py**.

C'est un exercice long et difficile qui vous prendra probablement plusieurs dizaines d'heures.

Conseil : procédez par étapes et n'essayez surtout pas de tout faire en même temps !

15.7. Bibliothèque d'ouvertures

En jouant contre le programme que nous avons étudié, on constate, si l'on est un joueur d'échecs, que l'ordinateur joue des débuts de parties très étranges (et en plus toujours les mêmes). Cela n'est pas étonnant : vu que la fonction d'évaluation est uniquement basée sur l'avantage matériel, et que généralement les prises interviennent plus tard dans la partie, tous les coups sont jugés plus ou moins équivalents par l'intelligence artificielle.

La solution est de doter ce programme d'une bibliothèque d'ouvertures (*opening book* en anglais). Il s'agit d'un fichier où sont répertoriées des suites de premiers coups classiques (il y a notamment l'ouverture italienne, l'espagnole, la défense Pirc, la nimzo-indienne, etc.). Tout joueur d'échecs se doit de bien connaître ces ouvertures et l'ordinateur devra faire de même.

On va donc procéder ainsi pour améliorer le début de partie :

- on va donner à l'ordinateur quelques centaines d'ouvertures ;
- au début, le programme sera en mode « ouverture » ;
- s'il commence la partie, le programme jouera le premier coup d'une de ces ouvertures prise au hasard ;
- le programme ira fouiller dans la liste d'ouvertures pour trouver celle qui est en train de se jouer, et il se contentera de jouer les coups de cette ouverture tant qu'il le pourra ;
- quand la liste de coups est terminée ou lorsque l'adversaire ne jouera plus le coup « classique », le programme quittera le mode ouverture et utilisera son I.A.



Exercice 15.3

Vous trouverez sur www.nymphomath.ch/pj/echecs/ un fichier nommé **book.txt** qui contient plusieurs centaines d'ouvertures données sous la forme :

```
g1f3 g8f6 c2c4 b7b6 g2g3
```

Chaque ligne correspond à une ouverture. Le nombre de coups est variable d'une ligne à l'autre. Cette bibliothèque provient de la page de Tom Kerrigan (www.tckerrigan.com/Chess/TSCP).

Implémentez cette bibliothèque d'ouvertures comme expliqué dans le § 15.7. Modifiez le fichier **main.py**.



Exercice 15.4

Le programme de l'ex. 15.3 n'est pas encore terminé (loin s'en faut, voir § 15.5). Il y a encore beaucoup de fonctionnalités qui ne sont pas implémentées. Par exemple, lorsqu'un pion est promu, il n'est pas automatiquement remplacé par une dame. Il est parfois plus intéressant de le remplacer par une tour ou un cavalier (ou même un fou).

Modifiez le programme de l'ex. 15.3 pour que le joueur puisse choisir la pièce qui remplacera le pion promu.

15.8. Stockfish

Il existe évidemment bien d'autres programmes d'échecs open source. Le plus performant d'entre eux s'appelle *Stockfish 8*. Il a en effet remporté la finale 2016 du *Thoresen Chess Engine Championship*, considéré comme le championnat du monde non-officiel des programmes d'échecs.

Il est écrit en C++ et est disponible à l'adresse <https://stockfishchess.org/>. Le programme provient de *Glaurung*, un moteur de jeu d'échecs open source créé par Tord Romstad et sorti en 2004. Marco Costalba a créé *Stockfish 1.0* en novembre 2008 en reprenant la version 2.1 de *Glaurung* en corrigeant quelques bugs et en incluant des améliorations.



15.9. Ce que vous avez appris dans ce chapitre

- Nous avons fait dans ce chapitre ce que tout programmeur sera amené à faire un jour : reprendre et modifier le programme d'un autre. Ce n'est pas un exercice facile, car chaque programmeur a son style d'écriture, un peu comme un écrivain ou un dessinateur. Il faut donc d'abord comprendre comment le programme fonctionne avant de pouvoir le modifier. Cela montre bien la grande importance de la documentation fournie avec le programme et des commentaires placés dans le code.
- Vous avez vu les grandes idées qui se cachent dans un programme de jeu d'échecs.

15.10. Références

15.10.1 Sources utilisées pour ce chapitre

- Jean-François Gazet, *Comment programmer un jeu d'échecs*, <<http://fr.jeffprod.com/blog/2014/comment-programmer-un-jeu-dechecs.html>>
- Tom Kerrigan's Simple Chess Program, <<http://www.tckerrigan.com/Chess/TSCP>>
- Wikipédia : Le Turc mécanique, <https://fr.wikipedia.org/wiki/Turc_m%C3%A9canique>
- Wikipédia : Licence publique générale GNU, <https://fr.wikipedia.org/wiki/Licence_publique_g%C3%A9n%C3%A9rale_GNU>
- Wikipédia : Intelligence artificielle, <https://fr.wikipedia.org/wiki/Intelligence_artificielle>
- Wikipédia : Top Chess Engine Championship, <https://fr.wikipedia.org/wiki/Top_Chess_Engine_Championship>
- Wikipédia : Stockfish (programme d'échecs), <[https://fr.wikipedia.org/wiki/Stockfish_\(programme_d%27%C3%A9checs\)](https://fr.wikipedia.org/wiki/Stockfish_(programme_d%27%C3%A9checs))>

15.10.2 Pour aller plus loin

- *Computer Games I & II*, édité par David N. L. Levy, 1988
- *Échecs et C : Initiation à l'analyse et à la programmation du jeu d'échecs*, Yann Takvorian, 1993
- *Techniques de programmation des jeux*, David Levy, Editions du PSI, 1983
- *How Computers Play Chess*, David N. L. Levy & Monty Newborn, Ishi Press, 2009
- *Intelligence artificielle : une approche ludique*, Tristan Cazenave, Ellipse, 2011



Chess IBM 704 (1957)